



Phaser 4

a guide for OO programmers

2D games in TypeScript, with Phaser 4 and Deno

Contents

PART 1 - THE IDEAS

1 Introduction: your first Phaser 4 game

2 A three-scene game

3 Preloading

4 The life of a scene

5 Audio

6 Scoring

7 Animations

8 Collisions

9 2D physics

10 Tilemaps

11 Tilemaps with Tiled

PART 2 - GAMES

12 Higher or lower

13 Memory match

14 Catch, avoid and shoot

15 Platformer

Phaser 4 - a guide for OO programmers

2D games in TypeScript, with Phaser 4 and Deno

A practical guide to making 2D browser games with [Phaser 4](#), written for programmers who know object-oriented programming from Java (and perhaps a little JavaScript), and who are learning TypeScript as they go. Every chapter comes with Celbridge projects to build, run and change, and ends with six challenges, from simple to advanced.

The first eleven chapters introduce the ideas every Phaser game is made of. The last four each build a complete game in a classic genre, in simple, intermediate and advanced versions.

Getting started

You need [Deno](#) 2.4 or later, and (optionally) Celbridge. Each project builds and runs the same way - open its `terminal.console`, press **build**, then **serve**, then open `game.webview` (or <http://127.0.0.1:8000/>). Chapter 1 explains it all. Phaser is the only package a project downloads; the first build needs an internet connection, after that everything works offline.

Chapters

Part 1 - The ideas

1. [Introduction: your first Phaser 4 game](#) What Phaser is, how a project is built and run with Deno and Celbridge, and the TypeScript you need as a Java programmer. Meets the three parts of every game - a config, scenes and game objects - the coordinate system, and the game loop, with a ball that moves itself at a speed in pixels per second.
2. [A three-scene game](#) A complete little game: press SPACE to start, click the bouncing ball to win, see your time. Covers starting one scene from another, passing data between scenes with `init(data)` and interfaces, clicking on game objects, and the registry - plus the trap that catches everyone: scenes are reused.
3. [Preloading](#) How Phaser's loader works: the boot-and-preload scene pattern, a progress bar driven by loader events, and loading every kind of asset - images, sprite sheets,

audio, JSON data and asset packs. Also loading more assets in the middle of a game, and what to do when a file fails to load.

4. **[The life of a scene](#)** Exactly what Phaser calls, and when: `init` , `preload` , `create` , `update` , and `preUpdate` on game objects, with `time` and `delta` . Scene events, and the scene manager's other powers - running scenes side by side, pausing, sleeping and waking - used for a pause menu and a HUD.
5. **[Audio](#)** Sound effects and music: loading and playing sounds, volume, looping, rate and detune, music that carries on across scenes, a mute button, fading with tweens, and the browser's rule that no sound plays until the player has interacted with the page.
6. **[Scoring](#)** Keeping score properly: a HUD in its own scene, running on top of the game; events and the registry's change events so the HUD updates itself; and a high-score table that survives closing the browser, using local storage.
7. **[Animations](#)** Making things move: sprite sheets and frame animations (`this.anims.create` , `play` , animation events), flipping a character to face its way, a small animation state machine, and tweens for smooth movement, scaling, fading and chains of effects - plus particle bursts.
8. **[Collisions](#)** Knowing when things touch: geometric tests by hand with `Phaser.Geom` , then Arcade Physics bodies - `overlap` versus `collider` , groups against groups, callbacks, and body sizes and shapes - building up to a breakout game.
9. **[2D physics](#)** Arcade Physics in depth: velocity, acceleration, gravity, drag, bounce, maximum speed, angular movement, static and immovable bodies, and wrapping round the world - with an Asteroids-style ship. Ends with a first look at Matter.js, Phaser's other physics engine, for stacking and tumbling shapes.
10. **[Tilemaps](#)** Building worlds from tiles: a tilemap from a 2D array, a tileset image, layers, collision by tile index, changing tiles while the game runs, and a camera that follows the player round a map bigger than the screen - with a little tile painter.
11. **[Tilemaps with Tiled](#)** Designing levels in [Tiled](#), the free map editor: tilesets, tile layers, object layers and custom properties, exporting to JSON, and loading the result into Phaser - with collision by property, spawn points and pickups placed in the editor, and doors between maps.

Part 2 - Games

12. [Higher or lower](#) A card game: guess whether the next card is higher or lower, and get five right to win. A `Card` and a `Deck` class, a proper shuffle, a flip animation, buttons, and game state - in simple, intermediate and advanced versions.
 13. [Memory match](#) Turn over two tiles; keep them if they match. A grid laid out in code, flip tweens, a small state machine that stops the player clicking while tiles are showing, moves and time, levels and star ratings.
 14. [Catch, avoid and shoot](#) The left-right game: a player at the bottom of the screen who catches good things, dodges bad ones, and fires at enemies. Spawning with timers, object pools, waves, power-ups, lives, and particle explosions.
 15. [Platformer](#) Run and jump: gravity and ground checks, a responsive jump, an animated hero, a tilemap level, collectables, patrolling enemies you can stomp, hazards, and - in the advanced version - moving platforms, checkpoints and several levels.
-

Also in this guide

- `assets/` - every picture, sprite sheet, tileset and sound the projects use, all generated by code, and free to use
- `teacher/` - for teachers: notes, worked solutions (with a runnable project for every challenge) and MARP slides for each chapter. Kept separate so the chapters can be given to students on their own
- `README_build_pdfs.md` - how to make PDFs of the book, the teacher guide, single chapters and slide decks (they are in `_PDFs/`)
- `template/` - the starting point for a new project
- `tools/` - how the guide is put together, and the tools used to make it



Introduction: your first Phaser 4 game

Phaser is a library for making 2D games that run in a web browser. In this chapter you set up the tools, look inside a Phaser project, meet just enough TypeScript to read it, and run two small games: one that shows pictures and text and responds to a key, and one with a ball that moves by itself.



What you will learn

- what Phaser is, and what is new in Phaser 4
- how a project is built and run with Deno, and opened in Celbridge
- the TypeScript you need, explained for Java programmers
- the three parts of every Phaser game: a **config**, **scenes**, and **game objects**
- how the screen's coordinates work, and what an **origin** is
- what the **game loop** is, and why speeds are measured in pixels per **second**

The projects

Project	What it shows
<code>ch01_hello_phaser</code>	a config, one scene, a loaded picture, text, and a key that does something
<code>ch01_moving_ball</code>	a class of our own that extends a Phaser game object and moves itself every frame

What is Phaser?

[Phaser](#) is a free, open source game framework for the web. You write the game's rules; Phaser does the rest - it draws everything (fast, using the graphics card), runs the game loop, loads pictures and sounds, reads the keyboard, mouse and touch screen, plays audio, runs the physics, and much more. A Phaser game is a web page, so it runs on anything with a browser: Windows, macOS, Linux, phones, tablets.

Phaser 4 came out in 2025. Almost everything you write is the same as in Phaser 3 - scenes, sprites, physics, input, tweens all work the way they did - but the drawing engine underneath was rewritten, and some effects changed. That matters when you search online: most tutorials and answers were written for Phaser 3. They are nearly always still right, but if something from a tutorial does not compile, check the [Phaser 4 migration notes](#) - a handful of features (such as `setTintFill`, bitmap masks and the old "FX" effects) work differently now.

Note - Phaser itself is written in JavaScript, but it comes with complete TypeScript type definitions. That means your editor, and the build, know the type of every Phaser class, method and parameter, and tell you when you get one wrong - long before you run the game.

The tools

Every project in this guide is built the same way, with **Deno** and nothing else to install.

- **Deno** runs TypeScript directly, checks types, bundles code for the browser, and serves web pages - all built in. The only package a project downloads is Phaser itself (the first build fetches it; after that everything works offline).
- **Celbridge** opens each project folder as a project. Two files in every project matter: `terminal.console` (a terminal with buttons for each command) and `game.webview` (a browser panel pointed at the running game).

Running a project

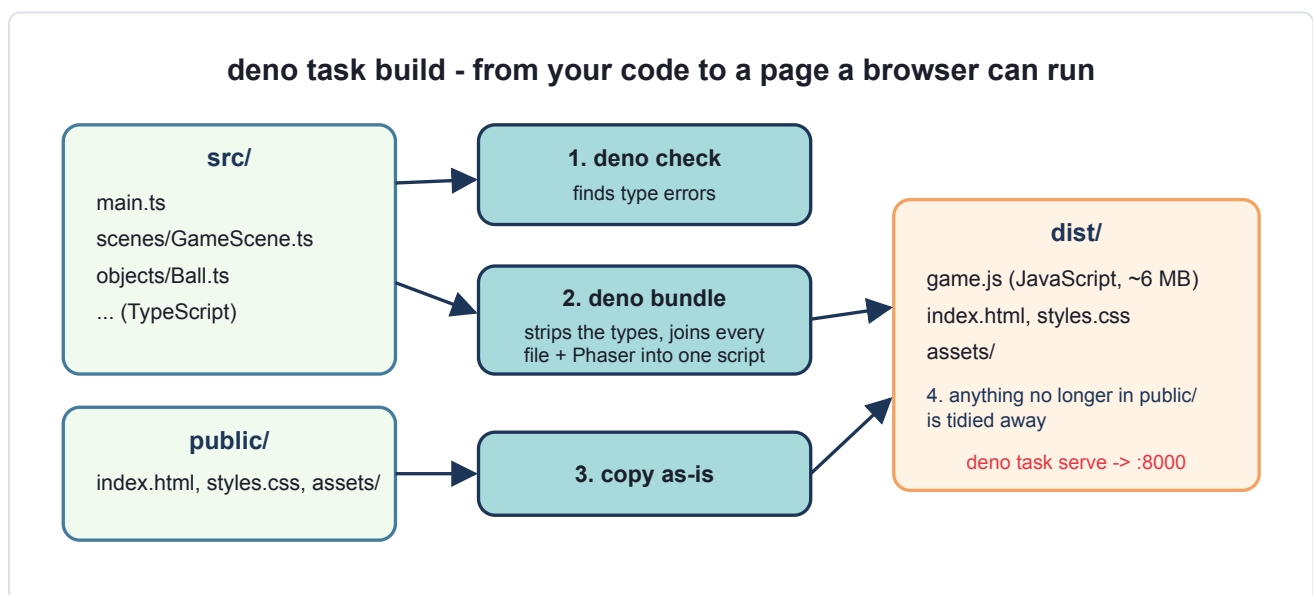
Open the project in Celbridge, open `terminal.console`, and press its buttons in order:

1. **build** runs `deno task build`
2. **serve** runs `deno task serve`, which serves the game at <http://127.0.0.1:8000/>

Then open `game.webview` to play. After you change the code, build again and refresh the game. (`deno task dev` rebuilds every time you save - handy in a second terminal.)

Without Celbridge, the same commands work in any terminal, in the project's folder, and any browser can open <http://127.0.0.1:8000/>.

What "build" does



Browsers cannot run TypeScript, and a game made of many files, plus Phaser, needs joining up. So `build.ts` (which you never need to edit) does four things:

1. **type checks** every `.ts` file - and lists any errors. The game is still built, so you can keep experimenting, but read the errors: they are nearly always a real bug

2. **bundles** `src/main.ts` , every file it imports, and Phaser, into one plain JavaScript file, `dist/game.js`
3. **copies** everything in `public/` (the web page, its CSS, and the game's pictures and sounds) into `dist/`
4. **tidies** `dist/` : anything that is no longer in `public/` (a picture you deleted, say) is removed. `dist/` itself is kept, not deleted and made again, so a page that is showing the game carries on working while you rebuild

Why the game has to be served

Phaser loads pictures and sounds by asking for them over the web, and browsers refuse to do that for a page opened straight from a file (`file:///...`). `deno task serve` runs a tiny web server on your own machine, so the page and its files come from

`http://127.0.0.1:8000/` instead. Nothing is sent anywhere - `127.0.0.1` means "this computer".

A project's files

```
ch01_hello_phaser/
  src/
    main.ts           the game's config - where it all starts
    scenes/
      HelloScene.ts   the scene - what the game does
  public/
    index.html        the web page: a <div id="game"> and a list of the
controls
    styles.css         how the page looks
    assets/images/     the pictures the game loads
  build.ts            the build script - never needs editing
  deno.json           the tasks (build, serve, dev, check) and the one
import: Phaser
  terminal.console     Celbridge's terminal, with a button per task
  game.webview         Celbridge's browser panel
  dist/               the built game - made by the build, never edited
by hand
```

TypeScript for Java programmers

TypeScript is JavaScript with **types** added. If you know Java, most of it will look familiar; the differences are small but they catch everybody out once. Here is what this guide uses.

Variables - `const` for a value that never changes, `let` for one that does. Never `var` .

```
const SPEED = 200;           // type worked out: number
let score = 0;               // type worked out: number
let name: string = "Ada";    // type written out
```

The type goes **after** the name, with a colon. TypeScript works types out for itself wherever it can, so you write them mostly on fields, parameters and return values.

One number type. There is no `int` , `float` or `double` - just `number` . `7 / 2` is `3.5` ; use `Math.floor(7 / 2)` for whole-number division.

Equality - always `===` and `!==` . (JavaScript's `==` converts types first, so `"1" == 1` is true. `===` never does.)

Strings - `"double"` or `'single'` quotes, or backticks for a **template string**, which puts values straight into the text:

```
this.message.setText(`Presses: ${this.presses}`);
```

Classes look like Java, with a few differences:

```

export class GameScene extends Phaser.Scene {
    private score: number; // a field: name, then type

    constructor() { // the constructor is
        // called `constructor`
        super("GameScene"); // must call super(...)
        // before using `this`
        this.score = 0; // fields are ALWAYS
        // reached through `this.`
    }

    override update(_time: number, delta: number): void { // return type
        // after the brackets
        this.score = this.score + delta / 1000;
    }
}

```

- fields and methods are reached with `this.` - always. There is no implicit `this` as in Java
- `private`, `protected` and `public` work as in Java (public is the default)
- `override` marks a method that replaces one in the parent class - here, `Phaser.Scene` 's own (empty) `update()` . These projects make it compulsory: leave it out and the build says `This member must have an 'override' modifier` . (And put it on a method that does *not* replace anything, and the build says that too)
- a parameter name starting with `_` (like `_time`) is the usual way to say "I have to accept this, but I do not use it"

Fields set later. A field that gets its value in `create()` , not in the constructor, is declared with a `!` :

```

private message!: Phaser.GameObjects.Text;

```

The `!` says "trust me, this will be set before it is used". Without it, TypeScript complains that the field might never be given a value. (A `!` after an expression, as in `this.input.keyboard!` , means something similar: "this is definitely not `null` ".)

Arrow functions are short functions - like Java's lambdas:

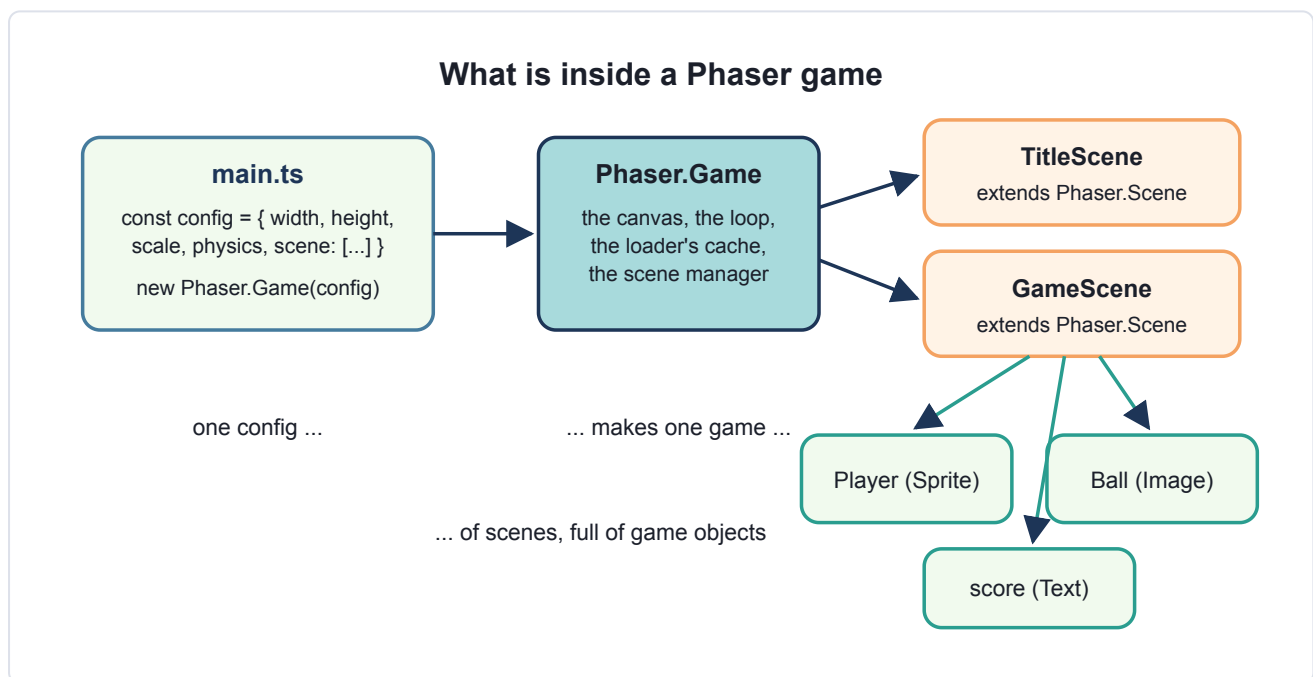
```
this.input.keyboard!.on("keydown-SPACE", () => {
  this.changeColour();
});
```

Inside an arrow function, `this` is still whatever it was outside - here, the scene. That is why the guide uses arrow functions for every callback.

Modules. Each file is a module. `export` lets something leave its file; `import` brings it in. Imports of your own files include the `.ts`:

```
import Phaser from "phaser";
import { HelloScene } from "../scenes/HelloScene.ts";
```

The three parts of a Phaser game



1. The config

`src/main.ts` describes the game and starts it:

`src/main.ts`


```
const config: Phaser.Types.Core.GameConfig = {
  title: "Hello Phaser",
  type: Phaser.AUTO,
  parent: "game",
  backgroundColor: "#1d2433",
  scale: {
    width: 800,
    height: 600,
    mode: Phaser.Scale.FIT,
    autoCenter: Phaser.Scale.CENTER_BOTH,
  },
  scene: [HelloScene],
};

new Phaser.Game(config);
```

- `type: Phaser.AUTO` - draw with WebGL (the graphics card) if possible, which it nearly always is
- `parent: "game"` - put the game's `<canvas>` inside `<div id="game">` in `index.html`
- `scale` - the game is **always 800 x 600 inside**, whatever size it appears on the page. `Phaser.Scale.FIT` stretches or shrinks it to fit its `<div>`, keeping its shape. Your code only ever deals with 800 x 600
- `scene` - the list of scenes. Phaser makes one of each, and starts the **first** one

The type `Phaser.Types.Core.GameConfig` means TypeScript checks the whole object: misspell `backgroundColor` as `backgroundColour` and the build tells you.

2. Scenes

A **scene** is one screen of the game - a title screen, a level, a game over screen. You write a class that extends `Phaser.Scene`, and fill in the methods Phaser calls for you:

Method	When Phaser calls it	What it is for
<code>constructor()</code>	once, when the game starts	give the scene its key (its name): <code>super("HelloScene")</code>
<code>preload()</code>	when the scene starts	load the pictures and sounds it needs
<code>create()</code>	once everything has loaded	make the game objects
<code>update(time, delta)</code>	every frame, from then on	the game's rules

`src/scenes/HelloScene.ts`

```
preload(): void {
  this.load.image(LOGO_KEY, LOGO_FILE);
  this.load.image(STAR_KEY, STAR_FILE);
}
```

`this.load.image(key, file)` asks the **loader** for a picture. The **key** is the name you use for it from then on; the file path is relative to `index.html`. Chapter 3 is all about loading.

3. Game objects

Everything you see is a **game object**: an image, a sprite, some text, a shape. `this.add` is the scene's **factory** for them - each call makes one, puts it in the scene, and hands it back:

```
const centreX = this.scale.width / 2;

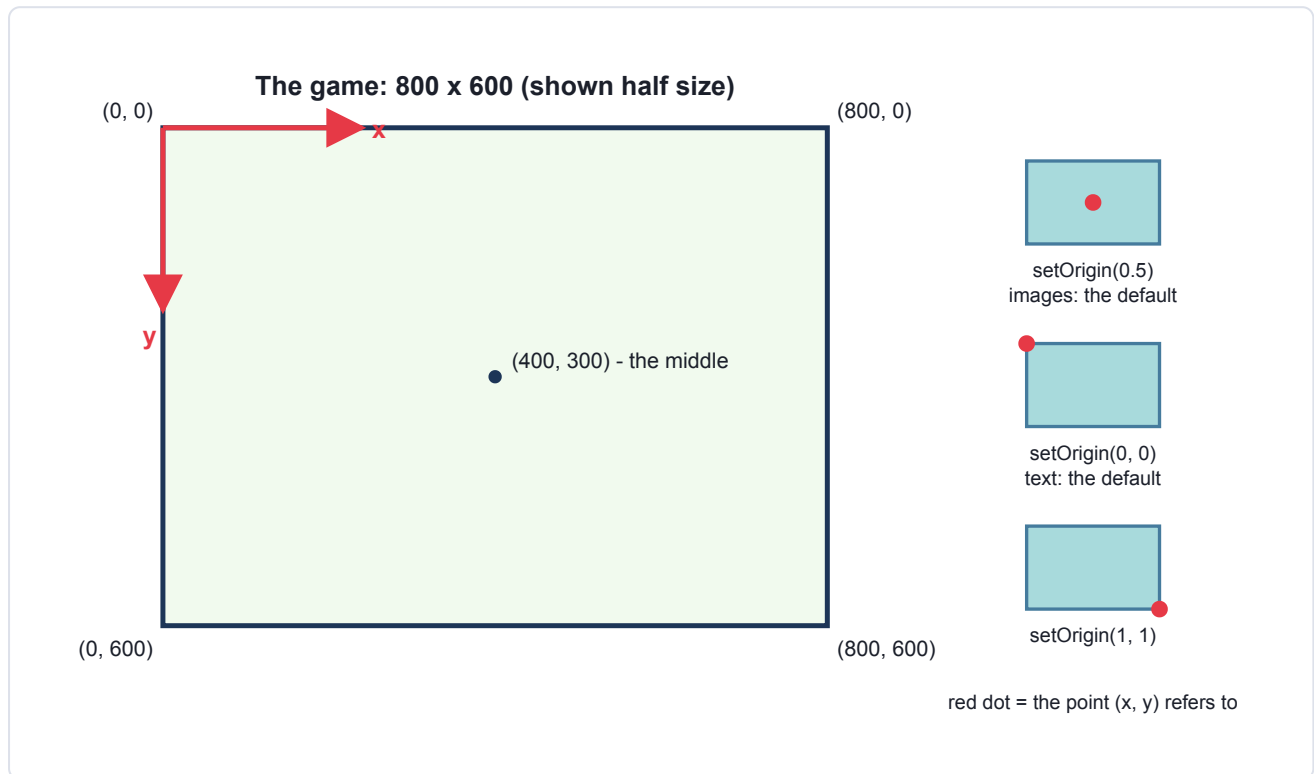
this.add.image(centreX, 200, LOGO_KEY);

this.message = this.add.text(centreX, 380, "Press SPACE", {
  fontFamily: "Arial",
  fontSize: "36px",
  color: "#ffffff",
});
this.message.setOrigin(0.5);
```

Once a game object is in the scene, **Phaser draws it, every frame, until it is removed**. You never draw anything yourself. To change what you see, change the game object - its position, its text, its picture - and the next frame shows the change.

The scene keeps the text in a field (`this.message`) because it changes it later. The logo never changes, so nothing keeps hold of it.

Coordinates and origins



(0, 0) is the **top left** corner. `x` grows to the right and `y` grows **downwards** - the opposite of the graphs you drew at school, and the same as Java's Swing and every other screen.

Every game object has an **origin**: the point on the object that (x, y) refers to. An image's origin starts in its middle - so `this.add.image(400, 300, ...)` centres the picture on the screen. Text starts with its origin at its top left. `setOrigin(x, y)` moves it: 0 is the left or top edge, 1 is the right or bottom edge, 0.5 is half way.

The stars in the Hello Phaser project use origins to sit exactly in each corner, with no sums:

```

this.add.image(0, 0, STAR_KEY).setOrigin(0, 0);
// top left
this.add.image(this.scale.width, 0, STAR_KEY).setOrigin(1, 0);
// top right
this.add.image(0, this.scale.height, STAR_KEY).setOrigin(0, 1);
// bottom left
this.add.image(this.scale.width, this.scale.height, STAR_KEY).setOrigin(1,
1);      // bottom right

```

Most methods that set something return the game object itself, so calls can be **chained** like this. The origin is also the point an object rotates and scales around.

Responding to a key

```

this.input.keyboard!.on("keydown-SPACE", () => {
    this.changeColour();
});

```

`this.input.keyboard` is the scene's keyboard. `.on(event, function)` says "when this happens, run this function" - an **event listener**, like adding an `ActionListener` in Java. The event name is `keydown-` plus the key's name (`SPACE`, `ENTER`, `A`, `LEFT`, ...).

```

private changeColour(): void {
    const colour = Phaser.Utls.Array.GetRandom(COLOURS);
    this.cameras.main.setBackgroundColor(colour);

    this.presses = this.presses + 1;
    this.message.setText(`Presses: ${this.presses}`);
}

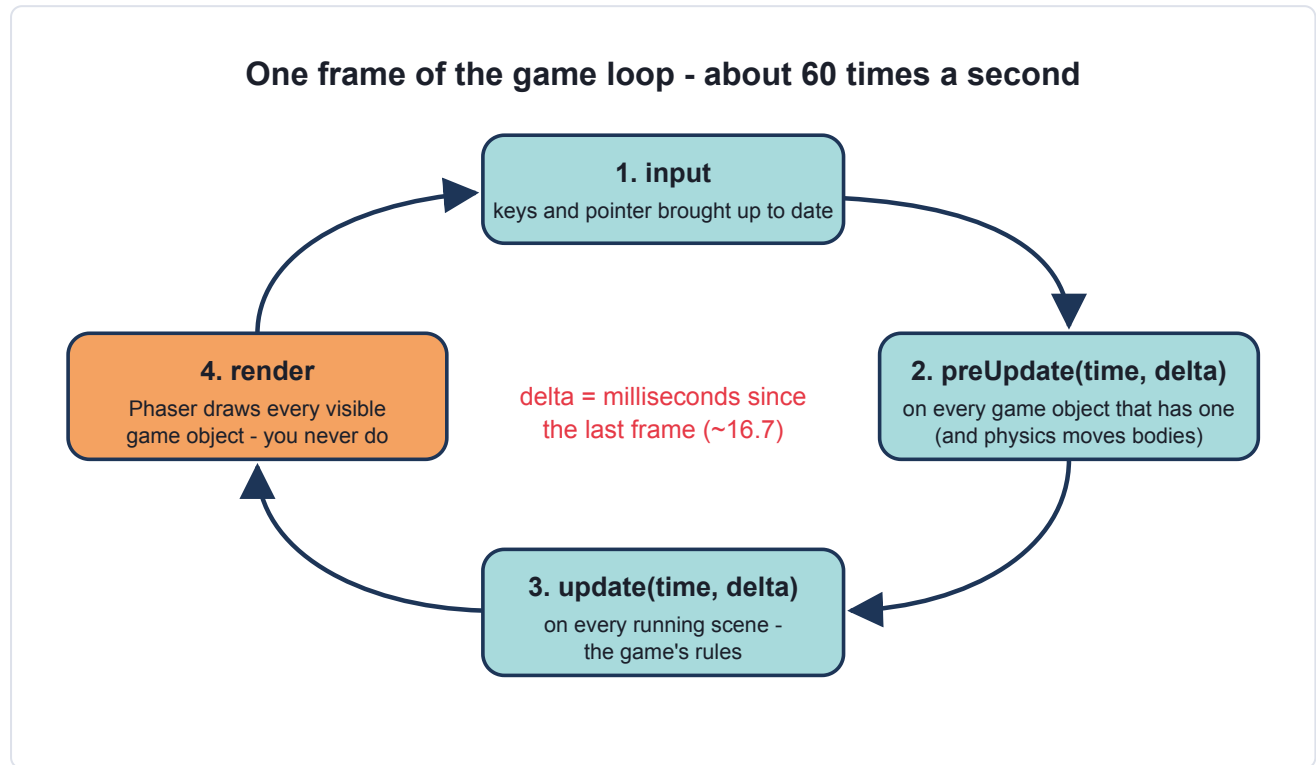
```

Every scene has a **camera** - `this.cameras.main` - which is what looks at the scene and shows it on screen. Its background colour is painted behind everything else.

Try it - build and serve `ch01_hello_phaser`, click on the game (so it has the keyboard), and press SPACE.

The game loop

A game that only changes when a key is pressed is not much of a game. Games change **all the time**: things move, timers count down, enemies think. That happens in the **game loop**.



Phaser runs the loop for you, in time with the screen - usually 60 times a second. Each trip round the loop is a **frame**. In each frame Phaser reads the input, lets every game object and every scene work out what has changed, and then draws everything where it now is.

You join in by writing methods that Phaser calls every frame:

- `update(time, delta)` in a **scene** - the game's rules
- `preUpdate(time, delta)` in a **game object** of your own - the object looks after itself



A game object that moves itself

`ch01_moving_ball` has a class of its own, `Ball`, which **extends** Phaser's `Image`. A ball is an image - it has a picture, a position and a size, and Phaser draws it - plus the ability to move:

`src/objects/Ball.ts`

```
export class Ball extends Phaser.GameObjects.Image {
  private speedX: number;
  private speedY: number;

  constructor(scene: Phaser.Scene, x: number, y: number, speedX: number,
    speedY: number) {
    super(scene, x, y, BALL_KEY);

    this.speedX = speedX;
    this.speedY = speedY;

    scene.add.existing(this);
  }
}
```

`this.add.image(...)` both makes an image and adds it to the scene. When you make a game object of your own with `new`, you have to add it yourself -

`scene.add.existing(this)` does that. It also notices that `Ball` has a `preUpdate()` method, and signs it up to be called every frame.

The scene makes a ball and forgets about it:

`src/scenes/GameScene.ts`

```
create(): void {  
    new Ball(this, 400, 300, 240, 180);  
    ...  
}
```

Speeds in pixels per second

```
preUpdate(_time: number, delta: number): void {  
    const seconds = delta / 1000;  
  
    this.x = this.x + this.speedX * seconds;  
    this.y = this.y + this.speedY * seconds;  
    ...  
}
```

(No `override` here: `Image` has no `preUpdate()` of its own for this one to replace. A `Sprite` does, and Chapter 4 shows what changes then.)

`delta` is how long the last frame took, in **milliseconds** - about 16.7 at 60 frames a second. Dividing by 1000 gives seconds, and **speed x time = distance**. So `speedX = 240` means 240 pixels every second, on every computer.

Why not just add a few pixels every frame? Because screens differ. Most run at 60 frames a second, but many laptops and gaming monitors run at 120 or 144. "Add 4 pixels a frame" would make the ball more than twice as fast on those screens. Measuring time instead of counting frames makes the game play the same everywhere. (From Chapter 8 onwards, Arcade Physics does this sum for you.)

Bouncing

```
const radius = this.width / 2;
const right = this.scene.scale.width - radius;
const bottom = this.scene.scale.height - radius;

if (this.x < radius || this.x > right) {
    this.speedX = -this.speedX;
    this.x = Phaser.Math.Clamp(this.x, radius, right);
}
```

The ball's `(x, y)` is its middle, so its edge touches the side of the screen when `x` is one radius from it. Turning round is just flipping the sign of the speed. `Clamp` puts the ball back inside the edge, in case it went a little past it during the frame.

`this.scene` - every game object knows which scene it is in, so the ball can ask the scene's scale manager how big the game is.

Where to find out more

- the Phaser site, <https://phaser.io/>, has the API documentation and hundreds of examples
- your editor knows Phaser's types: hover over any Phaser method to see what it takes and returns, or type `this.add.` and look at the list
- when a tutorial written for Phaser 3 does not work, the type checker usually points straight at the line that changed

Summary

- A Phaser game is a **config** (in `main.ts`), a list of **scenes**, and the **game objects** in them. Phaser draws every game object, every frame
- a scene is a class that extends `Phaser.Scene`; Phaser calls its `preload()`, `create()` and `update()` at the right times
- `this.add.image(...)`, `this.add.text(...)` make game objects; `setOrigin(...)` says which point of the object `(x, y)` refers to
- `(0, 0)` is the top left; `y` grows downwards; the game is always 800 x 600 inside

- `this.input.keyboard!.on("keydown-SPACE", () => { ... })` responds to a key
 - the **game loop** calls `preUpdate()` on game objects and `update()` on scenes every frame; `delta` is the length of the last frame, in milliseconds - use it to move things in pixels per second
 - build with `deno task build`, serve with `deno task serve`, play at <http://127.0.0.1:8000/>
-

Challenges

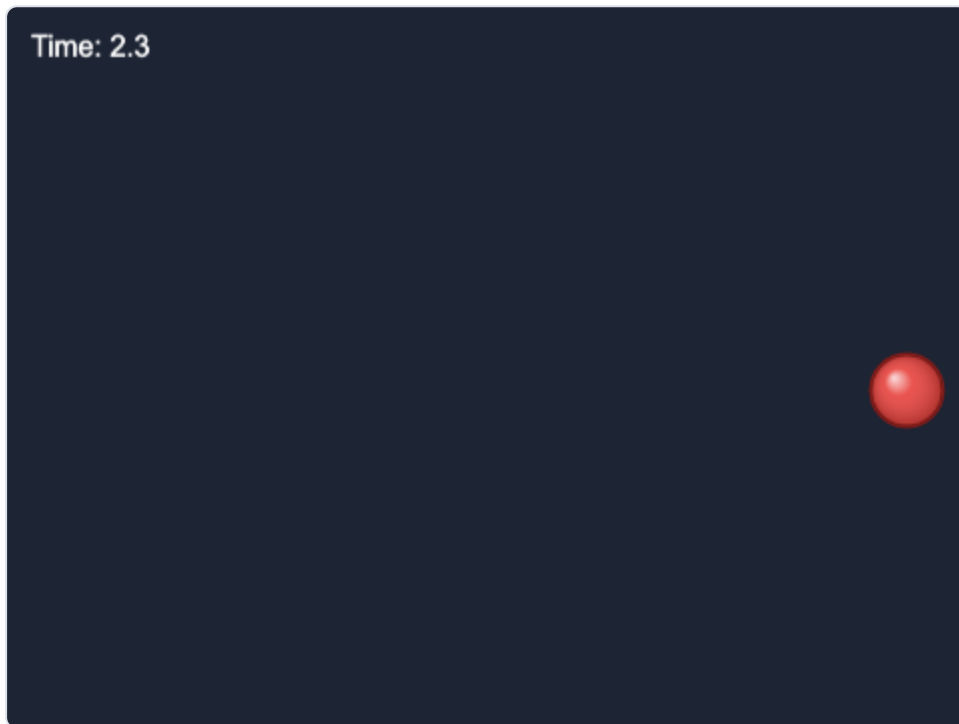
1. **Make it yours** (*ch01_hello_phaser*) - Change the game's title (in the config and in `index.html`), the starting background colour, and the message's size and colour. Add two colours of your own to the `COLOURS` list.
 2. **Corner labels** (*ch01_hello_phaser*) - Next to each corner star, add a small piece of text showing that corner's coordinates - `(0, 0)`, `(800, 0)` and so on - lined up against the edges of the screen. Use origins rather than working out text widths.
 3. **Reset key** (*ch01_hello_phaser*) - Add a second key, R, that sets the press count back to 0, puts the background back to its starting colour, and changes the message back to "Press SPACE".
 4. **More balls** (*ch01_moving_ball*) - Make five balls, each starting in a random place and moving at a random speed in a random direction. Use a loop, and `Phaser.Math.Between(min, max)`. *Hint*: a speed can be negative - that is a direction.
 5. **Bounce counter** (*ch01_moving_ball*) - Count how many times the ball has hit an edge, and show the total on screen. The ball knows when it bounces; the scene owns the text. *Hint*: give `Ball` a way for the scene to ask it how many times it has bounced, and have the scene ask in `update()`.
 6. **Steer the ball** (*ch01_moving_ball*) - Let the arrow keys **push** the ball: each arrow held down adds speed in its direction, a little every frame (so holding it longer makes the ball faster), up to a top speed of 600 pixels a second in each direction. Show the ball's current speed on screen. *Hint*: the scene reads the keys; give `Ball` a public method such as `push(amountX, amountY)` so the scene can ask the ball to speed up - and remember that "a little every frame" needs `delta` too.
-

Next: [Chapter 2 - A three-scene game](#)



A three-scene game

Most games are more than one screen: a title screen, the game itself, and a screen at the end. In Phaser each of those is a **scene**. In this chapter you build a complete little game in three scenes - press SPACE to start, click a bouncing ball as fast as you can, and see how long you took - and learn how scenes start each other, how they pass data along, and how a game object responds to the mouse.



What you will learn

- how a game is split into scenes, and how one scene starts another
- how to pass data from one scene to the next with `init(data)`, checked by an `interface`
- why a scene's fields keep their old values when it starts again, and how `init()` fixes that
- how to make a game object clickable, with `setInteractive()` and `"pointerdown"`
- the difference between `on` and `once` for events
- how to keep values that every scene can see, in the game's **registry**

The projects

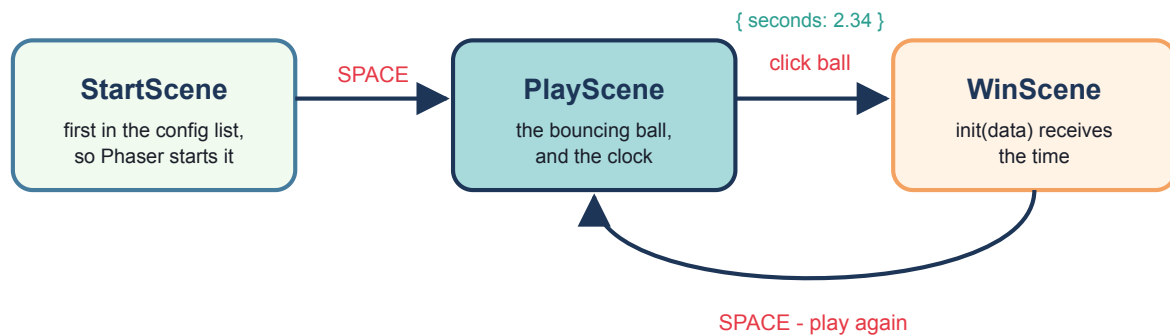
Project	What it shows
<code>ch02_click_the_ball</code>	the three-scene game at its simplest: start, click the ball, see your time
<code>ch02_click_the_ball_plus</code>	the same game, made better: five hits to win, a ball that speeds up, misses that cost time, sounds, and a best time

Three scenes



Here is the whole game, as a picture:

Three scenes, and what moves the game from one to the next



each arrow is `this.scene.start(KEY, data?)` - stop this scene, start that one

Each box is a class that extends `Phaser.Scene`, in its own file in `src/scenes/`. The config lists all three:

`src/main.ts`

```
scene: [StartScene, PlayScene, WinScene],
```

When the game begins, Phaser makes **one object of each scene class**, and starts the **first** one in the list. The others wait. From then on, the scenes themselves decide what comes next.

Scene keys

Every scene has a **key** - its name - which it gives to `super(...)` in its constructor. Scenes start each other by key, so the keys are kept together as constants:

`src/scenes/keys.ts`

```
export const START_SCENE = "StartScene";
export const PLAY_SCENE = "PlayScene";
export const WIN_SCENE = "WinScene";
```

`src/scenes/PlayScene.ts`

```
constructor() {
  super(PLAY_SCENE);
}
```

Why bother? Because `this.scene.start("Playscene")` - one letter wrong - does nothing at all, and says nothing about it. Misspell a constant - `this.scene.start(PLAYSCENE)` - and it is a build error you cannot miss. Whenever a string is used as a name in more than one place, make it a constant.

Starting a scene

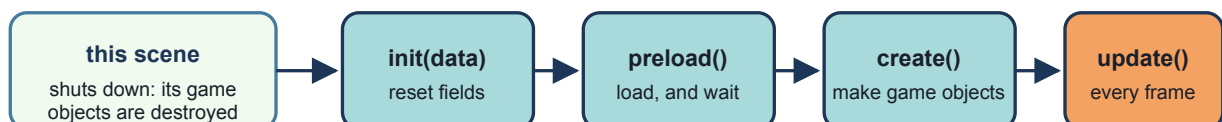
`src/scenes/StartScene.ts`

```
this.input.keyboard!.once("keydown-SPACE", () => {  
    this.scene.start(PLAY_SCENE);  
});
```

`this.scene` is the scene's **scene plugin** - its way of talking to Phaser's scene manager. `this.scene.start(key)` does two things:

1. it **shuts down** this scene - every game object in it is destroyed, its listeners are removed, and Phaser stops calling its `update()`
2. it **starts** the scene called `key` - which runs that scene's `init()`, `preload()`, `create()`, and then `update()` every frame

What `this.scene.start(PLAY_SCENE, data)` does



The scene OBJECT is not new: Phaser made it once, when the game began, and starts it again.
Its fields keep their old values - which is why `init()` puts them back.

on and once

Chapter 1 used `on(...)` : "every time this happens, run this". Here it is `once(...)` : "the first time this happens, run this, then stop listening". SPACE should start the game one time, and `once` says exactly that.

In this game `on` would happen to work too: `this.scene.start(...)` shuts the start scene down, and a scene's listeners are removed when it shuts down. But `this.scene.start` does not switch scenes instantly - it happens at the start of the next frame - and a scene that *keeps running* (a pause menu on top of the game, say, or a HUD)

would react to every press. Use `once` whenever something should only happen once; it says what you mean.

Loading once, for everyone

The start scene runs first, so it loads everything:

`src/scenes/StartScene.ts`

```
preload(): void {  
    this.load.image(LOGO_KEY, LOGO_FILE);  
    this.load.image(BALL_KEY, BALL_FILE);  
}
```

Loaded pictures and sounds belong to the **game**, not to the scene that loaded them. Once "ball" is loaded, `PlayScene` can use it without loading it again - so `PlayScene` has no `preload()` at all. (Chapter 3 gives loading a scene of its own, with a progress bar.)

Making the ball clickable

`PlayScene` uses the `Ball` class from Chapter 1, unchanged: an `Image` that moves itself in `preUpdate()` and bounces off the edges. The scene makes one, heading off in a random direction:

`src/scenes/PlayScene.ts`

```
const angle = Phaser.Math.Angle.Random();  
const ball = new Ball(this, 400, 300, Math.cos(angle) * BALL_SPEED,  
    Math.sin(angle) * BALL_SPEED);
```

`Angle.Random()` is a random angle (in radians); `cos` and `sin` split the speed into its across and down parts, so the ball always moves at `BALL_SPEED` whichever way it goes.

Game objects ignore the mouse until you ask:

```
ball.setInteractive({ useHandCursor: true });

ball.on("pointerdown", () => {
    this.win();
});
```

- `setInteractive()` tells Phaser's input system to watch this game object. By default Phaser checks the pointer against the object's rectangle (its picture's bounds); Chapter 8 shows other shapes
- `{ useHandCursor: true }` shows a pointing hand while the mouse is over it
- `ball.on("pointerdown", ...)` - every game object can have listeners. The **pointer** is Phaser's word for the mouse *or* a finger on a touch screen, so the same code works on a phone

Other pointer events you can listen for on a game object: `"pointerup"`, `"pointerover"` (the pointer has moved onto it), `"pointerout"` (and off again), and `"pointermove"`.

Notice that `Ball` does not know it can be clicked. The **scene** decides that - so the same `Ball` class works in Chapter 1, where it cannot be clicked, and here, where it can.

Timing the round

`src/scenes/PlayScene.ts`

```
create(): void {
    ...
    this.startTime = this.time.now;
}

override update(time: number, _delta: number): void {
    const seconds = (time - this.startTime) / 1000;
    this.timeText.setText(`Time: ${seconds.toFixed(1)}`);
}
```

`this.time` is the scene's **clock**, and `this.time.now` is the time in milliseconds. The `time` passed to `update()` is the same clock. `toFixed(1)` turns a number into a string with one decimal place - `2.3412` becomes `"2.3"`.

Passing data to the next scene

When the ball is clicked, `WinScene` needs to know how long the player took. The second argument to `this.scene.start(...)` is handed to the new scene's `init()`:

`src/scenes/PlayScene.ts`

```
private win(): void {
    const seconds = (this.time.now - this.startTime) / 1000;

    const data: WinData = { seconds: seconds };
    this.scene.start(WIN_SCENE, data);
}
```

`src/scenes/WinScene.ts`

```
export interface WinData {
    seconds: number;
}

export class WinScene extends Phaser.Scene {
    private seconds = 0;

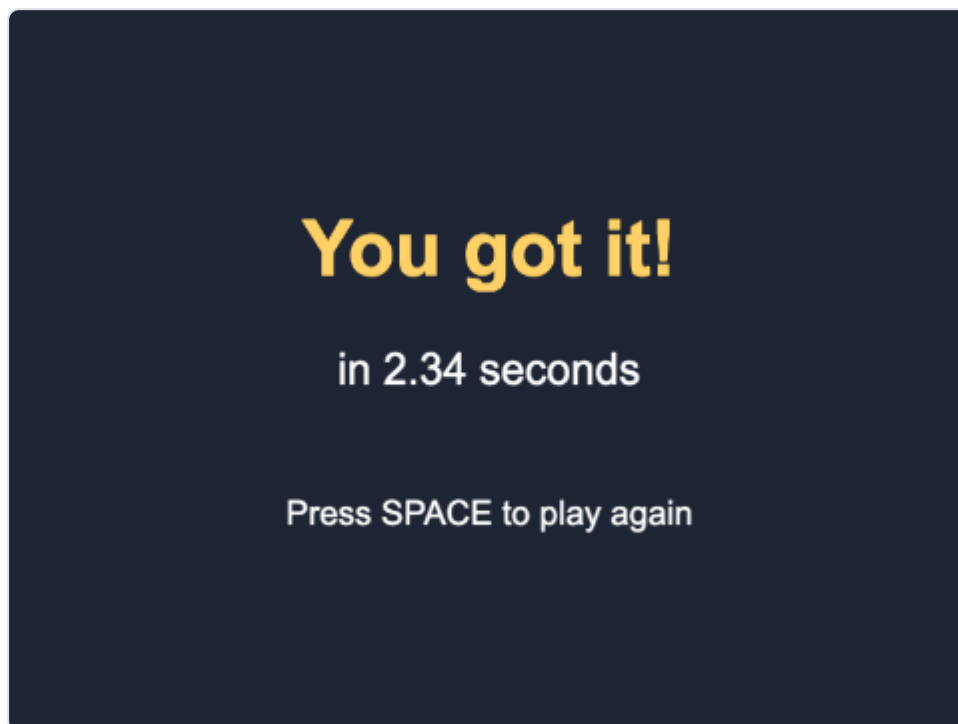
    init(data: WinData): void {
        this.seconds = data.seconds;
    }

    create(): void {
        ...
        this.add.text(centreX, 300, `in ${this.seconds.toFixed(2)} seconds`, {
            ... });
    }
}
```

`{ seconds: seconds }` is an **object literal** - JavaScript's quick way of making an object with named values, with no class needed.

An **interface** describes the shape an object must have - like a Java interface, but for data as well as methods. `WinData` says "an object with a `seconds` that is a number". Both scenes use it, so if `PlayScene` sent `{ secs: 2.3 }` or `{ seconds: "fast" }`, the build would say so.

`PlayScene` imports it with `import type { WinData } from "../WinScene.ts";`. Types disappear when the code is built, so `import type` brings in only the type, and costs nothing when the game runs.



Playing again - and the trap in it

`WinScene` starts `PlayScene` again when SPACE is pressed. It works first time, because `PlayScene` sets everything it needs in `create()`. But this is the place where almost everyone, once, writes a bug - and the plus version of the game shows why.

Phaser does not make a new scene object when a scene starts again. It made one `PlayScene` when the game began, and `this.scene.start(PLAY_SCENE)` starts that *same object* again. Its game objects are all new - `create()` makes them - but its **fields still hold whatever they held at the end of the last round.**

In the plus version, `PlayScene` counts hits and misses in fields:

```
ch02_click_the_ball_plus/src/scenes/PlayScene.ts
```

```
private hits = 0;
private misses = 0;
```

`= 0` there only happens **once**, when the object is made. Play a second round and `hits` would already be 5 - so the first click would win. The fix is `init()` , which runs every time the scene starts:

```
init(): void {  
    this.hits = 0;  
    this.misses = 0;  
}
```

Rule of thumb: anything that should start fresh every time a scene starts is set in `init()` or `create()` - not just where the field is declared.

The plus version



`ch02_click_the_ball_plus` is the same three scenes, with more game. It is worth reading all of it; here are the new ideas.

Hits, misses, and clicks on nothing

```
this.ball.on("pointerdown", () => {
    this.hit();
});

this.input.on("pointerdown", (_pointer: Phaser.Input.Pointer, over:
Phaser.GameObjects.GameObject[]) => {
    if (over.length === 0) {
        this.miss();
    }
});
```

`this.input.on("pointerdown", ...)` listens to **every** click in the scene, wherever it lands. Phaser passes the listener the pointer, and a list of the interactive game objects that were under it. An empty list means the click hit nothing - a miss. (Clicking the ball fires **both** listeners; the second sees the ball in the list and ignores it.)

The ball does what it is told

`Ball` gained two public methods, and the scene calls them after each hit:

`ch02_click_the_ball_plus/src/objects/Ball.ts`

```
public speedUp(factor: number): void {
    this.speedX = this.speedX * factor;
    this.speedY = this.speedY * factor;
}

public teleport(): void {
    const radius = this.width / 2;
    this.x = Phaser.Math.Between(radius, this.scene.scale.width - radius);
    this.y = Phaser.Math.Between(radius, this.scene.scale.height - radius);
}
```

The speeds stay private. The scene does not reach in and change them; it asks the ball to speed up, and the ball decides how. That is ordinary OO encapsulation - and it pays off the moment you want the ball to, say, flash when it speeds up: one place to change.

Sounds

```
// StartScene.preload()
this.load.audio(POP_KEY, POP_FILE);

// PlayScene.hit()
this.sound.play(POP_KEY);
```

Loading a sound is like loading a picture; `this.sound.play(key)` plays it. That is all this chapter needs - Chapter 5 is all about audio. (Browsers will not play any sound until the player has clicked or pressed a key on the page. Here the first sound comes after a click, so that is never a problem.)

The registry: values every scene can see

The best time has to outlive the scene that works it out - the start scene shows it. Every Phaser game has a **registry**: a store of named values shared by all its scenes.

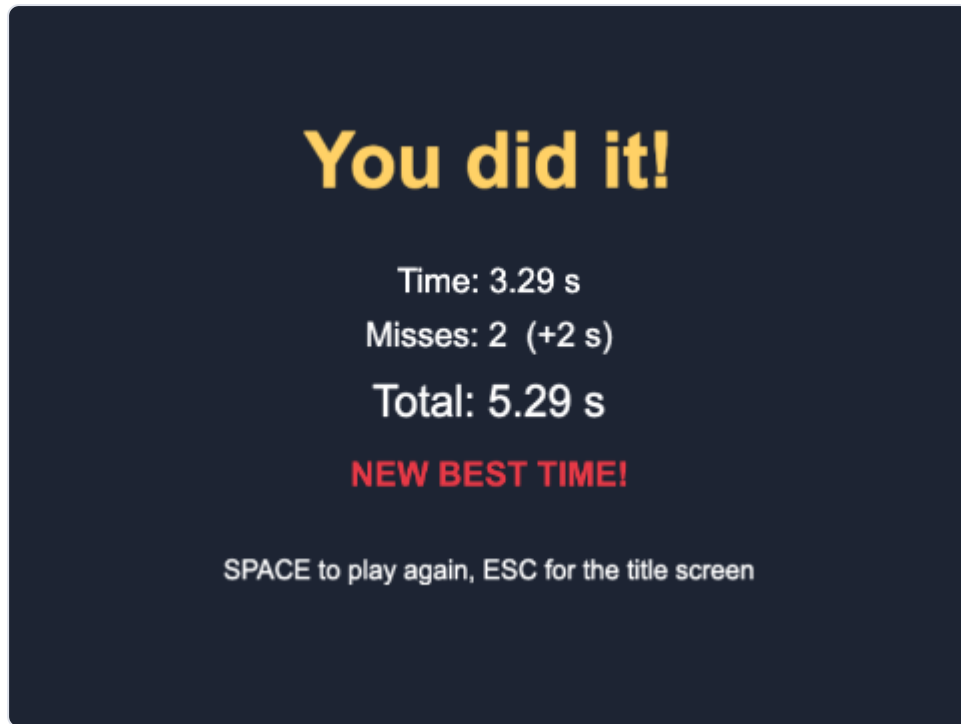
ch02_click_the_ball_plus/src/scenes/WinScene.ts

```
const best: number | undefined = this.registry.get(BEST_TIME);
const isNewBest = best === undefined || total < best;
if (isNewBest) {
    this.registry.set(BEST_TIME, total);
}
```

ch02_click_the_ball_plus/src/scenes/StartScene.ts

```
const best: number | undefined = this.registry.get(BEST_TIME);
const bestMessage = best === undefined ? "No best time yet" : `Best time:
${best.toFixed(2)} seconds`;
```

- `number | undefined` is a **union type**: "a number, or nothing". Until a game has been won, there is no best time, and `registry.get` gives back `undefined` - JavaScript's "no value here"
- `condition ? a : b` is the **conditional operator**, as in Java
- the registry lasts as long as the game - until the page is refreshed. Chapter 6 keeps scores for good, in the browser's storage



Spreading a style

```
const style = { fontFamily: "Arial", fontSize: "28px", color: "#ffffff" };

this.add.text(centreX, 130, "You did it!", { ...style, fontSize: "64px",
fontStyle: "bold", color: "#ffd166" });
this.add.text(centreX, 230, `Time: ${this.result.seconds.toFixed(2)} s`,
style);
```

`{ ...style, fontSize: "64px" }` makes a **new** object with everything from `style`, then changes `fontSize`. The `...` is the **spread** operator. It saves writing the same style out six times, without changing `style` itself.

Note - Java has nothing quite like object literals, spreading, or union types. They are three of the TypeScript features you will use most in Phaser code, because Phaser takes so many of its settings as plain objects.

Summary

- a game is a list of scenes in the config; Phaser starts the first

- `this.scene.start(KEY, data)` shuts this scene down and starts another; `data` arrives in the new scene's `init(data)`
- keep scene keys as constants; describe the data passed between scenes with an `interface`
- Phaser reuses the same scene object each time a scene starts: reset fields in `init()`
- `setInteractive()` makes a game object clickable; listen for `"pointerdown"` on it. Listen on `this.input` to hear every click in the scene
- `once` listens for the first time only; `on` listens every time
- `this.registry` holds values every scene can see, for as long as the game runs

Challenges

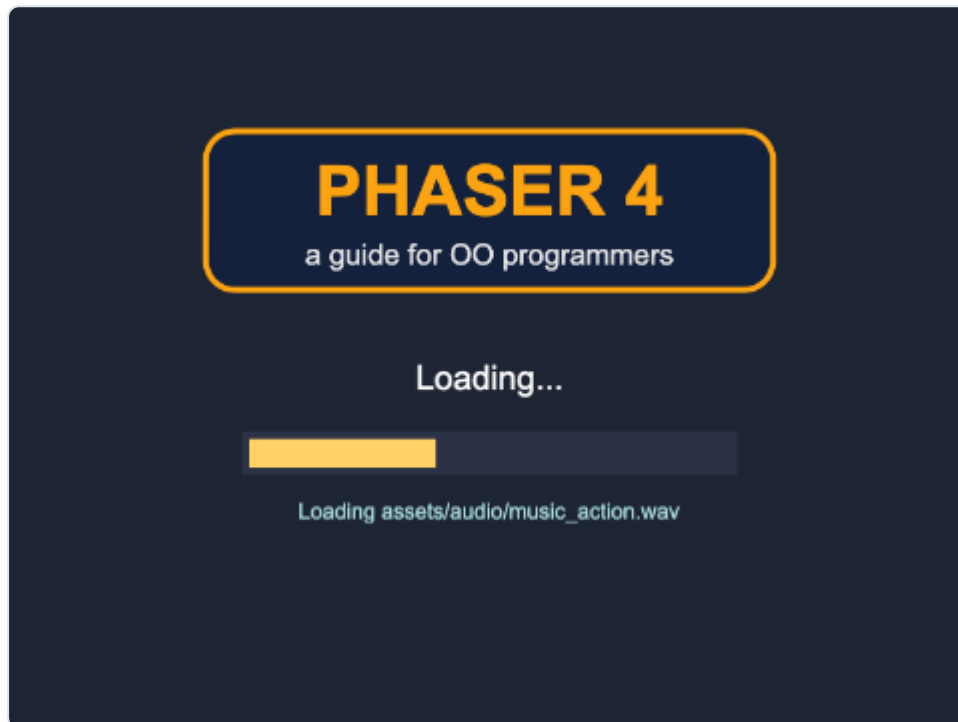
1. **Restyle** (*ch02_click_the_ball*) - Use the blue ball (`assets/images/ball_blue.png` from the asset library - copy it into `public/assets/images/`), make it faster, and change the wording and colours on the start and win screens.
2. **Enter works too** (*ch02_click_the_ball*) - Let ENTER do everything SPACE does: start the game from the title screen, and play again from the win screen.
3. **Give up** (*ch02_click_the_ball*) - Pressing ESC during play goes back to the title screen. The title screen should then work exactly as it did the first time.
4. **Too slow!** (*ch02_click_the_ball*) - Give the player five seconds. If they have not clicked the ball by then, go to the win scene anyway - but it should say "Too slow!" instead of showing a time. Do it without adding a fourth scene. *Hint:* what you pass to `WinScene` can say whether the player won. Look up `this.time.delayedCall(milliseconds, callback)` .
5. **The decoy** (*ch02_click_the_ball_plus*) - Add a second, blue ball that also bounces around. Clicking it counts as **two** misses. Clicking the red ball still counts as a hit. *Hint:* `Ball` will need to be told which picture to use. Think about what the "missed everything" listener sees when the blue ball is clicked.
6. **Top five** (*ch02_click_the_ball_plus*) - Keep the five best totals, not just one. Show them as a table on the title screen, and on the win screen say which place (if any) the new time took. *Hint:* the registry can hold an array. `array.sort((a, b) => a - b)` sorts numbers smallest first, and `array.slice(0, 5)` keeps the first five.

Previous: [Chapter 1 - Introduction](#) · Next: [Chapter 3 - Preloading](#)



Preloading

A game is made of files - pictures, sprite sheets, sounds, music, level data - and every one has to come over the network before the game can use it. In this chapter you look inside Phaser's **loader**: how it queues files, where it keeps them, and what it tells you as it goes. You build a proper loading screen with a progress bar, load one of every common kind of file, deal with files that fail to load, and load a room's files only when the player first goes into it.



What you will learn

- what the loader does: the **queue**, **keys**, and the game's **caches** where loaded files are kept
- how file paths work, and how `setPath()` and `setBaseURL()` save typing
- the **Boot -> Preload -> Menu** pattern, and how to draw a progress bar with `Graphics`, driven by the loader's events
- how to load images, sprite sheets, audio, JSON, text, and an **asset pack**, and read each back
- what happens when a file is missing, and how to find out which one
- how to load more files in the middle of a game, and only when they are needed

The projects

Project	What it shows
<code>ch03_loading_screen</code>	Boot -> Preload -> Menu: a loading screen with a progress bar and the file being loaded
<code>ch03_asset_types</code>	one of each kind of file - image, sprite sheet, audio, JSON, text and an asset pack - each shown on screen, and a missing file
<code>ch03_load_on_demand</code>	a menu of rooms; each room's files are loaded the first time it is chosen

What the loader does

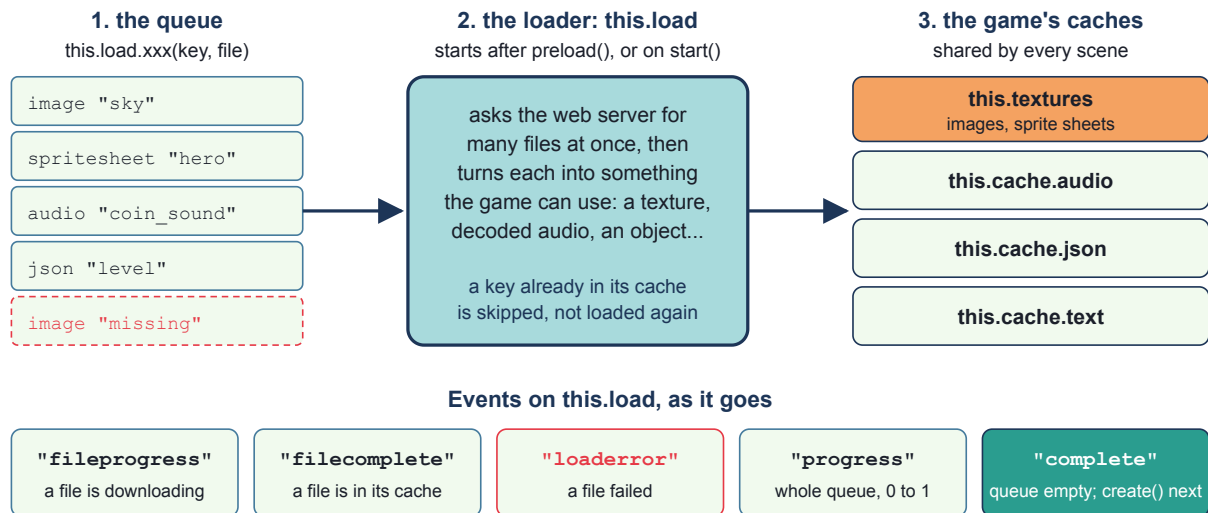
Every scene has a **loader**, `this.load`. You have used it since Chapter 1:

```
preload(): void {  
    this.load.image(LOGO_KEY, LOGO_FILE);  
}
```

It is worth knowing exactly what that line does - because it is not what it looks like. It does **not** load the picture. It adds the picture to a **queue**: "when you start, fetch this file, and keep it under this name". Nothing is fetched yet.

When `preload()` returns, Phaser starts the loader for you. The loader asks the web server for the queued files - many at once - and turns each into something the game can use: a picture becomes a **texture** the graphics card can draw, a sound is decoded, a JSON file becomes an object. Each one goes into a **cache**. When the queue is empty, Phaser calls `create()`. That is why `create()` can always use everything `preload()` asked for.

What the loader does with a queue of files



Keys and caches

The first argument to every `this.load...` method is the file's **key** - the name it is kept under. From then on the rest of the game uses only the key; the file name is never mentioned again.

There is a cache for each kind of thing:

What was loaded	Where it is kept	Read it back with
image , sprite sheet	the texture manager, this.textures	this.add.image(x, y, key) - or this.textures.get(key)
audio	this.cache.audio	this.sound.play(key) - or this.cache.audio.get(key)
json	this.cache.json	this.cache.json.get(key)
text	this.cache.text	this.cache.text.get(key)

Three things follow from this:

- **The caches belong to the game, not to the scene.** A picture loaded by one scene can be used by every scene, for as long as the game runs. Chapter 2's `PlayScene` relied on this
- **A key only has to be unique within its own kind.** A picture called `"coin"` and a sound called `"coin"` can live side by side (in different caches). It is still kinder to the reader to name them differently - this chapter's projects use `"coin"` and `"coin_sound"`

- **A key that is already loaded is skipped.** Ask for "sky" a second time and the loader does not fetch it again - it quietly leaves it out of the queue. That is usually what you want (and the last project in this chapter depends on it), but it also means you cannot replace a picture just by loading a different file under the same key

Where files come from

`build.ts` copies everything in `public/` into `dist/`, and `dist/` is what the web server serves. So a file in `public/assets/images/sky.png` ends up at `dist/assets/images/sky.png`, and the page asks for it as `assets/images/sky.png`. File names in the loader are **relative to the web page** (`index.html`) - not to the TypeScript file doing the loading.

When many files share a folder, `setPath()` saves writing it out every time:

`src/scenes/PreloadScene.ts` (ch03_loading_screen)

```
// setPath() is put in front of every file name that follows, until it is changed again
this.load.setPath("assets/images/");
this.load.image(SKY_KEY, "sky.png");
this.load.image(SPACE_KEY, "space.png");
...
this.load.setPath("assets/audio/");
this.load.audio(COIN_SOUND_KEY, "coin.wav");
```

The full address the loader asks for is **base URL + path + file name**. The base URL is empty unless you set one with `this.load.setBaseURL("https://example.com/my-game/")` - useful when a game's files are kept on a different server from the game. Both can also be set for the whole game in the config, with `loader: { baseURL: "...", path: "..." }`. A file name that is already a full address (`https://...`) ignores both.

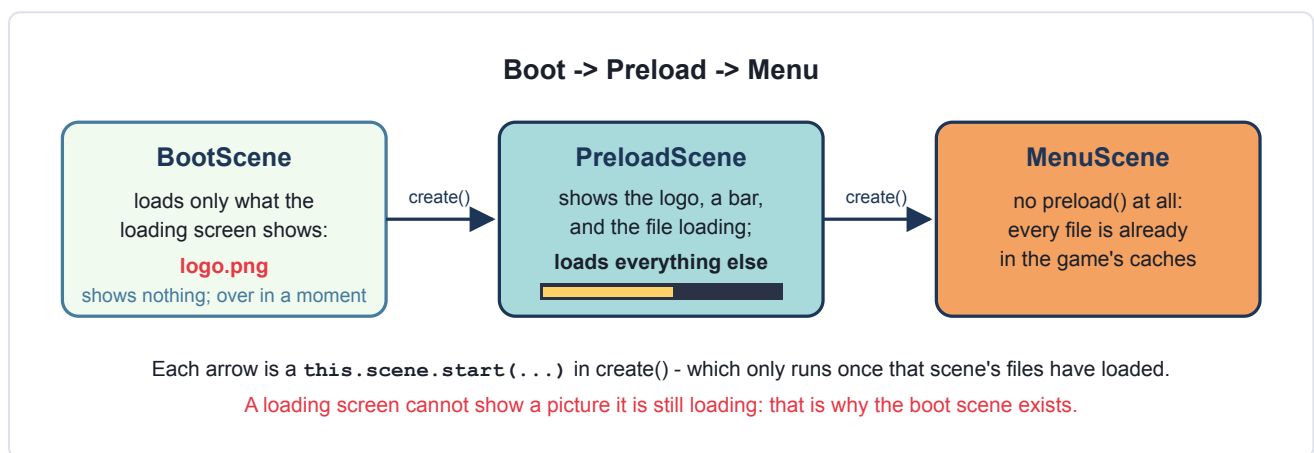
Note - `setPath()` lasts until you change it - including for loads later in the same scene. That is why `queueFiles()` ends with `this.load.setPath("")`.

A loading screen

A handful of small files load in a blink. A real game has hundreds, and some are big - music, large backgrounds - so on a slow connection the player can wait seconds with nothing but a black screen. Every finished game shows a **loading screen** instead.

Boot, Preload, Menu

There is a catch. The loading screen wants to show the game's logo - but the logo is a file too, and a loading screen cannot show a picture it is still loading. The standard answer is three scenes:



src/main.ts

```
scene: [BootScene, PreloadScene, MenuScene],
```

BootScene loads only the few, small files the loading screen needs, and starts the next scene:

src/scenes/BootScene.ts

```
preload(): void {
    this.load.image(LOGO_KEY, LOGO_FILE);
}

// create() only runs once everything queued in preload() has loaded
create(): void {
    this.scene.start(PRELOAD_SCENE);
}
```

It shows nothing, and is over in a moment.

`PreloadScene` is the loading screen. Its `preload()` does three things, in order: it draws the screen, it listens to the loader's events, and it queues every file. Then `MenuScene` - and the rest of the game - never needs a `preload()` at all.

Drawing the bar

The bar is not a picture; it is drawn by Phaser, with a **Graphics** game object. A `Graphics` is a list of drawing instructions - "fill with this colour", "fill a rectangle here" - that Phaser draws every frame:

`src/scenes/PreloadScene.ts`

```
// the box is drawn once; the bar is redrawn every time the progress changes
const box = this.add.graphics();
box.fillStyle(BOX_COLOUR);
box.fillRect(barX - BOX_PADDING, BAR_Y - BOX_PADDING, BAR_WIDTH +
BOX_PADDING * 2, BAR_HEIGHT + BOX_PADDING * 2);

const bar = this.add.graphics();
```

- `fillStyle(colour)` chooses the colour for the fills that follow. Colours are numbers written in hexadecimal - `0x2b3245` is the same colour as `"#2b3245"` in CSS
- `fillRect(x, y, width, height)` - note: `x` and `y` are the rectangle's **top-left** corner, not its centre
- `clear()` throws away everything drawn so far, ready to draw again

There are others you will meet later: `lineStyle(thickness, colour)` and `strokeRect` for outlines, `fillCircle`, `fillRoundedRect`, and more. The bar starts empty. The loader fills it in.

Listening to the loader

The loader is an **event emitter**, like the keyboard in Chapter 1 and game objects in Chapter 2. As it works through the queue, it announces what is happening:

`src/scenes/PreloadScene.ts`

```
// "progress": the whole queue - value goes from 0 (nothing loaded) to 1
// (everything loaded)
this.load.on("progress", (value: number) => {
    bar.clear();
    bar.fillStyle(BAR_COLOUR);
    bar.fillRect(barX, BAR_Y, BAR_WIDTH * value, BAR_HEIGHT);
});

// "fileprogress": one file is on its way - Phaser passes the File, which
// knows its key and URL
this.load.on("fileprogress", (file: Phaser.Loader.File) => {
    fileText.setText(`Loading ${file.src}`);
});

// "complete": the queue is empty - every file has loaded (or failed)
this.load.on("complete", () => {
    fileText.setText("All done!");
});
```

Each time the progress changes, the bar is cleared and drawn again, `BAR_WIDTH * value` wide: no width at 0, all of it at 1, and half of it at 0.5.

Notice the listeners are added **before** the files are queued, and all of it happens inside `preload()`. The loader only starts after `preload()` returns, so nothing can be missed.

The loader's most useful events:

Event	Phaser passes	When
"start"	the loader	the loader starts
"fileprogress"	the <code>File</code> , and how much of it has arrived (0 to 1)	part of a file has arrived
"filecomplete"	the key, the type, and the data	a file is loaded and in its cache
"loaderror"	the <code>File</code>	a file could not be loaded
"progress"	a number from 0 to 1	the queue as a whole has moved on
"complete"	the loader, how many loaded, how many failed	the queue is empty

Java and TypeScript - the listener's parameter types (`value: number` , `file: Phaser.Loader.File`) are written out by you. `this.load.on` accepts *any* function, so TypeScript cannot check that you asked for the right thing - the table above, and Phaser's documentation, say what each event passes. Leave the type off and the build complains that the parameter "implicitly has an 'any' type".

When the loading is done

`src/scenes/PreloadScene.ts`

```
// create() runs once the loader's queue is empty
create(): void {
  const data: MenuData = {
    files: this.load.totalComplete,
    seconds: (performance.now() - this.startTime) / 1000,
  };

  // leave the full bar up for a moment, so the player sees it finish
  this.time.delayedCall(FINISHED_PAUSE, () => {
    this.scene.start(MENU_SCENE, data);
  });
}
```

`this.load.totalComplete` is how many files loaded (and `this.load.totalFailed` how many did not). The menu is handed both numbers through `init(data)` , as in Chapter 2, and shows them. `performance.now()` is the browser's own clock, in milliseconds - `preload()` notes it at the start.

Seeing it work

Try it: build and run `ch03_loading_screen` . You will probably see the loading screen for about a second - and only because of this:

```
// Files on your own computer load far too quickly to see a progress bar. So
// that you can watch it
// work, the game also loads COPIES extra copies of some small pictures,
// each under its own key.
// A real game would not do this - set it to 0 and the loading screen is
// gone in a blink.
const COPIES = 150;
```


Each copy is loaded under its own key (`copy_0` , `copy_1` , ...), so none is skipped: 170 files in all. Set `COPIES` to 0, build, and the loading screen barely appears.

The honest way to see what your players see is to **slow the network down**. Open the game in Chrome or Edge at <http://127.0.0.1:8000/>, open the developer tools (F12), and go to the **Network** tab:

1. tick **Disable cache**, so every file really is fetched again
2. in the throttling menu (it says **No throttling**), choose **Fast 4G**, or **Slow 4G** for a long wait
3. refresh the page

Now the bar crawls, and you can watch the big files - the three music tracks are over 1.5 MB between them - arrive. (Firefox has the same menu.) Switch throttling off afterwards.



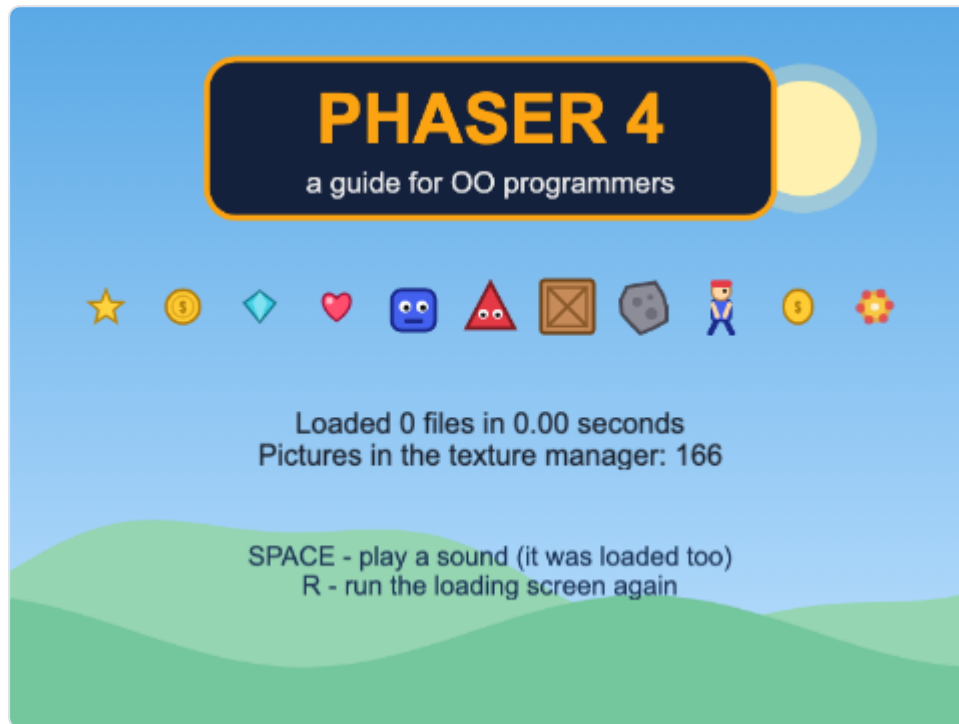
Loading again

The menu shows a few of the loaded pictures. It has no `preload()` - and pictures from a sprite sheet are shown by giving the frame as the fourth argument:

`src/scenes/MenuScene.ts`

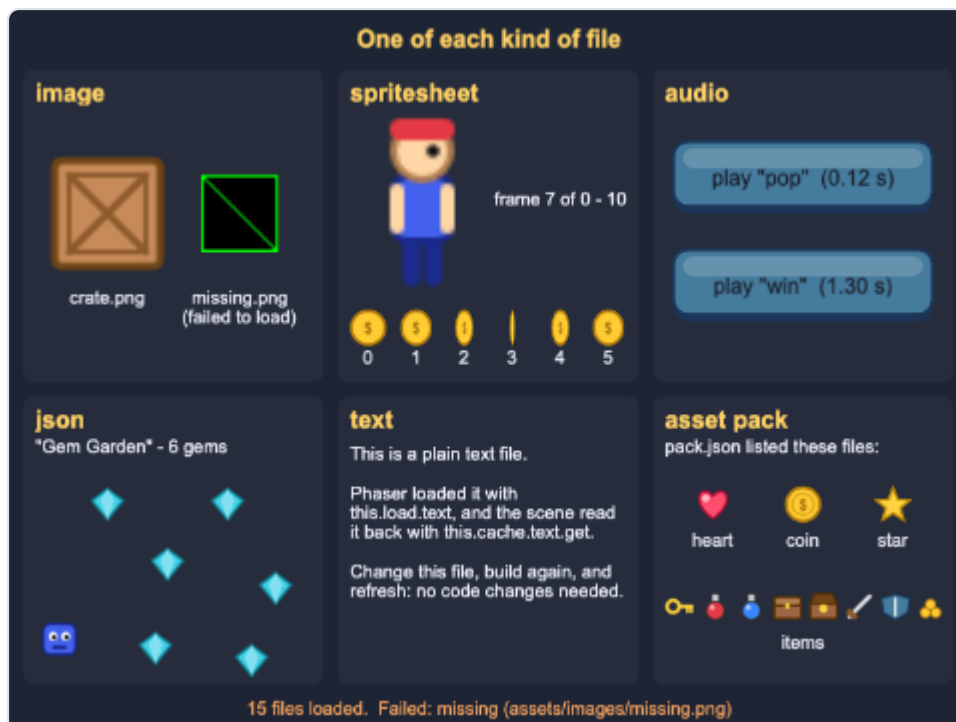
```
SHOW.forEach((shown, index) => {  
    this.add.image(firstX + index * SHOW_SPACING, 250, shown.key,  
    shown.frame);  
});
```

Press **R** in the menu. It starts `BootScene` again, which runs the whole loading screen again. But this time the menu says **Loaded 0 files in 0.00 seconds**: every key is already in its cache, so the loader skipped every file, found its queue empty, and finished at once. Loaded files stay loaded until the page is closed - or until you remove them (see the last project).



Note - there is another way to load the loading screen's own files: a scene can list files in its constructor, `super({ key: PRELOAD_SCENE, pack: { files: [ ... ] } })`, and Phaser loads them before that scene's `preload()` runs. It does the same job as a boot scene. The boot scene is easier to read, and has room to grow.

Every kind of file



`ch03_asset_types` loads one file of each kind Phaser games use most, and shows each in a panel. All of them are queued in `preload()`, and read back from their caches in `create()`.

Images and sprite sheets

`src/scenes/AssetTypesScene.ts`

```
// an image: one picture
this.load.image(CRATE_KEY, CRATE_FILE);

// a sprite sheet: one picture, cut into frames of the size given
this.load.spritesheet(HERO_KEY, HERO_FILE, { frameWidth: 32, frameHeight: 48 });
this.load.spritesheet(COIN_SPIN_KEY, COIN_SPIN_FILE, { frameWidth: 32,
frameHeight: 32 });
```

A **sprite sheet** is many pictures of the same size in one file - usually the frames of an animation. Phaser cuts it into frames of `frameWidth` x `frameHeight`, and numbers them from 0, left to right, then row by row. `coin_spin.png` is 192 x 32, so it is six frames, 0 to 5. The asset library's `README.md` lists the frame size of every sheet; get it wrong and every frame is cut in the wrong place.

The panel shows every frame of the coin, and steps the hero through its frames, one every quarter of a second:

```
// frameTotal counts every frame, plus one extra, "__BASE" - the whole picture  
this.heroFrames = this.textures.get(HERO_KEY).frameTotal - 1;
```

```
override update(time: number, _delta: number): void {  
    const frame = Math.floor(time / HERO_FRAME_TIME) % this.heroFrames;  
    this.hero.setFrame(frame);  
    ...  
}
```

`time / HERO_FRAME_TIME` counts up by one every 250 ms; `Math.floor` drops the fraction; `%` (the remainder, as in Java) wraps it round to 0 after the last frame. Chapter 7 does this properly, with Phaser's animations.

Audio

```
this.load.audio(POP_KEY, POP_FILE);
```

Sounds are decoded as they load, and kept in `this.cache.audio`. The panel reads one back, to show how long it is:

```
// with Web Audio (every modern browser), the audio cache holds a decoded AudioBuffer  
const buffer: AudioBuffer = this.cache.audio.get(key);
```

You rarely need the cache directly - `this.sound.play(key)` finds the sound by its key. This guide's sounds are WAV files. Games often use MP3 or OGG, which are much smaller, and `this.load.audio(key, [oggFile, mp3File])` lets the browser choose the first format it can play. Chapter 5 is all about audio.

JSON: data for the game

JSON is how games keep data outside the code - levels, enemy types, dialogue, settings. Here is a tiny level:

```
public/assets/data/level.json
```

```
{
  "name": "Gem Garden",
  "player": { "x": 30, "y": 150 },
  "gems": [
    { "x": 70, "y": 40 },
    { "x": 120, "y": 90 },
    ...
  ]
}
```

```
this.load.json(LEVEL_KEY, LEVEL_FILE);
```

The loader parses it, and `this.cache.json.get(key)` gives back an ordinary JavaScript object. But what *type* is it? Phaser has no idea what is in the file, so `get` returns `any` - TypeScript's word for "anything; I will not check". Left like that, a typo such as `level.gem` would not be caught. So the project describes the file with interfaces:

```
src/LevelData.ts
```

```
export interface Point {
  x: number;
  y: number;
}

export interface LevelData {
  name: string;
  player: Point;
  gems: Point[];
}
```

```
src/scenes/AssetTypesScene.ts
```

```
// cache.json.get gives back whatever the file held - TypeScript cannot know
what that is, so
// we say what we expect: a LevelData (see LevelData.ts)
const level: LevelData = this.cache.json.get(LEVEL_KEY);

this.add.text(x + 10, y + 34, `${level.name} - ${level.gems.length} gems`,
SMALL_STYLE);
```

```
level.gems.forEach((gem) => {  
    this.add.image(originX + gem.x, originY + gem.y, GEM_KEY);  
});
```

From here on, `level.gem` or `level.name * 2` is a build error.

Java and TypeScript - this is like reading JSON into a class with a Java library such as Gson or Jackson - with one big difference. Those libraries check the file against the class as they read it. TypeScript's types disappear when the game is built, so **nothing checks the file**. If `level.json` said `"gemz"`, `level.gems` would be `undefined`, and `level.gems.length` would crash the game when it ran. The interface is a promise you make about the file; keep the two in step.

Change a position in `level.json`, build, and refresh: the level changes, with no change to the code. That is the point of data files.

Text

```
this.load.text(NOTES_KEY, NOTES_FILE);  
...  
const notes: string = this.cache.text.get(NOTES_KEY);
```

Any plain text file comes back as one string - line breaks and all. Useful for credits, help screens, dialogue, or your own simple file formats.

Asset packs

A game with hundreds of files ends up with hundreds of `this.load...` lines. An **asset pack** moves that list out of the code into a JSON file:

```
public/assets/pack.json
```

```

{
  "pickups": {
    "path": "assets/",
    "files": [
      { "type": "image", "key": "heart", "url": "images/heart.png" },
      { "type": "image", "key": "coin", "url": "images/coin.png" },
      { "type": "image", "key": "star", "url": "images/star.png" },
      {
        "type": "spritesheet",
        "key": "items",
        "url": "spritesheets/items.png",
        "frameConfig": { "frameWidth": 32, "frameHeight": 32 }
      }
    ]
  }
}

```

A pack is divided into **sections** (here there is one, "pickups"). Each section has a list of files - each with the `type` of loader method to use, its `key` and its `url` - and can set its own `path` (and `baseUrl` and `prefix`) for the files in it.

`src/scenes/AssetTypesScene.ts`

```

// a pack: a JSON file that is a LIST of files to load. Phaser loads the
// pack, then queues
// everything in the section named, and loads those too
this.load.pack(PACK_KEY, PACK_FILE, PACK_SECTION);

```

Leave off the third argument and every section is loaded. The files arrive under the keys the pack gave them, so the code still has to know those keys (they are in `src/assets.ts`). What the pack changes is *where the list of files lives*: add a file to the game by adding a line to the pack, with no code to change - and a tool can write the pack for you.

The count at the bottom of the screen - **15 files loaded** - includes the pack file itself.

When a file is missing

Look at the image panel again. `assets.ts` asks for `assets/images/missing.png` , which does not exist - on purpose. The game does not stop. The web server answers "404 Not

Found", the loader gives up on that file, and carries on with the rest.

Two things happen that you can see. First, the **"loaderror"** event, which the project listens for:

src/scenes/AssetTypesScene.ts

```
// "loaderror" fires once for each file that could not be loaded
this.load.on("loaderror", (file: Phaser.Loader.File) => {
  this.failed.push(`${file.key} (${file.src})`);
});
```

The status line at the bottom lists what failed. (The browser's developer console shows the failed request too, in red.) Second, when `create()` asks for the picture, **"missing"** is not in the texture manager - so Phaser draws its **missing texture** instead: a black square with a green outline and a green diagonal line. You will see it again. It always means "there is no picture with that key", and the cause is one of:

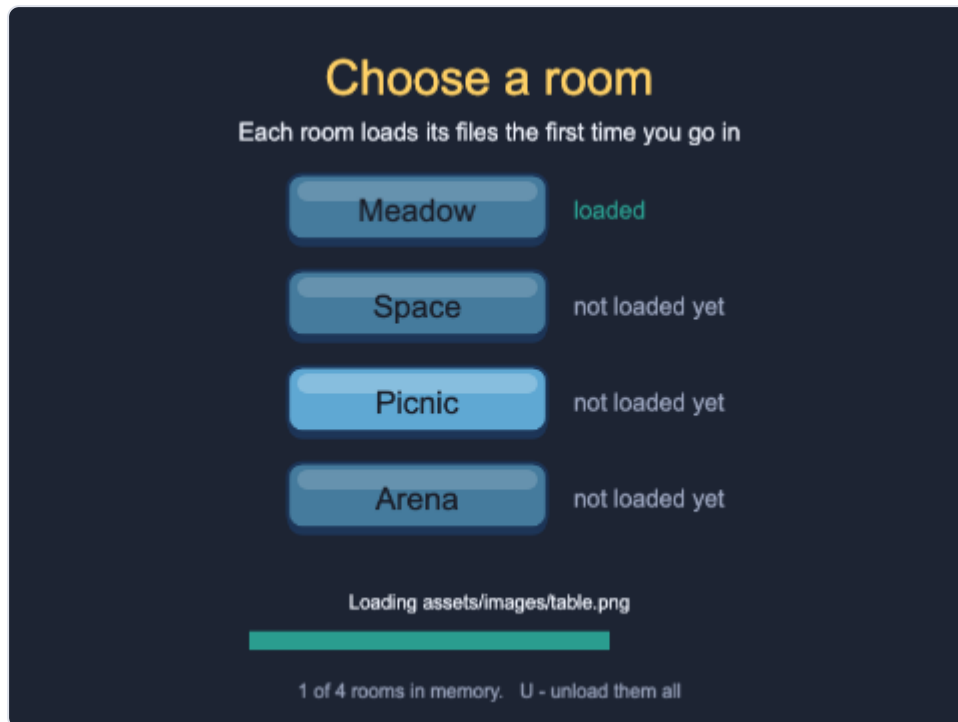
- the file is not in `public/` - for a picture from the asset library, it has to be copied into the project first
- the file name or folder is spelled differently in the code - `Sky.png` is not `sky.png`. Your own computer may not mind about capitals; a web server usually does, so a game can work at home and fail once it is uploaded
- the key used in `create()` is not the key used in `preload()` - one more reason for key constants

Pictures fail quietly. Other kinds of file fail less politely:

Missing	What happens
a picture	the missing-texture square is drawn; the game carries on
a sound	<code>this.sound.play(key)</code> throws <code>Error: Audio key "pop" not found in cache</code> , and the scene stops there
JSON or text	<code>this.cache.json.get(key)</code> returns <code>undefined</code> , and the first use of it crashes

So when something is not there, look at the developer console first: the red 404 line names the file the game asked for, exactly as it asked for it.

Loading later



Loading everything at the start is simplest, and right for most small games. But a big game might have forty levels, each with its own backgrounds and music. Loading all of it up front makes every player wait for levels they may never reach, and keeps all of it in memory at once. Instead, a game can load each part **when it is needed**.

`ch03_load_on_demand` is a menu of four rooms. At the start, only the menu's own files are loaded. The first time you choose a room, the menu loads that room's pictures and sound - with a progress bar - and then goes in.

The rooms as data

`src/rooms.ts`

```
export interface Room {
  name: string;
  background: RoomFile;
  pictures: RoomFile[];
  props: RoomProp[];
  sound: RoomFile;           // played as you walk in
}
```

```

{
  name: "Space",
  background: { key: "space", url: "assets/images/space.png" },
  pictures: [
    { key: "player_ship", url: "assets/images/player_ship.png" },
    { key: "enemy_ship", url: "assets/images/enemy_ship.png" },
    { key: "rock", url: "assets/images/rock.png" },
  ],
  props: [
    { key: "player_ship", x: 400, y: 500 },
    ...
  ],
  sound: { key: "shoot", url: "assets/audio/shoot.wav" },
},

```

The menu reads this to know what to load. `RoomScene` reads it to know what to show. One `RoomScene` class serves all four rooms; the data it is started with (`{ index: 2 }`) says which.

Loading from create()

In `preload()`, Phaser starts the loader for you. Anywhere else, you queue the files and start it yourself:

```
src/scenes/MenuScene.ts
```

```

private chooseRoom(index: number): void {
  // one load at a time: ignore clicks while a room is loading
  if (this.load.isLoading()) {
    return;
  }
  this.sound.play(CLICK_KEY);

  const room = ROOMS[index];
  if (this.isLoaded(room)) {
    this.enterRoom(index);
    return;
  }

  // queue the room's files, say what to do when they have all loaded, and
  start the loader
  this.queueRoom(room);
  this.load.once("complete", () => {
    this.enterRoom(index);
  });
  this.load.start();
}

```

- `this.load.start()` - the step that is easy to forget. Without it, the files sit in the queue for ever, and nothing happens: no error, no room
- `this.load.once("complete", ...)` - there is no `create()` to wait for this time, so the "complete" event is how the menu knows the room is ready. `once`, because each load wants its own, single, "then go in"
- `this.load.isLoading()` - a second click while a room is loading is ignored

The progress bar uses the same "progress" and "fileprogress" events as the loading screen, with listeners added in `create()`. A scene's loader removes all its listeners when the scene shuts down, so adding them every time the menu starts does not pile them up.

Is it loaded already?

`src/scenes/MenuScene.ts`

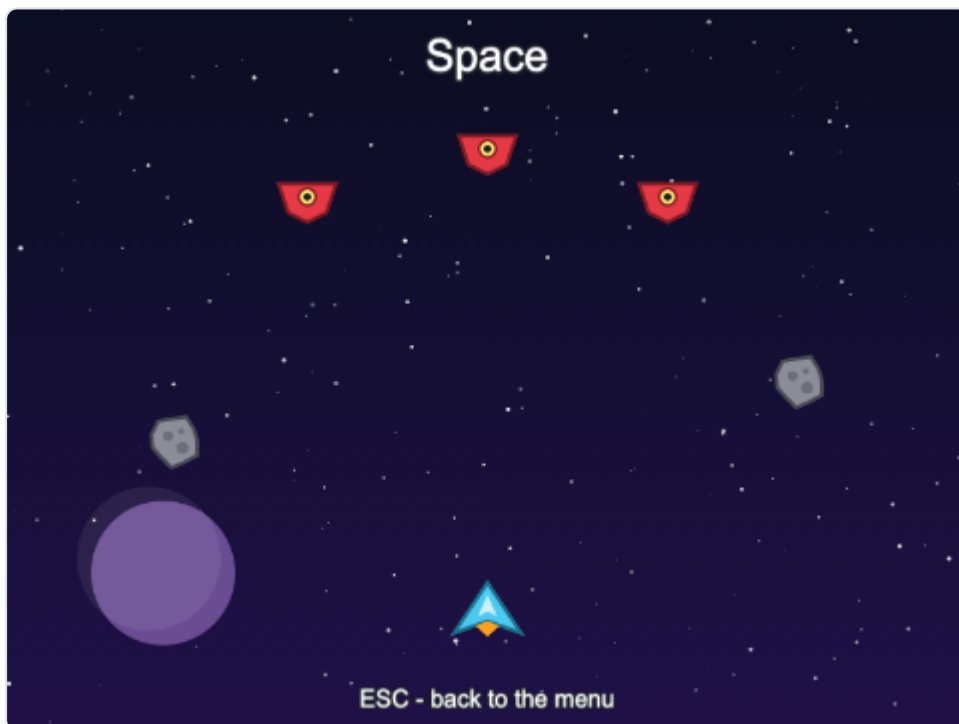
```
// Is every file the room needs already in the game's caches?
private isLoading(room: Room): boolean {
  const pictures = [room.background, ...room.pictures];
  const picturesLoaded = pictures.every((picture) =>
this.textures.exists(picture.key));
  return picturesLoaded && this.cache.audio.exists(room.sound.key);
}
```

`this.textures.exists(key)` and `this.cache.audio.exists(key)` ask the caches directly. `array.every(test)` is `true` if the test is true for every item.

`[room.background, ...room.pictures]` is Chapter 2's spread, on an array: the background followed by the other pictures.

`queueRoom()` then queues only the files that are missing. Two rooms share the player's picture, under the same key, so whichever of them you visit second finds it already there and loads one file fewer. (The loader would skip it anyway, as you saw - checking first just makes plain what is happening.)

Visit a room, press ESC, and choose it again: it is **loaded**, and you go straight in.



Letting go

Loaded files take memory - a picture takes width x height x 4 bytes on the graphics card, however small its file is: the 800 x 600 backgrounds are nearly 2 MB each. A game that

loads a level's files when the level starts might remove them when it ends. Press **U** in the menu:

```
for (const room of ROOMS) {
  for (const picture of [room.background, ...room.pictures]) {
    if (this.textures.exists(picture.key)) {
      this.textures.remove(picture.key);
    }
  }
  this.cache.audio.remove(room.sound.key);
}
```

Every room says **not loaded yet** again, and the next visit loads it again. Only remove what no game object is still using - the rooms are not running while the menu is, so here it is safe.

Which to choose? Load up front if the game is small, or if a pause in the middle of play would spoil it. Load on demand when the game is big, or when players only see part of it. Many games do both: a loading screen at the start for what every player needs, and a short "Loading..." between levels.

Common mistakes

Common mistakes

- **A path starting with / :** `"/assets/images/sky.png"` means "from the top of the web server", not "from the page". It happens to work with `deno task serve`, and breaks as soon as the game is put in a folder of a website. Leave the `/` off
- **Forgetting** `this.load.start()` outside `preload()` : nothing loads, and nothing says why
- **Using a file in** `preload()` : `preload()` only queues. Nothing is loaded until it returns - anything that uses a file belongs in `create()` (or in a loader event)
- **The wrong frame size** for a sprite sheet: every frame is cut in the wrong place. Check the size in the asset library's `README.md`
- **Loading a new file under an old key:** the loader skips it, and the old picture stays. Remove the old one first, or use a new key

Summary

- `this.load.image(key, file)` and friends only **queue** a file. In `preload()`, Phaser starts the loader when `preload()` returns, and calls `create()` when everything has loaded
- loaded files are kept, by key, in the game's caches - `this.textures`, `this.cache.audio`, `this.cache.json`, `this.cache.text` - shared by every scene. A key that is already loaded is skipped
- file names are relative to the web page; `setPath()` and `setBaseURL()` go in front of them
- **Boot -> Preload -> Menu**: a boot scene loads what the loading screen shows; the loading scene draws a bar with `Graphics`, moved by the loader's `"progress"` event
- `image`, `spritesheet` (with `frameWidth`, `frameHeight`), `audio`, `json` (describe it with an interface), `text`, and `pack` (a JSON list of files, in sections)
- a missing file fires `"loadererror"`; a missing picture is drawn as the missing texture; a missing sound throws an error
- outside `preload()`: queue the files, listen for `"complete"`, call `this.load.start()`. Check with `this.textures.exists(key)` first; remove with `this.textures.remove(key)`

Challenges

1. **Restyle the bar** (*ch03_loading_screen*) - Make the loading screen your own: a bigger bar in a box with rounded corners and an outline, new colours and wording, and a bar that changes colour as it fills - red for the first third, orange for the second, green for the last.
2. **Percentage** (*ch03_loading_screen*) - Show how far the loading has got as a percentage, in the middle of the bar: `0%` at the start, `100%` at the end, and whole numbers only. Make sure it never says `100%` before everything really has loaded.
3. **Hearts in the level** (*ch03_asset_types*) - Add a list of hearts to `level.json`, and draw them in the JSON panel, alongside the gems. The panel's heading should say how many of each there are. Keep TypeScript in the picture: the build should know about the hearts too.

4. **Keep it moving** (*ch03_loading_screen*) - A bar that stops moving while a big file loads looks like a game that has crashed. Add a spinner beside the bar - a picture that keeps turning round all the time the files are loading. *Hint:* `update()` is not called until after `create()`, so it cannot turn anything during loading. A tween can: `this.tweens.add({ targets: picture, angle: 360, duration: 1000, repeat: -1 })` (Chapter 7 has the details). And think about *which scene* has to load the spinner's picture.
5. **When files fail** (*ch03_loading_screen*) - If any file fails to load, show a new screen instead of the menu, listing each file that failed - its key and the address Phaser tried - with **R** to try again and **SPACE** to carry on to the menu anyway. Test it by asking for a file that is not there. *Hint:* collect failures from the `"loaderror"` event, and pass them to the new scene in its data. Remember that the loading scene may run more than once. What does trying again from the boot scene load the second time?
6. **Rooms from a pack** (*ch03_load_on_demand*) - Move the list of every room's files out of the code into one asset pack, `assets/rooms_pack.json`, with a section for each room. `rooms.ts` keeps only what the code needs - each room's name, section, keys and where things go - and the menu loads a room with a single call. U must still work: after unloading, every room has to load again properly. *Hint:* the third argument to `this.load.pack` picks one section. Each call needs its own key - and a loaded pack is kept in the JSON cache under that key. What does the loader do with a key that is already in its cache?

Previous: [Chapter 2 - A three-scene game](#) · Next: [Chapter 4 - The life of a scene](#)

The life of a scene

So far you have written `preload()`, `create()` and `update()` and trusted Phaser to call them. This chapter looks at exactly **what Phaser calls, and when**, the events a scene announces as it goes, and what the scene manager can do besides `start` - run scenes side by side, pause them, put them to sleep and wake them. Then it puts those powers to work in a game with a HUD and a pause menu.

DemoScene

update() calls: 106
game steps seen: 106
time: 2593 ms
this.time.now: 2593 ms
delta: 16.6 ms



```

- constructor()
0 init()
0 preload()
2 loader: complete
2 create()
2 CREATE event
2 PRE UPDATE event
2 Spinner.preUpdate()
2 UPDATE event
2 update()
2 POST UPDATE event
2 RENDER event

```

DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
fps: 56.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

What you will learn

- the exact order of a scene's life: `constructor` (once), `init`, `preload`, `create`, then `update` every frame - and where game objects' `preUpdate` fits in
- the scene events (`CREATE`, `PRE_UPDATE`, `UPDATE`, `POST_UPDATE`, `PAUSE`, `RESUME`, `SLEEP`, `WAKE`, `SHUTDOWN`, `DESTROY`) and how to listen for them
- why a `Sprite` subclass must call `super.preUpdate(time, delta)`

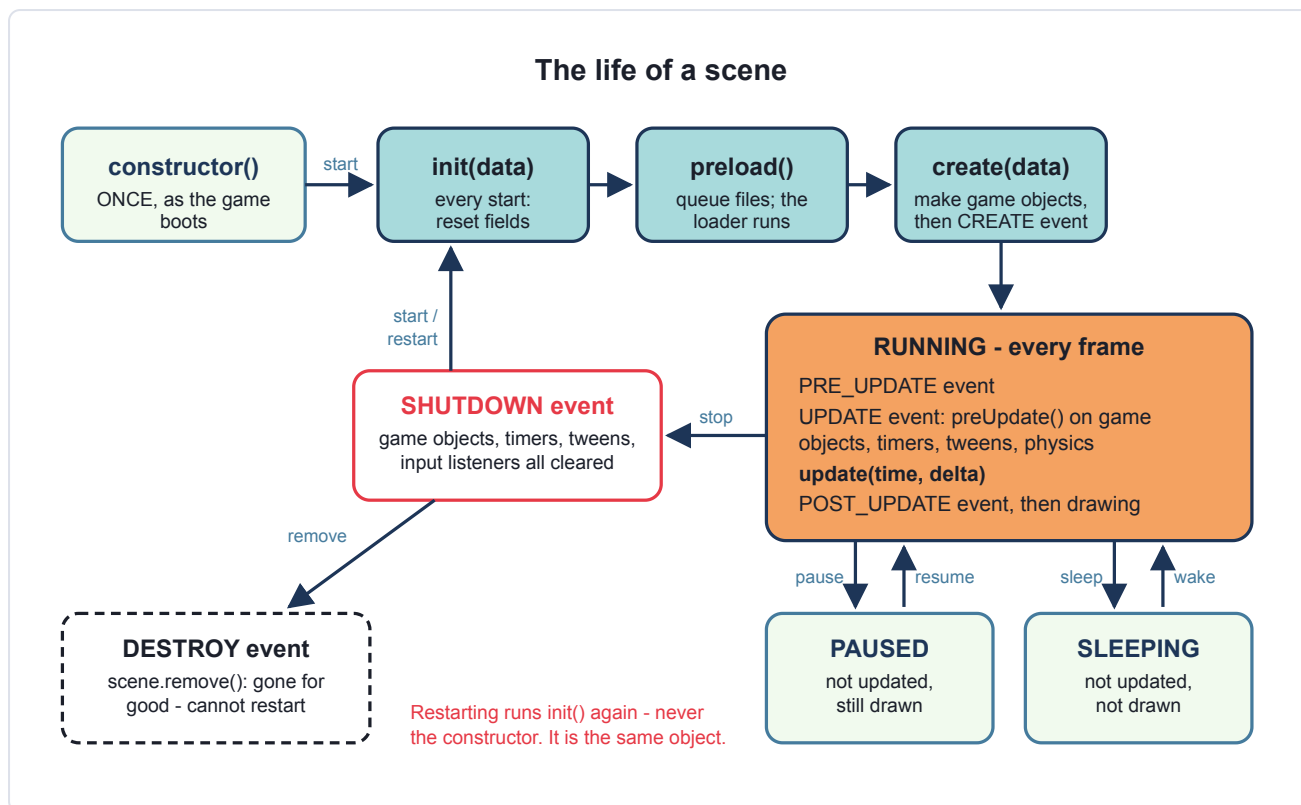
- the scene manager: `start`, `restart`, `launch`, `stop`, `pause / resume`, `sleep / wake`, `remove`, `bringToTop`, `isActive`, `get` - and what "paused" and "asleep" really mean
- how scenes running side by side are drawn, and how to use that for a HUD and a pause menu
- which listeners Phaser tidies up when a scene shuts down, which it does not, and how to clean up the rest
- `time`, `this.time.now` and `delta`, and the game's frame rate settings

The projects

Project	What it shows
<code>ch04_lifecycle_logger</code>	every lifecycle call and scene event written to an on-screen log, with keys to pause, resume, sleep, wake, restart, stop, launch and remove the scene being watched
<code>ch04_pause_and_hud</code>	Coin Rush: a game scene with a HUD scene launched on top, and a pause menu scene launched with P that pauses the game

The life of a scene

Here is everything that happens to a scene, from the moment the game starts:



`ch04_lifecycle_logger` lets you watch it happen. It has two scenes. `DemoScene`, on the left, is the scene being watched: every one of its methods writes a line to the log on the right. `LogScene` draws the log, shows `DemoScene`'s state along the bottom, and reads the keys.

The constructor - once

`src/scenes/DemoScene.ts`

```

constructor() {
  super(DEMO_SCENE);
  eventLog.add("constructor()");
}

```

Phaser makes **one object** of each scene class in the config's list, when the game starts - that is when the constructor runs, and it never runs again. (Chapter 2 showed what that means for fields.) At this point the scene has no loader, no clock, no events and no game objects - none of the things `this.` usually reaches. So the constructor gives the scene its key, and very little else. (That is why the log is a plain class of its own, `EventLog`, that any file can import.)

init, preload, create - every time the scene starts

```
// Runs every time the scene starts: the place to reset fields
init(): void {
    this.log("init()");
    this.updates = 0;
    this.gameSteps = 0;
    this.framesToLog = 0;
}

// Runs after init(). create() waits until everything queued here has loaded.
preload(): void {
    this.log("preload()");
    this.load.spritesheet(COIN_KEY, COIN_FILE, { frameWidth: 32, frameHeight: 32 });

    this.load.once(Phaser.Loader.Events.COMPLETE, () => {
        this.log(" loader: complete");
    });
}
```

When a scene starts - because it is first in the list, or something called `start` or `launch` - Phaser calls:

1. `init(data)` - with the data passed to `start / launch`, if any. Reset fields here
2. `preload()` - queue files to load. The loader then runs, and Phaser **waits** for it
3. `create(data)` - everything is loaded; make the game objects. The same `data` again
4. the scene's `CREATE` event, just after `create()` returns

Any of the three can be left out. Each log line starts with the frame number

(`game.loop.frame`): `init()` and `preload()` happen in frame 0, but `create()` waits until frame 2, when the coin's sprite sheet has arrived - with big files on a real server, that could be seconds.

Now press **5** to restart `DemoScene`, and read the log again: `SHUTDOWN`, then `init()`, `preload()`, `create()` - but no `constructor()`. It is the same object, started again. And `preload()` finishes at once, in the same frame: the sprite sheet is already loaded, and loaded files belong to the game, not the scene.

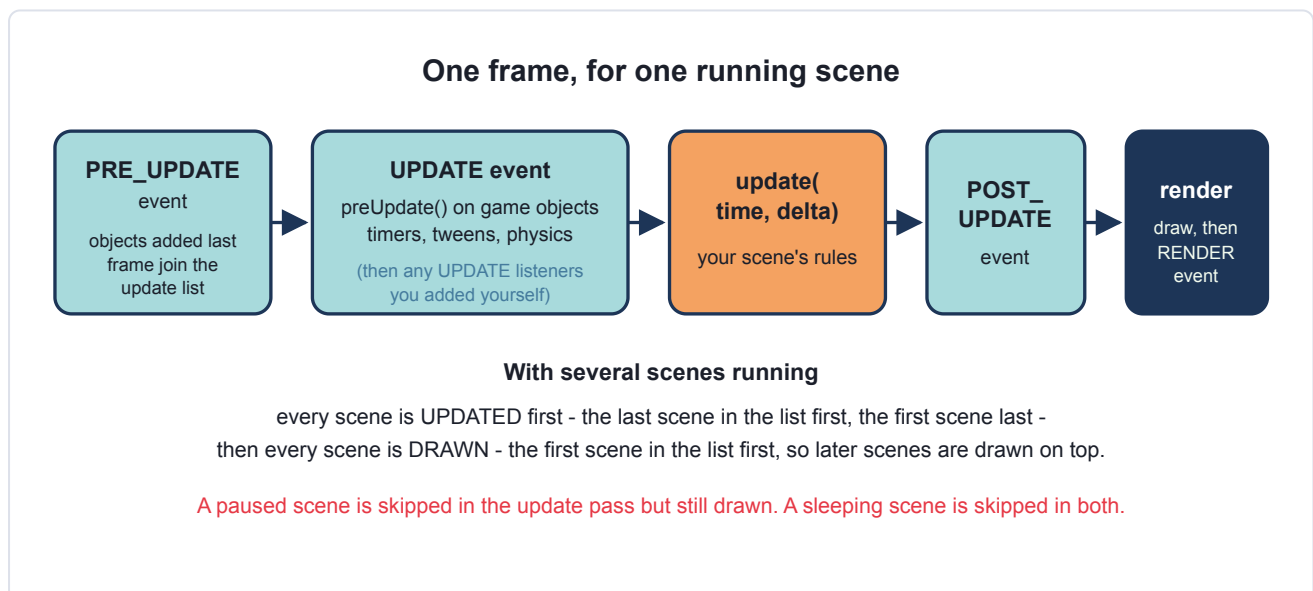
Animations belong to the game too (Chapter 7 is all about them). `DemoScene` makes one in `create()` - which runs on every restart - so it only makes it the first time:

```
if (!this.anims.exists(SPIN_ANIM)) {
  this.anims.create({
    key: SPIN_ANIM,
    frames: this.anims.generateFrameNumbers(COIN_KEY, { start: 0, end: 5 }),
    frameRate: 10,
    repeat: -1,
  });
}
```

Anything that belongs to the whole game needs the same thought: "what happens the second time `create()` runs?"

One frame, in order

Once `create()` has finished, the scene is **running**, and Phaser steps it once every frame. A scene's step has more in it than `update()` :



The logger writes the first frame out in full (after that, 60 lines a second would be too many to read, so it only counts):

```

2 PRE_UPDATE event
2   Spinner.preUpdate()
2   UPDATE event
2   update()
2   POST_UPDATE event
2   RENDER event

```

- **PRE_UPDATE** - Phaser tidies up: game objects added during the last frame join the scene's update list
- **UPDATE** - Phaser's own systems work: every game object on the update list has its `preUpdate(time, delta)` called (the coin, `Spinner`, is one), timers tick, tweens move, physics bodies move. Listeners you add to **UPDATE** come after Phaser's, which were there first
- `update(time, delta)` - your scene's method, once every game object has moved itself - so the rules of the game (did the player touch a coin?) see everything where it will be this frame
- **POST_UPDATE** - after `update()`
- **render** - the scene is drawn, then its **RENDER** event

Listening for scene events

A scene announces all of these on its own **event emitter**, `this.events`. `DemoScene` signs up for them in `create()` :

```

private listen(): void {
  this.events.on(Phaser.Scenes.Events.CREATE, this.onCreate, this);
  this.events.on(Phaser.Scenes.Events.PRE_UPDATE, this.onPreUpdate, this);
  ...
  this.events.on(Phaser.Scenes.Events.PAUSE, this.onPause, this);
  this.events.on(Phaser.Scenes.Events.RESUME, this.onResume, this);
  ...
}

```

`Phaser.Scenes.Events.PAUSE` is just the string `"pause"`, kept as a constant - use the constants, for the same reason Chapter 2 used them for scene keys.

These listeners are **methods**, not arrow functions, with a third argument, `this`: the **context** - the object the method should be called on.

Note - In Java, `this::onClick` remembers which object `onClick` belongs to. In JavaScript, `this.onPause` on its own is just the function: called later, it would not know its `this`. So Phaser's `on(event, method, context)` keeps the object with the method. (An arrow function keeps the `this` from where it was written.) The method style has one big advantage, as you will see: the listener can be taken off again with exactly the same three arguments.

A Sprite's own `preUpdate`

In Chapter 1, `Ball` extended `Image`, and its `preUpdate()` had no `override`, because an `Image` has no `preUpdate()` of its own. A `Sprite` does. A sprite is an image that can play **animations**, and its `preUpdate()` is the method that moves the animation on to its next frame. So when a class of yours extends `Sprite` and has a `preUpdate()`, it is **replacing** Phaser's:

`src/objects/Spinner.ts`

```
protected override preUpdate(time: number, delta: number): void {
    ...
    // Sprite's own preUpdate() plays the animation. Press S in the game to
    // leave it out, and the
    // coin still slides (that is our code, below) but stops spinning.
    if (this.scene.registry.get(SKIP_SUPER) !== true) {
        super.preUpdate(time, delta);
    }

    this.x = this.x + this.speed * delta / 1000;
    ...
}
```

Three things to get right:

- `override` - it replaces `Sprite`'s method. Leave it out and the build says `This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.`
- `protected` (or `public`) - `Sprite` declares `preUpdate` as `protected`. A subclass may make it more visible, never less: `private` gives `Class 'Spinner' incorrectly`

extends base class 'Sprite'. Property 'preUpdate' is private in type 'Spinner' but not in type 'Sprite'.

- `super.preUpdate(time, delta)` - the compiler does *not* check this one. Leave it out and everything still builds, the sprite still moves, and its animation silently freezes. Press **S** in the logger to see it: the coin keeps sliding, stuck on one frame

The rule is the same as in Java: when you override a method, call `super` 's version unless you really mean to replace everything it did. For a `Sprite`, call it first.

The scene manager

`this.scene` is the scene's **scene plugin** - its handle on Phaser's scene manager, which owns the list of every scene in the game. Chapter 2 used `start`. Here is the rest of the family, all of them in `LogScene`, each on a number key:

`src/scenes/LogScene.ts`

```
keyboard.on("keydown-ONE", () => {
  this.logCommand("scene.pause(DEMO_SCENE)");
  this.scene.pause(DEMO_SCENE);
});
...
keyboard.on("keydown-THREE", () => {
  this.logCommand("scene.sleep(DEMO_SCENE)");
  this.scene.sleep(DEMO_SCENE);
});
```

Method	What it does to the scene	Updated?	Drawn?	Keeps its game objects?
<code>start(key, data)</code>	stops this scene, and starts <code>key</code> (restarting it if it is already running)	yes	yes	new ones, from <code>create()</code>
<code>launch(key, data)</code>	starts <code>key</code> alongside this one - this scene carries on	yes	yes	new ones
<code>restart(data)</code>	stops and starts this scene again	yes	yes	new ones
<code>pause(key) / resume(key)</code>	freezes it / unfreezes it	no	yes	yes
<code>sleep(key) / wake(key)</code>	puts it to sleep / wakes it	no	no	yes
<code>stop(key)</code>	shuts it down: <code>SHUTDOWN</code>	no	no	no - all destroyed
<code>remove(key)</code>	shuts it down for good: <code>DESTROY</code> ; it cannot be started again	-	-	-

Leave the key out and most of them act on the scene itself: `this.scene.pause()` pauses this scene. `restart()` only ever restarts its own scene - which is why `LogScene` restarts `DemoScene` by asking `DemoScene` 's own scene plugin: `demo.scene.restart()`.

Paused or asleep?

Press **1** to pause `DemoScene`. The coin stops, the numbers stop - `update()` is no longer called, and neither is the coin's `preUpdate()` - but you can still see it all. A paused scene is **frozen, not hidden**. Press **2** to resume.

Press **3** to put it to sleep. Now it disappears, and you can see what `LogScene` drew underneath:

DemoScene is
not being drawn

```
- constructor()
0 init()
0 preload()
2 loader: complete
2 create()
2 CREATE event
2 PRE UPDATE event
2 Spinner.preUpdate()
2 UPDATE event
2 update()
2 POST UPDATE event
2 RENDER event
112 >> scene.pause(DEMO_SCENE)
112 PAUSE event
148 >> scene.resume(DEMO_SCENE)
148 RESUME event
148 PRE UPDATE event
148 Spinner.preUpdate()
148 UPDATE event
148 update()
148 POST UPDATE event
148 RENDER event
174 >> scene.sleep(DEMO_SCENE)
174 SLEEP event
```

DemoScene: SLEEPING isActive: no isPaused: no isSleeping: yes isVisible: no
fps: 53.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

A sleeping scene is neither updated nor drawn - but, like a paused one, it keeps everything: its game objects, its fields, its timers. Press **4** to wake it, and the coin carries on from exactly where it stopped.

Pause a scene that should still be seen - a game behind a pause menu. **Sleep** a scene you will come back to but do not want on screen - a world map while the player is inside a building. **Stop** a scene you have finished with; starting it again rebuilds it from `init()`.

While the scene is paused or asleep its `update()` count stops, but "**game steps seen**" keeps going: the game never stops, only this one scene. And a paused scene does not hear the keyboard or the mouse - which is why the keys are in `LogScene`. If `DemoScene` read them, pressing 1 would pause it, and nothing could ever resume it.

Asking questions

The bottom of the logger reports on `DemoScene` with the scene plugin's questions:

`this.scene.isActive(DEMO_SCENE)`, `isPaused`, `isSleeping`, `isVisible` and `getStatus`. `isActive` means "running and being updated" - it is `false` while paused.

`getStatus` gives the state as a number (`Phaser.Scenes.RUNNING` is 5, `PAUSED` 6, `SLEEPING` 7, ...).

`this.scene.get(key)` hands you the scene object itself - handy for a tool like this, but a game scene that reaches into another scene's fields is tied to it. It returns `null` for a key it does not know (once **8** has removed `DemoScene`), though its type does not say so - hence the `as` :

```
private getDemo(): DemoScene | null {  
    return this.scene.get(DEMO_SCENE) as DemoScene | null;  
}
```

Requests wait their turn

`pause` , `start` , `launch` , `stop` , `sleep` , `wake` and `restart` do not happen there and then: the request is **queued**, and the scene manager carries it out as its next update begins. (Keys are read between frames, so in the log a command, marked `>>` , and the event it causes share a frame number.)

The exceptions are `add` and `remove` , which happen at once when they can. The logger's **8** key shows why that matters: removing a scene while Phaser is handing a key press to every scene, including the one being removed, makes Phaser trip over itself (`TypeError: Cannot read properties of null (reading 'queue')`). So `LogScene` waits a frame:

```
this.logCommand("scene.remove(DEMO_SCENE)");  
this.time.delayedCall(0, () => {  
    this.scene.remove(DEMO_SCENE);  
});
```

After a remove, **7** makes a brand new `DemoScene` with `this.scene.add(DEMO_SCENE, DemoScene, true)` , and this time the log shows `constructor()` again: a new object.

Cleaning up

When a scene shuts down, Phaser clears away a great deal: its game objects are destroyed, its timers and tweens thrown away, and listeners on its input (`this.input` , `this.input.keyboard` , game objects) removed.

But two kinds of listener are **not** removed:

1. **listeners on the scene's own** `this.events` . The scene object is reused, and so is its emitter. A listener added in `create()` is still there after a restart - and the next `create()` adds another
2. **listeners on anything that lives longer than the scene** - the game's emitter (`this.game.events`), the registry's (`this.registry.events`), or another scene's `events`

`DemoScene` uses both. Besides its own events, it listens to the **game's** `STEP` event, which happens once every frame whatever state the scene is in - that is the "game steps seen" count:

```
this.game.events.on(Phaser.Core.Events.STEP, this.onGameStep, this);
```

So when it shuts down, it takes everything off again:

```
private onShutdown(): void {  
    ...  
    this.events.off(Phaser.Scenes.Events.CREATE, this.onCreate, this);  
    this.events.off(Phaser.Scenes.Events.PRE_UPDATE, this.onPreUpdate, this);  
    ...  
    this.game.events.off(Phaser.Core.Events.STEP, this.onGameStep, this);  
}
```

`off(event, method, context)` removes the listener that was added with exactly those three things. And `onShutdown` itself was added with `once`, which removes itself after it has run.

A leak you can see

Press **L** to turn on **leak mode** - `onShutdown()` "forgets" to clean up - then **5** twice:

DemoScene

update() calls: 68
 game steps seen: 201
 time: 7776 ms
 this.time.now: 7776 ms
 delta: 20.0 ms



```

244 POST_UPDATE event
244 POST_UPDATE event
244 RENDER event
263 >> demo.scene.restart()
263 SHUTDOWN event
263 leak mode: listeners left behind!
263 init()
263 preload()
263 loader: complete
263 create()
263 CREATE event
263 CREATE event
263 CREATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 UPDATE event
263 UPDATE event
263 Spinner.preUpdate()
263 UPDATE event
263 update()
263 POST_UPDATE event
263 POST_UPDATE event
263 POST_UPDATE event
263 RENDER event

```

DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
 fps: 45.8 listeners for CREATE: 3 for the game's STEP: 3 leak mode: ON super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
 L leak mode on/off S skip super.preUpdate() on/off C clear the log

Nearly every event is now logged three times: one listener from each run of `create()` .

"game steps seen" climbs three times faster than `update()` is called, and the status line counts three listeners for `CREATE` and three for the game's `STEP` .

Here a leak just makes the log noisy. In a real game every leftover listener does its work again - a score added twice, a sound played three times - or touches a game object that has since been destroyed, and crashes.

The rule: **whatever you add to an emitter that outlives your scene, take off in `SHUTDOWN` .** Listen with `once` where one time is enough.

Note - `off` needs the *same* function that `on` was given. This does nothing at all, because each arrow function is a new, different function:

```
this.game.events.on(Phaser.Core.Events.STEP, () => this.count());
this.game.events.off(Phaser.Core.Events.STEP, () => this.count()); //
removes nothing
```

Pass a method and `this` (as `DemoScene` does), or keep the arrow function in a variable.

SHUTDOWN or DESTROY ?

`SHUTDOWN` is for a scene that stops but may start again. `DESTROY` is for a scene removed for good. Phaser empties the scene's own `this.events` on `DESTROY`, but the game's emitter is still yours to tidy - and a running scene that is removed gets `DESTROY` **without** a `SHUTDOWN` first, so `DemoScene` tidies the game's emitter in `onDestroy()` as well.

Time, delta and the frame rate

`update(time, delta)` - and every `preUpdate(time, delta)` - gets two numbers:

- `time` - the game's clock, in milliseconds. It counts from when the page loaded, not from when the scene started. `this.time.now` is the same value (the logger shows both, side by side)
- `delta` - how long the last frame took, in milliseconds (Phaser smooths it a little, so one slow frame does not make things jump)

The difference shows when a scene is paused. `time` **keeps going** - the game is still running. `delta` **only arrives while the scene runs**. Pause for ten seconds and `time` has jumped ten seconds when you resume, while anything that adds up `delta` has not moved. `ch04_pause_and_hud` shows why that matters.

Frame rate

The logger shows `this.game.loop.actualFps` : the frames per second the browser is giving Phaser - usually the screen's refresh rate, 60, or 120 or 144 on faster screens. The config decides what Phaser does with them:

src/main.ts

```
fps: {  
  target: 60,  
  limit: 0,  
},
```

`target` is the rate Phaser expects (60 is the default). `limit` caps it: with `limit: 30`, Phaser skips browser frames so that the game steps at most 30 times a second (`actualFps` still counts the browser's frames, so it does not drop). Try `limit: 10` in the logger: the coin moves at the same speed, in bigger jumps, because it moves by `delta` - the payoff for Chapter 1's pixels per second.

Pause and HUD

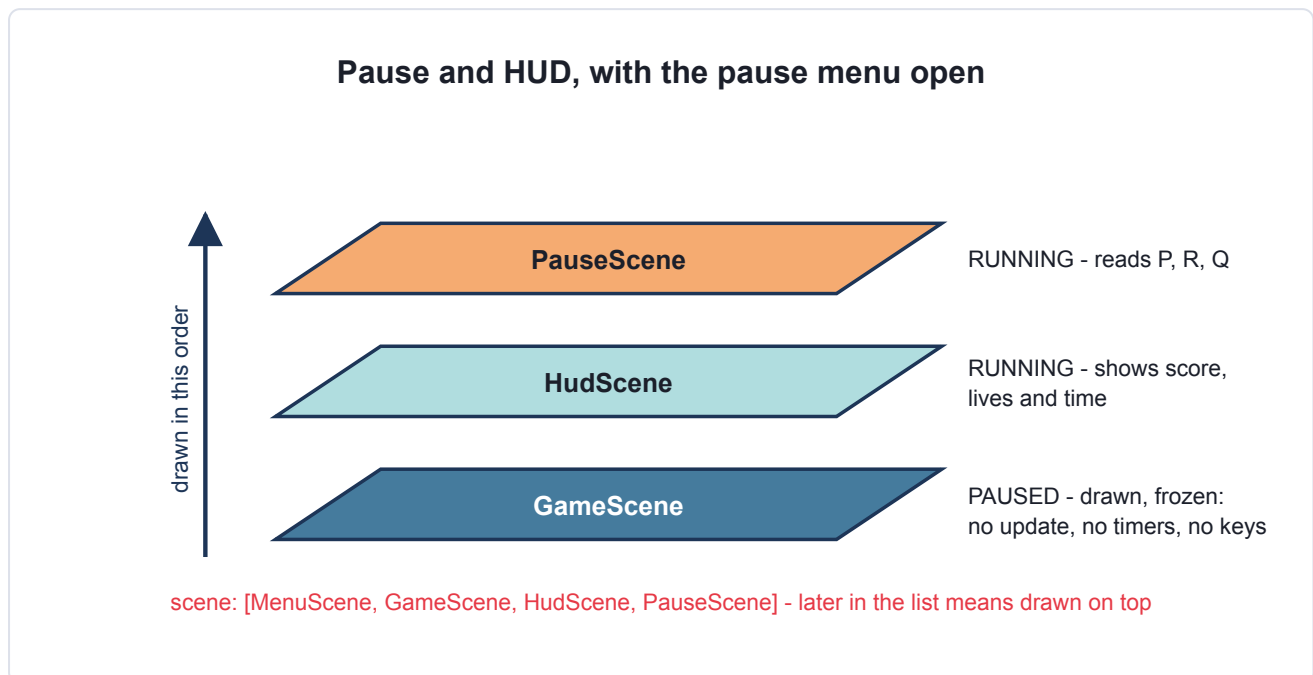


`ch04_pause_and_hud` is a small game, Coin Rush: move the blue blob with the arrow keys, collect the spinning coins before they fade, keep away from the triangles, and see how much you can collect in 45 seconds. The game is written as four scenes:

src/main.ts

```
scene: [MenuScene, GameScene, HudScene, PauseScene],
```

`MenuScene` loads everything and waits for SPACE. `GameScene` is the game. It does not draw the score, lives or time - `HudScene` does, running **on top of it**. And P opens `PauseScene`, on top of both, while `GameScene` is paused underneath:



Scenes that are running at the same time are drawn in the order of the scene list - the first at the back. (They are *updated* in the opposite order: the top scene first.)

A HUD in its own scene

Why not just add the text to `GameScene` ? Each scene has its own camera, so when a later chapter makes the game's camera follow the player round a big world, or shake, the HUD stays put. And the HUD's code lives in its own class, not tangled into the game's.

`GameScene` launches it at the end of `create()` :

`src/scenes/GameScene.ts`

```
const hudData: HudData = { score: this.score, lives: this.lives, seconds:
this.secondsShown };
this.scene.launch(HUD_SCENE, hudData);
```

`launch` starts the HUD **alongside** `GameScene` - `start` would stop `GameScene`. And because the launch is queued, the HUD's `create()` will not run until after `GameScene`'s has finished, so it is handed the starting values as data rather than left to wait for the first change.

The HUD listens

From then on, `GameScene` never touches the HUD. When something changes, it **emits an event** on its own emitter:

```
private collect(coin: Coin): void {
    coin.destroy();
    this.sound.play(COIN_SOUND_KEY);
    this.score = this.score + COIN_POINTS;
    this.events.emit(SCORE_CHANGED, this.score);
}
```

`this.events.emit(name, value)` calls every listener for `name`, passing them `value`. The names are constants in `src/scenes/gameEvents.ts`. The HUD listens:

`src/scenes/HudScene.ts`

```
// Listen to GameScene's own emitter. That emitter belongs to GameScene, not
to this scene,
// so Phaser will not remove these listeners when the HUD shuts down - we
must.
const gameScene = this.scene.get(GAME_SCENE);
gameScene.events.on(SCORE_CHANGED, this.showScore, this);
gameScene.events.on(LIVES_CHANGED, this.showLives, this);
gameScene.events.on(TIME_CHANGED, this.showTime, this);

this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
    gameScene.events.off(SCORE_CHANGED, this.showScore, this);
    gameScene.events.off(LIVES_CHANGED, this.showLives, this);
    gameScene.events.off(TIME_CHANGED, this.showTime, this);
});
```

This is the cleaning-up rule at work. The listeners are on `GameScene`'s emitter, which outlives every run of the HUD: without the `SHUTDOWN` handler, after two restarts every score change would be handled three times. (Here that does no visible harm - which is what makes leaks easy to miss. Chapter 6 compares this way of feeding a HUD with two others.)

The HUD belongs with the game, so when `GameScene` shuts down - a restart, or quitting to the menu - it stops the HUD too:

`src/scenes/GameScene.ts`


```
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
    this.scene.stop(HUD_SCENE);
});
```

Pausing

P (or ESC) calls this:

```
private pauseGame(): void {
    const data: PauseData = { gameOver: false, title: "Paused", score:
this.score };
    this.scene.launch(PAUSE_SCENE, data);
    this.scene.pause();
}
```



Launch the pause menu, pause this scene. Everything in `GameScene` freezes: the player and the triangles (their `preUpdate()` is not called), the coins (their animation is in `preUpdate()` too), the round's timer and the coin spawner (the scene's clock is not updated), the keyboard (a paused scene hears nothing). And it all stays on screen, behind the darkened panel. The HUD is still running, but nothing is emitting events, so it has nothing to change.

`PauseScene` reads its own keys:

src/scenes/PauseScene.ts

```
keyboard.once("keydown-R", () => {  
    // start() stops THIS scene and starts GameScene - which, being paused, is  
    // shut down and  
    // started again from init()  
    this.scene.start(GAME_SCENE);  
});  
keyboard.once("keydown-Q", () => {  
    this.scene.stop(GAME_SCENE);  
    this.scene.start(MENU_SCENE);  
});
```

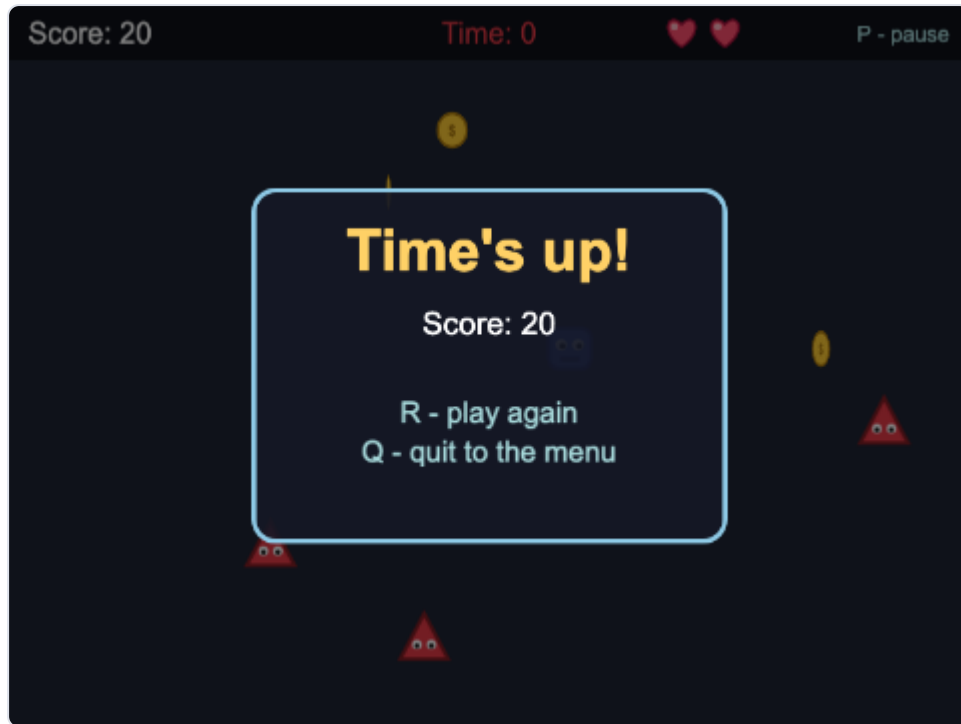
```
// resume the game, and take this scene away  
private carryOn(): void {  
    this.scene.resume(GAME_SCENE);  
    this.scene.stop();  
}
```

`this.scene.stop()` with no key stops the pause scene itself; next time P is pressed it starts fresh. Its `create()` begins with `this.scene.bringToTop()`, which moves it to the end of the scene list - it is last already, but this keeps it on top whatever order the list is later given. (`sendToBack`, `moveAbove` and `moveBelow` do the other moves.)

`GameScene` 's P listener uses `on`, not `once`: that scene keeps running after a pause, and P must work every time. `PauseScene` uses `once`: it is stopped after one choice.

Game over, with the same scene

When the time runs out, or the last life goes, `endRound()` does almost the same thing - pauses the game and launches `PauseScene` - but with `gameOver: true` in the data, so the pause scene leaves out "P - carry on", and its P key. One scene, two uses, told apart by its data.



Time that survives a pause

After the player is hit, they are safe for a second and a half. The obvious way is to remember the time and compare:

```
// NOT what the game does
this.safeUntil = this.time.now + SAFE_TIME;
...
if (this.time.now > this.safeUntil) { ... }
```

But `this.time.now` is the game's clock, and it keeps going while the scene is paused. Pause during the safe time, wait, carry on - and the safe time has already gone. So the game uses a **timer**, which belongs to the scene's clock and only counts while the scene runs:

src/scenes/GameScene.ts

```
this.player.setAlpha(0.4);
this.time.delayedCall(SAFE_TIME, () => {
  this.safe = false;
  this.player.setAlpha(1);
});
```

The round's 45 seconds are a timer too (`this.time.addEvent(...)` , and `roundTimer.getRemaining()` for the HUD). And each coin counts its own age with `delta` :

`src/objects/Coin.ts`

```
protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);

    // Age by delta, not by the clock: delta only arrives while the scene is
    // running, so a
    // coin's four seconds do not tick away while the game is paused.
    this.age = this.age + delta;

    const timeLeft = LIFETIME - this.age;
    if (timeLeft <= 0) {
        this.destroy();
    } else if (timeLeft < FADE_TIME) {
        this.setAlpha(timeLeft / FADE_TIME);
    }
}
```

`Coin` is a `Sprite` (it spins), so here is `super.preUpdate(time, delta)` again, first.

Rule of thumb: in a game that can be paused, measure time with **timers** and `delta` , not by comparing `time` or `this.time.now` .

Common mistakes

Note - "My animation does not play." A `Sprite` subclass with a `preUpdate()` that does not call `super.preUpdate(time, delta)`. It builds, it moves, it never changes frame.

"Everything happens twice after I restart." A listener added in `create()` to `this.events`, `this.game.events`, `this.registry.events` or another scene's `events`, and never removed. Take it off in `SHUTDOWN`.

"this.scene.start(PAUSE_SCENE) made my game disappear." `start` stops the scene that called it. To run a scene alongside, use `launch`.

"The timer ran out while I was paused." Time measured with `time` or `this.time.now`. Use a timer, or add up `delta`.

Summary

- the constructor runs **once**, when the game starts; `init`, `preload` and `create` run every time the scene starts; `update` runs every frame while it is running
- each frame, for each running scene: `PRE_UPDATE`, `UPDATE` (game objects' `preUpdate`, timers, tweens, physics), your `update()`, `POST_UPDATE`, then drawing
- listen for scene events with `this.events.on(Phaser.Scenes.Events.X, this.method, this)`
- a `Sprite` subclass overrides `protected preUpdate` - and must call `super.preUpdate(time, delta)`
- `launch` runs a scene alongside; `pause` freezes a scene but still draws it; `sleep` hides it too; `stop` shuts it down; `remove` destroys it. Requests are queued until the next frame
- running scenes are drawn in the order of the scene list; `bringToTop` moves one to the front
- Phaser clears a scene's game objects, timers, tweens and input listeners when it shuts down - but not listeners on `this.events` or on longer-lived emitters. Remove those in `SHUTDOWN`

- `time` and `this.time.now` keep going while a scene is paused; timers and `delta` do not
 - `fps: { target, limit }` sets the frame rate; things that move by `delta` do not care
-

Challenges

1. **Count the frames drawn** (*ch04_lifecycle_logger*) - Count how many times `DemoScene` has been drawn (its `RENDER` event) since it started, and show it with the number of `update()` calls in the status lines at the bottom - not in `DemoScene`'s own text (why not?). Pause the scene, then put it to sleep: which numbers stop?
2. **Thirty frames a second** (*ch04_pause_and_hud*) - Limit the game to 30 frames a second, and show on the HUD how many frames the game *really* runs each second (`actualFps` counts the browser's). Check that the player and the triangles move just as fast as before.
3. **How long was I away?** (*ch04_pause_and_hud*) - The pause menu shows how long the game has been paused, counting up in tenths of a second. When the player carries on, the HUD shows the total time paused this round. A pause must count once, however many times the round has been restarted.
4. **Settings** (*ch04_pause_and_hud*) - Add "S - settings" to the pause menu. It opens a new `SettingsScene` over the paused game while the pause menu sleeps; S or ESC closes it and wakes the pause menu. One setting: H turns the HUD on and off, at once, and it stays that way after a restart. *Hint:* `this.scene.setVisible(false, key)` hides a scene without pausing it. Where can a setting live that outlasts every scene?
5. **Fade in, fade out** (*ch04_pause_and_hud*) - The menu fades to black before the game starts, and the game and HUD fade in; restarting or quitting from the pause menu fades out first too. A key pressed during a fade must not start another. *Hint:* `this.cameras.main.fadeOut(ms)`, `fadeIn(ms)` and the camera event `Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE`. A camera's effects are updated with its scene - so which camera can fade while the game is paused?
6. **Auto-pause, without a leak** (*ch04_pause_and_hud*) - Open the pause menu by itself when the window loses focus or the tab is hidden - but only while a round is being played. Then prove there is no leak: show on the HUD how many listeners the game's emitter has for those events, and check the number does not grow however often you

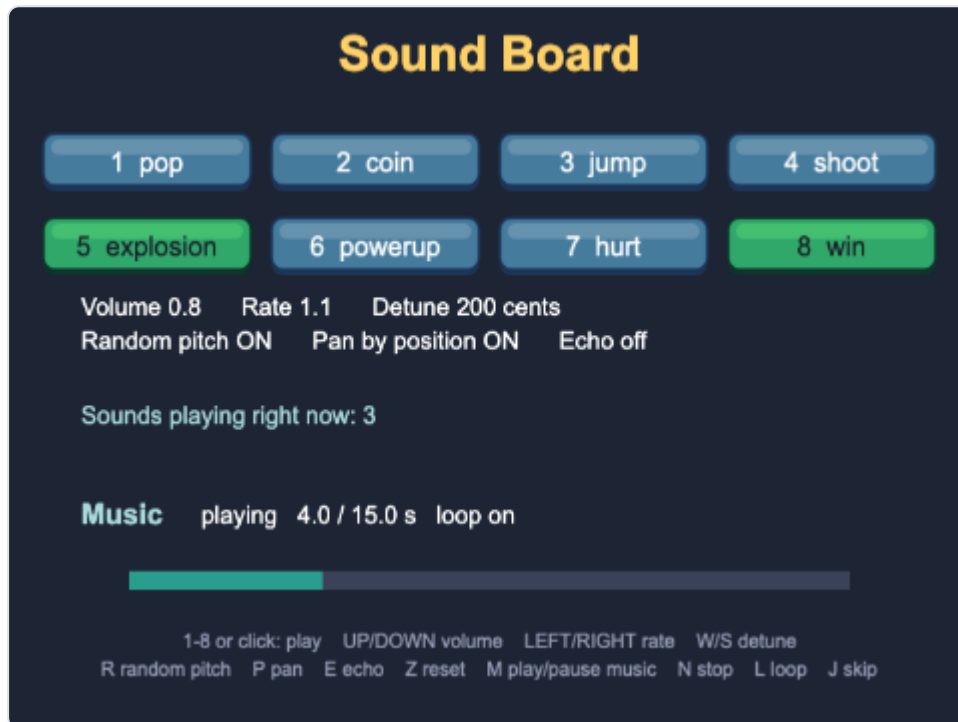
restart, or quit and play again. *Hint:* the events are `Phaser.Core.Events.BLUR` and `HIDDEN` on `this.game.events`, which lives as long as the game; `listenerCount(event)` counts (Phaser has listeners of its own there). To test, click on another window, or switch tab and back.

Previous: [Chapter 3 - Preloading](#) · Next: [Chapter 5 - Audio](#)



Audio

A game without sound feels strangely empty: a click that makes no noise, a jump with no *boing*, a menu with no music. In this chapter you learn how Phaser plays sound effects and music - loading them, playing them in different ways, changing them while they play - and how to deal with the two things about audio that surprise everyone: music does not stop when the scene that started it does, and a browser will not let a page make a sound until the player has clicked on it.



What you will learn

- how to load sounds, and the two ways to play one: `this.sound.play()` and `this.sound.add()`
- how to change a sound with a config object - volume, rate, detune, pan, loop, delay - and with a sound object's methods while it plays
- a sound object's life: playing, paused, stopped, the "complete" event, and `destroy()`
- why the sound manager belongs to the **game**, not to a scene - and how to keep one copy of the music playing as the scenes change
- how to fade and cross-fade music with tweens, and mute every sound in the game
- the browser's autoplay rule, `this.sound.locked`, and "click to start" screens

The projects

Project	What it shows
<code>ch05_sound_board</code>	eight pads that play sound effects, keys that change how they sound, and a music track to play, pause, stop, loop and skip through
<code>ch05_music_across_scenes</code>	a title screen, a menu, credits and a tiny game, with music that carries on between scenes, cross-fades, and a mute button that works everywhere

Loading sounds

Sounds are loaded like pictures (Chapter 3), in `preload()`, with a key and a file:

`ch05_sound_board/src/scenes/SoundBoardScene.ts`

```
preload(): void {
  for (const pad of PAD_SOUNDS) {
    this.load.audio(pad.key, pad.file);
  }
  this.load.audio(MUSIC_KEY, MUSIC_FILE);
  this.load.image(BUTTON_KEY, BUTTON_FILE);
  this.load.image(BUTTON_OVER_KEY, BUTTON_OVER_FILE);
}
```

`PAD_SOUNDS` is an array of `{ key, file }` objects in `src/assets.ts`, so the list of sounds is data, and the loop loads them all. Once loaded, a sound is **decoded** - turned from a file into raw audio the browser can play - and kept in the game's audio cache under its key, for every scene to use.

The guide's sounds are all WAV files, which every browser plays. Real games usually use smaller, compressed formats - MP3, OGG, M4A - and not every browser plays every format (Safari has no OGG, for example). So `load.audio` also takes an **array** of files, and Phaser loads the first one this browser can play:

```
this.load.audio("theme", ["assets/audio/theme.ogg",
"assets/audio/theme.mp3"]);
```

Note - Music files are big: `music_game.wav` is 660 KB, bigger than every picture in the project put together. Load music in a loading scene (Chapter 3), so the player is not left staring at a black screen.

Playing a sound: fire and forget

The quickest way to play a sound is the one Chapter 2 used:

```
this.sound.play("coin");
```

`this.sound` is the **sound manager**. `play(key)` makes a sound, plays it, and - when it has finished - throws it away. You never see the sound object, so you cannot stop it or change it once it has started. For most sound effects that is exactly right: a coin pings, and that is the end of it.

`play` takes a second argument: a **config** object that says *how* to play it. Only the settings you give change; the rest keep their defaults.

Setting	Default	What it does
<code>volume</code>	1	0 is silent, 1 is full volume
<code>rate</code>	1	speed: 2 is twice as fast (and an octave higher), 0.5 half speed (an octave lower)
<code>detune</code>	0	pitch, in cents : 100 cents is a semitone, 1200 an octave; -1200 to 1200
<code>pan</code>	0	-1 is fully left, 0 the middle, 1 fully right
<code>loop</code>	false	start again from the beginning each time it ends
<code>delay</code>	0	wait before starting - in seconds
<code>seek</code>	0	start this far in - in seconds
<code>mute</code>	false	silent, but still "playing"

`SoundConfig` is a Phaser type (`Phaser.Types.Sound.SoundConfig`), but the object itself is just an **object literal** (Chapter 2). The sound board's echo uses one - the same sound again, quieter and a quarter of a second later:

ch05_sound_board/src/scenes/SoundBoardScene.ts

```
if (this.echo) {  
  this.sound.play(pad.soundKey, { ...config, volume: this.volume *  
    ECHO_VOLUME, delay: ECHO_DELAY });  
}
```

`{ ...config, volume: ..., delay: ... }` is the **spread** from Chapter 2: a copy of the pad's config, with a new volume and a delay.

Note - Almost everything in Phaser is timed in milliseconds - but `delay` and `seek` in a sound config are in **seconds**. `delay: 250` does not wait a quarter of a second: it waits four minutes. There is no error, just a sound that never seems to play.

Rate and detune

Both change the pitch, and both change the speed - in Web Audio, a sound played higher is played faster, like a record at the wrong speed. The difference is the units:

- `rate` is a multiplier: `rate: 2` is twice as fast and an octave higher
- `detune` is in cents, the unit musicians use: `detune: 1200` is also an octave higher - and so also twice as fast. `detune: 100` is one semitone, the next note up on a piano

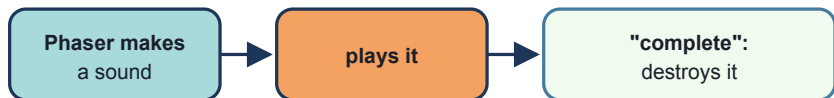
Use `detune` when you think in notes ("two semitones up"), and `rate` when you think in speed ("a little faster"). Phaser multiplies them together, so they can be used at the same time. Try them on the sound board: LEFT/RIGHT change the rate, W/S the detune, a semitone per press.

Keeping hold of a sound: `add`

When you need to control a sound after it starts - stop it, change its volume, know when it ends - use `this.sound.add(key, config)` instead. It makes the sound object and **gives it to you**. It does not start it: you call `play()` when you want it.

Two ways to play a sound

this.sound.play(key, config)
"fire and forget"



You never get the sound object: you cannot stop it, change it, or know when it ends.

const s = this.sound.add(key, config)
"keep it"



It stays in the sound manager - played or not - until you call s.destroy().

Use it for music, and for any sound you need to control or listen to.

Each pad on the sound board lights up while its sound plays, and goes out when it ends - so it needs the sound object, to listen for its end:

ch05_sound_board/src/objects/SoundPad.ts

```

public play(config: Phaser.Types.Sound.SoundConfig): void {
    // add() makes a Sound object and keeps it in the sound manager until it
    // is destroyed.
    // Unlike this.sound.play(key), it gives us the object - so we can listen
    // for its end
    const sound = this.scene.sound.add(this.soundKey);

    sound.once(Phaser.Sound.Events.COMPLETE, () => {
        this.playing = this.playing - 1;
        this.showState();
        // a sound made with add() stays in the manager until destroyed - and we
        // will not need
        // this one again (the next press makes a new one)
        sound.destroy();
    });

    sound.play(config);
    this.playing = this.playing + 1;
    this.showState();
}
  
```

- a sound is an event emitter, like a game object: `sound.once(event, callback)` listens for one event. `Phaser.Sound.Events.COMPLETE` is the constant for "complete", which fires when the sound reaches its end. (There are others: "play", "pause", "resume", "stop", "looped")

- `sound.play(config)` plays it, with the settings the scene worked out
- the pad counts how many of its sounds are playing, because you can press it again before the first one has finished - both play at once, and the pad should only go out when the last one ends

Sounds made with `add()` are not thrown away for you. `this.sound.play()` destroys its sound when it completes; `add()` leaves it in the sound manager until you call `destroy()`. Forget, and every press of a pad would leave another dead sound object behind, for as long as the game runs.

What type is a sound?

The sound board keeps its music in a field. What type is it? When a game starts, Phaser picks one of **three** sound managers: Web Audio (nearly every browser), HTML5 Audio (very old browsers), or "no audio" (no sound hardware, or sound turned off in the config). Each makes its own kind of sound, so the honest answer is "one of three" - a **union type** (Chapter 2):

`ch05_sound_board/src/GameSound.ts`

```
export type GameSound = Phaser.Sound.WebAudioSound |
    Phaser.Sound.HTML5AudioSound | Phaser.Sound.NoAudioSound;
```

`type` gives a type a name - like a very short interface. All three classes have the same methods (`play` , `setVolume` , ...), so TypeScript lets you call any of them on a `GameSound` , whichever one Phaser picked. `SoundPad` does not need the type written out at all: `const sound = this.scene.sound.add(...)` lets TypeScript work it out.

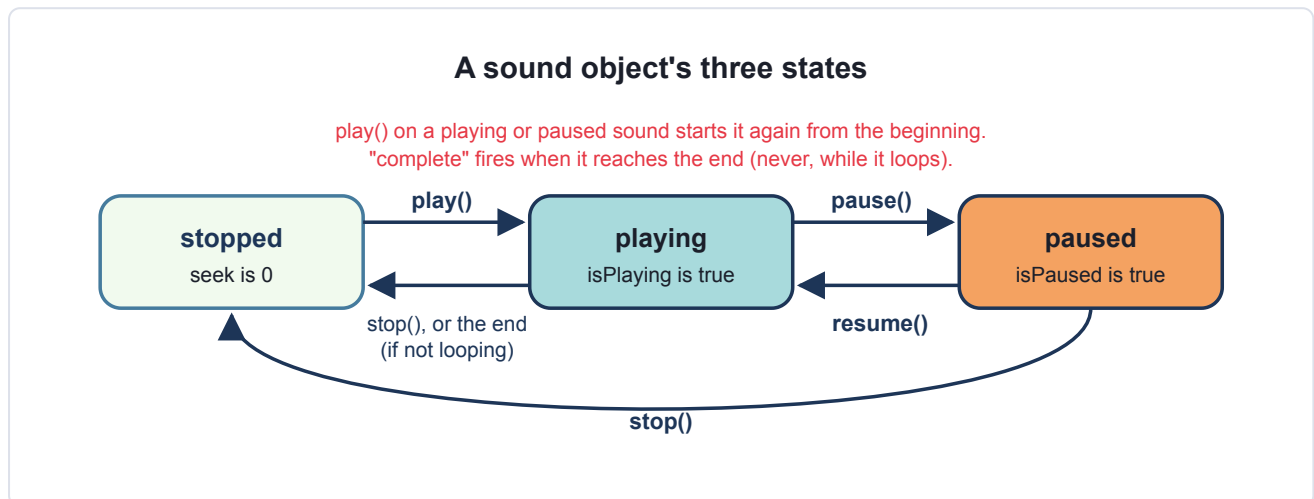
Controlling a sound while it plays

The sound board's music is **one** sound object, made once in `create()` and kept:

`ch05_sound_board/src/scenes/SoundBoardScene.ts`

```
// The music is made ONCE, with add(), and kept: it is the same sound object
// every time it is
// played, paused or changed. Adding it does not start it.
this.music = this.sound.add(MUSIC_KEY, { loop: true, volume: MUSIC_VOLUME
});
```

A sound object is always in one of three states:



M plays, pauses and carries on - three jobs for one key, depending on the state:

```
private playOrPauseMusic(): void {
  if (this.music.isPlaying) {
    this.music.pause();
  } else if (this.music.isPaused) {
    this.music.resume();
  } else {
    this.music.play();
  }
}
```

`pause()` remembers where the sound had got to, and `resume()` carries on from there; `stop()` goes back to the start. Calling `play()` on a sound that is already playing starts it again from the beginning - which is why the code checks first.

Every setting in the config table can be changed while a sound plays, with a `set...` method or by setting the property:

```
private changeRate(step: number): void {
  this.rate = Phaser.Math.Clamp(this.rate + step, MIN_RATE, MAX_RATE);
  this.music.setRate(this.rate);
  this.showSettings();
}
```

`Phaser.Math.Clamp(value, min, max)` keeps a value between a minimum and a maximum. The `set...` methods return the sound itself, so they can be **chained**, as Phaser's game object methods can:

```
this.music.setVolume(MUSIC_VOLUME).setRate(1).setDetune(0);
```

The others: `setLoop(true)` (the L key), `setPan(-1)`, `setMute(true)`, and `setSeek(seconds)`, which jumps to a place in the sound (the J key). Seeking only works while a sound is playing:

```
private skipMusic(): void {
  if (this.music.isPlaying) {
    const seek = (this.music.seek + MUSIC_SKIP) % this.music.duration;
    this.music.setSeek(seek);
  }
}
```

`music.seek` is how far through the sound it is, in seconds, and `music.duration` its length. The `%` wraps the new position round to the start if it would go past the end. `update()` uses the same two values every frame to draw the progress bar.

Making repeated sounds less boring

Click the same pad ten times, and by the fifth it sounds like a machine - because it is. Real sounds are never quite the same twice. Two cheap tricks help:

- **random pitch:** a small random `detune` each time. Press R on the sound board, then play a pad over and over
- **pan by position:** play a sound from the side of the screen it came from. Press P, then play the pads on the far left and far right (use headphones)

The music project's game uses both. Every star you click plays a coin sound:

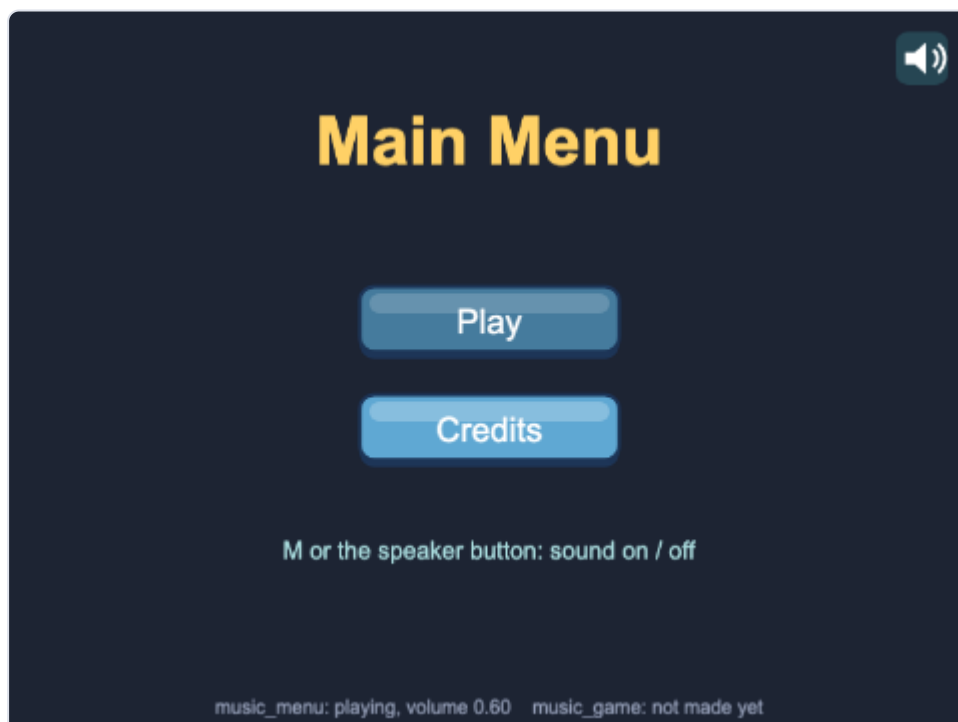
```
ch05_music_across_scenes/src/scenes/GameScene.ts
```

```
const centreX = this.scale.width / 2;
this.sound.play(COIN_KEY, {
  detune: Phaser.Math.Between(-RANDOM_DETUNE, RANDOM_DETUNE),
  pan: (star.x - centreX) / centreX * MAX_PAN,
});
```

`Phaser.Math.Between(min, max)` is a random whole number from `min` to `max`. The `pan` sum turns the star's `x` (0 to 800) into a number from -1 to 1, then `MAX_PAN` (0.8) keeps it from going fully to one side, which sounds odd.

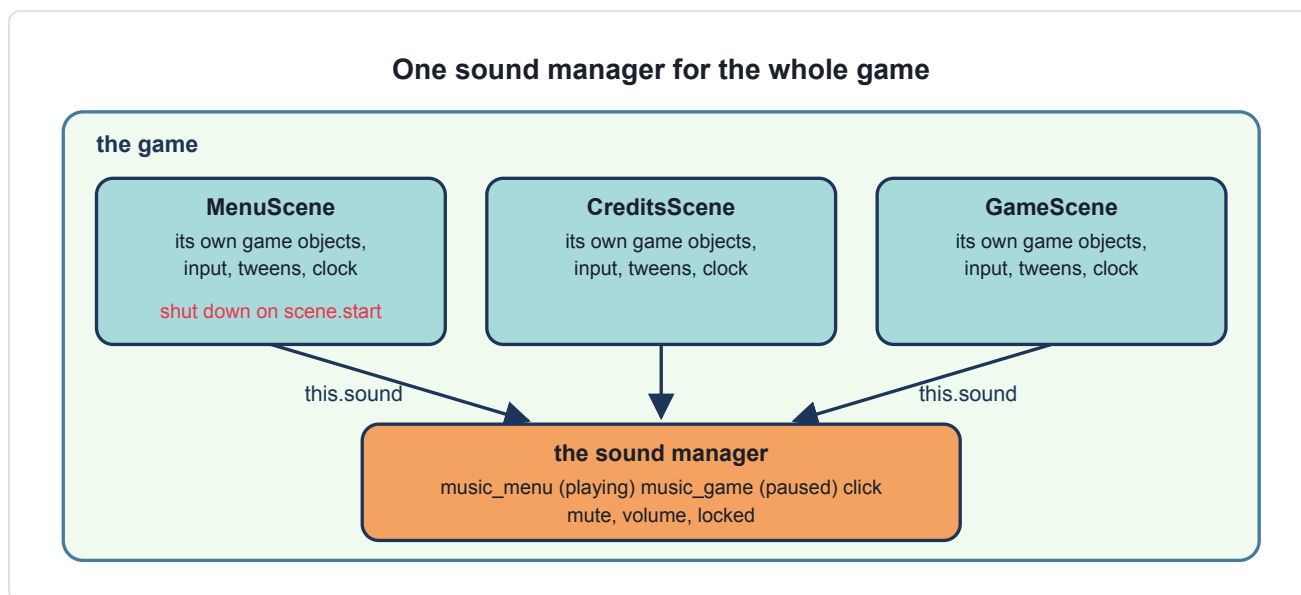
Note - Panning needs Web Audio's stereo panner, and Phaser does not use it on iPhones and iPads (older versions of Safari had none): there, `pan` is quietly ignored. Make it a finishing touch, never something the game depends on.

The sound manager belongs to the game



Here is the fact that shapes every game with music: `this.sound` is the same object in every scene. There is one sound manager, and it belongs to the game. Each scene has its

own game objects, input, clock and tweens - all thrown away when the scene shuts down - but it shares the sound manager with every other scene.



So when `this.scene.start(...)` shuts a scene down, its sounds **keep playing**. That is what you want for music: the menu music carries on while the player looks at the credits. It is also why the click in `MenuButton` still plays after the button's scene has gone:

`ch05_music_across_scenes/src/objects/MenuButton.ts`

```
this.on("pointerdown", () => {
    // fire and forget: the click carries on playing even though onClick()
    // usually starts
    // another scene - sounds belong to the game, not to the scene that played
    // them
    scene.sound.play(CLICK_KEY);
    onClick();
});
```

But it causes a bug in almost every first attempt at music. Suppose the menu's `create()` did this:

```
this.sound.add(MUSIC_MENU_KEY, { loop: true }).play();
```

Menu, credits, menu: the menu's `create()` runs again - it runs every time the scene starts - and adds a **second** copy of the music, on top of the first, which never stopped. Go back and forth a few times and there are five copies, slightly out of step, getting louder and

louder. (That is not a guess: with that change, after menu - credits - menu - credits - menu, `this.sound.getAllPlaying()` holds five `music_menu` sounds.)

The fix is to ask the sound manager first. `this.sound.get(key)` returns the sound with that key if there is one - and `null` if there is not.

One place for the music

In the music project, every scene says which music it wants, in its `create()` :

```
ch05_music_across_scenes/src/scenes/MenuScene.ts
```

```
Music.play(this, MUSIC_MENU_KEY);
```

`CreditsScene` asks for the same track, and `GameScene` for `MUSIC_GAME_KEY`. `Music` is a small class of our own that decides what to do:

```
ch05_music_across_scenes/src/Music.ts
```

```

public static play(scene: Phaser.Scene, key: string): void {
    // fade out every OTHER track that is playing
    for (const otherKey of MUSIC_TRACKS) {
        if (otherKey !== key) {
            Music.fadeOut(scene, otherKey);
        }
    }

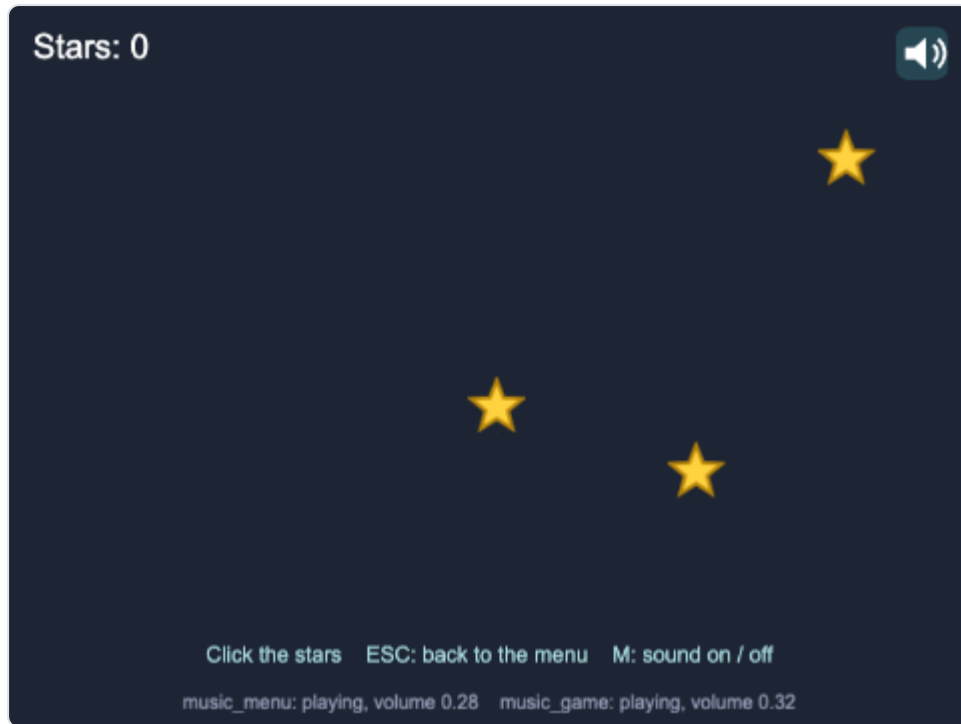
    // Is this track in the sound manager already? Phaser's types say get()
    // always returns a
    // sound, but it returns null when there is no sound with that key - so we
    // say so ourselves.
    let music: Phaser.Sound.BaseSound | null = scene.sound.get(key);

    if (music === null) {
        // the first time: make it (silent), and start it
        music = scene.sound.add(key, { loop: true, volume: 0 });
        music.play();
    } else if (music.isPaused) {
        // faded out earlier: carry on from where it was
        music.resume();
    } else if (!music.isPlaying) {
        music.play();
    }

    // (and if it is already playing, leave it alone - it just gets faded up
    // to full volume)

```

- `static` means the same as in Java: `play` belongs to the class, and is called as `Music.play(...)`, with no `new Music()`. The music does not belong to any scene, so it is given a home of its own - but the scene is passed in, because Phaser's sound manager and tweens are reached through a scene
- `Phaser.Sound.BaseSound | null`: Phaser's type for `get` forgets to say it can return `null`. Writing the union ourselves makes TypeScript insist on the `=== null` check
- menu to credits: `music_menu` is already playing, so nothing starts - it just carries on.
Menu to game: `music_game` is made (the first time) or resumed (after that), and `music_menu` fades out



Fading with tweens

A **tween** changes a number on an object smoothly, over time. Chapter 7 covers tweens properly; here is all you need for fading. Volume is a number on the sound, so a tween can fade it:

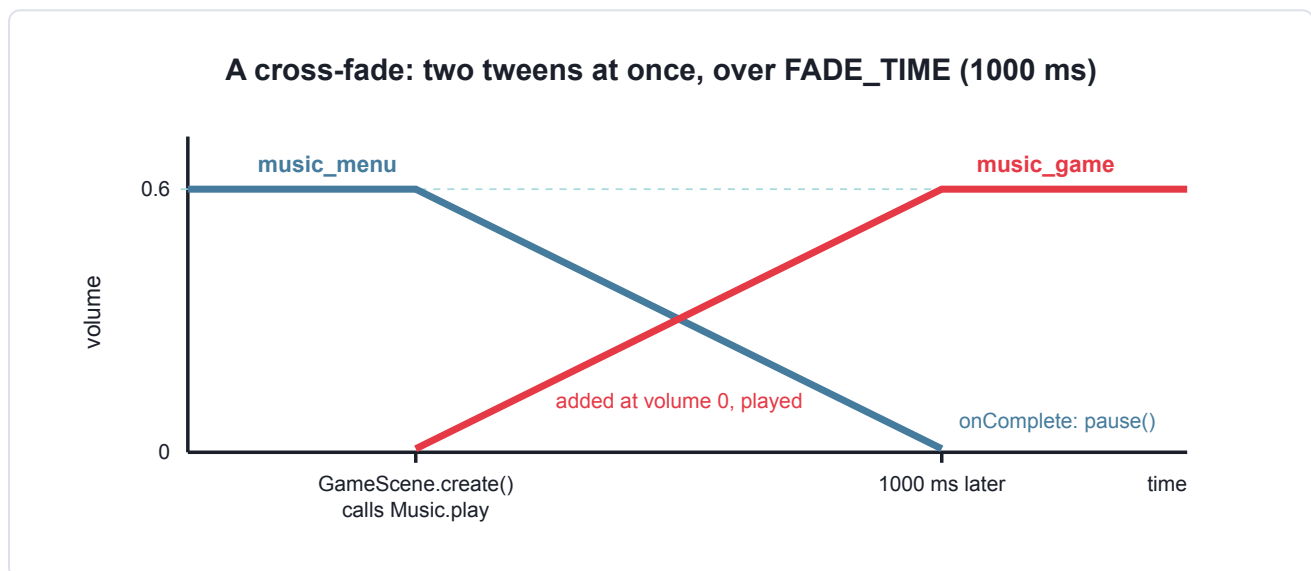
ch05_music_across_scenes/src/Music.ts

```
// stop any fade that is still running on it (it might be fading OUT), then  
fade it in  
scene.tweens.killTweensOf(music);  
scene.tweens.add({  
  targets: music,  
  volume: MUSIC_VOLUME,  
  duration: FADE_TIME,  
});
```

"Take `music`, and move its `volume` from whatever it is now to `MUSIC_VOLUME`, over `FADE_TIME` milliseconds." Fading out is the same, towards 0, with one more thing - an `onComplete` function that the tween calls when it has finished:

```
scene.tweens.add({
  targets: music,
  volume: 0,
  duration: FADE_TIME,
  onComplete: () => {
    music.pause();
  },
});
```

A silent track is still playing, so it is paused at the end of the fade - and resumed, from the same place, when the player comes back to that scene. Both tweens run at the same time, so one track fades out while the other fades in: a **cross-fade**.



`killTweensOf(music)` stops any tween this scene already has running on the music. Without it, a second fade could start while the first was still going: two tweens pulling one volume in opposite directions - and a fade-out's `onComplete` pausing music that is meant to be playing. In this project each scene calls `Music.play` once, so it is a safety net; the moment anything else tweens the music's volume (challenge 4 does), it earns its place.

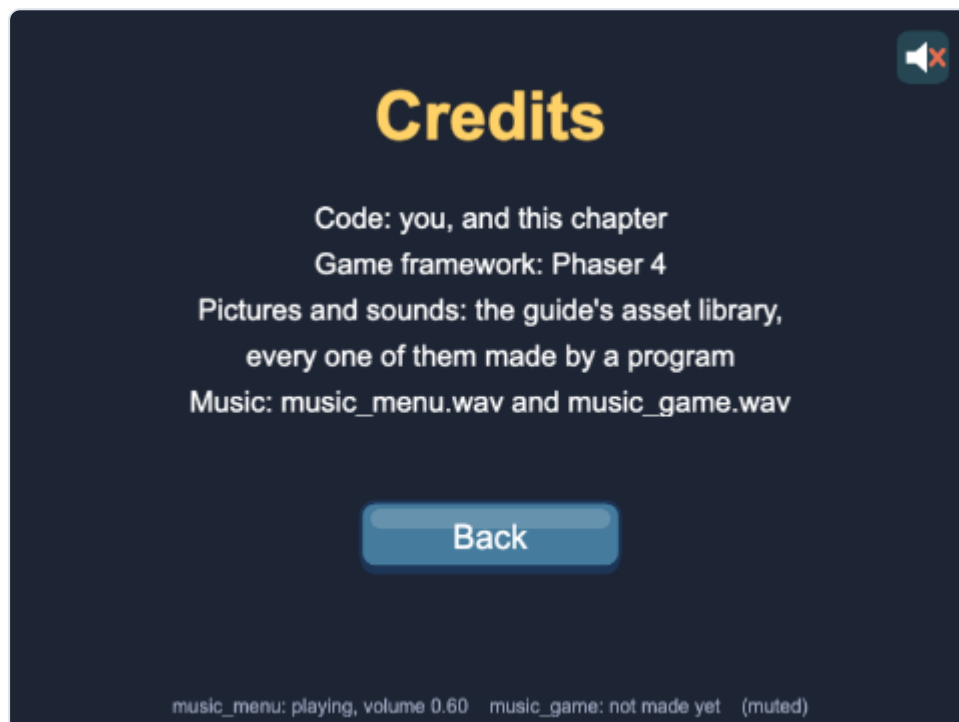
Whose tweens?

The music belongs to the game, but **tweens belong to a scene** - `scene.tweens` - and when a scene shuts down, its tweens are thrown away, finished or not. So a menu that started fading its own music out, then called `this.scene.start(GAME_SCENE)`, would leave the menu music stuck half-way, playing quietly underneath the game for ever.

That is why `Music.play` is called by the scene that is **starting**, in its `create()`, and makes both fades with that scene's tweens: that scene will be around to finish them.

The `MusicMonitor` line at the bottom of each scene shows each track's state and volume every frame, so you can watch the fades happen - and see that `music_game` is only ever paused, never made twice.

Mute, everywhere



Two properties of the sound manager affect every sound in the game:

- `this.sound.mute = true` silences everything - sounds playing now, and sounds started later
- `this.sound.volume = 0.5` is a master volume: every sound's own volume is multiplied by it

Each scene makes its own `MuteButton` - a game object belongs to one scene - but they all show and change the same setting:

```
ch05_music_across_scenes/src/objects/MuteButton.ts
```

```

private isMuted(): boolean {
    return this.scene.registry.get(MUTED) === true;
}

private toggle(): void {
    const muted = !this.isMuted();
    this.scene.registry.set(MUTED, muted);
    // this.sound.mute silences every sound in the game - playing ones, and
    // ones not yet started
    this.scene.sound.mute = muted;
    this.showState();
}

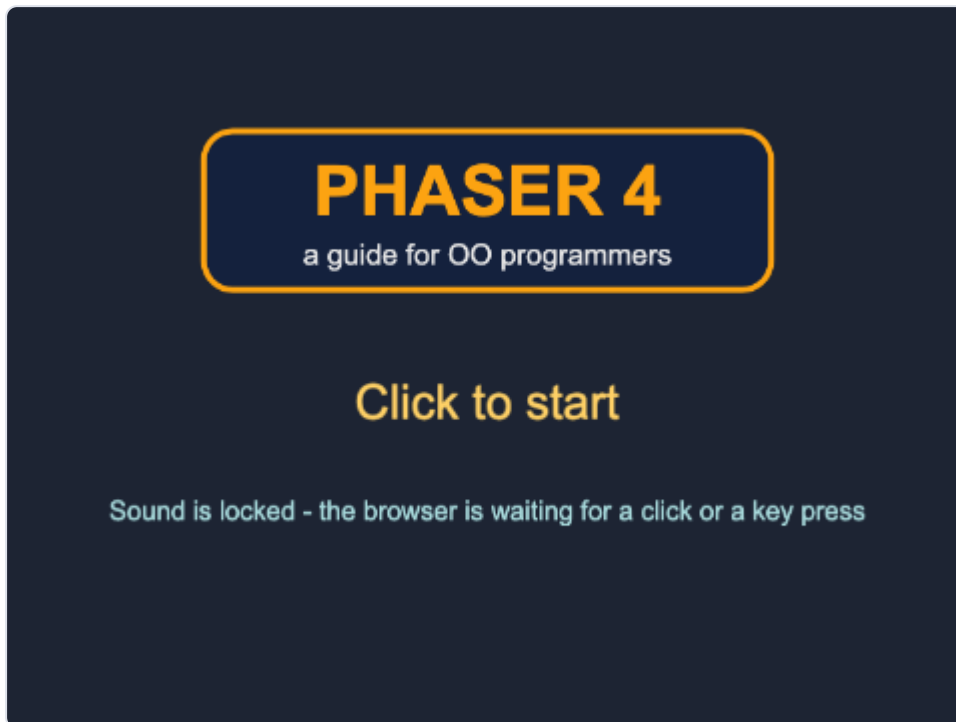
private showState(): void {
    this.setTexture(this.isMuted() ? SOUND_OFF_KEY : SOUND_ON_KEY);
}

```

`this.sound.mute` is itself game-wide, so strictly the registry is not needed. It is there because the registry (Chapter 2) is where the game keeps its **settings**: the button reads the setting to choose its picture (`sound_on.png` or `sound_off.png`), and when a setting needs to outlive the page - saved in the browser, say - the registry is the one place to save it from. Until the button is first pressed there is nothing in the registry (`undefined`), so `=== true` counts that as "not muted".

The button also listens for the M key, on its own scene's keyboard. Those listeners go when the scene shuts down, and the next scene's button adds its own - so M always works, and never twice.

The browser's autoplay rule



Imagine opening a web page and having it blare music at you. Browsers decided no page may do that: **a page may not play any sound until the player has interacted with it** - clicked, tapped or pressed a key. Until then, Phaser's sound manager is **locked**. Phaser listens for that first click or key press itself, and unlocks the sound as soon as it comes.

What happens to a sound played while the sound is locked? Phaser keeps it waiting, and it starts when the sound unlocks. That sounds fine, but anything timed goes wrong: a fade-in happens in silence, so the music starts at full volume - and every sound effect "played" before the click comes out at once, the moment it is unlocked. (Try it: three effects played a second apart while locked all start together on the first click.)

So most browser games start with a **"click to start"** screen. It has no sound, and the click that leaves it is the one that unlocks the sound - so by the time the menu starts its music, the sound is ready.

```
ch05_music_across_scenes/src/scenes/TitleScene.ts
```



```

if (this.sound.locked) {
    status.setText("Sound is locked - the browser is waiting for a click or a
key press");

    // The sound manager belongs to the game, so a listener on it outlives
this scene. The
    // click that unlocks the sound also starts the menu - and "unlocked"
arrives a frame
    // later, when this scene and its text are gone. So take the listener off
at SHUTDOWN.
    const onUnlocked = (): void => {
        status.setText("Sound is unlocked");
    };
    this.sound.once(Phaser.Sound.Events.UNLOCKED, onUnlocked);
    this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
        this.sound.off(Phaser.Sound.Events.UNLOCKED, onUnlocked);
    });
} else {
    status.setText("Sound is ready");
}

```

- `this.sound.locked` is `true` until the browser allows sound
- the sound manager emits `"unlocked"` (`Phaser.Sound.Events.UNLOCKED`) when it has been unlocked

Notice the `SHUTDOWN` listener - the lesson of Chapter 4, met for real. The first version of this scene did not have it. The click that unlocks the sound also starts the menu, which shuts the title scene down and destroys its text. The `"unlocked"` event comes a frame later - and the listener, which lives on the game-wide sound manager, not the scene, tried to change text that no longer existed:

```

TypeError: Cannot read properties of null (reading 'drawImage')

```

Any listener you put on something that outlives the scene - the sound manager, `this.game.events`, the registry's events - must be taken off when the scene shuts down. A listener on the scene's own objects, or `this.input`, is cleaned up for you.

Whether you ever see "Sound is locked" depends on the browser, and on the page's history: some browsers let a page play sound straight away when you have clicked on it

before, or often play sound on that site. Test with a fresh browser window, as a new player would see it.

Common mistakes

- **Two copies of the music:** playing the music in a scene's `create()` without checking `this.sound.get(key)` first. Every visit to the scene adds another copy
- **Music that never stops:** forgetting that sounds carry on when their scene shuts down. Stop or fade out music on purpose, when a scene that should be quiet starts
- `delay: 500` **does nothing:** sound delays and seeks are in seconds, not milliseconds
- **Sounds that pile up:** `add()` without `destroy()`. Use `this.sound.play()` for effects you do not need to control
- **A fade that stops half-way:** a tween made in a scene that is shutting down is thrown away with it
- `Audio key "music_menu" not found in cache`: the sound was not loaded under that key - a misspelled key, or a file that failed to load (look for a red 404 in the browser's console)

Summary

- `this.load.audio(key, file)` loads a sound; an array of files lets the browser pick a format
- `this.sound.play(key, config)` plays a sound and throws it away when it ends - fire and forget
- `this.sound.add(key, config)` gives you a sound object to keep: `play`, `pause`, `resume`, `stop`, `isPlaying`, `isPaused`, `seek`, `duration`, and `setVolume`, `setRate`, `setDetune`, `setPan`, `setLoop`, `setSeek`. Destroy it when you are done with it
- a config object sets `volume`, `rate`, `detune` (cents), `pan`, `loop`, `delay` and `seek` (seconds)
- `sound.once("complete", ...)` tells you when a sound has finished
- a little random detune and a pan by position keep repeated sounds from sounding mechanical

- the sound manager belongs to the game: sounds keep playing across scenes, so check `this.sound.get(key)` before starting music; `this.sound.mute` and `this.sound.volume` affect everything
- tween a sound's `volume` to fade it; tweens belong to the scene that made them
- sound is locked until the player interacts with the page: `this.sound.locked`, the "unlocked" event, and a "click to start" screen

Challenges

1. **More pads** (*ch05_sound_board*) - Add four more pads in a third row, for `door`, `whoosh`, `correct` and `wrong` (copy the files from the asset library's `audio/` folder into `public/assets/audio/`). Pad 9 should also play with the 9 key; the others can be click-only.
2. **Turn it down** (*ch05_music_across_scenes*) - The V key changes the master volume of the whole game, in every scene except the title: 100%, then 60%, then 30%, then back to 100%. Show the current level as a small percentage next to the mute button.
3. **Remember the mute** (*ch05_music_across_scenes*) - Make the mute setting survive refreshing the page. Browsers keep small strings for a site in `localStorage`: `localStorage.setItem("muted", "true")` saves one, and `localStorage.getItem("muted")` gives it back (or `null` if there is nothing saved). Chapter 6 says more about it. The game should start muted if it was muted when the page was closed.
4. **Duck the music** (*ch05_music_across_scenes*) - In the game, every tenth star plays the power-up sound. Make the music "duck" under it: drop to a fifth of its volume while the power-up plays, and come back up when it has finished. *Hint*: you need to know when the power-up ends, so `this.sound.play()` will not do. And the music is in the sound manager: `this.sound.get(...)` will find it.
5. **On the beat** (*ch05_music_across_scenes*) - `music_game.wav` is 128 beats per minute. Add a circle to the game that pulses - grows, then shrinks back - on every beat. It must stay in time after the music has been paused and resumed (leave the game and come back). *Hint*: do not count beats with a timer - it drifts, and it does not know about pauses. Work out, every frame, which beat the music is on from its `seek`, and pulse when the beat number changes.

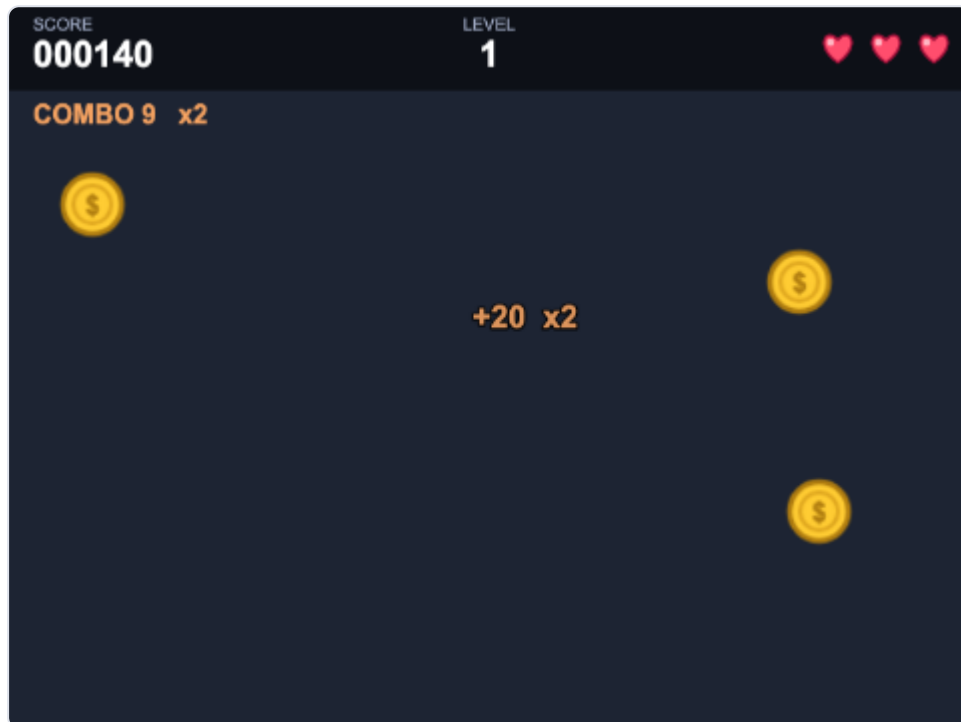
6. **Jukebox** (*ch05_sound_board*) - Turn the music strip into a playlist of three tracks - `music_menu`, `music_game` and `music_action` - that plays them in order, one after another, and goes back to the first after the last. K skips to the next track. Show the name of the track that is playing. *Hint*: a track that does not loop fires `"complete"` when it ends. Think about what should happen to the playing track when K is pressed, and whether its `"complete"` listener should still fire.

Previous: [Chapter 4 - The life of a scene](#) · Next: [Chapter 6 - Scoring](#)



Scoring

Almost every game keeps score, and players look at the score more than at anything else on the screen. In this chapter the score gets a proper home: a **HUD** (heads-up display) in a scene of its own, running on top of the game; a `ScoreManager` class that owns the numbers and **announces** every change as an event; and a high score table that is still there tomorrow, because it is saved in the browser.



What you will learn

- how to run a HUD as a second scene, launched on top of the game, and why that is better than drawing it in the game scene
- three ways to tell the HUD that something changed - an event on the game scene, an event on the game, and the registry's `changedata-` events - and how to choose between them
- how to put the rules of scoring in a plain class of their own that sends events, and why the scenes should only *tell* it things and *listen* to it
- how to make scoring feel good: a "+10" that floats away, and a score that counts up
- how to keep high scores for good with `localStorage`, `JSON.stringify` and `JSON.parse`, and how to check what comes back before trusting it

- how to read typed letters from the keyboard, for entering initials

The projects

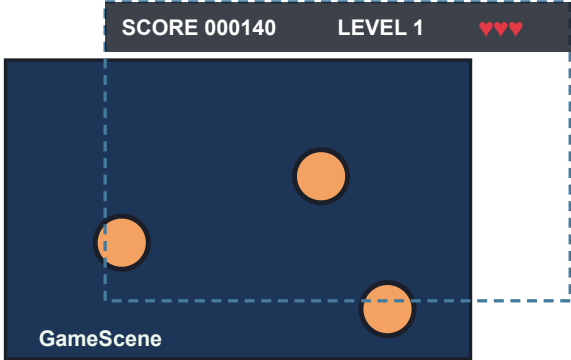
Project	What it shows
<code>ch06_hud_events</code>	a HUD scene fed three different ways - restart either scene and see what each way does
<code>ch06_coin_collector</code>	a complete little game: click coins before they vanish; a HUD scene listening to a <code>ScoreManager</code> ; combos, a multiplier, levels and lives
<code>ch06_high_score_table</code>	the same game with a top-ten table saved in the browser, and initials typed on the keyboard

A HUD is a scene of its own

In Chapter 2 the score was a piece of text in the play scene. That works, but it mixes two jobs in one class - *playing the game* and *showing information about it* - and it causes small problems as a game grows: shake the camera when the player is hit, and the score shakes too; make the camera follow the player, and the score scrolls off the screen. So the HUD gets a scene of its own. **Two scenes run at the same time**, and Phaser draws them one on top of the other:

Two scenes running at once, one drawn on top of the other

HudScene (see-through, except its bar)



GameScene

main.ts

```
scene: [TitleScene, GameScene,
        HudScene, GameOverScene]
```

Later in the list = drawn later = on top.

GameScene

```
this.scene.launch(HUD_SCENE, data)
starts the HUD beside it, on the next frame

this.scene.stop(HUD_SCENE)
takes it away again when the game ends
```

The game scene starts the HUD with `launch` :

ch06_coin_collector/src/scenes/GameScene.ts

```
const hudData: HudData = { scoreManager: this.scoreManager };  
this.scene.launch(HUD_SCENE, hudData);
```

`this.scene.start(key)` (Chapter 2) shuts *this* scene down and starts another;
`this.scene.launch(key, data)` starts another scene **beside** this one. Chapter 4 covers the scene manager in full; three things matter for a HUD:

- **Drawing order is the order of the scene list in the config.** `HudScene` comes after `GameScene`, so it is drawn later - on top. Its background is see-through except for what it draws, so the game shows through
- **Each scene has its own camera.** `GameScene` shakes its camera when a coin gets away (`this.cameras.main.shake(150, 0.005)`) - and the HUD, with its own camera, stays perfectly still
- **A launched scene starts on the next frame**, not straight away. Keep that in mind: it is the reason for one of the traps below

When the game ends, the game scene takes the HUD away again before moving on:

```
this.scene.stop(HUD_SCENE);  
this.scene.start(GAME_OVER_SCENE, result);
```

Telling the HUD: three routes

The HUD could look at the game scene's fields sixty times a second in `update()`, but that ties the two scenes tightly together, and wastes work: the score changes perhaps once a second. Much better: when something changes, the game **announces** it, and the HUD, which is **listening**, updates itself.

You have used this already. `ball.on("pointerdown", ...)` listens for an event that the ball sends. The ball is an **event emitter**: an object you can `emit` events on, and listen to with `on`, `once` and `off`. Phaser has emitters everywhere, and three of them are candidates for talking to a HUD. The `ch06_hud_events` project uses all three side by side - one for each thing you can click.



The event names - `COINS_CHANGED` and `GEMS_CHANGED` - are constants in `src/events.ts`, for the same reason as scene keys: a misspelt event name is a listener that never hears anything, with no error to tell you.

Route 1: the game scene's own events

Every scene has an emitter, `this.events`. Phaser uses it for the scene's own events (update, shutdown and so on), and you can send events of your own on it:

`ch06_hud_events/src/scenes/GameScene.ts`

```
private addCoin(): void {
  this.coins = this.coins + 1;
  this.events.emit(COINS_CHANGED, this.coins);
}
```

`emit(name, ...values)` calls every listener for that name, straight away, passing it the values. To listen, the HUD needs the game scene itself, which it gets from the scene manager by key:

`ch06_hud_events/src/scenes/HudScene.ts`

```
this.gameScene = this.scene.get<GameScene>(GAME_SCENE);
this.gameScene.events.on(COINS_CHANGED, this.showCoins, this);
```


`this.scene.get(key)` returns the scene object; `<GameScene>` tells TypeScript which class it is (Phaser only knows it is "a `Scene`"). The third argument to `on` is the **context** - the object that `this` will mean inside `showCoins` when the emitter calls it. More on that in a moment.

Route 2: the game's events

The game has an emitter too, `this.game.events`, and every scene can reach it:

```
// GameScene.addGem()
this.game.events.emit(GEMS_CHANGED, this.gems);

// HudScene.create()
this.game.events.on(GEMS_CHANGED, this.showGems, this);
```

No reference to the game scene is needed: anyone can send, anyone can listen. That is convenient, and also the danger - it is one noticeboard for the whole game, and nothing on it says who sent what.

Route 3: the registry's change events

Chapter 2 kept the best time in the **registry**, the game-wide store of named values. The registry has an emitter as well, `this.registry.events`, and it announces changes **by itself**: whenever a key it already holds is set, it sends `changedata-` followed by the key's name.

```
// GameScene.addStar()
this.registry.inc(STARS, 1);

// HudScene.create()
this.registry.events.on(`changedata-${STARS}`, this.onStarsChanged, this);
```

`inc(key, amount)` adds to a number in the registry (`set` works too). The game scene sends no event itself - it just changes the value. The listener is given the game, the new value and the old value.

Note - `changedata-stars` is only sent when a key that **already exists** changes. The very first `set` of a new key sends a different event, `setdata`. That is why `GameScene.init()` sets the stars to 0 before anything else happens: from then on, every change is a `changedata`.

News and state

Start the project and look at the gems: the HUD says `Gems: ?`. Press H to restart the HUD, and it says `?` again, even though you have collected some.



An event is **news**: it tells whoever is listening *at that moment* that something changed. A listener that starts later has missed it. And a HUD always starts late - it was launched, so it starts on the frame after the game scene's `create()`. So as well as listening, the HUD has to find out how things stand **now**:

`ch06_hud_events/src/scenes/HudScene.ts`

```
this.showCoins(this.gameScene.getCoins()); // route 1: ask the game scene
this.gemsText.setText("Gems: ?");          // route 2: no way to ask -
wait for the next event
this.showStars(this.registry.get(STARS));   // route 3: the registry holds
the value itself
```

With route 1 the HUD has the game scene, so it can ask it. With route 2 it has nobody to ask. The registry, route 3, is the only one of the three that is **state and news together**: it holds the value, and announces changes to it.

Cleaning up

This is the part people forget. A scene's own game objects, timers and input listeners are cleared away when it shuts down - but the HUD's listeners are not on the HUD. They are on three emitters that **outlive the HUD**, and Phaser has no idea they belong to it. The HUD must remove them itself, when it shuts down:

ch06_hud_events/src/scenes/HudScene.ts

```
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  this.gameScene.events.off(COINS_CHANGED, this.showCoins, this);
  this.game.events.off(GEMS_CHANGED, this.showGems, this);
  this.registry.events.off(`changedata-${STARS}`, this.onStarsChanged,
this);
});
```

`off` takes exactly what `on` was given - the event, the function *and* the context - and removes that listener. (Chapter 4 covers `SHUTDOWN` and the other scene events.)

The HUD shows each event's `listenerCount` under its number. Press H as often as you like: it stays at 1. Take the clean-up away and every restart adds one more - and **nothing looks wrong**. Phaser reuses the scene object (Chapter 2), so the old listeners call the same `HudScene`, which updates its new text perfectly well. Every change is simply handled two, three, ten times - which you *would* notice the day the HUD plays a sound on each change.

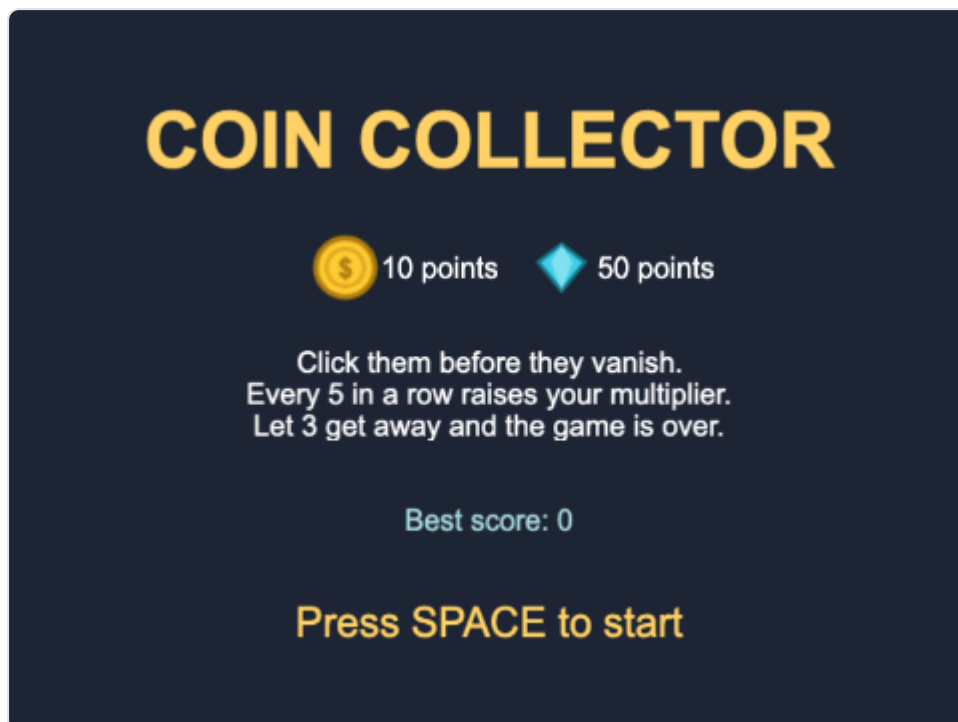
Note - why pass `this` as the third argument? A method passed as a value, like `this.showCoins`, forgets which object it belongs to: when the emitter calls it, `this` inside it is **the emitter**, and `this.coinsText` is `undefined`. Passing the context fixes that. (So does an arrow function - but then you cannot name that same function again for `off`.) In Java, `this::showCoins` stays bound to its object; in JavaScript a method does not.

Which one?

	Scene events	Game events	Registry
Sent with	<code>this.events.emit(...)</code>	<code>this.game.events.emit(...)</code>	<code>this.registry.set(/ inc(...)</code>
The listener needs	the sending scene (<code>this.scene.get</code>)	nothing	nothing
A late listener can find the current value?	only by asking the scene	no	yes - <code>this.registry.get(</code>
Good for	events that belong to one scene	game-wide happenings: "paused", "settings changed"	a few values shown in several places

All three work, and you will see all three in other people's games. For a score, though, there is a fourth answer, and it is the one the rest of this chapter uses: an emitter that belongs to **the score itself**.

A ScoreManager



`ch06_coin_collector` is a real game: click coins and gems before they vanish. Five in a row raises a multiplier, every ten pickups is a faster level, and three that get away end the game. The score, level, combo and lives belong together, and so do the rules that change them - in OO terms, one object:

```
ch06_coin_collector/src/ScoreManager.ts
```

```

// The events a ScoreManager sends, and what each one passes to its
// listeners.
export const SCORE_CHANGED = "score-changed"; // (score: number, added:
// number)
export const LIVES_CHANGED = "lives-changed"; // (lives: number)
export const COMBO_CHANGED = "combo-changed"; // (combo: number,
// multiplier: number)
export const LEVEL_CHANGED = "level-changed"; // (level: number)
export const GAME_OVER = "game-over"; // ()

export class ScoreManager extends Phaser.Events.EventEmitter {
  private score = 0;
  private lives = START_LIVES;
  private level = 1;
  private combo = 0;
  private bestCombo = 0;
  private pickups = 0;

```

It is a **plain class** - not a scene and not a game object. It knows nothing about text, pictures, sounds or scenes. It extends `Phaser.Events.EventEmitter`, the class behind `this.events`, `this.game.events` and every game object's `on(...)`, so it gets `on`, `once`, `off` and `emit` for free.

Its public methods are the things that can **happen** in the game, and each one applies the rules and announces the results:

```

public collect(basePoints: number): number {
  if (this.isGameOver()) {
    return 0;
  }

  this.combo = this.combo + 1;
  this.bestCombo = Math.max(this.bestCombo, this.combo);
  const points = basePoints * this.getMultiplier();
  this.score = this.score + points;
  this.pickups = this.pickups + 1;

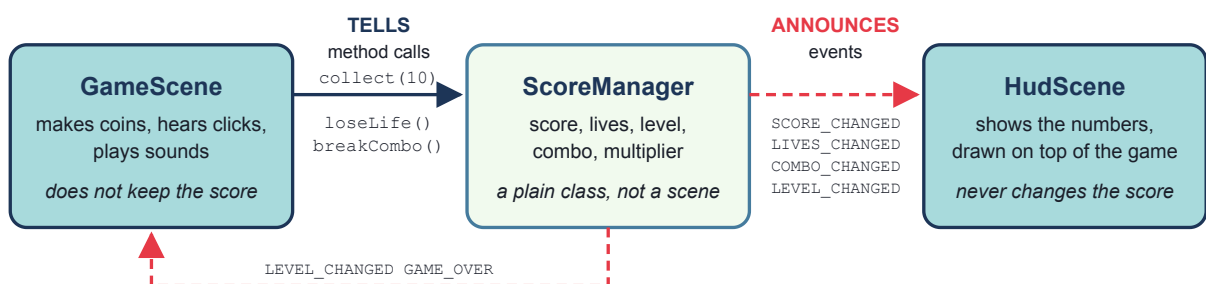
  this.emit(SCORE_CHANGED, this.score, points);
  this.emit(COMBO_CHANGED, this.combo, this.getMultiplier());

  if (this.pickups % PICKUPS_PER_LEVEL === 0) {
    this.level = this.level + 1;
    this.emit(LEVEL_CHANGED, this.level);
  }
  return points;
}

```

`getMultiplier()` works the multiplier out from the combo: 1 for the first four in a row, 2 from the fifth, 3 from the tenth, up to 5. `loseLife()` and `breakCombo()` work like `collect()`, and `loseLife()` sends `GAME_OVER` when the last life goes. The fields are private and there are no setters: the only way to change the score is to report something that happened.

Who talks to whom in Coin Collector



Solid arrow: "do this" - the caller knows exactly who it is talking to.

Dashed arrow: "this happened" - the ScoreManager does not know who (if anyone) is listening.

This is worth pausing on, because it is the design idea of the chapter. There are two directions, and they are different kinds of message:

- **the game scene tells the ScoreManager** what happened - `collect(10)` , `loseLife()` . These are ordinary method calls: the scene knows exactly what it is talking to
- **the ScoreManager announces** the results as events. It does not know who is listening - the HUD, the game scene, both, or nobody - and it does not need to

Because of that, each class has one job. Want the multiplier to go up every three in a row instead of five? One constant in `ScoreManager.ts` ; nothing else changes. Want a second display of the score? It just listens. Want to test the scoring rules? They are in one class that needs no scene.

Note - in Java this is the **observer** pattern, and you have met it as Swing's `addActionListener` . TypeScript does not check events, though: `emit(SCORE_CHANGED, this.score, points)` accepts any values at all, and a listener declared as `(score: number)` is a promise you make, not one the compiler checks. That is why the comment beside each event name says what it sends.

A new ScoreManager for every game

Chapter 2's trap was a scene reused with old values in its fields. Here the whole score is one object, so `GameScene.init()` starts again in one line, `this.scoreManager = new ScoreManager();` , and the old one - with any listeners still attached to it - is thrown away.

The game scene reports; the HUD listens

When a coin is clicked, the game scene reports it, and shows the points it was worth:

`ch06_coin_collector/src/scenes/GameScene.ts`

```
// the ScoreManager applies the multiplier, and says how many points that
// was worth
const points = this.scoreManager.collect(coin.points);
const multiplier = this.scoreManager.getMultiplier();
const message = multiplier > 1 ? `+${points} x${multiplier}` :
`+${points}`;
const colour = multiplier > 1 ? "#f4a261" : "#ffd166";
new FloatingText(this, coin.x, coin.y - 20, message, colour);
```

A coin that gets away sends an event of its own, `COIN_EXPIRED` - a `Coin` is a game object, so it is an emitter too - and the game scene reports that as `loseLife()` . The game scene also listens to the `ScoreManager`, for the two things that are its business:


```
this.scoreManager.on(LEVEL_CHANGED, () => this.sound.play(LEVEL_SOUND_KEY));  
this.scoreManager.once(GAME_OVER, () => this.endGame());
```

The HUD is handed the ScoreManager when it is launched, as its scene data (an interface, `HudData`, as in Chapter 2), and in `create()` it does the three things every HUD must: show how things stand now (`this.showScore(this.scoreManager.getScore())` and so on), listen for changes (`this.scoreManager.on(SCORE_CHANGED, this.countUpTo,` `this)` and so on), and stop listening on `SHUTDOWN`.

Making the score feel good

A number that changes is information. A number that *reacts* is a reward. Two small touches make a big difference.

Floating points

Where the coin was, a "+10" drifts upwards and fades away. It is a class of its own, which does everything itself:

```
ch06_coin_collector/src/objects/FloatingText.ts
```

```

export class FloatingText extends Phaser.GameObjects.Text {
  constructor(scene: Phaser.Scene, x: number, y: number, message: string,
    colour: string) {
    super(scene, x, y, message, {
      ...
    });
    this.setOrigin(0.5);
    scene.add.existing(this);

    // two properties in one tween: it rises quickly at first then slows
    (Cubic.easeOut), and fades
    // slowly at first then quickly (Cubic.easeIn) - so it can be read
    before it goes
    scene.tweens.add({
      targets: this,
      y: y - RISE,
      alpha: { value: 0, ease: "Cubic.easeIn" },
      duration: DURATION,
      ease: "Cubic.easeOut",
      onComplete: () => this.destroy(),
    });
  }
}

```

A **tween** changes properties of an object smoothly over time - here `y` (upwards by `RISE` pixels) and `alpha` (to transparent) over `DURATION` milliseconds; `ease` says *how*. When the tween finishes, `onComplete` destroys the text. So the game scene makes one with `new FloatingText(...)` and never thinks about it again. Chapter 7 covers tweens properly.

A score that counts up

In the HUD the score does not jump from 120 to 140 - it counts up, 121, 123, 127 ... 140, in less than half a second. A **counter tween** does it. It tweens a number rather than a property of an object, and on every step the HUD shows the number it has reached:

```
ch06_coin_collector/src/scenes/HudScene.ts
```

```

private countUpTo(score: number): void {
    this.countUp?.stop(); // a new score while still counting? start again
    from where it got to
    this.countUp = this.tweens.addCounter({
        from: this.shownScore,
        to: score,
        duration: COUNT_UP_TIME,
        ease: "Cubic.easeOut",
        onUpdate: (tween) => {
            // getValue() is number | null: null only if the tween has no value
            yet - so 0
            this.showScore(Math.round(tween.getValue() ?? 0));
        },
    });
}

private showScore(score: number): void {
    this.shownScore = score;
    this.scoreText.setText(String(score).padStart(SCORE_DIGITS, "0"));
}

```

- the tween always runs from `shownScore`, the score on the screen, so collecting three coins quickly never makes the number jump backwards
- `this.countUp?.stop()` - `?.` is **optional chaining**: "if `countUp` is not `undefined`, call `stop()` on it". Before the first score there is no tween yet
- `tween.getValue()` returns `number | null`, and the compiler will not let a possible `null` into `Math.round`. `??` is the **nullish coalescing** operator: `a ?? b` is `a`, unless `a` is `null` or `undefined`, in which case it is `b`
- `padStart(6, "0")` pads a string on the left: `"140"` becomes `"000140"` - the arcade look

The combo is simpler: the HUD shows `COMBO 9 x2` whenever it hears `COMBO_CHANGED`, and hides it when the combo is broken. Clicking empty space breaks it (the `over` list from Chapter 2); letting a coin escape breaks it and costs a life.

High scores that last

The registry forgets everything when the page is closed. For a high score table you need the browser to keep something for you - and it will, in **local storage**. `localStorage` is part of the browser, not Phaser, and has three methods that matter: `setItem(key, text)` keeps some text under a key, `getItem(key)` gives it back (or `null` if there is nothing), and `removeItem(key)` forgets it. What you keep stays - through refreshes, closing the tab, and restarting the computer - until the player clears their browser's data. Three things to know:

- **it keeps strings only.** Anything else has to be turned into a string first
- **it belongs to the web site** - the address the page came from. Every project in this book is served from <http://127.0.0.1:8000/>, so as far as the browser is concerned they are all *one* web site, sharing *one* storage. Give your keys a name that says which game they belong to
- **each browser keeps its own.** Celbridge's `game.webview` and Chrome on the same machine do not share scores

Objects to text, and back: JSON

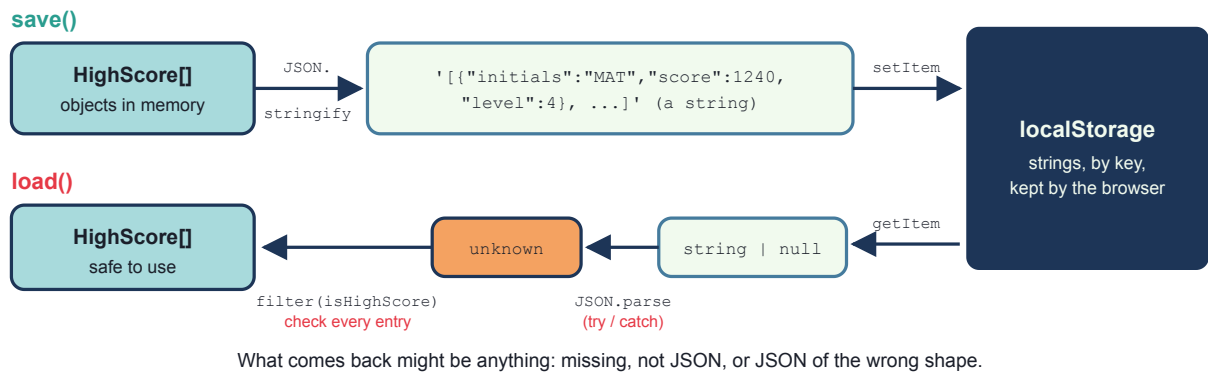
ch06_high_score_table/src/HighScoreTable.ts

```
export interface HighScore {  
  initials: string;  
  score: number;  
  level: number;  
}
```

The table is an array of these. `JSON.stringify(value)` turns objects and arrays into a string of **JSON** (JavaScript Object Notation) - the table above becomes

`[{"initials":"MAT","score":1240,"level":4}, ...]` - and `JSON.parse(text)` turns the string back into objects.

Saving and loading the high score table



Saving is short. `try / catch` is there because `setItem` throws an error if storage is full or turned off - and a score that cannot be saved is no reason to stop the game:

```
public save(): void {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(this.entries));
  } catch {
    // storage full or turned off: the scores just are not kept - not worth
    // stopping the game for
  }
}
```

`STORAGE_KEY` is `"phaser4-guide.coin-collector.high-scores"` - a name no other game will use.

Never trust what comes back

Loading is where the care goes. The text in storage could be **anything**: missing (the first time the game runs), not JSON at all, JSON of some other shape saved by an older version of the game or another game using the same key, or edited by a curious player - it is all visible in the browser's developer tools. If the game just trusted it, one bad save would stop the game from ever starting. So `load()` checks every step, and if anything is wrong it starts with an empty table:

`ch06_high_score_table/src/HighScoreTable.ts`

```

public static load(): HighScoreTable {
    let text: string | null;
    try {
        text = localStorage.getItem(STORAGE_KEY);    // null if nothing has been
        saved yet
    } catch {
        return new HighScoreTable([]);                // storage turned off in
        this browser
    }
    if (text === null) {
        return new HighScoreTable([]);
    }

    let data: unknown;
    try {
        data = JSON.parse(text);
    } catch {
        return new HighScoreTable([]);                // not JSON at all
    }
    if (!Array.isArray(data)) {
        return new HighScoreTable([]);                // JSON, but not a list
    }

    // keep only the entries that really are high scores, best first, ten at
    most
    const entries = data.filter(HighScoreTable.isHighScore);
    entries.sort((a, b) => b.score - a.score);
    return new HighScoreTable(entries.slice(0, TABLE_SIZE));
}

```

- `getItem` returns `string | null`, and the compiler will not let that into `JSON.parse`, which wants a `string` - you must deal with `null` first
- `JSON.parse` **throws** a `SyntaxError` on text that is not JSON - so it is inside `try`
- `data` is `unknown`: "could be anything". Unlike `any`, TypeScript lets you do nothing with an `unknown` until you have checked what it is - `Array.isArray(data)` is the first check
- the constructor is `private`, so the only way to get a table is `HighScoreTable.load()`, which always checks: a **static factory method**, as in Java

Then each entry is checked by a **type guard** - a function whose return type, `value is HighScore`, tells the compiler that a `true` means "this is a `HighScore`":

```
private static isHighScore(value: unknown): value is HighScore {
  if (typeof value !== "object" || value === null) {
    return false;
  }
  // `as` - it is an object, but TypeScript cannot know what properties it
  // has: look at them
  const entry = value as Record<string, unknown>;
  return typeof entry.initials === "string" && /^[A-Z]
{1,3}$/.test(entry.initials) &&
  typeof entry.score === "number" && Number.isInteger(entry.score) &&
  entry.score >= 0 &&
  typeof entry.level === "number" && Number.isInteger(entry.level) &&
  entry.level >= 1;
}
```

`typeof x` gives the type of a value as a string: `"string"`, `"number"`, `"object"` and so on. (`typeof null` is `"object"` - a famous JavaScript oddity - hence the extra check.) `/^[A-Z]{1,3}$/` is a regular expression for "one to three capital letters" - more on those in the next section. Because `isHighScore` is a type guard, `data.filter(HighScoreTable.isHighScore)` has the type `HighScore[]`: the compiler knows every entry that survived the filter is safe.

Try it: in the browser's developer tools, change the saved value to `this is not json`, or to a list with a few broken entries, and reload. The game starts anyway, keeping what it can.

Getting into the table

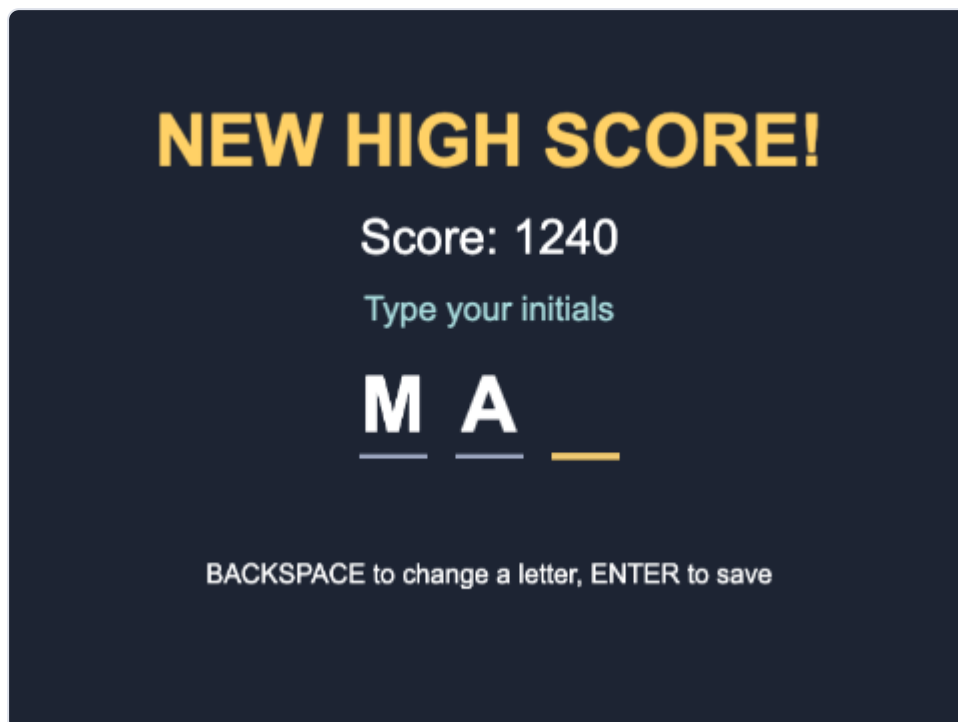
`qualifies(score)` says whether a score makes the table: it does if the table is not full yet, or if it beats the lowest score in it. `add(entry)` finds the first entry the new score beats, inserts it there with `splice`, cuts the list back to ten, and returns the place it went. A score that *ties* goes below the one already there - whoever got it first keeps their place.

When the game ends, the game scene uses `qualifies` to decide where to go next:

`ch06_high_score_table/src/scenes/GameScene.ts`

```
const madeTheTable = HighScoreTable.load().qualifies(result.score);
this.scene.stop(HUD_SCENE);
this.scene.start(madeTheTable ? NAME_ENTRY_SCENE : GAME_OVER_SCENE, result);
```

Typing your initials



So far every key has had its own listener - `keydown-SPACE` , `keydown-ESC` . For typing, that would mean 26 listeners, one per letter. Instead, `NameEntryScene` listens for **every** key press:

`ch06_high_score_table/src/scenes/NameEntryScene.ts`

```
this.input.keyboard!.on("keydown", (event: KeyboardEvent) => {  
  this.handleKey(event);  
});
```

Phaser passes the browser's own `KeyboardEvent` , and its `key` property says what the key **types**: `"a"` , `"A"` , `"7"` , `"Enter"` , `"Backspace"` .


```

const LETTER = /^[a-z]$/i;  // one letter, a to z, upper or lower case (the
i)
...
private handleKey(event: KeyboardEvent): void {
  if (LETTER.test(event.key) && this.initials.length < MAX_INITIALS) {
    this.initials = this.initials + event.key.toUpperCase();
    this.sound.play(CLICK_SOUND_KEY);
  } else if (event.key === "Backspace") {
    this.initials = this.initials.slice(0, -1);  // everything but the last
letter
  } else if (event.key === "Enter" && this.initials.length > 0) {
    this.save();
    return;
  }
  this.showInitials();
}

```

- `/^[a-z]$/i` is a **regular expression**: `^` start, `[a-z]` one letter, `$` end, and `i` for "ignore case". `test` says whether a string matches. It rejects `"Enter"` and `"Shift"` - they are more than one character - as well as digits and punctuation
- `event.key` follows the player's keyboard layout. There is also `event.code`, which names the key's *position* (`"KeyA"`, whatever is printed on it) - right for game controls like WASD, wrong for typing
- `slice(0, -1)` is "from the start up to, but not including, the last character"

Enter loads the table, adds the score, saves it, and starts the title screen with the new row's place as its data - `{ highlight: place }` - so that row can flash.



`TitleScene` shows the table whether or not there is a row to highlight - the very first time the game starts, there is none. Its data says so with an **optional property**:

`ch06_high_score_table/src/scenes/TitleScene.ts`

```
export interface TitleData {
  highlight?: number; // "?" - optional: the first time the game starts,
  there is no new row
}
...
init(data: TitleData): void {
  this.highlight = data.highlight ?? -1;
}
```

`highlight?: number` means "a number, or not there at all", and `?? -1` turns "not there" into "no row". The table is drawn in a **monospaced** font (Courier New), where every character is the same width, so `padStart` and `padEnd` line the columns up.

Common mistakes

- not removing listeners from another object's emitter on `SHUTDOWN` . No error; they pile up
- `on(SCORE_CHANGED, this.countUpTo)` without the context: when the event comes you get `TypeError: Cannot read properties of undefined (reading 'addCounter')`
- launching a HUD that is already running: `launch` on a running scene **restarts** it. Check `this.scene.isActive(HUD_SCENE)` first, as `ch06_hud_events` does
- `this.scene.start(TITLE_SCENE)` with no data, to a scene that takes optional data: a scene started without data is handed **the data it was last started with** - here, an old `{ highlight: 4 }`. `GameOverScene` passes an empty `TitleData`, `{}`, for that reason
- reading storage with no checks - fine until the day the saved text is wrong, and then the game never starts again

Summary

- a HUD is a scene of its own, started with `this.scene.launch(key, data)`, drawn on top because it is later in the config's `scene` list, with its own camera
- emitters you can use: `this.events`, `this.game.events`, `this.registry.events` (`changedata-<key>`), and your own classes that extend `Phaser.Events.EventEmitter`
- events are news, not state: a scene that starts late must read the current values too; and it must remove listeners from longer-lived emitters on `SHUTDOWN`
- put the rules of scoring in one plain class; scenes **tell** it what happened and **listen** to what it announces
- a tween floats a "+10" away; a counter tween (`this.tweens.addCounter`) counts a score up
- `localStorage` keeps strings for good; `JSON.stringify` and `JSON.parse` turn objects into strings and back. Check everything you read back
- `keyboard.on("keydown", ...)` hears every key; `event.key` is what it types

Challenges

1. **A star is born** (*ch06_coin_collector*) - Add a third kind of pickup: a star (`assets/images/star.png` from the asset library - copy it into `public/assets/images/`), worth 100 points, which appears only one time in twenty and vanishes faster than a gem. Show it on the title screen with the other two.
2. **Bonus life** (*ch06_coin_collector*) - Each time the player reaches a new level, give back one lost life (never more than three). The HUD's hearts should fill in again without you changing `HudScene.ts` at all. Why does that work?
3. **Combo meter** (*ch06_coin_collector*) - Under the combo text, show a bar that fills up as the combo grows towards the next multiplier - a fifth full after one pickup, full at five, then empty again as the multiplier goes up - and empties when the combo is broken. Change only the HUD.
4. **Wipe the slate** (*ch06_high_score_table*) - On the title screen, pressing R asks "Clear all high scores? Y / N". Y empties the table - in storage too - and shows the empty table; N (or any other key) puts the screen back as it was. *Hint*: give `HighScoreTable` a method for this, and look up `localStorage.removeItem` . `this.scene.restart()` redraws a scene from scratch.
5. **Chasing the record** (*ch06_high_score_table*) - Show the top score in the HUD - `HI 002450` - and when the player's score passes it during a game, make `HI` follow the score and flash a "NEW HIGH SCORE!" message, once. *Hint*: the HUD can load the table itself when it starts. It already hears every `SCORE_CHANGED` ; a boolean field can remember whether it has celebrated yet.
6. **Easy, normal, hard** (*ch06_high_score_table*) - Let the player choose a difficulty on the title screen with 1, 2 and 3. Harder games have coins that vanish sooner and come faster, and each difficulty has **its own** high score table, saved separately; the title screen shows the table for the difficulty chosen. *Hint*: the difficulty has to travel from the title screen to the game, to the name entry scene, and back - scene data can carry it. What should `HighScoreTable.load()` take as a parameter, and what should it do with it?

Previous: [Chapter 5 - Audio](#) · Next: [Chapter 7 - Animations](#)



Animations

A game where things only ever slide about looks dead. In this chapter you bring it to life in three ways: **frame animations**, where a sprite flicks through the pictures of a sprite sheet like a flip book; **tweens**, where Phaser changes a value - a position, a size, a transparency - smoothly over time; and **particles**, hundreds of tiny images thrown out at once for sparks and explosions. Along the way you build a hero who runs, jumps and falls, driven by a small state machine.



What you will learn

- how a sprite sheet is turned into named animations with `this.animations.create(...)` and `generateFrameNumbers(...)`, and why they are made once, for the whole game
- how to play, stop, pause, speed up, chain and queue animations on a sprite, and how to listen for animation events such as `ANIMATION_COMPLETE`
- how `setFlipX` lets one set of frames face both ways
- how a **state machine** keeps a character's animations under control (idle, run, jump, fall, hurt)
- how to tween any value with `this.tweens.add(...)`: duration, ease, yoyo, hold, repeat and delay

- what easing is, and how to chain, stagger and count with tweens
- how to make a burst of particles with `this.add.particles(...)` and `explode(...)`

The projects

Project	What it shows
<code>ch07_animation_lab</code>	five sprite sheets animating, and a big hero whose animation you control from the keyboard, with its state and events shown as they happen
<code>ch07_hero_animations</code>	the hero on a strip of ground: run, flip, jump, fall and get hurt, driven by a state machine
<code>ch07_tweens_and_particles</code>	a playground of buttons that set off different tweens, explosions made from an animation and particles, and a gallery of easing curves

Two kinds of animation

Phaser has two quite different tools that both get called "animation":

- a **frame animation** changes *which picture* a sprite shows: frame 2, then 3, then 4... - a running character, a spinning coin. The code only decides which frames, in what order, how fast
- a **tween** changes *a number* smoothly over a time you choose: `x` from 100 to 500 in 800 milliseconds, `alpha` from 1 to 0. It works on any object with numbers in it

A character usually needs both: frames to move its legs, tweens (or physics) to move it across the screen.

Sprite sheets and frame animations

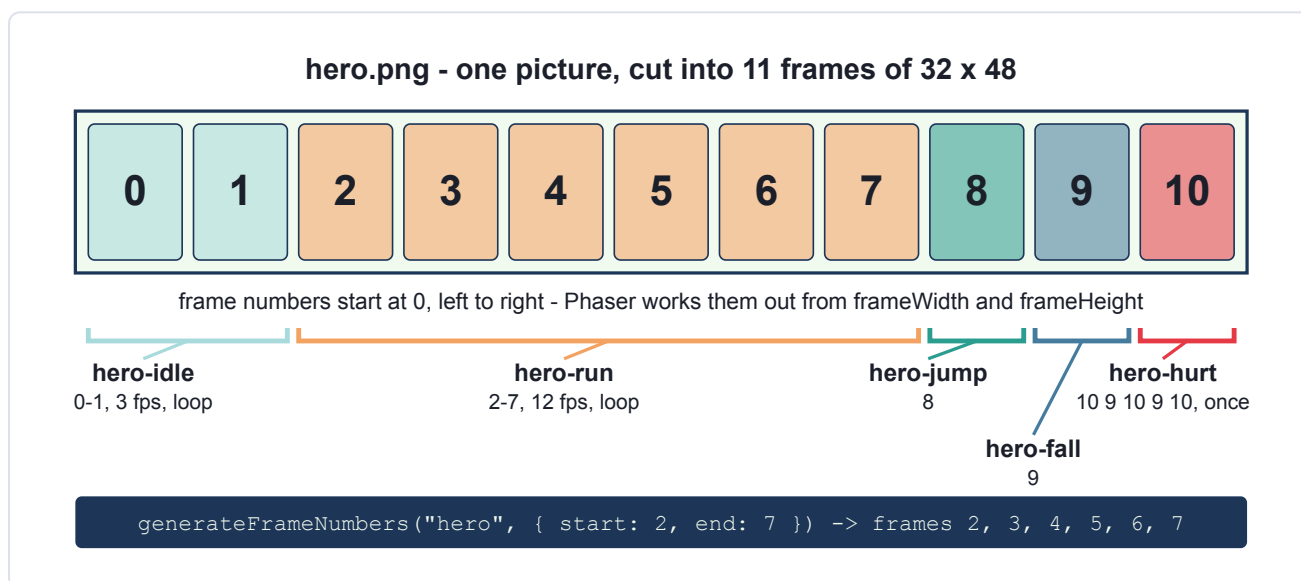
Sprite sheets

Chapter 3 loaded sprite sheets; here is a reminder. A **sprite sheet** is one picture made of equal-sized frames, side by side. Phaser cuts it up for you when you tell it the size of a frame:

```
ch07_animation_lab/src/scenes/PreloadScene.ts
```

```
this.load.spritesheet(HERO_SHEET, HERO_FILE, { frameWidth: HERO_WIDTH,
frameHeight: HERO_HEIGHT });
```

`hero.png` is 352 x 48 pixels, and each frame is 32 x 48, so it has eleven frames, numbered from 0:



Making an animation

An animation is a list of frames, a speed, and what to do at the end. You make one with the scene's animation manager, `this.anims`, and give it a key of its own:

```
ch07_animation_lab/src/scenes/PreloadScene.ts
```

```

this.anims.create({
  key: HERO_RUN,
  frames: this.anims.generateFrameNumbers(HERO_SHEET, { start: 2, end: 7 }),
  frameRate: 12,
  repeat: -1,
});

```

- `key` - the animation's name, `"hero-run"`. It is a different key from the sheet's (`"hero"`): one sheet usually holds several animations
- `frames` - `generateFrameNumbers(sheet, { start, end })` makes the list of frames 2, 3, 4, 5, 6, 7 of the sheet. To pick frames in any order, list them instead: `{ frames: [10, 9, 10, 9, 10] }`
- `frameRate` - frames per second. Six frames at 12 a second is one stride every half second
- `repeat` - how many times to play it again after the first time. `-1` means for ever

A few more settings you will meet:

Setting	What it does
<code>yoyo: true</code>	play forwards, then backwards: 0 1 2 3 2 1 0 - the slime and ghost in the lab use it
<code>repeatDelay</code>	milliseconds to wait before each repeat - the lab's explosion waits 800 between bangs
<code>delay</code>	milliseconds to wait before it starts
<code>duration</code>	the whole animation's length, in milliseconds, instead of a <code>frameRate</code>

A single-frame "animation" is fine too: `hero-jump` is just frame 8, so the code can always say "play the animation for this state".

Made once, for the whole game

Animations belong to the **game**, not to the scene that made them - like loaded pictures (Chapter 2). `this.anims` in any scene is the same, game-wide animation manager. So every project in this chapter makes its animations in `PreloadScene.create()`, once, and every later scene just plays them.

Make them in a scene that starts more than once - a `GameScene` that restarts for every level - and the second time round the browser's console says `AnimationManager` key already exists: `hero-idle`, and Phaser keeps the old one. Harmless, but a sign the code is in the wrong place. (A sprite can also have animations of its own, `sprite.anims.create(...)`, but global ones are what you want nearly always.)

Playing an animation

Only a **Sprite** can play an animation. An `Image` shows one frame and that is all:

`ch07_animation_lab/src/scenes/LabScene.ts`

```
// a Sprite, not an Image: only a Sprite can play an animation
const sprite = this.add.sprite(x, 72, sheet);
...
sprite.play(anim);
```

Make an image and try to `play` it, and the build stops you with `Property 'play' does not exist on type 'Image'`. An image *can* show any single frame, though - the strip of frames along the bottom of the lab is eleven images, each given a frame number:

```
this.add.image(x, 505, HERO_SHEET, frame)
```



In `ch07_animation_lab`, try the keys: the panel shows the hero's animation state, and the log shows the events it sends.

play, and play(key, true)

`this.hero.play(HERO_RUN)` starts the run animation **from its first frame** - even if it is already running. Press 2 again and again in the lab and watch the hero stutter: every press restarts the stride. Hold the key down (so it repeats) and the hero is stuck on frame 2.

`this.hero.play(HERO_RUN, true)` - key 3 - means "unless it is already playing". Pressed again, it does nothing, and the stride carries on. Remember this one: it is the first thing that goes wrong when you play animations from `update()`, which runs sixty times a second.

Finding out what is playing

A sprite's animation state is `sprite.ans`. The lab's `update()` shows it every frame:

- `ans.currentAnim` - the animation it is playing (or last played), or `null` if none yet; `ans.currentAnim.key` is its name
- `ans.currentFrame.textureFrame` - which frame of the sheet is on show
- `ans.isPlaying`, `ans.isPaused` - `true` or `false`

Controlling it

`ch07_animation_lab/src/scenes/LabScene.ts`

```
// chain: queue up what plays when this one completes (or is stopped)
keyboard.on("keydown-C", () => {
  this.hero.play(HERO_HURT);
  this.hero.chain(HERO_IDLE);
});

// playAfterRepeat: let the current loop finish, THEN change - no jump in
// the middle of a stride
keyboard.on("keydown-A", () => this.hero.ans.playAfterRepeat(HERO_IDLE));
```

- `chain(key)` queues an animation to play when this one ends. `hurt` plays once, then `idle` takes over with no more code. (`chain` can take an array to queue several)
- `playAfterRepeat(key)` waits until the current animation reaches the end of its loop, then switches. Start the hero running (2), then press A: the stride finishes before the hero stops

- `stop()` stops it where it is; `animations.pause()` and `animations.resume()` freeze it and carry on
- `animations.timeScale` speeds it up or slows it down: 2 is twice as fast, 0.5 half. It changes this sprite only; the animation itself - and every other sprite playing it - is unchanged

Animation events

A sprite **emits events** as its animation goes along, and you can listen for them with `on` and `once`, exactly as for `"pointerdown"` in Chapter 2:

`ch07_animation_lab/src/scenes/LabScene.ts`

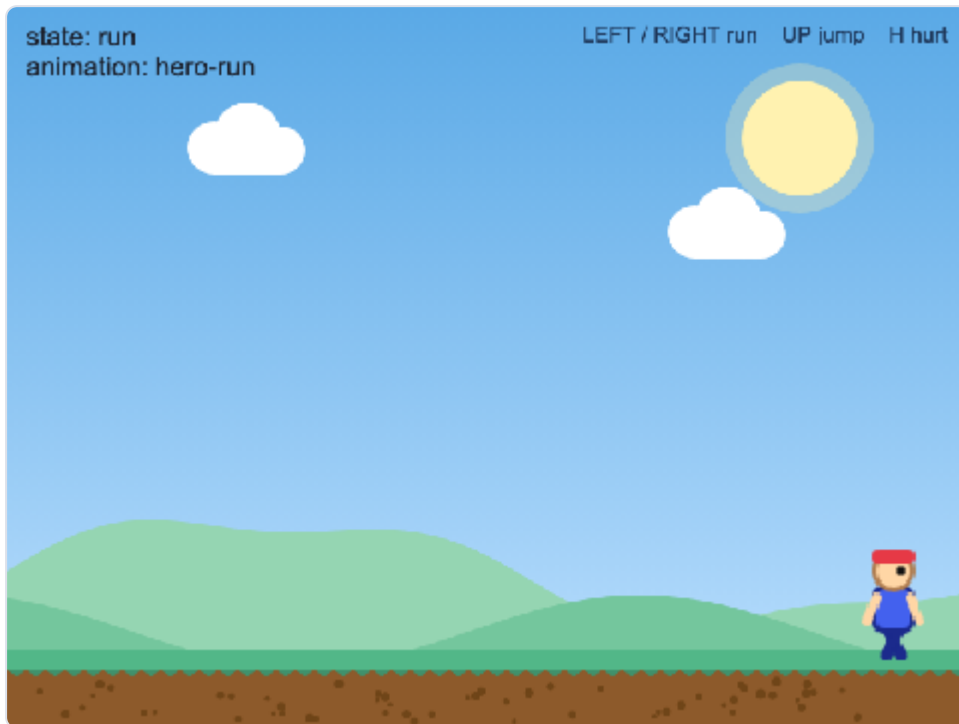
```
const events = [
  Phaser.Animations.Events.ANIMATION_START,
  Phaser.Animations.Events.ANIMATION_REPEAT,
  Phaser.Animations.Events.ANIMATION_COMPLETE,
  Phaser.Animations.Events.ANIMATION_STOP,
  Phaser.Animations.Events.ANIMATION_RESTART,
];
for (const eventName of events) {
  this.hero.on(eventName, (animation: Phaser.Animations.Animation) => {
    this.addToLog(`${eventName.padEnd(18)} ${animation.key}`);
  });
}
```

The constants are just strings (`"animationstart"` ...), but a constant cannot be misspelled. Each listener is given the animation, the frame, the sprite and the frame's key. The event you will use most is `ANIMATION_COMPLETE` : "that has finished - now do the next thing". For just one animation, add its key to the end:

```
this.hero.on(Phaser.Animations.Events.ANIMATION_COMPLETE_KEY + HERO_HURT, ()
=> {
  this.addToLog("  (hurt is over)");
});
```

Note - an animation with `repeat: -1` never completes, so it never sends `ANIMATION_COMPLETE`. Only animations that end - `repeat: 0` (the default) or a number - complete. Stopping one with `stop()` sends `ANIMATION_STOP` instead.

A hero with a state machine



`ch07_hero_animations` puts a hero on a strip of ground. Arrow keys run and jump; H hurts the hero. It has no physics engine (that is Chapters 8 and 9) - the hero moves itself, in `preUpdate()`, with a speed and a pretend gravity, just as the ball did in Chapter 1.

Facing left and right

The frames in `hero.png` all face right. There is no need for a second set facing left: a sprite can be mirrored.

`ch07_hero_animations/src/objects/Hero.ts`

```
// the frames face right; flipping the picture makes them face left
if (this.velocityX !== 0) {
  this.setFlipX(this.velocityX < 0);
}
```

`setFlipX(true)` draws the sprite mirrored left to right; `false` puts it back. The animation carries on playing, mirrored. Notice the `if`: when the hero stops, it keeps facing whichever way it was going.

Why a state machine?

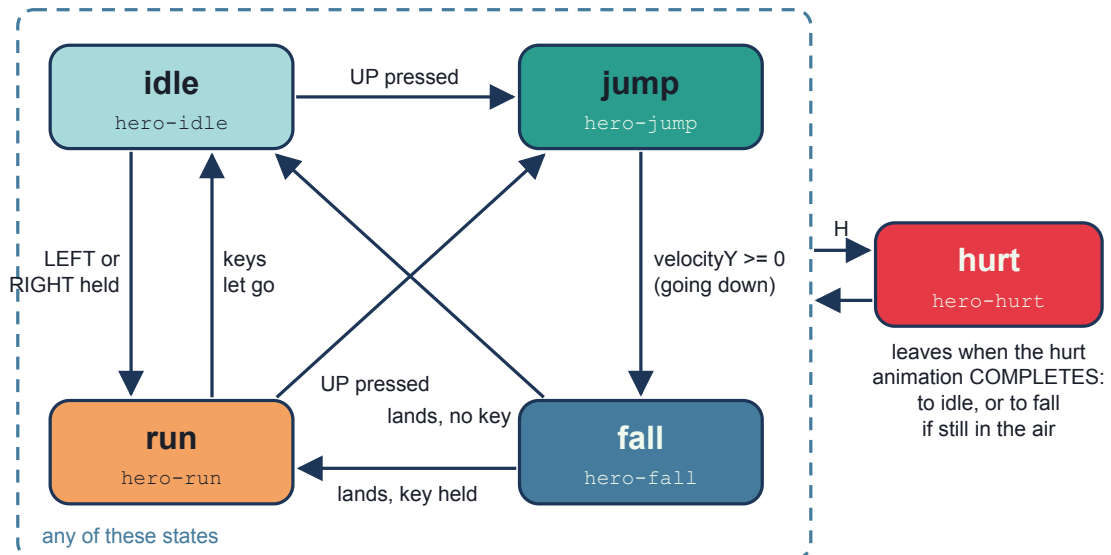
The obvious way to choose an animation is a pile of `if` s in the update:

```
// a sketch - NOT what the project does
if (onGround && movingSideways) {
  this.play(HERO_RUN, true);
} else if (onGround) {
  this.play(HERO_IDLE, true);
} else if (goingUp) {
  this.play(HERO_JUMP, true);
} ...
```

It works - for about a day. Then you add being hurt, which running must *not* interrupt, and ducking, and attacking, and every new animation needs checks in every other `if` , until nobody can say what the hero is doing.

A **state machine** turns that round. The hero is always in exactly **one state**; each state knows its animation and knows the rules for leaving it:

The hero's states - always exactly one; each plays its own animation



Changing state is the ONLY time an animation starts - so an animation never restarts every frame

The states

ch07_hero_animations/src/objects/Hero.ts

```
export type HeroState = "idle" | "run" | "jump" | "fall" | "hurt";

// which animation each state plays. Record<HeroState, string> is "an object
// with one string for
// EVERY HeroState" - leave a state out, and the build says so
const ANIMATIONS: Record<HeroState, string> = {
  idle: HERO_IDLE,
  run: HERO_RUN,
  jump: HERO_JUMP,
  fall: HERO_FALL,
  hurt: HERO_HURT,
};
```

`HeroState` is a **union of string literals**: one of those five strings and nothing else, so `this.changeState("runing")` is a build error. It does the job of a Java `enum`, with less ceremony.

`Record<HeroState, string>` is an object with a `string` for every state - a little like Java's `EnumMap`. Add a sixth state and forget its animation, and the build says `Property 'duck' is missing in type ... but required in type 'Record<HeroState, string>'`.

Changing state - the one place an animation starts

`ch07_hero_animations/src/objects/Hero.ts`

```
private changeState(next: HeroState): void {
  if (next === this.currentState) {
    return; // already in that state: do NOT restart its
    animation
  }
  this.currentState = next;
  this.play(ANIMATIONS[next]);
}
```

This is the whole trick. The hero never calls `play` anywhere else, and only calls it when the state really **changes**. So animations never restart by accident, and there is no need for `play(key, true)`.

The rules for leaving each state

ch07_hero_animations/src/objects/Hero.ts

```
private updateState(): void {
  switch (this.currentState) {
    case "idle":
    case "run":
      this.changeState(this.velocityX === 0 ? "idle" : "run");
      break;
    case "jump":
      if (this.velocityY >= 0) {
        this.changeState("fall");
      }
      break;
    case "fall":
      if (this.isOnGround()) {
        this.changeState(this.velocityX === 0 ? "idle" : "run");
      }
      break;
    case "hurt":
      // nothing: the ANIMATION_COMPLETE listener in hurt() ends this state
      break;
  }
}
```

A `switch` on strings works just as it does in Java. Read each `case` as "while in this state, here is when to leave it". Jumping is started from `readKeys()`, which only jumps from the ground:

```
// JustDown: true only on the first frame the key is down, so holding UP is one jump
if (Phaser.Input.Keyboard.JustDown(this.cursors.up) && this.isOnGround()) {
  this.velocityY = -JUMP_SPEED;
  this.changeState("jump");
}
```

`this.cursors` are the arrow keys, made by the scene with `this.input.keyboard!.createCursorKeys()` and handed to the hero; each has an `isDown`.

Hurt - ended by an animation event

Being hurt is the state where animation events earn their keep. The hero should stay hurt exactly as long as the hurt animation lasts - so the animation decides:

ch07_hero_animations/src/objects/Hero.ts

```
public hurt(): void {
    if (this.currentState === "hurt") {
        return;
    }
    this.changeState("hurt");
    this.setTint(HURT_TINT);

    // knocked away from the way the hero is facing, and a little way up
    this.velocityX = this.flipX ? KNOCKBACK_X : -KNOCKBACK_X;
    this.velocityY = -KNOCKBACK_Y;

    // hurt has no repeat, so it completes - and this listener hears it, once
    this.once(Phaser.Animations.Events.ANIMATION_COMPLETE_KEY + HERO_HURT, ()
=> {
        this.clearTint();
        this.changeState(this.isOnGround() ? "idle" : "fall");
    });
}
```

`once` , not `on` : with `on` , every hurt would add another listener, and they would pile up. `preUpdate()` skips `readKeys()` while the hero is hurt, so the player cannot run out of it early. Change the hurt animation's frame rate and the hero stays hurt longer or shorter, with no other change: the animation and the game rule cannot get out of step.

preUpdate, and super.preUpdate

ch07_hero_animations/src/objects/Hero.ts


```
protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);

    const seconds = delta / 1000;
    if (this.currentState !== "hurt") {
        this.readKeys();
    }
    this.move(seconds);
    this.updateState();
}
```

In Chapter 1 the ball was an `Image`, and its `preUpdate()` needed no `override`: `Image` has none of its own. A `Sprite` does have one - it is where the sprite's animation moves on to the next frame (Chapter 4 has the details). So here the method needs `override` (leave it out and the build says `This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'`) and, more importantly, it must call `super.preUpdate(time, delta)`.

Leave that line out and nothing complains - but the hero glides across the screen frozen on the first frame of its run. If an animation "plays" but never moves, look for a missing `super.preUpdate`.

A pretend jump

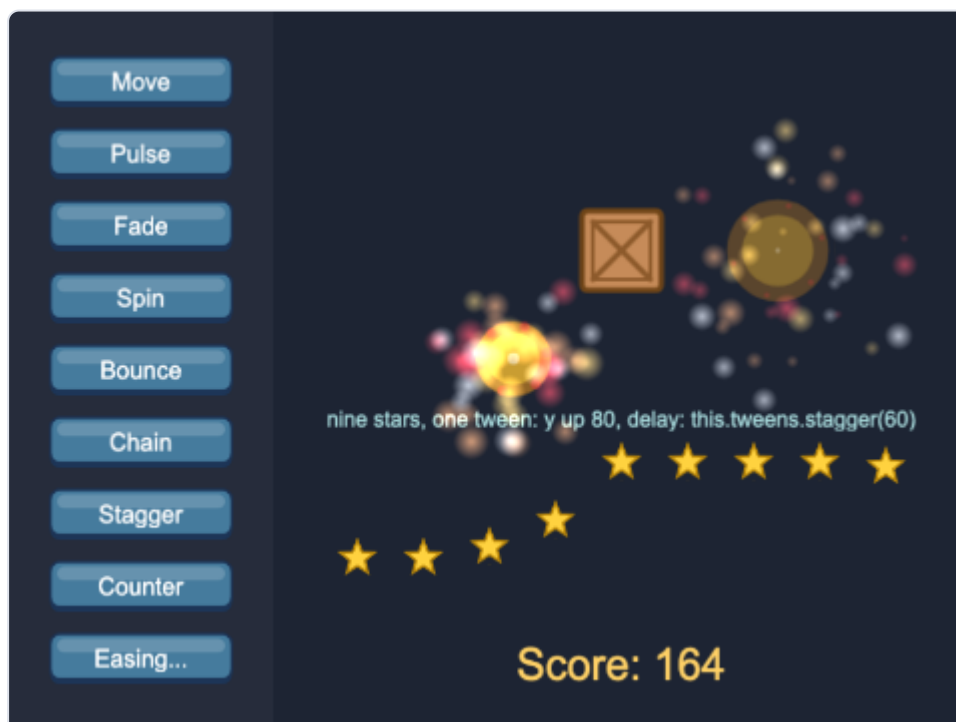
`move()` in `Hero.ts` is Chapter 1's "speed x time = distance", plus gravity: a jump sets `velocityY` to a big upward (negative) speed, and gravity adds to it every frame until the hero falls back to the ground. The hero's origin is `(0.5, 1)` - the middle of its feet - so "standing on the ground" is simply `y === groundY`. Arcade Physics (Chapters 8 and 9) will do this for you; the state machine will not need to change.

Common mistakes with frame animations

- the sprite shows one frame and never moves: a missing `super.preUpdate(time, delta)`, or an `Image` where a `Sprite` was needed
- the animation is stuck on its first frame while a key is held: `play(key)` from `update()` restarts it every frame. Use a state machine, or `play(key, true)`
- the sprite carries on with its old animation, and the browser's console says `Missing animation: hero-run`: a misspelled key. Keep animation keys as constants
- a field called `state` in a sprite class: every game object already has a `state`, and the build says `Class 'Hero' incorrectly extends base class 'Sprite'`. The hero's is called `currentState`

Tweens

A **tween** (from "in between") changes numbers on an object smoothly, from what they are now to what you ask for, over a time you choose. You describe it once; Phaser's tween manager, `this.tweens`, does the changing every frame and throws the tween away when it is done.



`ch07_tweens_and_particles` is a playground: each button on the left - or the keys 1 to 8 - sets off a different tween on the crate or the stars. Here is the first:

`ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts`

```
this.tweens.add({
  targets: this.crate,
  x: this.restX,
  y: this.restY,
  duration: 800,
  ease: "Cubic.easeInOut",
});
```

- `targets` - what to change: one object, or an array of them
- then **the values to change, and where they should end up**: here `x` and `y`. Any number property works: `x`, `y`, `scale`, `scaleX`, `alpha`, `angle`, ... - even a property of an object of your own
- `duration` - how long, in milliseconds
- `ease` - how the value gets from start to end (see Easing, below)

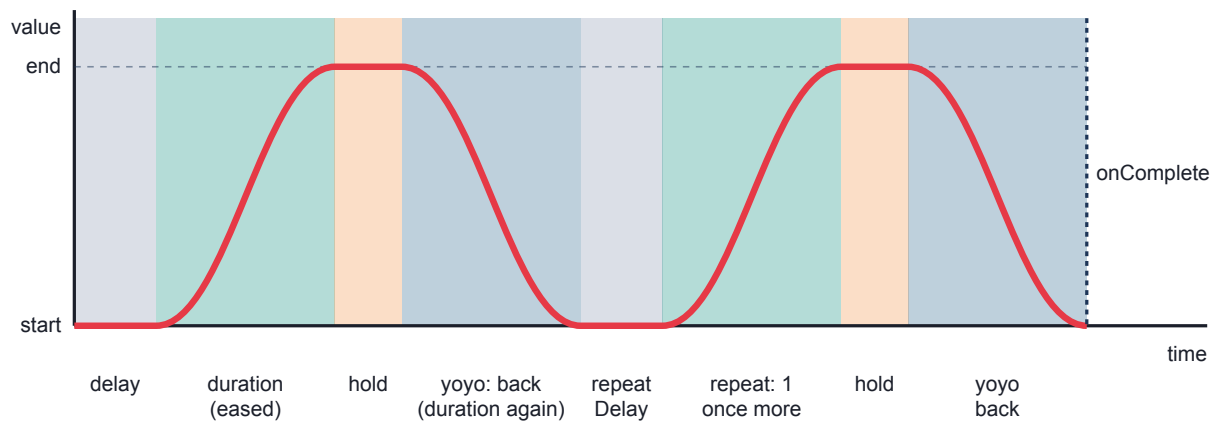
`this.tweens.add` returns straight away; the crate moves over the next 800 milliseconds while the game carries on - more like handing a job to a timer than writing a loop.

Yoyo, hold, repeat and delay

`ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts`

```
this.tweens.add({
  targets: this.crate,
  alpha: 0,
  duration: 500,
  hold: 400,           // wait this long at alpha 0 before the yoyo
  yoyo: true,
});
```

One tween, over time: { delay, duration, hold, yoyo: true, repeat: 1, repeatDelay }



Setting	What it does
delay	wait this long before starting
yoyo: true	when it gets there, come back again (taking duration again)
hold	wait this long at the end value before the yoyo
repeat	play it this many more times (-1 for ever). repeat: 2 is three times in all
repeatDelay	wait this long before each repeat
onComplete	a function to call when it has finished

The clouds in the hero project use `repeat: -1` to drift across the sky for ever. The **Bounce** button uses `yoyo: true, repeat: 2` to hop three times.

A value can also be **relative**: `angle: "+=360"` means "360 more than it is now", so the Spin button spins the crate one more whole turn every time, wherever it had got to.

Doing something when it finishes

The **Counter** button shows a "+123" that floats up and fades out. Once it has faded it is no use to anyone, so the tween removes it when it is done:

```
ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts
```

```

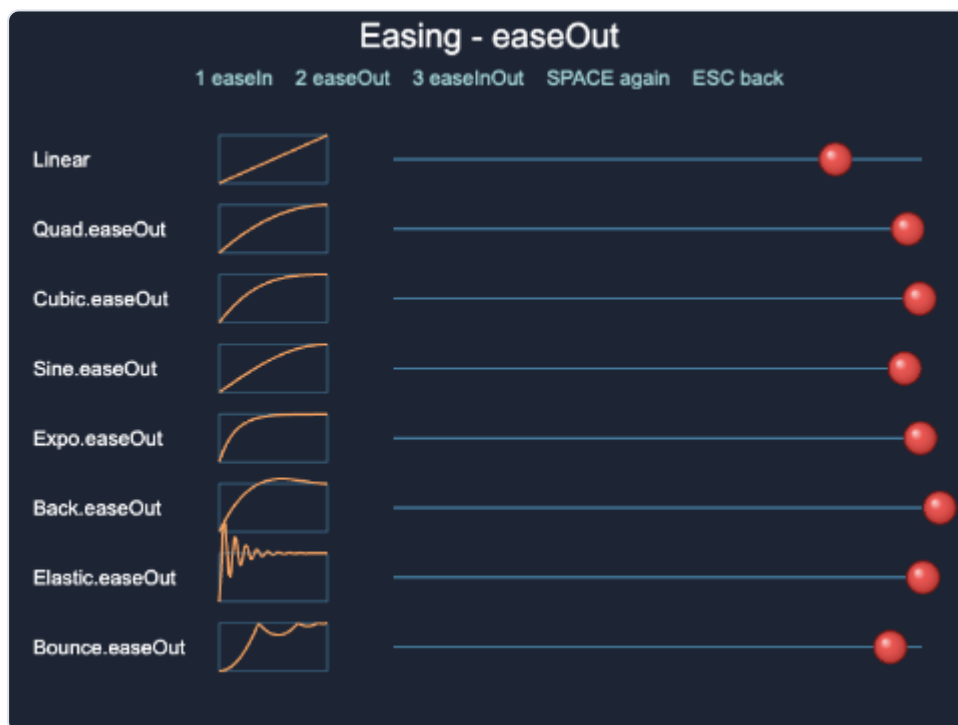
this.tweens.add({
  targets: popup,
  y: popup.y - 70,
  alpha: 0,
  duration: 900,
  ease: "Quad.easeOut",
  onComplete: () => {
    popup.destroy();           // finished with: remove it from the scene
  },
});

```

Anything made just for an effect should remove itself when the effect is over, or the scene slowly fills with invisible objects.

Easing

Press 9 in the playground for the easing gallery: eight balls, each making the same trip in the same two seconds, with eight different **eases**.



An ease is a function from "how far through the time" (0 to 1) to "how far along the way" (0 to 1). **Linear** goes at a steady speed - which looks mechanical, because nothing in the real world starts or stops instantly. The others speed up, slow down, overshoot or bounce. Next to each name is its curve: time goes across, distance up.

Each family comes in three versions - press 1, 2 and 3 in the gallery to switch:

- `.easeIn` - starts slowly, finishes fast (like something falling)
- `.easeOut` - starts fast, finishes slowly (like something sliding to a stop). The name on its own, `"Cubic"`, means `"Cubic.easeOut"`
- `.easeInOut` - slow at both ends (like a lift between floors)

`Quad`, `Cubic`, `Quart`, `Quint` and `Expo` are more and more dramatic versions of one idea; `Sine` is gentle; `Back` overshoots; `Elastic` wobbles; `Bounce` bounces. Rule of thumb: `easeOut` for things arriving, `easeIn` for things leaving, `easeInOut` for moving from place to place.

Note - an ease is a string, and a misspelled one (`"Bounce.easout"`) is not an error: Phaser quietly uses `Linear` instead. If a tween looks flat, check the spelling.

The gallery draws each curve by asking Phaser for the ease function and calling it:

`ch07_tweens_and_particles/src/scenes/EasingScene.ts`

```
// Phaser's types only say GetEaseFunction returns a Function; it is really this
const easeFunction = Phaser.Tweens.Builders.GetEaseFunction(ease) as (t: number) => number;
```

That `as` is a **type assertion**: Phaser's types say only "a function", and `as` says which kind. Use it only where you know more than the types do, and say why. (The gallery also restarts itself with `this.scene.restart(data)`, read in `init(data)` as in Chapter 2.

`Partial<EasingData>` makes every property optional, and `data.variant ?? "easeOut"` means "the variant, or `"easeOut"` if there is none".)

When tweens fight

Press Pulse twice quickly and, without care, two tweens would be changing the crate's scale at once, and the second would "yoyo back" to a size half way through the first. So the playground stops any tween still running on the crate, and puts it back as it was, before starting another:

`ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts`

```
private resetCrate(): void {
    this.tweens.killTweensOf(this.crate);
    this.crate.setPosition(this.restX, this.restY);
    this.crate.setScale(CRATE_SCALE).setAlpha(1).setAngle(0);
}
```

`this.tweens.killTweensOf(target)` stops and removes every tween working on it.

`this.tweens.isTweening(target)` asks whether any are. Whenever a tween can be set off again while it is still going - by a button, a key, a collision - decide what should happen, and write it.

Chains

A **chain** is a list of tweens played one after another: each starts when the one before ends.

ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts

```
this.tweens.chain({
    targets: this.crate,
    tweens: [
        { y: this.restY - 120, duration: 400, ease: "Quad.easeOut" },
        { angle: 360, duration: 500, ease: "Cubic.easeInOut" },
        { y: this.restY, duration: 700, ease: "Bounce.easeOut" },
        { scaleX: CRATE_SCALE * 1.4, scaleY: CRATE_SCALE * 0.6, duration: 120,
        yoyo: true },
    ],
});
```

The crate jumps up, spins, drops with a bounce, and squashes. Every tween in the chain uses its `targets`. Without a chain, each tween's `onComplete` would have to start the next.

Stagger

ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts

```

this.tweens.add({
  targets: this.stars,
  y: STAR_Y - 80,
  duration: 300,
  ease: "Quad.easeOut",
  yoyo: true,
  delay: this.tweens.stagger(60),
});

```

One tween, nine targets. `delay: this.tweens.stagger(60)` gives each target a delay 60 milliseconds longer than the one before, so a wave runs along the row - the trick behind menus that slide in item by item.

Counting

A tween does not need a game object at all. `addCounter` tweens a plain number, and hands it to you every frame:

`ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts`

```

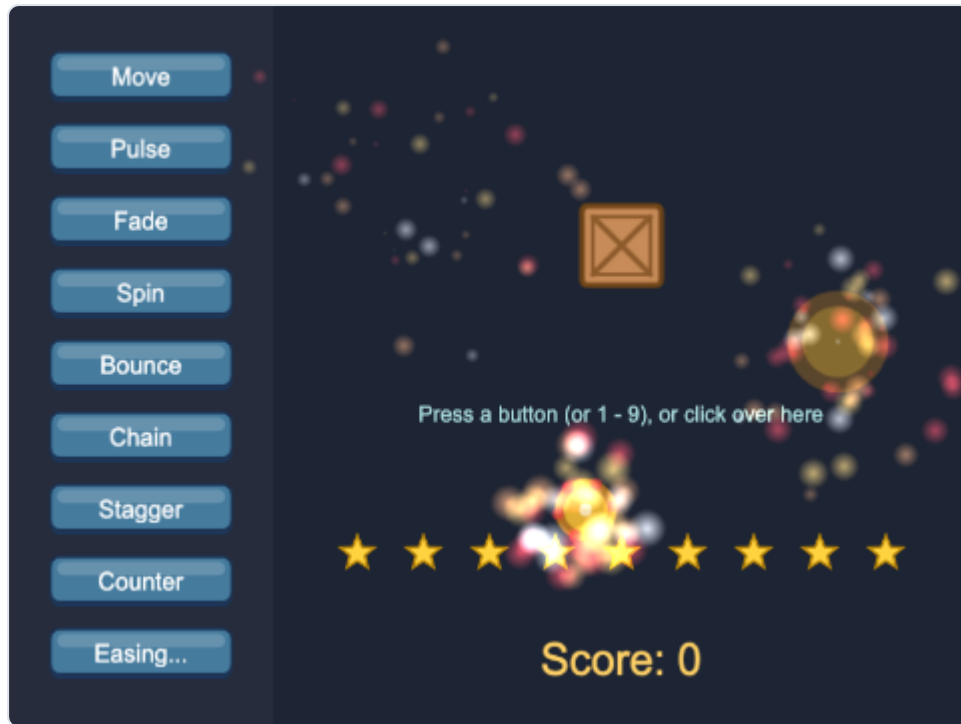
this.tweens.addCounter({
  from: from,
  to: this.score,
  duration: 1200,
  ease: "Cubic.easeOut",
  onUpdate: (tween) => {
    // getValue() is number | null; "?? 0" means "or 0, if it is null"
    this.scoreText.setText(`Score: ${Math.round(tween.getValue() ?? 0)}`);
  },
});

```

The score really goes up at once; the **display** catches up over 1.2 seconds, fast at first and slowing as it arrives. `onUpdate` is called every frame the tween changes.

Particles

Click anywhere in the playground (not on a button) for an explosion.



A **particle emitter** throws out lots of small images - particles - each with its own speed, direction, size, colour and lifetime. Particles are much simpler than game objects, so one emitter can have hundreds alive at once.

ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts

```
// ONE emitter, made now, emitting nothing (emitting: false) until explode()
// is called
this.sparks = this.add.particles(0, 0, PARTICLE_KEY, {
  speed: { min: 80, max: 360 },           // a random speed for each
  particle...,
  angle: { min: 0, max: 360 },           // ...in a random direction
  lifespan: { min: 400, max: 900 },       // milliseconds each particle
  lives
  scale: { start: 2.5, end: 0 },          // shrinks away...
  alpha: { start: 1, end: 0 },            // ...and fades, over its life
  tint: [0xffd166, 0xf4a261, 0xe63946, 0xffffffff], // one of these colours
  each
  gravityY: 300,
  blendMode: "ADD",                      // overlapping particles add up
  to brighter light
  emitting: false,
});
```

The settings take several kinds of value:

- a number - every particle gets the same
- `{ min, max }` - each particle gets a random value in the range
- `{ start, end }` - the value changes over the particle's life: here each one shrinks to nothing and fades out as it dies
- an array - each particle gets one of the values, at random

Left to itself, an emitter **flows**: it emits particles every frame, for ever - right for smoke or a fountain. `emitting: false` stops that. Then, on each click:

```
this.sparks.explode(SPARKS, x, y);
```

`explode(count, x, y)` throws out `count` particles at once, from that point. One emitter serves every explosion: there is no need for a new one each time.

An animation that tidies up after itself

The fireball in the middle of each explosion is a frame animation, from `explosion.png`. Every click makes a new sprite, which destroys itself when its animation completes:

`ch07_tweens_and_particles/src/scenes/PlaygroundScene.ts`

```
// a new sprite for every explosion, which removes itself when its animation completes
const boom = this.add.sprite(x, y, EXPLOSION_SHEET).setScale(1.5);
boom.play(EXPLODE);
boom.once(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {
    boom.destroy();
});

this.sparks.explode(SPARKS, x, y);
this.cameras.main.shake(150, 0.006);
```

The `EXPLODE` animation has no `repeat`, so it completes (with `repeat: -1` the explosions would never go away). The last line shakes the camera for 150 milliseconds - a cheap way to make anything feel bigger.

Summary

- a **sprite sheet** is one picture of equal-sized frames, numbered from 0
- `this.animations.create({ key, frames: this.animations.generateFrameNumbers(sheet, { start, end }), frameRate, repeat })` makes an animation. Animations belong to the game: make them once
- only a `Sprite` plays animations: `play(key)` restarts, `play(key, true)` does not; `chain`, `playAfterRepeat`, `stop`, `animations.pause`, `animations.timeScale`
- `animations.currentAnim` and `currentFrame` say what is showing; events such as `ANIMATION_COMPLETE` say when it changes. A `Sprite` subclass's `preUpdate` must call `super.preUpdate(time, delta)`
- `setFlipX` mirrors a sprite; a **state machine** - one state at a time, one place that changes it - keeps a character's animations under control
- `this.tweens.add({ targets, ...values, duration, ease, yoyo, hold, repeat, onComplete })` changes numbers smoothly; `chain`, `stagger`, `addCounter` and `killTweensOf` do the rest
- `this.add.particles(x, y, key, config)` makes an emitter; `emitting: false` plus `explode(count, x, y)` makes bursts

Challenges

1. **Different speeds** (*ch07_animation_lab*) - Make the bat flap twice as fast and the ghost bob more slowly, and update their labels. Then add a new animation, `hero-sprint`, made from the same frames as `hero-run` but at 20 frames a second, played on the 7 key. Add it to the help line at the bottom of the screen.
2. **Coin run** (*ch07_hero_animations*) - Put a row of spinning coins in the air above the ground (use `coin_spin.png` from the asset library - frames 0-5, 32 x 32). When the hero touches a coin, it floats upwards and fades out with a tween, then is removed; a count of coins collected is shown on screen. You can decide "touches" by the distance between the hero and the coin.
3. **Title sequence** (*ch07_tweens_and_particles*) - Pressing T plays a title sequence: the word "ANIMATIONS" drops in from above the screen and lands with a bounce, grows a little and shrinks back, wobbles side to side, holds for a moment, then fades away. Use

one `this.tweens.chain` , and make sure pressing T again part way through starts the sequence cleanly from the beginning.

4. **Sparkle trail** (*ch07_tweens_and_particles*) - Make a trail of particles that follows the mouse pointer around the playground: small, short-lived and fading. The trail should stop while the pointer is over the buttons, and start again when it comes back. *Hint:* this emitter should flow, not explode. Look up `emitter.startFollow(...)` - anything with an `x` and a `y` can be followed - and `emitter.emitting` .
5. **Speed you can see** (*ch07_hero_animations*) - Make the hero speed up and slow down instead of starting and stopping instantly: holding an arrow key accelerates up to the running speed, and letting go slows the hero to a stop. The run animation should play at a frame rate that matches: slow steps when moving slowly, a full stride at top speed. The hero should only be "idle" once it has actually stopped. *Hint:* the hero's speed can be changed a little each frame, using `delta` . `anim.timeScale` changes how fast a sprite's animation plays without changing the animation.
6. **Stomp** (*ch07_hero_animations*) - Add three slimes (`slime.png` , 32 x 32, frames 0-3) that patrol back and forth along the ground, animated. If the hero lands on a slime from above, the slime is squashed: it is removed with an explosion animation (`explosion.png`) and a burst of particles, and the hero bounces up off it. If the hero walks into a slime from the side, the hero is hurt, and cannot be hurt again until the hurt state is over. *Hint:* for "touching", compare the two sprites' rectangles - `Phaser.Geom.Intersects.RectangleToRectangle(a.getBounds(), b.getBounds())` - Chapter 8 explains it. "From above" means the hero is falling at the moment they touch. Give `Hero` public methods for the scene to call, rather than letting the scene change its fields.

Previous: [Chapter 6 - Scoring](#) · Next: [Chapter 8 - Collisions](#)



Collisions

Nearly every game rule starts with "when this touches that": the player picks up a coin, a bullet hits a ship, a ball bounces off a wall. In this chapter you first work out whether two shapes touch by hand, with Phaser's geometry classes, and then hand the job to **Arcade Physics** - Phaser's built-in physics engine - which gives game objects **bodies** and checks them for you every frame. It ends with a complete breakout game.



What you will learn

- how to test whether two rectangles or two circles overlap, with `Phaser.Geom` and `Phaser.Geom.Intersects`, and why rectangles are unfair to round things
- how to give a circle a circular hit area, so clicks outside the circle miss it
- how to turn on Arcade Physics, give game objects **bodies**, and see them with `debug`
- how to size and shape a body to fit its picture: `setSize`, `setOffset` and `setCircle`
- the difference between `this.physics.add.overlap` and `this.physics.add.collider`
- how to collide whole **groups** - dynamic and static - with each other, and what the callbacks receive

- how to decide whether a collision counts, with a **process callback**, and how to switch collisions and bodies off

The projects

Project	What it shows
<code>ch08_hand_made_collisions</code>	rectangles and circles to drag with the mouse; they turn red while they touch. No physics - just geometry
<code>ch08_collect_and_avoid</code>	the first Arcade Physics game: collect stars, avoid enemies, bump into crates. Bodies, groups, colliders and overlaps
<code>ch08_breakout</code>	a complete breakout: a paddle, a ball, and a static group of bricks; a process callback, and aiming the ball by where it hits the paddle

Part 1: collisions by hand

To ask whether two things touch, you need shapes with numbers you can compare.

Phaser's geometry classes, in `Phaser.Geom - Rectangle`, `Circle`, `Line`, `Triangle`, `Polygon`, `Point` - are exactly that: just numbers, never drawn.

`Phaser.Geom.Intersects` has a function for each pair of shapes. The two you will use most:

RectangleToRectangle

they overlap when $a.left < b.right$, $a.right > b.left$, $a.top < b.bottom$ and $a.bottom > b.top$ - all four

CircleToCircle

they overlap when the distance between the centres is less than the radii added: $d < r1 + r2$

- **two rectangles** overlap if each one starts before the other one ends - across *and* down. Four comparisons, and no square roots, so it is very fast
- **two circles** overlap if the distance between their centres is less than their two radii added together

`CircleToRectangle` finds the point of the rectangle nearest the circle's centre, and checks whether that is inside the circle.

The shapes project



`ch08_hand_made_collisions` has three rectangles and three circles, which you can drag around with the mouse. They are Phaser **shape game objects**: `this.add.rectangle(...)` makes a `Phaser.GameObjects.Rectangle`, and `this.add.circle(...)` makes a `Phaser.GameObjects.Arc` (a circle is an arc that goes all the way round). Neither knows anything about collisions. That is the job of one function:

```
src/geometry.ts
```

```
// a union type: a shape on screen is a rectangle OR a circle
export type DragShape = Phaser.GameObjects.Rectangle |
Phaser.GameObjects.Arc;

// the circle a round shape covers, in game coordinates
export function circleOf(shape: Phaser.GameObjects.Arc): Phaser.Geom.Circle
{
    return new Phaser.Geom.Circle(shape.x, shape.y, shape.radius);
}
```

```
export function touching(a: DragShape, b: DragShape, boxesOnly: boolean):
boolean {
    const intersects = Phaser.Geom.Intersects;

    // getBounds() gives the smallest rectangle, lined up with the screen,
    that holds the whole shape
    if (boxesOnly) {
        return intersects.RectangleToRectangle(a.getBounds(), b.getBounds());
    }

    // instanceof checks the class of an object while the game runs, as in
    Java - and TypeScript
    // then knows that `a` is an Arc inside the if, so a.radius is allowed
    if (a instanceof Phaser.GameObjects.Arc && b instanceof
Phaser.GameObjects.Arc) {
        return intersects.CircleToCircle(circleOf(a), circleOf(b));
    }
    if (a instanceof Phaser.GameObjects.Arc) {
        return intersects.CircleToRectangle(circleOf(a), b.getBounds());
    }
    if (b instanceof Phaser.GameObjects.Arc) {
        return intersects.CircleToRectangle(circleOf(b), a.getBounds());
    }
    return intersects.RectangleToRectangle(a.getBounds(), b.getBounds());
}
```

- `type DragShape = ... | ...` gives a union type a name - a **type alias**
- `getBounds()`, on every game object with a size, returns a new `Phaser.Geom.Rectangle` - the **bounding box**: the smallest rectangle, square to the screen, that the object fits inside. For a circle, `circleOf()` builds a `Phaser.Geom.Circle` from the arc's centre and radius

- `instanceof` works as in Java, and TypeScript **narrows** the type: inside `if (a instanceof Phaser.GameObjects.Arc)` it knows `a` is an `Arc`, so `a.radius` compiles

Testing every pair

Every frame, the scene tests every shape against every other shape - once:

`src/scenes/ShapesScene.ts`

```
override update(): void {
    // test every PAIR once: shape 0 with 1, 2, 3...; shape 1 with 2, 3...;
    // and so on
    const touched = new Set<DragShape>();
    let pairs = 0;

    for (let i = 0; i < this.shapes.length; i++) {
        for (let j = i + 1; j < this.shapes.length; j++) {
            const a = this.shapes[i];
            const b = this.shapes[j];
            if (touching(a, b, this.bboxesOnly)) {
                touched.add(a);
                touched.add(b);
                pairs++;
            }
        }
    }
    ...
}
```

Starting `j` at `i + 1` tests each pair once, and never a shape against itself. Six shapes make 15 pairs; 100 would make 4,950 - every frame - which is why physics engines work hard to skip pairs that are obviously far apart. `Set` is like Java's `HashSet`; the rest of `update()` colours each shape by asking `touched.has(shape)`.

Why rectangles are unfair to round things

Press SPACE, and the project tests only the shapes' bounding boxes - which it outlines in orange:



The two circles on the right are now "touching" each other and the tall rectangle, although there is clear space between them. Their boxes overlap in the corners:

Why rectangles are unfair to round things

The boxes overlap (red),
so a box test says "hit" -
but the balls are nowhere
near each other.

about a fifth of a circle's box
is empty corner (the red)

Players never forgive a bullet that visibly missed but still killed them. For anything round - balls, coins, bubbles - test circles. For anything boxy - platforms, crates, bricks - rectangles are exact, and faster.

Point in shape: clicking a circle

Chapter 2 said that `setInteractive()` checks the pointer against a game object's rectangle. A circle's rectangle has the same four empty corners, so by default a click just *outside* a circle, in one of its corners, would pick it up. The fix is a different **hit area**:

```
private addCircle(x: number, y: number, radius: number): void {
    const circle = this.add.circle(x, y, radius, IDLE_COLOUR, SHAPE_ALPHA);

    // A circle's hit area would be its rectangle too - so clicking just
    // outside the circle, in a
    // corner of its box, would pick it up. Give it a CIRCLE hit area instead.
    // The hit area is in
    // the object's own coordinates, where (0, 0) is its top-left corner - so
    // the centre is at
    // (radius, radius). Circle.Contains is the test Phaser runs: is this
    // point inside the circle?
    circle.setInteractive({
        hitArea: new Phaser.Geom.Circle(radius, radius, radius),
        hitAreaCallback: Phaser.Geom.Circle.Contains,
        draggable: true,
        useHandCursor: true,
    });
    this.shapes.push(circle);
}
```

Every geometry class has a static `Contains(shape, x, y)` - "is this point inside?" - and you can call them yourself, too. `draggable: true` makes the object draggable, and the scene listens for drags on all of them at once:

```
this.input.on("drag", (_pointer: Phaser.Input.Pointer, shape: DragShape,
dragX: number, dragY: number) => {
    shape.setPosition(dragX, dragY);
});
```

`dragX` and `dragY` are where the object should be now, allowing for where you grabbed it.

Part 2: Arcade Physics

A real game has dozens of moving things, and you want them to *respond* - stop at a wall, bounce off each other. **Arcade Physics** does all of that. It is fast because it only knows rectangles and circles that never rotate - which suits most 2D games. (Chapter 9 looks at it in depth, and at Matter.js, Phaser's other engine.)

Turning it on

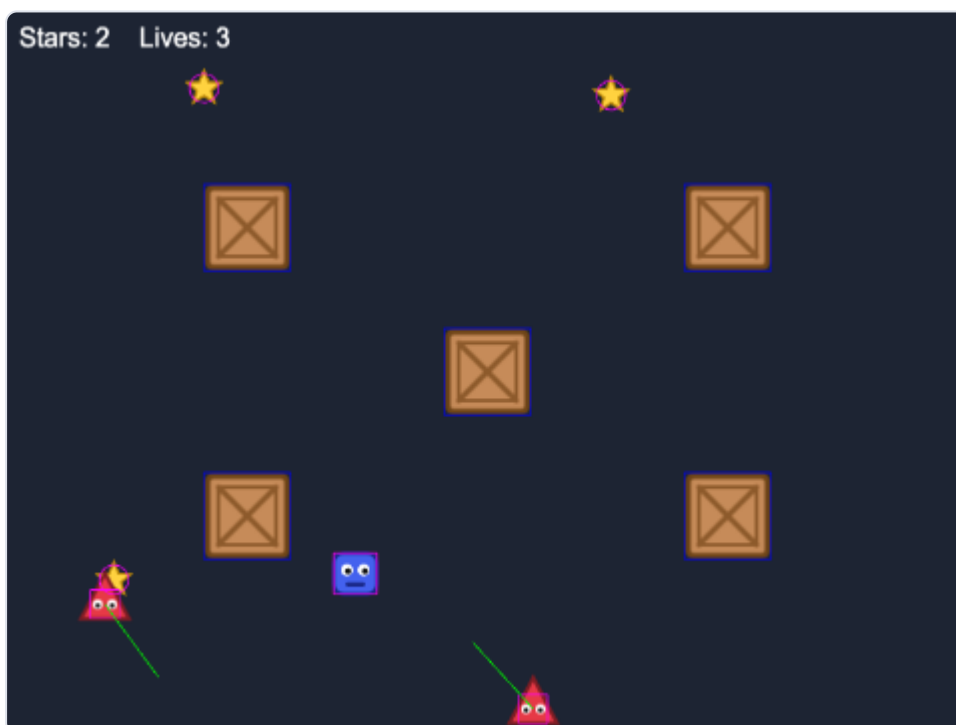
Physics is switched on in the game's config:

ch08_collect_and_avoid/src/main.ts

```
physics: {  
  default: "arcade",  
  arcade: {  
    debug: false,  
  },  
},
```

This gives every scene a `this.physics` - the Arcade Physics **plugin** - and a physics **world**, the size of the game, that it runs every frame. Nothing in the world has a body yet, though: game objects only get one when you ask.

`debug: true` draws every body's outline, and a line showing which way and how fast it is moving. It is the single most useful thing to turn on when a collision does not do what you expect:



Pink outlines are **dynamic** bodies (they move); blue ones are **static** bodies (they never move). Notice the stars' round bodies, and the small boxes inside the triangles - more on those soon.

Bodies

A **body** is the physics engine's idea of a game object: a rectangle or circle, with a position, a velocity, and settings such as bounce. Each frame, the world moves every body, checks for collisions, and moves the game objects to match - the `speed * delta` maths from Chapter 1, done for you.

There are three ways to get a game object with a body:

```
this.physics.add.image(x, y, key);           // a new Image, with a body
this.physics.add.sprite(x, y, key);          // a new Sprite (animations), with
a body
this.physics.add.existing(gameObject);      // give a body to a game object you
already have
```

The player in `ch08_collect_and_avoid` is a class of its own, so it uses the third:

`src/objects/Player.ts`

```
export class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene: Phaser.Scene, x: number, y: number) {
    super(scene, x, y, PLAYER_KEY);

    scene.add.existing(this);           // show it: add it to the scene's
display list
    scene.physics.add.existing(this);   // give it a dynamic physics body
```

`Phaser.Physics.Arcade.Image` is an `Image` with the physics methods added - `setVelocity`, `setBounce`, `setCollideWorldBounds` and many more - so the player can call them on itself. Extending it does not give the object a body, though; `scene.physics.add.existing(this)` does. Forget that line and the first physics call fails as the game starts:

```
TypeError: Cannot read properties of null (reading 'setSize')
```

The player's `move()` method, called from the scene's `update()`, works out a speed across and down from the arrow keys and calls `this.setVelocity(speedX, speedY)`. It never changes its own `x` and `y`, and that matters: if a crate is in the way, the physics engine stops the body at its edge. If the code set `x` itself, it could put the player straight

through the crate. And `setCollideWorldBounds(true)` keeps the body inside the world - the edges of the game.

Fitting the body to the picture

A body starts as the size of the picture. That is rarely right: pictures have empty space round their edges. `player.png` is 48 x 48, but the blob drawn in it is 36 x 34:

`src/objects/Player.ts`

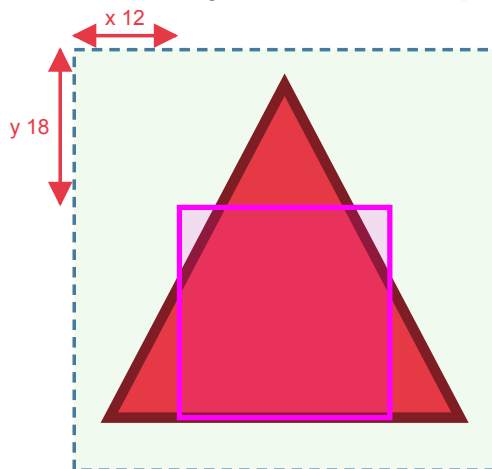
```
// the body starts as the size of the picture; shrink it to fit the blob,  
and move it to  
// where the blob is. The offset is measured from the picture's top-left  
corner.  
this.setSize(BODY_WIDTH, BODY_HEIGHT);  
this.setOffset(BODY_OFFSET_X, BODY_OFFSET_Y);
```

The enemies are triangles - the worst shape for a rectangle. A body the size of the picture would hurt the player when they touched an empty corner, so each enemy gets a smaller box in the middle, a little low, where most of the triangle is:

`src/scenes/GameScene.ts`

```
enemy.setSize(24, 24);  
enemy.setOffset(12, 18);
```

A body smaller than its picture (enemy.png, drawn 5 times full size)



dashed: the picture, 48 x 48

pink: the body - what the physics engine sees

```
enemy.setSize(24, 24);  
enemy.setOffset(12, 18);
```

setSize: the body's width and height

setOffset: where its top-left corner is,
measured from the picture's top-left corner

A slightly *small* hit box is the usual choice for things that hurt the player: a near miss feels like skill, and a hit that looks like a miss feels like cheating. (Challenge 6 makes the triangles exactly fair.)

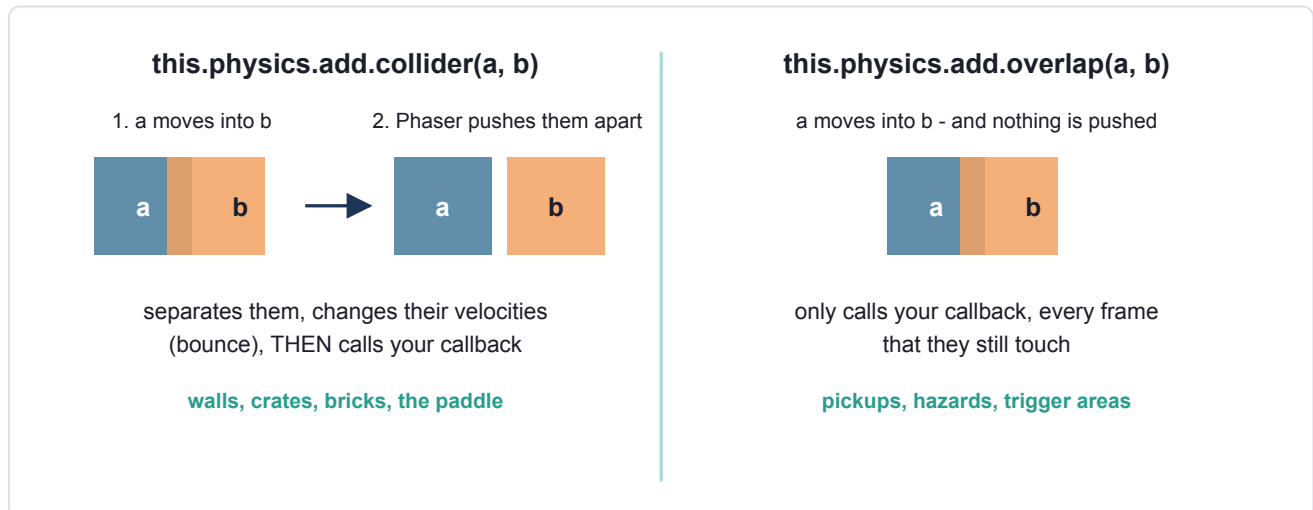
Round things get round bodies. `setCircle(radius, offsetX, offsetY)` turns a body into a circle; the offset is where the circle's bounding box starts, from the picture's top-left corner:

```
// a round body, smaller than the picture: radius 12, its top-left 4 across
and 5 down
star.setCircle(12, 4, 5);
```

Turn on `debug` whenever you size a body: guessing numbers blind is slow work.

Overlap or collider?

Arcade Physics has two ways to react to two things touching, and choosing the right one is most of the work:



- `this.physics.add.collider(a, b, callback)` - `a` and `b` must not pass through each other. When their bodies overlap, Phaser **separates** them (pushes them back apart), changes their velocities - bouncing them if they have bounce - and then calls your callback, if you gave one
- `this.physics.add.overlap(a, b, callback)` - Phaser only tells you they touched. Nothing is pushed, and the callback is called **every frame** that they still overlap

Here are all the collisions in `ch08_collect_and_avoid` :

```

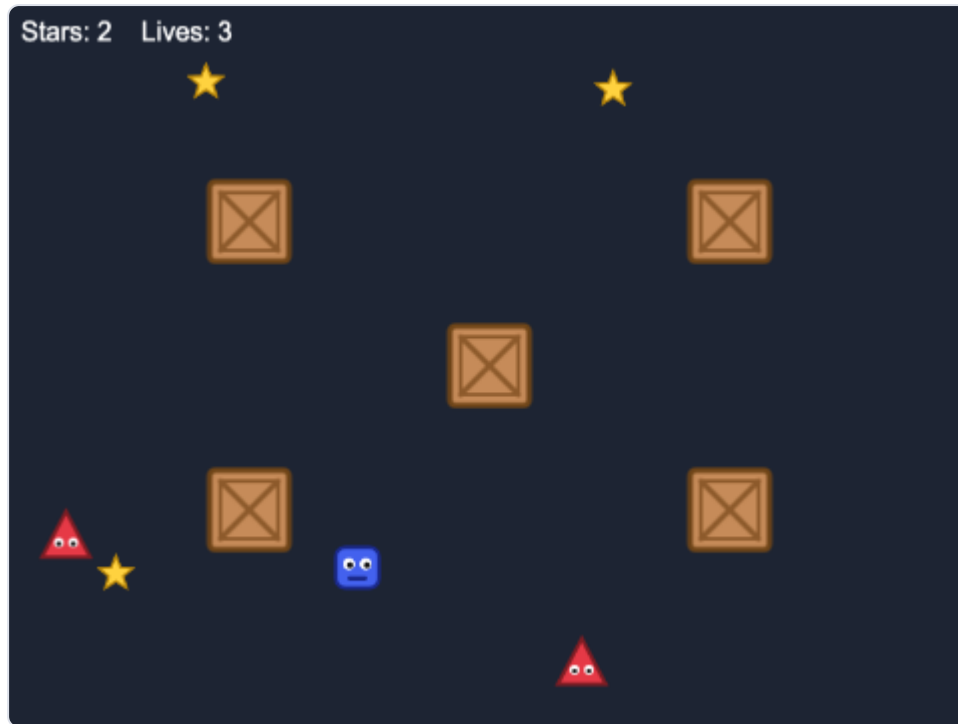
// COLLIDERS: these must not pass through each other. Phaser pushes them
// apart
this.physics.add.collider(this.player, this.crates);
this.physics.add.collider(this.enemies, this.crates);
this.physics.add.collider(this.enemies, this.enemies); // a group against
// itself

// OVERLAPS: only tell us that they touched - nothing is pushed
this.physics.add.overlap(this.player, this.stars, (_player, star) => {
  // Phaser's types allow for a body or a tile here too; we know it is a
  // star, because the
  // second thing we passed was the stars group
  this.collectStar(star as Phaser.Physics.Arcade.Sprite);
});
this.enemyOverlap = this.physics.add.overlap(this.player, this.enemies, ()
=> {
  this.hurtPlayer();
});

```

These are set up **once**, in `create()`. Each call makes a `Collider` object and adds it to the world, which checks it every frame from then on. There is no collision code in `update()` at all.

Why an overlap for the stars? With a collider, the player would *bump* the star before the callback removed it; you want to walk straight through a pickup. And an enemy should hurt, not shove.



Groups against groups

A collider or overlap can take a single game object, a **group**, or an array. Given a group, it checks every member - so one line handles all the stars, however many there are, including ones added later. `this.physics.add.collider(this.enemies, this.enemies)` checks every enemy against every other enemy (each pair once, as in the shapes project), so they bounce off each other.

Physics groups come in two kinds:

- `this.physics.add.staticGroup()` gives its members **static bodies**: they never move, whatever hits them, and cost almost nothing to check. Perfect for walls, platforms and crates
- `this.physics.add.group(config)` gives its members **dynamic bodies**, which move with their velocity and can be pushed. The config sets defaults for every member it makes: the enemies group has `{ bounceX: 1, bounceY: 1, collideWorldBounds: true }`, so every enemy keeps all its speed when it bounces, and stays inside the game

`group.create(x, y, key)` makes a new member, with a body, and adds it to the group and the scene:

```

for (const [x, y] of CRATE_PLACES) {
    // create() makes a member of the group. It is typed `any`, so say what it
    // is: a physics
    // group makes Arcade Sprites unless told otherwise
    const crate = this.crates.create(x, y, CRATE_KEY) as
    Phaser.Physics.Arcade.Sprite;
    crate.setScale(CRATE_SCALE);
    // a static body does not follow its game object: after scaling, make the
    // body match again
    crate.refreshBody();
}

```

`const [x, y] of CRATE_PLACES` takes each two-number array in turn and **destructures** it into `x` and `y`. And note `refreshBody()`: a static body is worked out once, when it is made, so if you move or scale a static object afterwards, you must tell its body to catch up. Forget, and the player bumps into an invisible crate-sized box inside a bigger crate.

What the callbacks receive

A callback receives the two objects that touched, in the order you passed them - except that a single object always comes before a group member, even if you passed the group first. TypeScript does not know what is inside a group, and colliders can also involve bare bodies and tilemap tiles (Chapter 10), so both parameters are typed as a union of all four. Use one directly:

```

this.physics.add.overlap(this.player, this.stars, (_player, star) => {
    star.disableBody(true, true);
});

```

and the build says:

```

TS2339 [ERROR]: Property 'disableBody' does not exist on type 'Body |
StaticBody | Tile | GameObjectWithBody'.

```

You know better - the stars group only holds stars - so you say so with `as`, TypeScript's cast (Java's `(Sprite) star`). It is exactly what `as` is for: a fact Phaser's types cannot know. Keep it right next to the collider that guarantees it.

Switching bodies off

When a star is collected, it should vanish - and stop being collectable:

```
private collectStar(star: Phaser.Physics.Arcade.Sprite): void {
  this.sound.play(COIN_SOUND);
  this.score++;
  this.showHud();

  // switch the star's body off and hide it: it can no longer be touched or
  // seen
  star.disableBody(true, true);

  // ...and bring it back somewhere else a moment later
  this.time.delayedCall(STAR_RESPAWN_TIME, () => {
    const spot = this.freeSpot();
    star.enableBody(true, spot.x, spot.y, true, true);
  });
}
```

`disableBody(disableGameObject, hideGameObject)` takes the body out of the simulation - no collisions at all - and with `true, true` also makes the star inactive and invisible. `enableBody(reset, x, y, enableGameObject, showGameObject)` brings it back at a new place. The same star object is reused, rather than destroyed and made again.

Where? `freeSpot()` picks random places until one is clear of the crates and the player - checking the crates with `RectangleToRectangle`, by hand, as in Part 1.

Switching collisions off

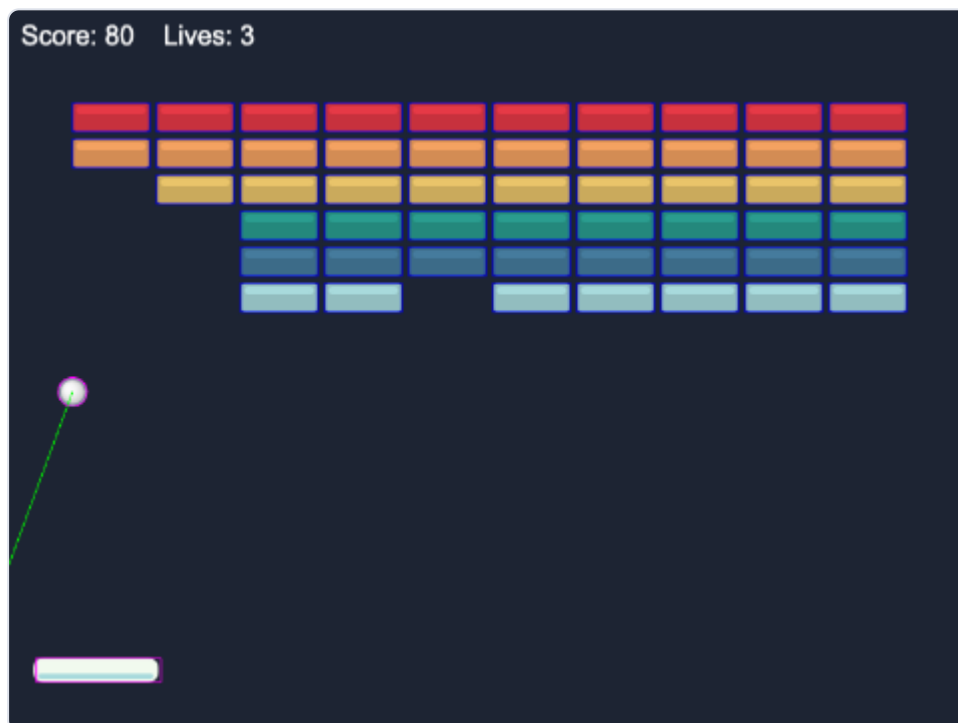
An overlap calls its callback every frame the two things touch. Standing on an enemy for half a second would cost 30 lives. So after a hit, the player is safe for a moment:

```
// a short time of safety: switch the overlap off, flash the player, and
// switch it on again.
// Without this, touching an enemy would cost a life EVERY FRAME it touched
this.enemyOverlap.active = false;
this.tweens.add({ ... });
this.time.delayedCall(INVULNERABLE_TIME, () => {
  this.player.setAlpha(1);
  this.enemyOverlap.active = true;
});
```

`this.physics.add.overlap(...)` returns the `Collider` it made; keeping it in a field lets you switch it off with `active = false` and on again with `active = true`. The player still overlaps the enemies - the world just stops checking. (To remove a collider for good, call its `destroy()`.) The flashing is a tween from Chapter 7, which fades the player in and out.

At the start of `hurtPlayer()` there is one more check - `if (!this.enemy0verlap.active)` - because two enemies can touch the player in the same frame, and only the first should count.

Breakout



`ch08_breakout` puts it all together. It has three kinds of object:

- the **paddle** - a `Paddle` class, with a dynamic body made **immovable**: when the ball hits it, the physics engine moves only the ball (Chapter 9 has more on immovable bodies)
- the **ball** - a `Ball` class, with a circular body, bounce 1, and world-bounds collision
- the **bricks** - a static group of 60, made in rows and columns

The paddle and bricks are not in the asset library, so `makeTextures()` draws them with `Graphics` and saves each as a texture with `generateTexture(key, width, height)` - usable by key, like a loaded image. The brick is white, and each row is tinted with `setTint(colour)`.

The ball

src/objects/Ball.ts

```
constructor(scene: Phaser.Scene, x: number, y: number) {
  super(scene, x, y, BALL_KEY);
  scene.add.existing(this);
  scene.physics.add.existing(this);

  // a round body the size of the picture - so the ball's corners never hit
  // anything
  this.setCircle(RADIUS);
  this.setCollideWorldBounds(true);
  this.setBounce(1);
}
```

Bounce is how much speed a body keeps when it hits something: 0 stops dead, 0.5 keeps half, 1 keeps it all. With bounce 1 and world-bounds collision on, the ball bounces round the edges of the game for ever - except that breakout needs a hole in the floor:

src/scenes/BreakoutScene.ts

```
// The ball bounces off the left, right and top edges of the world - but not
// the bottom:
// there it falls out, and a life is lost
this.physics.world.setBoundsCollision(true, true, true, false);
```

The four values are left, right, top, bottom. `update()` then watches for the ball falling below the screen.

Ball against bricks

```
// ball and bricks: a COLLIDER, so the ball bounces. The callback runs after
// the bounce
this.physics.add.collider(this.ball, this.bricks, (_ball, brick) => {
  // Phaser's types allow for a body or a tile too; we passed the bricks
  // group, so it is one
  // of its members - and a group makes Arcade Sprites unless told otherwise
  this.hitBrick(brick as Phaser.Physics.Arcade.Sprite);
});
```

A collider this time: the ball must bounce off the brick. The physics engine does the bounce - it works out which side was hit and reverses the right part of the velocity - and then the callback removes the brick:

```
// gone for good: destroy() removes the brick from the scene, its body from  
the physics world,  
// and the brick from the group  
brick.destroy();  
...  
if (this.bricks.countActive() === 0) {  
    this.endGame("You cleared the wall!", WIN_SOUND);  
}
```

Stars were disabled, because they come back; bricks are destroyed, because they do not. `countActive()` counts the group's active members - none left means the wall is cleared.

Ball against paddle: a process callback

The paddle is where the player gets control, and it needs two extra ideas:

```
// ball and paddle: a collider with a PROCESS CALLBACK (the second  
function). Phaser asks it  
// first, and if it answers false the two do not collide at all. Only a  
FALLING ball bounces  
// off the paddle - so a ball that clips the paddle's end on its way up is  
not turned round  
this.physics.add.collider(  
    this.ball,  
    this.paddle,  
    () => {  
        this.ball.bounceOff(this.paddle);  
        this.sound.play(PADDLE_SOUND);  
    },  
    () => {  
        return this.ball.isFalling();  
    },  
);
```

A collider (or overlap) can take a second function, the **process callback**. When the bodies overlap, Phaser calls it *first*; if it returns `false`, the pair is ignored - no separation, no bounce, no collide callback - for this frame. It is how you say "these two collide, *but only*

when...": only when the ball is falling; only when the player is not invulnerable; only from above, for a platform you can jump up through.

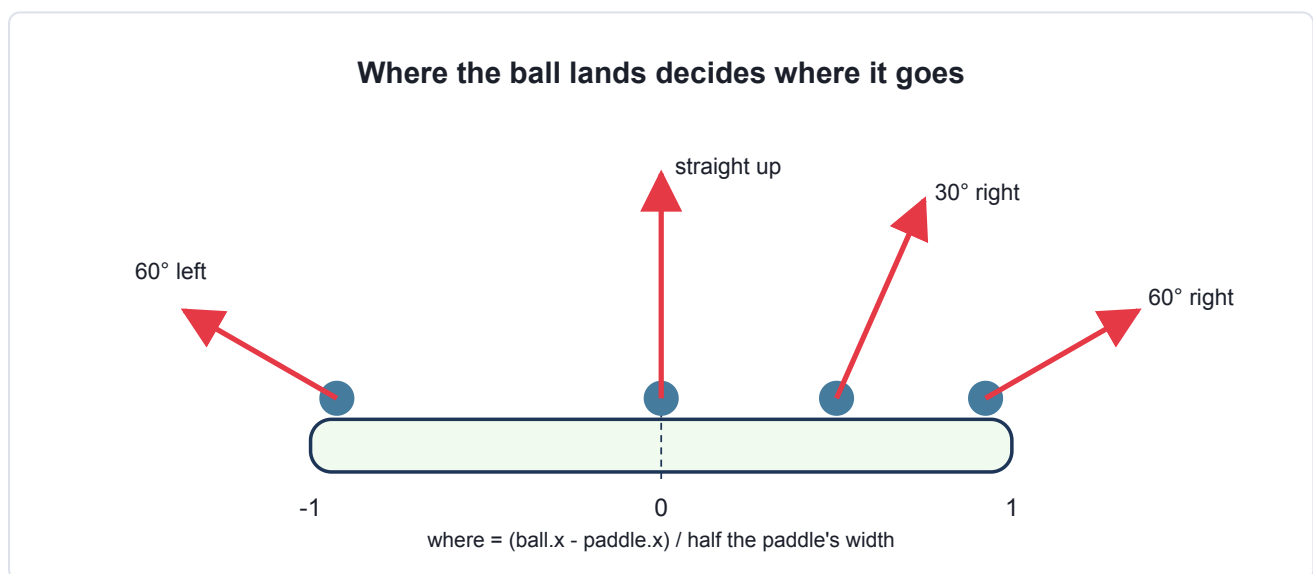
Here it fixes a classic breakout bug: without it, a paddle moved sideways into a rising ball turns the ball round, down into the paddle, where it can stick.

Note - Phaser only skips the collision when the process callback returns exactly `false`. A process callback that forgets its `return` gives `undefined` - and everything collides as if there were no process callback at all. TypeScript will not warn you: Phaser's types say the callback returns nothing.

`Ball.isFalling()` returns `this.body!.velocity.y > 0`. A game object's `body` is typed as possibly `null` - it only has one once physics gives it one - hence the `!`: "I know this is not null".

Aiming the ball

If the ball simply bounced off the paddle like a wall, the player could never aim. Every breakout game instead sends the ball off at an angle that depends on **where** it hit:



`src/objects/Ball.ts`

```
public bounceOff(paddle: Paddle): void {  
    const halfWidth = paddle.displayWidth / 2;  
    const where = Phaser.Math.Clamp((this.x - paddle.x) / halfWidth, -1, 1);  
    this.setDirection(where * MAX_BOUNCE_ANGLE);  
}
```

```
// move at BALL_SPEED, at `degrees` from straight up (negative = to the
left)
private setDirection(degrees: number): void {
    const radians = Phaser.Math.DegToRad(degrees);
    this.setVelocity(Math.sin(radians) * BALL_SPEED, -Math.cos(radians) *
BALL_SPEED);
}
```

where `is` is -1 at the paddle's left end, 0 in the middle and 1 at the right (`Clamp` keeps it there when the ball catches a corner), and becomes an angle of up to 60 degrees from straight up. `sin` and `cos` split the speed into across and up, as in Chapter 2, so the ball always moves at exactly `BALL_SPEED`. This runs in the **collide** callback - *after* Phaser has bounced the ball - so it replaces the plain bounce.

Moving the paddle

`src/objects/Paddle.ts`

```
// the mouse: jump straight to the pointer, keeping the whole paddle on
screen
public moveTo(x: number): void {
    const halfWidth = this.displayWidth / 2;
    this.x = Phaser.Math.Clamp(x, halfWidth, this.scene.scale.width -
halfWidth);
}
```

The arrow keys move the paddle by velocity (`moveWithKeys`); the mouse sets `x` directly, so the paddle is exactly where the pointer is. That is safe: at the start of every step, a dynamic body takes its position from its game object.

Common mistakes

Note - "Cannot read properties of undefined (reading 'add')" as the game starts: the config has no `physics` section, so `this.physics` does not exist.

A body in the wrong place: the offset in `setOffset` and `setCircle` is measured from the picture's **top-left corner**, not its centre. Turn on `debug` and look.

Things bump that should pass through, or the other way round: a `collider` where you meant `overlap`, or the other way round.

Summary

- `Phaser.Geom` has shapes made only of numbers; `Phaser.Geom.Intersects` tests whether two of them overlap. `getBounds()` gives any game object's bounding rectangle
- rectangles are exact for boxy things and unfair to round ones: use circles for round things - and a circular `hitArea` for clicking them
- turn on Arcade Physics in the config; give game objects bodies with `this.physics.add.image / sprite`, or `this.physics.add.existing`. `debug: true` shows the bodies
- fit a body to its picture with `setSize + setOffset`, or `setCircle`
- `collider` separates and bounces; `overlap` only reports. Set them up once, in `create()`
- colliders work with single objects and whole groups: `staticGroup` for things that never move, `group` for things that do. Call `refreshBody()` after moving or scaling a static body
- callbacks get the two objects that touched (a single object before a group member); cast them with `as`
- a process callback decides whether a collision counts; `collider.active` switches one off for a while, `destroy()` for good; `disableBody` / `enableBody` switch one object's body off and on

Challenges

1. **Debug key** (*ch08_collect_and_avoid*) - Press D to turn the physics debug drawing on and off while the game runs, and show "DEBUG" in the corner while it is on. The game should start with it off, as now.
2. **Hover highlight** (*ch08_hand_made_collisions*) - When the pointer is over a shape - without clicking - give that shape a thick white outline. The test must be fair: over a circle's empty corner is not over the circle. Do the point-in-shape tests yourself, with `Contains`, rather than with `setInteractive`'s events.
3. **Tough bricks** (*ch08_breakout*) - Make the top row of bricks take two hits. After the first hit, a tough brick should change colour (or look cracked) so the player knows; the second hit removes it. Tough bricks are worth more points.
4. **Wide paddle** (*ch08_breakout*) - Some bricks, chosen at random, drop a power-up when they are hit. It falls down the screen; if the paddle catches it, the paddle is half as wide again for ten seconds. If the paddle misses it, it falls off the bottom and is gone. *Hint*: the power-up needs a body with a velocity, and an overlap with the paddle. A dynamic body follows its game object's scale by itself.
5. **Glass bricks** (*ch08_breakout*) - Make one row out of glass: the ball smashes glass bricks as it touches them, but does **not** bounce off them - it carries straight on. Every other brick works as before. *Hint*: a process callback that returns `false` stops the collision - but it can still do something first. Remember which object is which in the callback.
6. **Fair triangles** (*ch08_collect_and_avoid*) - The enemies' small square bodies are a compromise: some hits are missed, and some touches still count that should not. Make it exact. Keep the square body (the physics engine cannot do triangles), but only count a hit when the player's body really touches the red triangle. *Hint*: the triangle's corners in `enemy.png` are (24, 4), (44, 42) and (4, 42), and `Phaser.Geom` has a `Triangle` and an `Intersects.RectangleToTriangle`. Where does the extra test go so that a near miss does not cost a life? Turn debug on to check, and make the enemies' bodies big enough to cover the whole triangle.

Previous: [Chapter 7 - Animations](#) · Next: [Chapter 9 - 2D physics](#)



2D physics

Chapter 8 gave game objects **bodies**, and let Arcade Physics decide when they touch. This chapter is about everything else a physics engine does: moving bodies with velocity and acceleration, gravity, drag, bounce, mass and friction, turning, speed limits, and a world that wraps round at the edges. You will play with all of them in a sandbox, fly an Asteroids-style ship built from nothing but physics settings, and finish with a first look at **Matter.js** - Phaser's other physics engine, where crates really stack and topple.



What you will learn

- what Arcade Physics does to every body in every step, and why you never need `delta` when it moves things for you
- gravity, bounce, drag (linear and damping), maximum speed, mass and friction - and how to change them while a game runs
- the kinds of body - dynamic, immovable, not pushable, static - and who moves when two collide
- how to find out what a body is resting on (`blocked` , `touching`) and when it hits the edge of the world (`onWorldBounds`)

- turning with angular velocity and acceleration, flying the way you point with `velocityFromRotation`, and wrapping round the world with `wrap`
- the fixed time step, and pausing or slowing down the physics world
- Matter.js: real shapes that rotate and stack, and when to choose it instead of Arcade

The projects

Project	What it shows
<code>ch09_physics_playground</code>	balls and crates in an Arcade world, with keys that change gravity, bounce, drag, friction and time while it runs
<code>ch09_space_ship</code>	an Asteroids-style game: rotation, thrust, damping, a top speed, spinning rocks, bullets and a wrapping world
<code>ch09_matter_stack</code>	a Matter.js world: a tower and a pyramid of crates to knock down with cannonballs, shapes to drop, and bodies to drag with the mouse

Why let physics move things?

In Chapter 1 the ball moved itself. Every frame, its `preUpdate()` did the arithmetic:

`ch01_moving_ball/src/objects/Ball.ts`

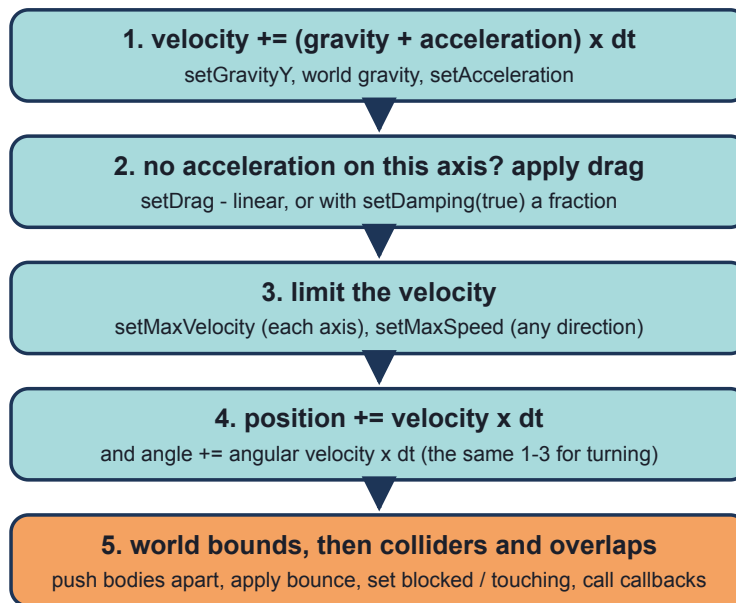
```
const seconds = delta / 1000;

// move: speed (pixels per second) x time (seconds) = distance (pixels)
this.x = this.x + this.speedX * seconds;
this.y = this.y + this.speedY * seconds;
```

Fine for one ball at a steady speed. But a ship that speeds up and drifts, a ball that falls faster and faster and bounces, a crate that slows to a stop - each is more arithmetic, on every object, every frame, and easy to get wrong. A physics body does it for you. You set **numbers** - a velocity, an acceleration, a drag, a bounce - and in every step Arcade Physics works out the rest:

What Arcade Physics does to each body, every step

dt = the length of one step in seconds (1/60 = 0.0167)



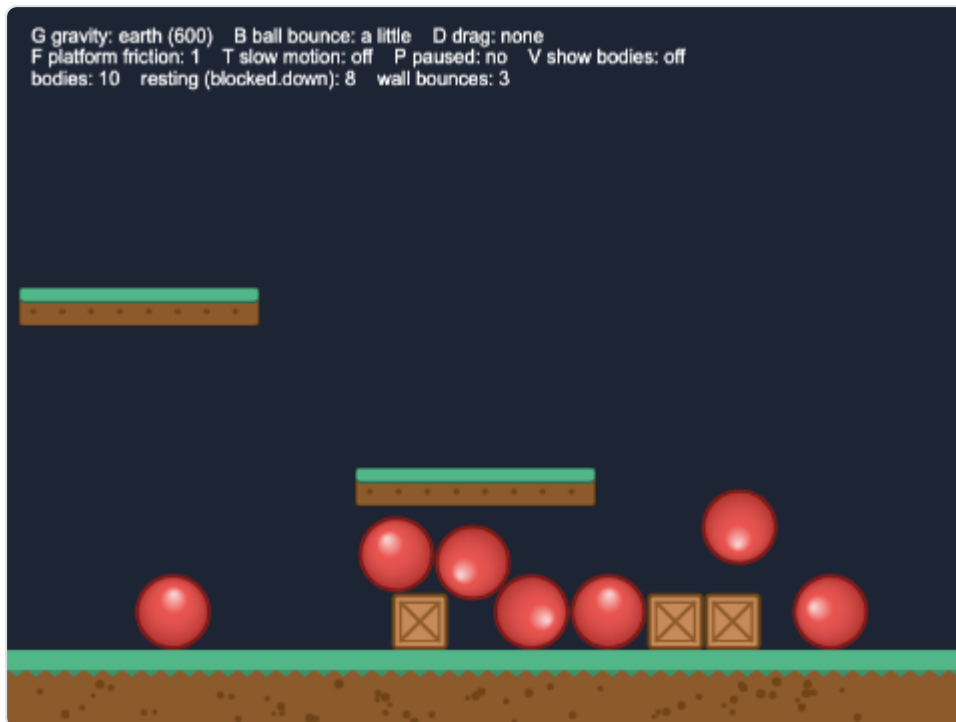
You set the numbers. Phaser does this arithmetic - with dt - for every body.
Then the game object is moved to where its body ended up.

- **velocity** is how fast something moves, and which way: pixels per second, across (x) and down (y). It changes the **position**
- **acceleration** is how fast the velocity changes: pixels per second, **per second**. It changes the **velocity**. Gravity is an acceleration that every body shares

`setVelocityX(200)` means "move right at 200 pixels per second";

`setAccelerationX(200)` means "go 200 pixels per second faster every second". Your code says what; Phaser does the `delta` maths.

The physics playground



`ch09_physics_playground` is a sandbox: a floor, a ledge, a sliding platform, and balls and crates. Click to drop a ball, C to drop a crate, and use the keys to change the laws of physics as it runs.

Gravity

Physics is switched on in the config, as in Chapter 8. This time the world has gravity:

```
src/main.ts
```

```

physics: {
  default: "arcade",
  arcade: {
    // pixels per second, per second: every second, a falling body goes 600
    px/s faster
    gravity: { x: 0, y: 600 },
    // physics steps per second (60 is the default - written here so you can
    see it)
    fps: 60,
    // debug drawing: ON, so that Phaser gets ready to draw every body - but
    the scene hides
    // it at the start, and V shows it
    debug: true,
  },
},

```

World gravity pulls on every dynamic body. G changes it while the game runs:

src/scenes/PlaygroundScene.ts

```

// G: the world's gravity - shared by every body (a body can add its own
with setGravityY)
keyboard.on("keydown-G", () => {
  this.gravityIndex = (this.gravityIndex + 1) % GRAVITY_CHOICES.length;
  this.physics.world.gravity.y = GRAVITY_CHOICES[this.gravityIndex].value;
});

```

`GRAVITY_CHOICES` is an array of `{ name, value }` objects (earth 600, jupiter 1500, off, moon 100). A body can also change the gravity it feels: `setGravityY(g)` **adds** `g` to the world's gravity for that body alone, and `body.setAllowGravity(false)` switches it off - as the moving platform does.

Bounce and the edges of the world

A body's **bounce** is how much of its speed it keeps when it hits something: 0 stops dead, 1 comes back at full speed, more than 1 gains speed. B cycles the balls through 0.5, 0.9, 1 and 0.

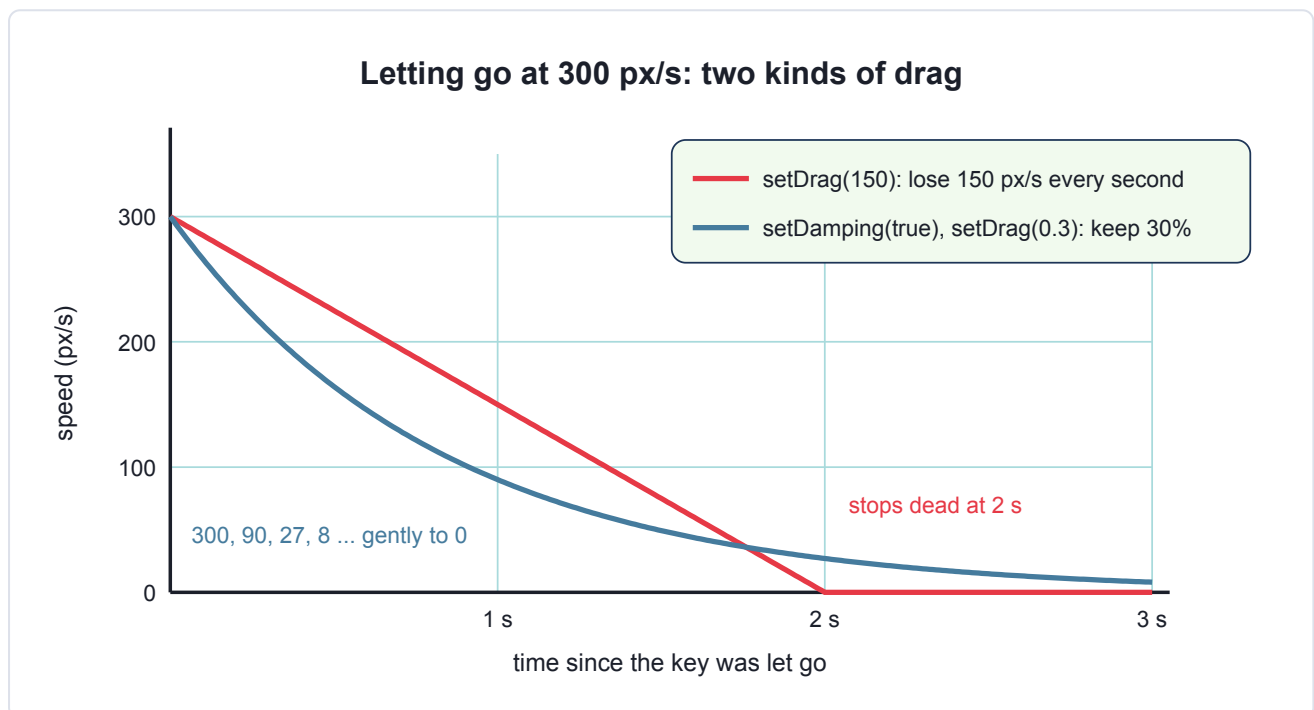
Balls bounce off other bodies, through the colliders, and off the **edges of the world**. The world starts as big as the game; the playground ends it at the top of the grass:

```
// the physics world starts as big as the game; stop it at the top of the
ground instead
this.physics.world.setBounds(0, 0, 800, FLOOR_Y);
```

`setCollideWorldBounds(true)` makes a body treat those edges as walls.

Drag, two ways

A ball pushed along an Arcade floor rolls for ever: there is no friction with the ground, no air. **Drag** slows things down, in two ways (D cycles through them):



- **linear drag** - `setDrag(150)` takes 150 pixels per second off the speed every second, until it reaches 0
- **damping** - `setDamping(true)` makes the drag number the **fraction of the speed kept** each second. `setDrag(0.3)` keeps 30%: fast things slow quickly, slow things gently, like a boat in water. It must be between 0 and 1, and *smaller* means *more* drag

`src/scenes/PlaygroundScene.ts`


```
private applySettings(thing: Phaser.Physics.Arcade.Image): void {
    const drag = DRAG_CHOICES[this.dragIndex];
    thing.setCollideWorldBounds(true);
    thing.setDamping(drag.damping);
    thing.setDrag(drag.value);
    if (thing instanceof Ball) {
        thing.setBounce(BOUNCE_CHOICES[this.bounceIndex].value);
    }
}
```

`instanceof` works as in Java: only balls get the bounce; crates keep the default, 0.

One rule catches everybody: **drag only works on an axis with no acceleration** - it is for slowing down once the engine is off. (Gravity does not count, so with damping a falling ball reaches a **terminal velocity**, like a skydiver.)

Mass: who gets pushed

When two dynamic bodies collide, Arcade shares the push by their **mass**. The default is 1; `addCrate()` calls `crate.setMass(CRATE_MASS)`, with `CRATE_MASS = 3`. A ball hardly moves a crate; a crate knocks a ball aside. Against anything that cannot move, mass makes no difference.

A trap: groups reset their members

Look at the order in `addBall()`:

```
private addBall(x: number, y: number): void {
    const ball = new Ball(this, x, y);
    // add to the group FIRST: a physics group resets the bodies it is given
    // to its own defaults
    // (no bounce, no velocity...), so anything set before this line would be
    // lost
    this.things.add(ball);
    bodyOf(ball).onWorldBounds = true;
    ball.setVelocityX(Phaser.Math.Between(-LAUNCH_SPEED, LAUNCH_SPEED));
    this.applySettings(ball);
}
```

In Chapter 8 a group's config set **defaults** for its members. The group applies them to every body added to it - and with no config, the defaults are velocity 0, bounce 0, drag 0, mass 1,

no world bounds. Set a ball moving, then add it to the group, and it stops - with no error or warning. So: **add to the group first, then set the body up.** (The circle body from `Ball`'s constructor survives: shape is not a group default.)

`body0f()` is a helper at the top of the scene. A game object's `body` is typed "dynamic, static, Matter, or `null`"; every body here is dynamic, so `body0f()` says so with `as`, once.

Immovable, static, pushable - and friction

Here is the whole family of bodies, and what happens when a normal dynamic body runs into each:

Body	Moves itself?	Pushed by collisions?	Made with
dynamic	yes - velocity, gravity, drag	yes, shared by mass	<code>physics.add.image</code> , <code>add.existing(obj)</code>
not pushable	yes	no - it pushes back, and the other body takes all the bounce	<code>setPushable(false)</code>
immovable	yes, if you give it a velocity	never	<code>setImmovable(true)</code>
static	never	never; the cheapest body of all	<code>physics.add.staticImage</code> , <code>staticGroup</code>

The ledge on the left is **static**. The platform is **immovable**: it moves by its own velocity, but nothing can push it about:

`src/objects/MovingPlatform.ts`

```
this.setImmovable(true);           // collisions never push it

// the Image has no setAllowGravity() - the body itself does.
physics.add.existing() made a
// dynamic body, but Phaser's types cannot know which kind, so `as` says so
const body = this.body as Phaser.Physics.Arcade.Body;
body.setAllowGravity(false);        // it floats
this.setVelocityX(SPEED);
// turn round at the edges of the world: bounce 1 keeps all of its speed
this.setCollideWorldBounds(true, 1, 1);
```

Drop a crate on it and the crate rides along. That is **friction** - and in Arcade, friction belongs to the *platform*: how much of its movement it passes to a rider. 1 (the default) carries it; 0 slides out from under it. F switches between them:

```
// F: friction belongs to the platform - it is how much of ITS movement it passes on
keyboard.on("keydown-F", () => {
  this.platformFriction = this.platformFriction === 1 ? 0 : 1;
  this.platform.setFriction(this.platformFriction, 0);
});
```

That is all Arcade friction does - not the grip of crate on crate (that needs Matter). It is also why crates have no bounce: a bouncing crate would never be carried.

What is it touching?

After every step, each body has two sets of flags - `up`, `down`, `left`, `right` and `none` :

- `body.blocked` - pushed against something that cannot give way: the edge of the world, a static or immovable body, or a body that is itself blocked
- `body.touching` - touching another body (of any kind) on that side

The playground's `update()` counts the bodies with `blocked.down` - resting on something solid. It is every platform game's question: "is the player standing on something?" (Chapter 15). The flags change every step, so check them in `update()`, not once in `create()`.

To hear about the world's edges, a body must **opt in**: the balls set `onWorldBounds = true`, and the world emits `"worldbounds"` whenever one hits an edge:

```

this.physics.world.on(
  Phaser.Physics.Arcade.Events.WORLD_BOUNDS,
  (body: Phaser.Physics.Arcade.Body, _up: boolean, down: boolean) => {
    // the floor does not count, and neither does a ball just leaning on a
    wall (it would
    // "hit" it every step): only a real bounce, that sends the ball back
    with some speed
    if (!down && body.velocity.length() > BOUNCE_SPEED) {
      this.wallBounces = this.wallBounces + 1;
      this.flash(body.gameObject as Phaser.Physics.Arcade.Image); // only
      images have onWorldBounds on
    }
  },
);

```

The listener gets the **body** (`body.gameObject` leads back to the ball) and a boolean per edge. Note the comment: the event fires for every *step* a body is against the edge, not just the first.

Pausing and slowing the world

The physics world has its own clock, separate from the game's:

```

// T: the physics clock runs slower - but the game loop, and everything
else, does not
keyboard.on("keydown-T", () => {
  const world = this.physics.world;
  world.timeScale = world.timeScale === 1 ? SLOW_MOTION : 1;
});

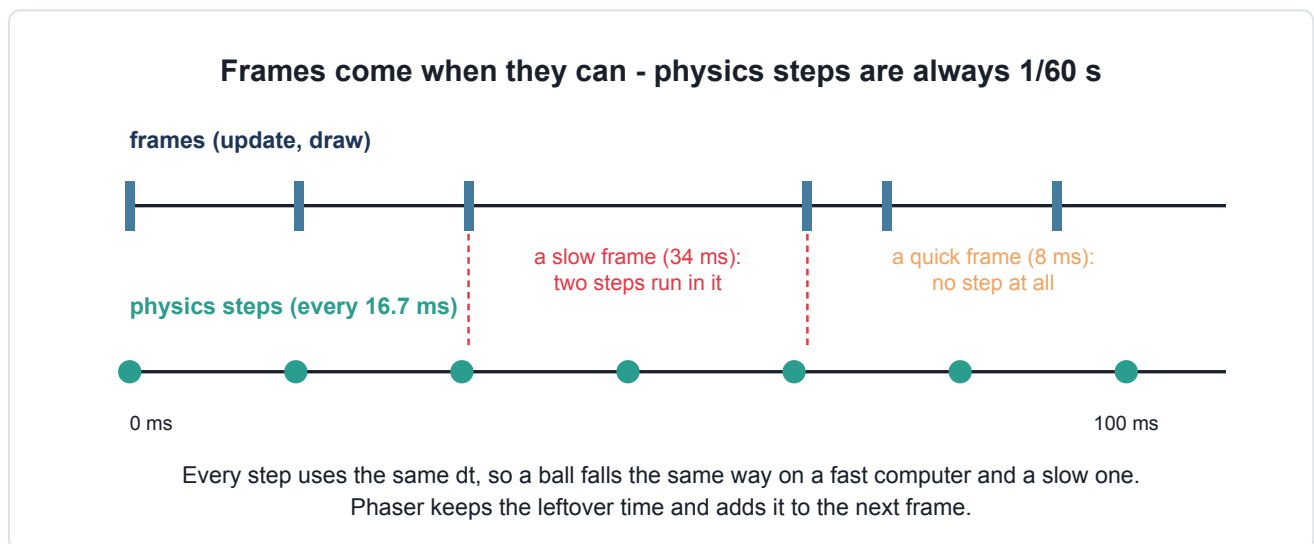
// P: freeze the physics world. Nothing moves, no colliders run - but the
scene still
// updates, draws, and listens to keys
keyboard.on("keydown-P", () => {
  if (this.physics.world.isPaused) {
    this.physics.resume();
  } else {
    this.physics.pause();
  }
});

```

`timeScale` is backwards: **bigger is slower** (2 is half speed). `this.physics.pause()` freezes every body, but unlike Chapter 4's `this.scene.pause()` the scene's `update()`, timers and tweens carry on - handy for a game-over screen that freezes the action (the space ship does this).

The fixed time step

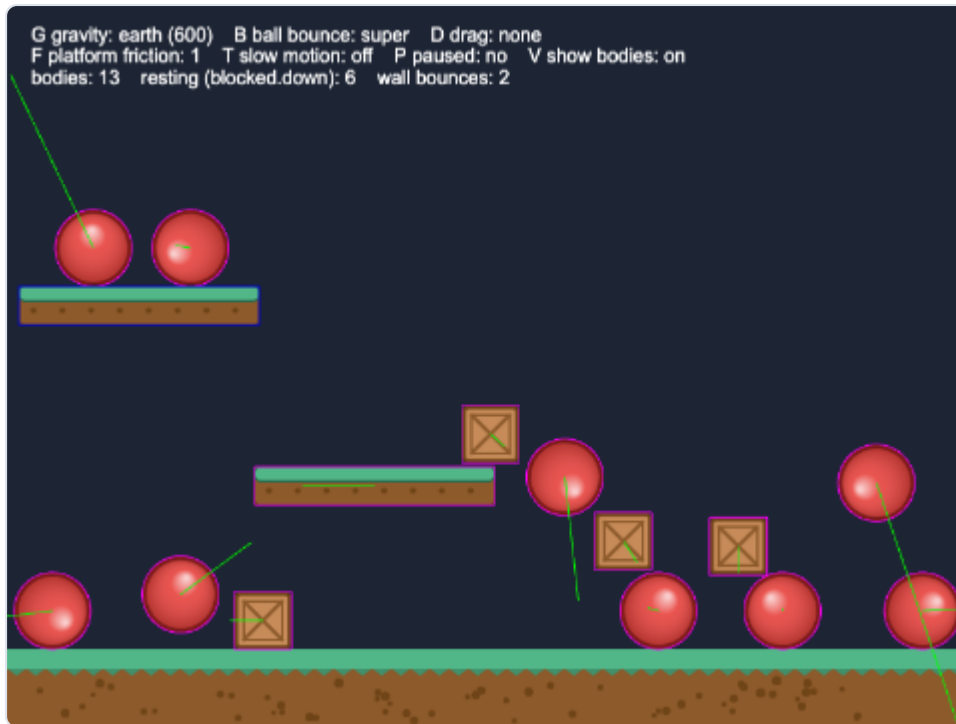
Frames do not arrive exactly every 16.7 milliseconds: one might take 8, the next 34. Moving bodies by `velocity x delta` once per frame, a slow frame would be one big jump - big enough to carry a fast ball through a thin wall - and the game would behave differently on every computer. So Arcade runs on a **fixed time step**: it moves the world in steps of exactly $1/60$ s (`fps: 60`):



A slow frame gets two steps, a quick one none. Every step is the same size, so the same throw lands in the same place on any computer. You never see this - but it is why a 120 Hz screen draws some frames where nothing has moved, and why `fps: 120` makes collisions of very fast things more reliable (for twice the work).

Seeing the bodies

Press V: pink outlines for dynamic bodies, blue for static, green lines for velocity.



```
// V: debug drawing on and off. The config switches debug on (so Phaser
// draws with, and gets every body ready to be drawn), and create() switches
// the drawing off.
private toggleDebug(): void {
    const world = this.physics.world;
    world.drawDebug = !world.drawDebug;
    world.debugGraphic.clear();           // or the last drawing would stay on
screen
}
```

Watch a ball roll. The picture turns - `Ball` 's `preUpdate()` sets an angular velocity of $\text{speed} / \text{radius}$ - but its body does not. **Arcade bodies never rotate.** A circle does not mind; a box does. That is Arcade's biggest limit, and the reason Matter.js exists.

The space ship

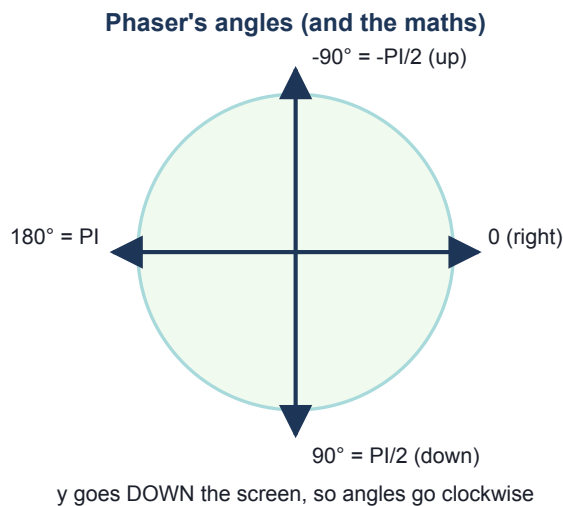


`ch09_space_ship` is a small Asteroids: LEFT and RIGHT turn, UP thrusts, SPACE fires. There is no gravity, and the ship's code never sets its own position or angle - only accelerations.

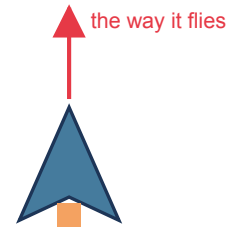
Which way is it pointing?

Phaser measures angles from **pointing right**, clockwise (`rotation` in radians, `angle` in degrees). But `player_ship.png` points **up** - so at rotation 0 the ship faces -90 degrees:

Which way is angle 0?



The ship's picture points up



drawn like this, its rotation is 0
- but it faces $-\pi/2$ (up)

facing = rotation - Math.PI / 2

Forget this and the ship flies sideways. Draw pictures pointing right, or - as here - subtract a quarter turn whenever a rotation becomes a direction.

Turning

Turning works like moving: **angular velocity** (degrees per second) turns the picture, **angular acceleration** changes it, **angular drag** slows it, and `maxAngular` caps it:

src/objects/Ship.ts

```
// turning works just like moving: an angular acceleration, a drag, and a maximum
this.setAngularDrag(TURN_DRAG);
this.arcadeBody.maxAngular = MAX_TURN;
```

```
// -1 turns left (anticlockwise), 1 turns right, 0 lets the turn die away
public turn(direction: number): void {
    this.setAngularAcceleration(direction * TURN_ACCELERATION);
}
```

The scene calls `this.ship.turn(left + right)`, with -1 for LEFT and 1 for RIGHT. Full turning speed comes in under a fifth of a second: quick, but with a little weight. (`arcadeBody` is the body, cast once in the constructor: `maxAngular` and `setMaxSpeed` live on the body, not the image.)

Thrust: flying the way you point

```
public thrust(on: boolean): void {
    if (on) {
        // the picture points UP when rotation is 0, but Phaser's angles start
        // pointing RIGHT
        // (0 = right, PI/2 = down). A quarter turn back puts them in line.
        const facing = this.rotation - Math.PI / 2;
        // write the result straight into the body's acceleration - no new
        // vector needed
        this.scene.physics.velocityFromRotation(facing, THRUST,
        this.arcadeBody.acceleration);
    } else {
        // no acceleration: now (and only now) drag slows the ship down
        this.setAcceleration(0);
    }
}
```

`velocityFromRotation(angle, speed, vector)` turns "this fast, that way" into an x and a y - the `cos` and `sin` trick from Chapter 2 - and writes them into `vector`. Despite its name, here the vector is the ship's **acceleration**: it speeds up the way it faces. Turn round and thrust, and you slow, stop and fly back. (`velocityFromAngle` is the same in degrees.)

Drifting, and a top speed

```
// damping: drag as a fraction of the speed kept each second, not pixels per
// second taken away
this.setDamping(true);
this.setDrag(DAMPING);
// a true speed limit. setMaxVelocity(x, y) limits each axis on its own, so
// going
// diagonally you could reach 1.4 times the limit
this.arcadeBody.setMaxSpeed(MAX_SPEED);
```

A coasting ship keeps 60% of its speed each second: it drifts, but does stop. Hold UP and the speed on screen climbs to exactly 350, and no further. `setMaxVelocity(x, y)` suits a platform game, where running and falling need different limits - but it limits x and y **separately**, so a diagonal could reach 424. `setMaxSpeed` limits the real speed, whichever way it points.

Wrapping round the world

Fly off the right edge, and you come back on the left:

src/scenes/SpaceScene.ts

```
// off one edge, on at the other. wrap() works with game objects and whole groups  
this.physics.world.wrap(this.ship, SHIP_WRAP);  
this.physics.world.wrap(this.rocks, ROCK_WRAP);  
this.physics.world.wrap(this.bullets, BULLET_WRAP);
```

`wrap(object, padding)` moves anything past one edge of the world to the opposite edge. The **padding** - about half its size - lets it slide off screen before it reappears, instead of popping. It is a check, so it runs in `update()`, every frame.

Rocks

src/objects/Rock.ts

```
public launch(): void {  
    // a random direction and speed, turned into an x and y velocity  
    const angle = Phaser.Math.Between(0, 359);  
    const speed = Phaser.Math.Between(MIN_SPEED, MAX_SPEED);  
    this.scene.physics.velocityFromAngle(angle, speed, (this.body as  
    Phaser.Physics.Arcade.Body).velocity);  
  
    // spin: angular velocity turns the PICTURE. The body is a circle, so it  
    does not matter  
    this.setAngularVelocity(Phaser.Math.Between(-MAX_SPIN, MAX_SPIN));  
  
    // rocks bounce off each other without losing any speed  
    this.setBounce(1);  
}
```

`launch()` is not in the constructor because of the group trap: the scene adds each rock to its group, then launches it. A collider between the rocks group and itself makes rocks bounce off each other, trading speeds rather like snooker balls.

Bullets

A bullet leaves at `BULLET_SPEED` the way the ship faces - **plus the ship's own velocity**, or a fast ship would overtake its bullets:

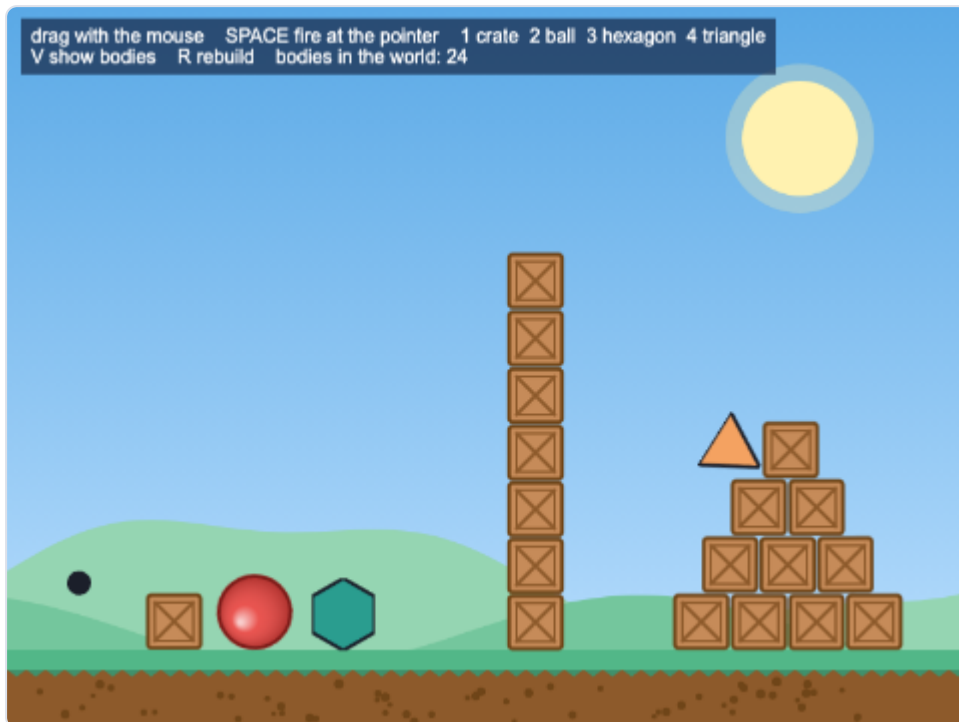
```
// velocity = the ship's own velocity + BULLET_SPEED in the direction the ship faces
const body = bullet.body as Phaser.Physics.Arcade.Body; // a physics group makes dynamic bodies
this.physics.velocityFromRotation(this.ship.rotation - Math.PI / 2,
BULLET_SPEED, body.velocity);
body.velocity.add((this.ship.body as Phaser.Physics.Arcade.Body).velocity);
```

`body.velocity` is a `Phaser.Math.Vector2`, with `add`, `scale`, `length` and more. A `delayedCall` destroys each bullet after `BULLET_LIFE` milliseconds.

Losing a life, and game over

Ship and rocks use an `overlap` with a process callback that ignores hits while a new ship blinks, and `disableBody` / `enableBody` for losing a life (both from Chapter 8). When the last life goes, `endGame()` calls `this.physics.pause()`: the rocks freeze, but the scene runs on, so R still restarts it - and `init()` resets the score (Chapter 2's trap).

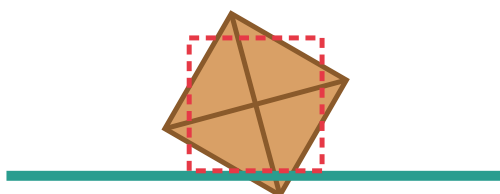
Matter.js



Arcade is fast because it is simple: upright boxes and circles. For most 2D games that is enough. But a tower of crates does not work: boxes cannot tip, and piles jitter and sink. Phaser's second engine, **Matter.js**, is a real **rigid-body** engine: any convex shape, rotation, friction between bodies, and stacks that stand and topple.

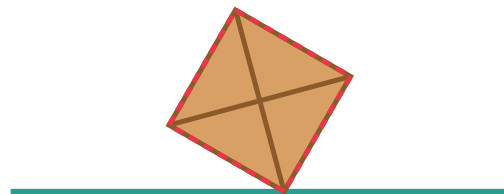
The same tilted crate, in each engine

Arcade Physics



the picture turns - the body (red) does not:
always an upright box, or a circle
it cannot tip over, and its corners poke through

Matter.js



the body turns with the picture: any angle,
any convex shape - so it rests on its corner
and topples over, as a real crate would

Turning Matter on

```
ch09_matter_stack/src/main.ts
```

```

physics: {
  default: "matter",
  matter: {
    // Matter's gravity is not in pixels per second: y: 1 is "normal"
    gravity
    gravity: { x: 0, y: 1 },
    // debug drawing: ON, so that Phaser gets ready to draw every body - but
    the scene hides
    // it at the start, and V shows it
    debug: true,
  },
},

```

Scenes now have `this.matter` instead of `this.physics`. A game normally uses one engine or the other.

Matter game objects

`src/objects/Crate.ts`

```

export class Crate extends Phaser.Physics.Matter.Image {
  constructor(scene: Phaser.Scene, x: number, y: number) {
    super(scene.matter.world, x, y, CRATE_KEY, undefined, {
      friction: 0.6, // wood on wood: grippy, so a stack stays up (0
= ice, 1 = rubber)
      restitution: 0.05, // bounce: crates hardly bounce at all
    });
    scene.add.existing(this);
  }
}

```

Two differences from Arcade: the constructor takes the **Matter world**, and the body is made straight away from the last argument, an object literal of settings (`undefined` is the frame). `this.matter.add.image(x, y, key, frame, options)` does the same without a class.

Restitution is Matter's word for bounce. **Friction** is real friction: how much two surfaces grip. With friction near 0 the tower slides apart at a touch. Without a `shape`, the body is a rectangle the size of the picture; other shapes are asked for by name:

`src/objects/Ball.ts`

```
super(scene.matter.world, x, y, BALL_KEY, undefined, {
  shape: { type: "circle", radius: RADIUS },
  restitution: 0.7,      // bouncy: keeps 70% of its speed after a hit
  friction: 0.05,
});
```

Polygon asks for `{ type: "polygon", sides: sides, radius: RADIUS }`, and - there being no hexagon in the asset library - draws its own picture with `Graphics` and `generateTexture`, corners exactly where Matter puts them. Press V: the green outlines are Matter's shapes, turning with the pictures.

A static ground, and walls

```
// the ground: a Matter image whose body is STATIC - it never moves,
// whatever hits it
this.matter.add.image(400, 600 - 32, GROUND_KEY, undefined, { isStatic: true
});

// walls at the left and right edges (none at the top or bottom - the ground
// is the bottom).
// setBounds(x, y, width, height, thickness, left, right, top, bottom)
this.matter.world.setBounds(0, -400, 800, 1000, 64, true, true, false,
false);
```

Matter has no "collide with world bounds": `setBounds` builds real static walls just outside the area you give.

Heavy, and fast: density and velocity

`src/objects/Cannonball.ts`

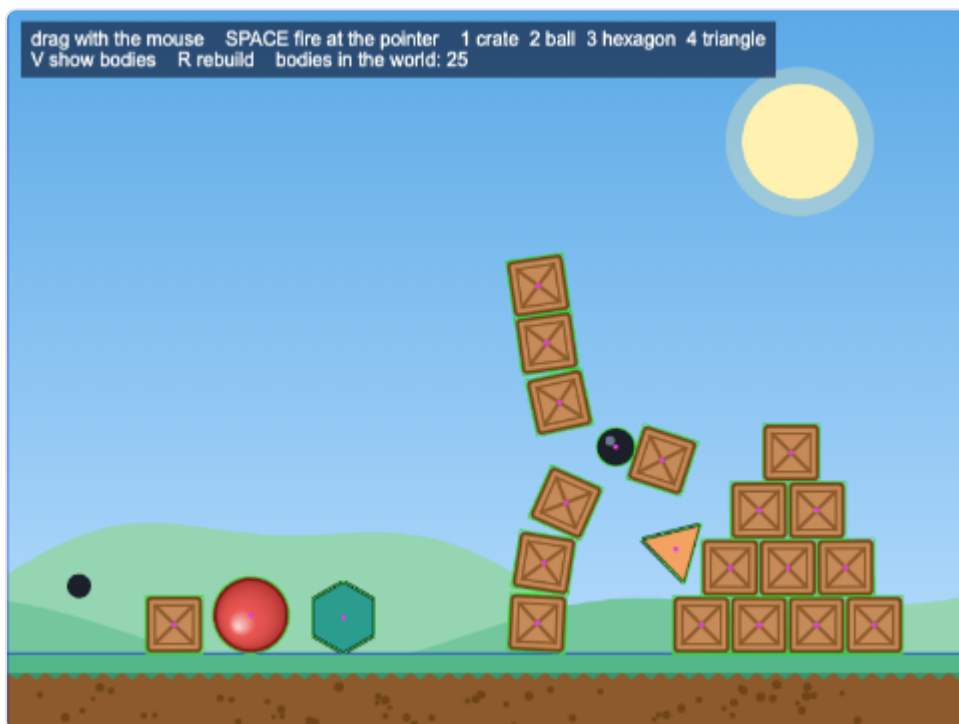
```
super(scene.matter.world, x, y, CANNONBALL_KEY, undefined, {
  shape: { type: "circle", radius: RADIUS },
  density: 0.01,
  restitution: 0.2,
});
```

Matter works out **mass** from area and **density** (default 0.001). The cannonball is small but ten times as dense, so the tower moves, not the cannonball.

src/scenes/StackScene.ts

```
// aim at the pointer: the angle from the cannon to the pointer, turned into  
an x and y speed  
const angle = Phaser.Math.Angle.Between(CANNON_X, CANNON_Y, pointer.worldX,  
pointer.worldY);  
cannonball.setVelocity(Math.cos(angle) * CANNON_SPEED, Math.sin(angle) *  
CANNON_SPEED);
```

`CANNON_SPEED` is 22 - because **Matter's velocities are pixels per step**, not per second (22 is about 1,300 px/s). Its gravity and forces have their own units too; Arcade numbers go very wrong.



Dragging with the mouse

One line:

```
// drag any body with the mouse or a finger: a springy "constraint" joins  
the body to the  
// pointer while the button is held  
this.matter.add.mouseSpring({ stiffness: 0.2 });
```

A **constraint** is Matter's joint: it holds two bodies, or a body and a point, together. The mouse spring joins the pointer to the body you press on; being a spring, the body swings

and knocks things over on its way. Chains, pendulums and wheels are constraints too (`this.matter.add.constraint(bodyA, bodyB, length, stiffness)`).

Collision callbacks

In Matter everything collides with everything - no colliders. To *hear* about a collision, give a body a callback:

```
// a thud, the first time it hits something
let hit = false;
cannonball.setOnCollide(() => {
  if (!hit) {
    hit = true;
    this.sound.play(HIT_KEY, { volume: 0.6 });
  }
});
```

`this.matter.world.on("collisionstart", ...)` hears every collision in the world.

Arcade or Matter?

	Arcade Physics	Matter.js
shapes	upright rectangles and circles	circles, rectangles, any convex polygon, shapes made of several parts
rotation	the picture turns; the body never does	bodies rotate, tumble and topple
stacking	poor: piles jitter and sink	good: towers stand until knocked over
units	pixels per second	pixels per step; gravity 1; tiny forces
collisions	only what you ask for, with <code>collider</code> and <code>overlap</code>	everything with everything, filtered by categories
speed	very fast: thousands of bodies	slower: hundreds of bodies
use it for	platformers, shooters, top-down games - most 2D games	physics puzzles, stacking, ragdolls, vehicles, pinball

Use Arcade unless the game is *about* the physics. The rest of this guide uses Arcade.

Common mistakes

Note - The group ate my velocity. Adding a body to a physics group resets it to the group's defaults. Add it to the group first, then set it up.

Note - Drag does nothing. Drag only slows an axis with no acceleration. Set it to 0 when the key is let go. And with damping, `setDrag(0.99)` is gentle; `setDrag(0.01)` is a brick wall.

Note - The ship flies sideways. The picture points up but angle 0 is right. Subtract `Math.PI / 2` (or draw the picture pointing right).

Note - Property `'setAllowGravity'` does not exist on type `...`. Some settings live only on the body: `(this.body as Phaser.Physics.Arcade.Body).setAllowGravity(false)`.

Note - Everything in Matter flies off the screen. Matter velocities are per step and its forces are tiny: 5 is a brisk velocity, 0.05 a strong force. Arcade numbers are far too big.

Summary

- a body moves itself: you set velocity (px/s) and acceleration (px/s per second); gravity is an acceleration every body shares. No `delta` maths in your code
- `setBounce`, `setCollideWorldBounds` and `world.setBounds` for bouncing off the world's edges
- `setDrag` takes speed away; with `setDamping(true)` it keeps a fraction. Drag needs zero acceleration. `setMaxVelocity` limits each axis, `setMaxSpeed` the real speed
- mass shares the push between dynamic bodies; immovable bodies are never pushed; static ones never move. Arcade friction is how much a moving immovable body carries its riders

- `body.blocked` / `body.touching` say what a body is up against; `onWorldBounds` and `"worldbounds"` report the world's edges
- angular velocity, acceleration and drag turn things; `velocityFromRotation` points a vector the way something faces; `world.wrap` wraps it round the world
- physics runs in fixed steps, and can be paused or slowed (`timeScale`)
- a physics group resets the bodies added to it: add first, then set up
- Matter.js: real shapes that rotate and stack, friction, density and constraints - in its own units. Use Arcade unless the physics is the game

Challenges

1. **Gravity switch** (*ch09_physics_playground*) - Make the arrow keys choose which way gravity pulls - down, up, left or right - at the strength G has chosen. Everything should pile up against that edge of the world. Show the direction on the screen.
2. **Bouncy and dead** (*ch09_physics_playground*) - Add two new kinds of ball. Key 1 drops a light blue super-ball (`ball_blue.png` from the asset library) that bounces perfectly and has half the normal mass; key 2 drops a heavy, dead grey ball (the red ball, tinted) with no bounce and five times the mass. They keep their own bounce whatever B is set to. Drop a heavy ball onto a pile of light ones.
3. **Brakes and reverse** (*ch09_space_ship*) - Holding DOWN fires the ship's retro-rockets: half the thrust, backwards. Holding SHIFT puts the brakes on: the ship slows to a stop in about half a second, far faster than it drifts. Show "BRAKES" on the screen while they are on.
4. **Rocks that split** (*ch09_space_ship*) - Shooting a big rock breaks it into two medium rocks, and a medium rock into two small ones; only small rocks are destroyed. Smaller rocks move faster and are worth more points. A new wave only starts when every piece is gone. *Hint:* give `Rock` a size. `setScale` changes the body too. The two halves should fly apart - at an angle either side of the direction the bullet was going, say - and remember the group trap.
5. **Demolition** (*ch09_matter_stack*) - Turn the stack into a game: five cannonballs to knock down as many crates as possible. A crate counts as down once it has tipped more than 30 degrees or dropped more than half a crate below where it started. Show "Crates down" and "Shots left", and when the last shot has been fired and a few seconds have

passed, show the final score. *Hint:* keep the crates in an array, and remember each one's starting `y`. `crate.angle` is in degrees, from -180 to 180.

6. **Pinball flipper** (*ch09_matter_stack*) - Add a flipper near the bottom left: a long thin Matter body that swings up round a pin at one end while Z is held, and falls back when it is let go - hard enough to flick a ball up into the tower. Drop balls onto it with 2. *Hint:* a `this.matter.add.worldConstraint(body, 0, 1, { pointA, pointB })` pins a point of a body to a point in the world; `pointB` is measured from the body's centre. `setAngularVelocity` spins a Matter body (in radians per step). Something must stop the flipper swinging all the way round: two small static bodies, or a check of its angle every frame.

Previous: [Chapter 8 - Collisions](#) · Next: [Chapter 10 - Tilemaps](#)

Tilemaps

Most 2D games with a world to walk round - dungeons, islands, platform levels, puzzle boards - are built from **tiles**: small square pictures laid out on a grid. In this chapter you build tile worlds in code: a map written as a 2D array of numbers, a tile painter that changes the map while the game runs, and a random cave far bigger than the screen, with a camera that follows the player and a minimap in the corner.



What you will learn

- why games use tiles, and what a **tileset** and a **tile index** are
- how to turn a 2D array into a map with `this.make.tilemap`, `addTilesetImage` and `createLayer`
- how to make some tiles solid with `setCollision`, and stop an Arcade Physics sprite walking into them
- how to find the tile at a point, and convert between pixels and tile columns and rows
- how to change the map while the game runs, with `putTileAt`, `removeTileAt` and `putTilesAt`, and what an empty tile (`-1`) is
- how to make a world bigger than the screen: camera bounds, `startFollow`, and a second camera as a minimap

- the difference between free movement and grid movement

The projects

Project	What it shows
<code>ch10_array_map</code>	a top-down map from a 2D array; a player who cannot walk into walls or water
<code>ch10_tilePainter</code>	click to paint tiles; number keys choose the tile; save and load in <code>localStorage</code> ; print the map as code
<code>ch10_scrolling_world</code>	a 60 x 40 random cave made in code, two layers, a camera that follows the player, and a minimap

Chapter 11 does the same things with maps drawn in [Tiled](#), a free map editor. Everything in this chapter still applies there - Tiled just writes the numbers for you.

Why tiles?

Imagine drawing a level as one big picture. A world 60 tiles by 40 is 1920 x 1280 pixels - about 10 MB of graphics memory for one level, and every change means repainting it. Worse, the game has no idea what is *in* the picture: which pixels are wall, which are water.

Tiles fix all three problems:

- **memory:** the pictures are drawn once, in a small **tileset** image, and reused as often as you like. `simple_tiles.png` , which the first two projects use, is 128 x 32 pixels - four tiles
- **level design:** a level is just a grid of numbers. Change a number, and the level changes. A new level is a new grid, not a new picture
- **collision and rules:** the game knows exactly what is at every place, because it knows the number there. "Is this a wall?" is a question about a number, not about pixels

A tileset is one picture; a map is a grid of numbers that pick from it

simple_tiles.png - 128 x 32 pixels, four 32 x 32 tiles



0

grass



1

water



2

wall



3

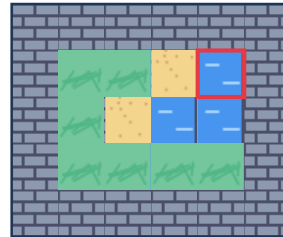
sand

each tile's INDEX is its place in the picture, counting from 0 - left to right, then row by row in a taller tileset

-1 means "no tile": nothing is drawn

```
const LEVEL: number[][] = [  
  [2, 2, 2, 2, 2, 2],  
  [2, 0, 0, 3, 1, 2],  
  [2, 0, 3, 1, 1, 2],  
  [2, 0, 0, 0, 0, 2],  
  [2, 2, 2, 2, 2, 2],  
];
```

make.tilemap



LEVEL[1][4] is 1:
row 1, column 4
is water

rows first: LEVEL[row][column] - the array looks like the map

Each tile in a tileset has an **index**: its place in the picture, counting from 0, left to right and then row by row. The asset library has three tilesets, all with 32 x 32 tiles (the full lists are in `assets/README.md`):

Tileset	Tiles	Good for
<code>simple_tiles.png</code>	4: 0 grass, 1 water, 2 wall, 3 sand	a first map; this chapter's first two projects
<code>dungeon_tiles.png</code>	16: floors, walls, doors, stairs, water, a void...	top-down dungeons; <code>ch10_scrolling_world</code>
<code>platform_tiles.png</code>	16: grass tops, dirt, bricks, spikes, ladders...	side-on platform games (Chapters 11 and 15)

A map from an array

The level is data

`ch10_array_map` keeps its map in a file of its own:

`src/level.ts`

```

export const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
  [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 0, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 3, 3, 1, 3, 3, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
  ...,
  [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
];

// where the player starts: a column and a row, counted in TILES from the
// top left (0, 0)
export const START_COLUMN = 7;
export const START_ROW = 7;

```

`number[][]` is an array of arrays of numbers - Java's `int[][]`. There is one inner array for each **row**, top to bottom, and one number for each **column**, left to right. Because it is written row by row, the code looks like the map: you can see the walls round the edge (2), the pond with its sandy beach (3 round a 1), and the lake. The map is 25 columns by 18 rows; at 32 pixels a tile that is 800 x 576 pixels, leaving a 24-pixel strip at the bottom of the game for a line of text.

What the numbers mean lives in another small file:

`src/tiles.ts`

```

export const TILES_KEY = "tiles";
export const TILES_FILE = "assets/tilesets/simple_tiles.png";

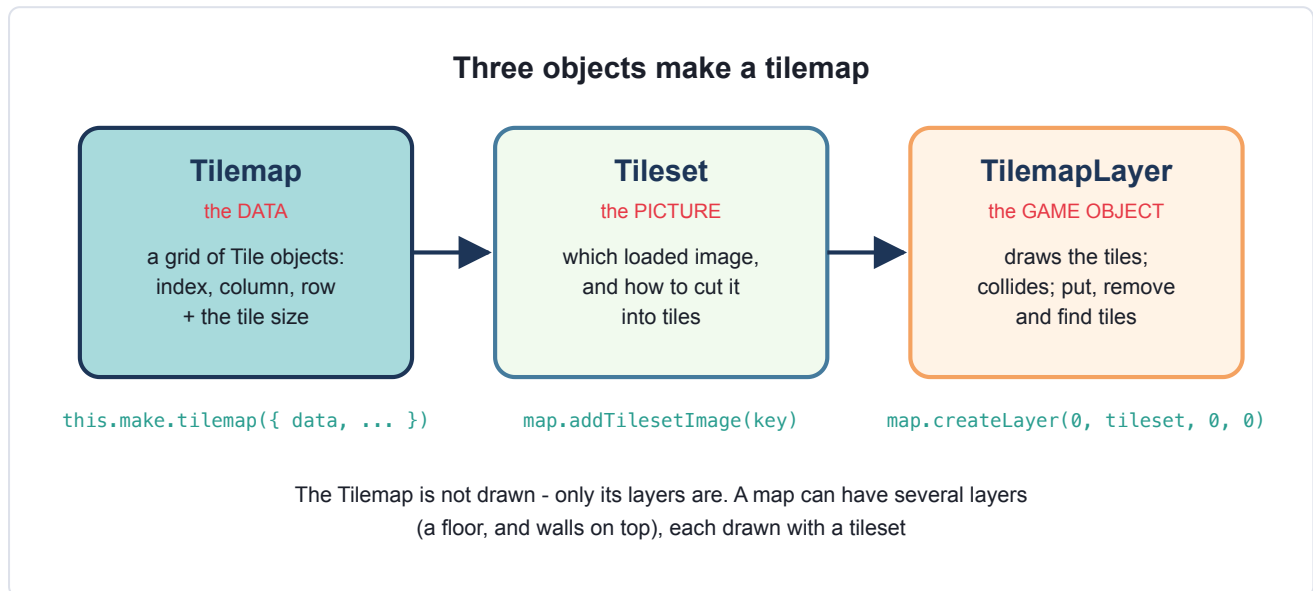
export const TILE_SIZE = 32;

export const GRASS = 0;
export const WATER = 1;
export const WALL = 2;
export const SAND = 3;

```

Note - rows first. `LEVEL[row][column]` - that is, `LEVEL[y][x]` - is the tile at a place. Almost everyone writes `LEVEL[x][y]` once. With a square map the mistake can hide for a while; with this 25 x 18 map it gives `undefined` as soon as `x` passes 17.

Three objects make a tilemap



A tileset is loaded like any other picture - Phaser cuts it into tiles later:

`src/scenes/GameScene.ts`

```
preload(): void {  
    // a tileset is loaded as a plain image - Phaser cuts it into tiles later  
    this.load.image(TILES_KEY, TILES_FILE);  
    this.load.spritesheet(HERO_KEY, HERO_FILE, { frameWidth: HERO_FRAME_SIZE,  
    frameHeight: HERO_FRAME_SIZE });  
}
```

Then `create()` builds the map in five steps:


```

create(): void {
    // 1. the map: the data, and the size of one tile in pixels
    const map = this.make.tilemap({ data: LEVEL, tileWidth: TILE_SIZE,
    tileHeight: TILE_SIZE });

    // 2. the tileset: which loaded image the tile indexes refer to
    const tileset = map.addTilessetImage(TILES_KEY)!;

    // 3. the layer: the game object that draws the tiles. A map made from an
    array has one layer, 0.
    // createLayer can make a TilemapLayer or (if asked) a GPU-only
    TilemapGPULayer, so its return
    // type is "either"; we did not ask for a GPU layer, so we tell
    TypeScript which one it is
    this.layer = map.createLayer(0, tileset, 0, 0) as
    Phaser.Tilemaps.TilemapLayer;

    // 4. collision: tiles with these indexes block physics bodies
    this.layer.setCollision([WATER, WALL]);

    // the player starts in the middle of a tile. tileToWorldXY gives the
    tile's TOP LEFT corner
    const start = map.tileToWorldXY(START_COLUMN, START_ROW)!;
    this.player = new Player(this, start.x + TILE_SIZE / 2, start.y +
    TILE_SIZE / 2);

    // 5. the player bumps into the colliding tiles
    this.physics.add.collider(this.player, this.layer);
    ...
}

```

1. `this.make.tilemap(...)` makes a `Tilemap`: the **data**. It turns every number in the array into a `Tile` object that knows its index, its column and its row. A `Tilemap` is *not* drawn - it is a model, not a view. (`this.add.tilemap(...)` makes one too, taking its settings as separate arguments instead of an object; a `Tilemap` is never on the display list, whichever you use)
2. `map.addTilessetImage(name, key)` makes a `Tilesset`: which loaded image the numbers refer to, and how to cut it up. The first argument is the tileset's *name* and the second is the image's key; leave the key out and the name is used as the key, as here. (In Chapter 11 the name matters: it must match the name the tileset has inside the Tiled map)

3. `map.createLayer(0, tileset, x, y)` makes a `TilemapLayer` : a **game object** that draws the tiles, at `(x, y)` . A map from an array has exactly one layer, number 0
4. `layer.setCollision([...])` marks every tile with those indexes as solid
5. `this.physics.add.collider(player, layer)` - the same collider as Chapter 8, with a tilemap layer on one side. Arcade Physics now separates the player from every solid tile it touches

The `!` after `addTilesetImage(...)` and `tileToWorldXY(...)` is TypeScript's **non-null assertion**. Both methods say they might return `null` (the tileset's image might not be loaded; the map might have no layer) - `!` says "not this time". If it is wrong, the error comes at the next line that uses the value, so keep it for things you know are there.

The `as Phaser.Tilemaps.TilemapLayer` is needed because `createLayer` can also make a `TilemapGPULayer` - a faster, WebGL-only layer, new in Phaser 4, made when you pass `true` as a fifth argument. The return type says "one or the other"; we know which, and say so. (The GPU layer is for enormous maps; it does not show `putTileAt` changes until you call `generateLayerDataTexture()` , so this chapter uses the ordinary one.)

One more setting, in `src/main.ts` : `pixelArt: true` . Tile art is small and blocky, and the game is scaled to fit the window; this tells Phaser to scale it without smoothing, so the tiles stay crisp.

The player

`src/objects/Player.ts` is an Arcade Physics sprite (Chapter 8) using `topdown_hero.png` , with a walk animation for each direction (Chapter 7). It does not move itself: it sets its **velocity** from the arrow keys, and the physics world moves it - which is what lets the collider stop it.

`src/objects/Player.ts`

```
// normalise, so that walking diagonally is not faster than walking straight
const velocity = new Phaser.Math.Vector2(dx, dy).normalize().scale(SPEED);
this.setVelocity(velocity.x, velocity.y);
```

`dx` and `dy` are -1, 0 or 1. Holding RIGHT and DOWN together gives `(1, 1)` , which is about 1.41 long - so without `normalize()` (which makes it length 1) the player would go 41% faster diagonally.

One more line matters for tiles:

```
// a body smaller than the picture, centred on it
this.setSize(BODY_SIZE, BODY_SIZE);
```

The picture is 32 x 32 - exactly one tile. A body that size would have to be lined up perfectly, to the pixel, to fit through the house's one-tile door; one pixel out and it catches on the wall. A 20 x 20 body slips through easily, and still looks right. Making bodies a little smaller than their tiles is normal practice in tile games.

Seeing what collides

Press **D** in the game to see which tiles are solid, and which of their edges Phaser checks:



src/scenes/GameScene.ts

```
// renderDebug draws every tile that collides, and its colliding edges
this.layer.renderDebug(this.debugGraphics, {
  tileColor: null, // tiles that do
  not collide: nothing
  collidingTileColor: new Phaser.Display.Color(230, 57, 70, 120),
  faceColor: new Phaser.Display.Color(255, 255, 255, 255),
});
```

The white lines are the **interesting faces**: only the edges of a solid tile that face a non-solid tile. The edge between two wall tiles can never be touched, so Phaser does not check it.

That is one reason tile collision is fast, and it is why a player sliding along a straight wall does not snag at the joins between tiles.

Which tile am I on?

The strip under the map says what the player is standing on, every frame:

src/scenes/GameScene.ts

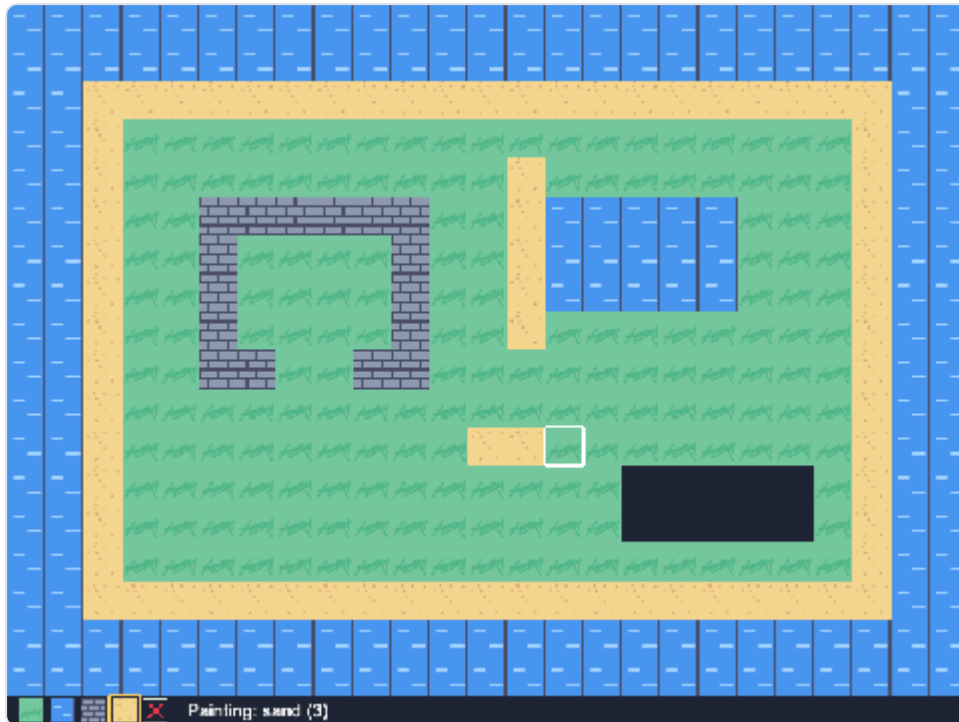
```
override update(): void {
    // which tile is under the middle of the player? getTileAtWorldXY takes
    // PIXELS...
    const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);

    // ...and the tile knows its own column and row (x and y, counted in
    // tiles) and index
    if (tile) {
        const name = TILE_NAMES[tile.index];
        this.infoText.setText(
            `Standing on: ${name} (tile ${tile.index}) at column ${tile.x}, row
            ${tile.y}` +
            `      Arrow keys: walk      D: show collision`,
        );
    }
}
```

`getTileAtWorldXY(x, y)` takes a position in **pixels** and gives back the `Tile` there - or `null` if there is no tile (off the map, or an empty cell). Watch the names: a `Tile`'s `x` and `y` are its column and row, **in tiles**. Its position in pixels is `pixelX` and `pixelY`. A `Tile` has plenty more - `collides`, `properties`, `tint`, `alpha` - but `index`, `x` and `y` are the ones you use most.

`if (tile)` is JavaScript's short way of saying `if (tile !== null)` - `null` counts as false.

Changing the map while the game runs



A tilemap is not fixed once it is made. `ch10_tilePainter` is a small level editor: choose a tile with the number keys (1-4, or 0 for an eraser) or the palette under the map, then click or drag.

An empty map

This time there is no array. The map is made empty, at the right size, and filled with grass:

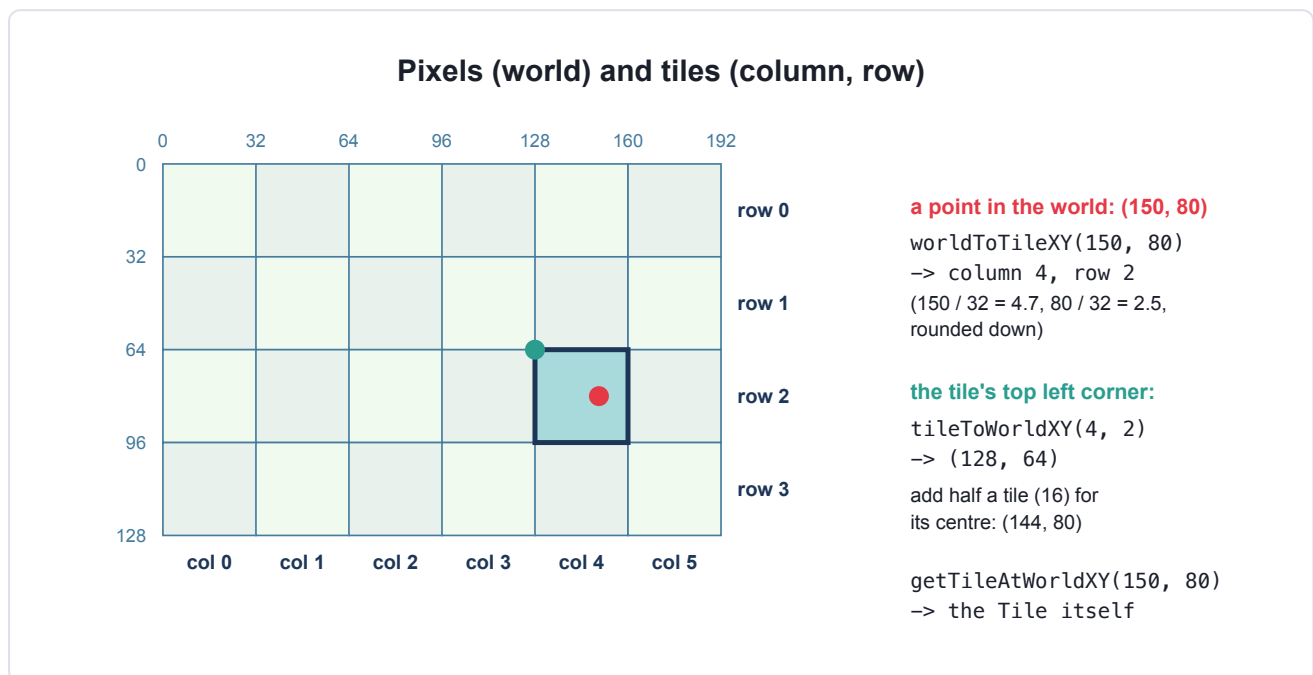
`src/scenes/PainterScene.ts`

```
// no data this time: an empty map of the right size, and a blank layer to
// paint on
const map = this.make.tilemap({
  tileWidth: TILE_SIZE,
  tileHeight: TILE_SIZE,
  width: MAP_COLUMNS,
  height: MAP_ROWS,
});
const tileset = map.addTilesetImage(TILES_KEY!);
this.layer = map.createBlankLayer("ground", tileset!);
this.layer.fill(GRASS);
```

`createBlankLayer(name, tileset)` makes a layer the size of the map, full of empty tiles. A map made this way can have as many layers as you like, each with its own name - `ch10_scrolling_world` has two. `fill(index)` sets every tile in the layer (or, with more arguments, a rectangle of it).

Pixels and tiles

A click happens at a position in pixels; the map works in columns and rows. Two methods convert:



`src/scenes/PainterScene.ts`

```
// which tile is under the pointer? worldToTileXY turns pixels into a column
// and a row
private tileUnder(pointer: Phaser.Input.Pointer): Phaser.Math.Vector2 | null
{
  const cell = this.layer.worldToTileXY(pointer.worldX, pointer.worldY);
  const onMap = cell.x >= 0 && cell.x < MAP_COLUMNS && cell.y >= 0 && cell.y
  < MAP_ROWS;
  return onMap ? cell : null;
}
```

- `worldToTileXY(x, y)` - pixels to tiles. It divides by the tile size and rounds **down**, so every pixel from 128 to 159 across is column 4. It gives a `Vector2` whose `x` is the column and `y` the row. It happily gives columns and rows outside the map (a click on the palette is row 18), so the code checks

- `tileToWorldXY(column, row)` - tiles to pixels: the tile's **top left corner**. The white frame that follows the mouse uses it:

```
const corner = this.layer.tileToWorldXY(cell.x, cell.y);
this.highlight.setPosition(corner.x, corner.y);
```

Why `pointer.worldX` rather than `pointer.x`? `x` is where the pointer is on the **screen**; `worldX` is where that is in the **world**, allowing for the camera. Here the camera never moves, so they are the same - but in a scrolling world they are not, and `worldX` is the one that matches the map. Get into the habit now.

Putting and removing tiles

`src/scenes/PainterScene.ts`

```
private paint(pointer: Phaser.Input.Pointer): void {
  const cell = this.tileUnder(pointer);
  if (cell === null) {
    return;           // below the map - the palette handles its own clicks
  }

  if (this.current === EMPTY) {
    this.layer.removeTileAt(cell.x, cell.y);           // leaves a hole:
    nothing is drawn there
  } else {
    this.layer.putTileAt(this.current, cell.x, cell.y);
  }
}
```

- `putTileAt(index, column, row)` changes one tile. The change shows on the next frame
- `removeTileAt(column, row)` empties it. Nothing is drawn there, so you see the game's background colour through the hole

An empty cell has index `-1` - "no tile". It is not the same as 0, which is a real tile (grass, here). In an array you write `-1` for an empty cell, and `ch10_scrolling_world` uses that to put one layer on top of another.

Painting is triggered by two events:

```
// paint when the button goes down, and while it is held and the mouse moves
this.input.on("pointerdown", (pointer: Phaser.Input.Pointer) =>
this.paint(pointer));
this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
  this.hover(pointer);
  if (pointer.isDown) {
    this.paint(pointer);
  }
});
```

"pointermove" fires whenever the pointer moves, button or no button; `pointer.isDown` says whether a button is held - so dragging paints a line.

Note - `setCollision` remembers the **indexes** you gave it, not the tiles that had them. A wall tile put down later with `putTileAt` is solid straight away; a wall removed is solid no more. The painter has no player to bump into things, but the same is true in any game: change the map, and the collisions change with it.

Reading the map back, saving and loading

To save the map, the painter turns the layer back into a 2D array - the same shape as `level.ts`:

`src/scenes/PainterScene.ts`

```
private toArray(): number[][] {
  const rows: number[][] = [];
  for (let row = 0; row < MAP_ROWS; row++) {
    const indexes: number[] = [];
    for (let column = 0; column < MAP_COLUMNS; column++) {
      const tile = this.layer.getTileAt(column, row); // null where
there is no tile
      indexes.push(tile === null ? EMPTY : tile.index);
    }
    rows.push(indexes);
  }
  return rows;
}
```


`getTileAt(column, row)` is `getTileAtWorldXY`'s twin, taking a column and row instead of pixels. Both give `null` for an empty cell - so the code writes `-1` there.

Saving and loading use the browser's `localStorage`, as the high-score table did in Chapter 6. S saves; L loads:

```
private saveMap(): void {
    localStorage.setItem(SAVE_KEY, JSON.stringify(this.toArray()));
    this.showMessage("Saved");
}
```

```
// what comes back is only text - check it really is a map of the right size
before using it.
// JSON.parse throws an exception if the text is not JSON at all
let data: unknown = null;
try {
    data = JSON.parse(saved);
} catch {
    // leave data as null - isMap says no
}
if (!this.isMap(data)) {
    this.showMessage("The saved map is damaged");
    return;
}

// putTilesAt puts a whole 2D array down at once, starting at column 0, row
0. -1 becomes empty
this.layer.putTilesAt(data, 0, 0);
```

`putTilesAt(array, column, row)` puts a whole 2D array of indexes down in one go, with its top left corner at that column and row. It is how you would stamp a ready-made room, or a tree two tiles tall, into a map.

`unknown` is TypeScript's type for "could be anything - check before you use it". It is the honest type for something read back from storage, which anyone can edit in the browser's developer tools. The check is a method with an unusual return type:

```
// true if value is MAP_ROWS arrays of MAP_COLUMNS tile indexes (or -1)
private isMap(value: unknown): value is number[][] {
  return Array.isArray(value) && value.length === MAP_ROWS &&
    value.every((row) =>
      Array.isArray(row) && row.length === MAP_COLUMNS &&
      row.every((index) => Number.isInteger(index) && index >= EMPTY &&
index < TILE_COUNT)
    );
}
```

`value is number[][]` is a **type guard**: a method that returns a boolean, and also tells the compiler "if this returns true, `value` is a `number[][]`". After `if (!this.isMap(data)) return;` the compiler knows `data` is a map, and lets it be passed to `putTilesAt` with no `as`. Java has nothing quite like it; the nearest is `instanceof` with a pattern (`if (o instanceof String s)`), which only works for classes. `every(test)` is true if the test is true for every element.

Printing a level for your code

Press **P** and the painter writes the map to the browser's console (F12, or Cmd+Option+I on a Mac, then the Console tab) as TypeScript:

```
// write the map to the browser's console as TypeScript, ready to paste into
level.ts
private print(): void {
  const lines = this.toArray().map((row) => ` [${row.join(", ")}],`);
  console.log(`export const LEVEL: number[][] =
[\n${lines.join("\n")}\n];`);
  this.showMessage("Printed to the browser's console");
}
```

The painter's map is the same size as `ch10_array_map`'s, with the same tiles - so you can paint a level, print it, and paste it over `LEVEL` in that project's `level.ts`. (Make sure the tile at `START_COLUMN`, `START_ROW` is one the player can stand on.) A tool that writes code for you is a game developer's oldest trick.

The palette - passing a function

The palette is a class of its own. It knows nothing about maps; when a button is clicked, it calls a function it was given:

src/objects/Palette.ts

```
constructor(scene: Phaser.Scene, x: number, y: number, onChoose: (index:
number) => void) {
  ...
  button.on("pointerdown", () => onChoose(index));
```

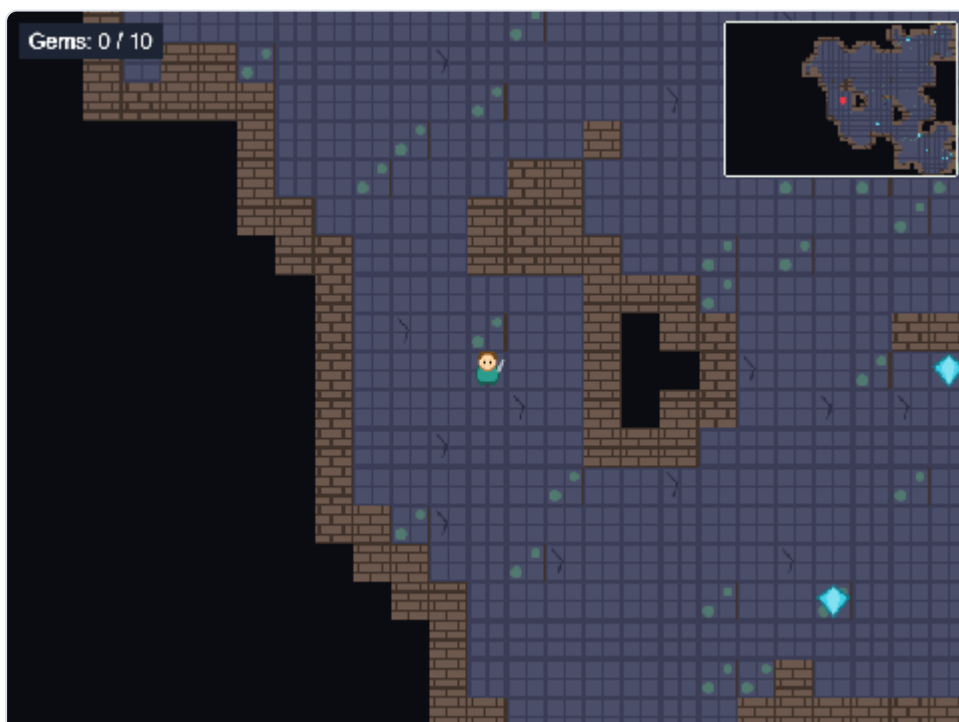
src/scenes/PainterScene.ts

```
this.palette = new Palette(this, 20, STRIP_Y, (index) =>
this.choose(index));
```

`(index: number) => void` is a **function type**: "a function that takes a number and returns nothing". In Java you would declare the parameter as an `IntConsumer` and pass a lambda; in TypeScript functions are ordinary values, and their types are written out like this.

The palette's buttons are small `Image`s, one per tile. To show one tile as an `Image`, the painter loads `simple_tiles.png` a **second** time, under another key, with `load.spritesheet` - so each tile is also a frame.

A world bigger than the screen



`ch10_scrolling_world` is a cave 60 tiles by 40 - 1920 x 1280 pixels, more than five screens - made in code, and different every time. Walk round it and find the ten gems; R makes a new cave.

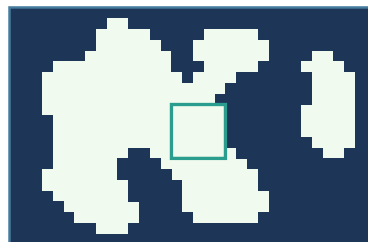
A cave made in code

`src/Cave.ts` makes the cave. It is a plain class, not a game object and not a scene: it knows nothing about Phaser, tiles or pictures, only which cells are rock and which are floor.

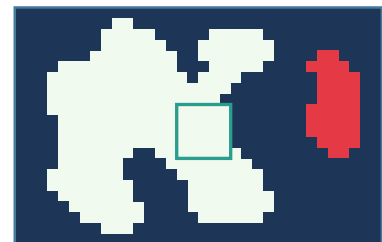
Making a cave: noise, smoothed, then cut-off caves filled in



1. random rock (47%)



2. smoothed 5 times



3. unreachable floor filled

dark = rock, light = floor, green = the cleared start, red = floor the player could never reach, now rock

It uses a well-known recipe called **cellular automata**:

1. fill the grid with random rock - a little under half of it (random "noise")
2. **smooth** it, five times: each cell becomes rock if 5 or more of the 9 cells around it (itself included) are rock. Lone rocks disappear, and clumps grow into walls. What was noise becomes smooth, natural-looking caves
3. clear a space for the player to start in; then **flood fill** from there - visit every floor cell that can be reached, one neighbour at a time - and turn any floor that was *not* reached into rock. Otherwise a gem could be put in a cave the player can never get to

Each step is a short method in `Cave.ts` - `randomRock()`, `smooth()`, `clearStart()` and `fillUnreachable()` - built from nothing more than nested `for` loops over rows and columns. The one subtle point is in `smooth()`: it builds a **new** grid, so that every cell is judged on the old one, not on cells already changed in this pass.

Keeping the cave (the **model**) apart from the tilemap (the **view**) is worth the extra class. The scene decides what the cave *looks* like; the cave decides what it *is*. You could draw the same cave with `platform_tiles.png`, or as coloured squares in a minimap, without touching `Cave.ts`.

Two layers

The scene draws the cave with two layers, one on top of the other:

`src/scenes/WorldScene.ts`

```
// layer 1, the floor: every cell, mostly plain, some cracked or mossy.
// weightedRandomize picks a
// random index for each tile - a weight of 12 is 12 times as likely as a
// weight of 1
const floor = map.createBlankLayer("floor", tileset!);
floor.weightedRandomize([
  { index: FLOOR, weight: 12 },
  { index: FLOOR_CRACKED, weight: 1 },
  { index: FLOOR_MOSSY, weight: 1 },
]);

// layer 2, the walls: drawn on top of the floor, and empty (-1) where the
// cave is open
const walls = map.createBlankLayer("walls", tileset!);
walls.putTilesAt(this.wallTiles(cave), 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not
// empty collides
```

The floor layer is floor everywhere - even under the rock, where it cannot be seen. The wall layer is **empty** (-1) wherever the cave is open, so the floor shows through. Layers are drawn in the order they are made, like any game objects, so walls are on top.

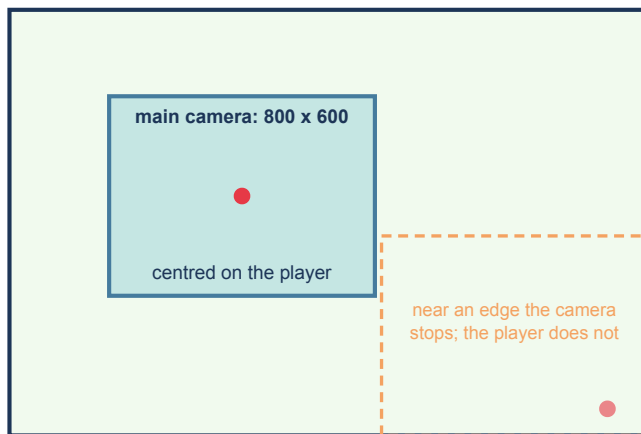
Only the wall layer collides. `setCollisionByExclusion([-1])` means "every index *except* these": whatever wall tiles the scene chose, they are all solid - with no list to keep up to date.

`wallTiles()` turns the cave into indexes. It looks at each rock cell's **neighbours** to choose its picture: bricks where the rock is next to the floor, black "void" deep inside the rock, and now and then a torch on a wall with floor just below it. Choosing a tile from its neighbours is called **auto-tiling**; bigger tilesets have a picture for every combination of neighbours.

The camera

A camera is a window onto the world. Until now the main camera has always looked at the rectangle from (0, 0) to (800, 600), which was the whole game. Now the world is bigger:

A world bigger than the screen

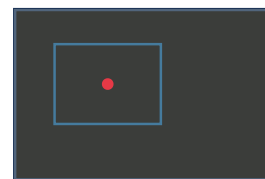


the world: 60 x 40 tiles = 1920 x 1280 pixels
`physics.world.setBounds` and `camera.setBounds(0, 0, 1920, 1280)`

`startFollow(player, true)`
 the camera's `scrollX` / `scrollY` change every frame to keep the player centred

`setBounds(...)`
 never shows anything outside the map

`hud.setScrollFactor(0)`
 stays in the same place on screen



a second camera,
 zoom 0.1: the
 whole world in
 192 x 128 pixels

`cameras.add(x, y, 192, 128)`

`src/scenes/WorldScene.ts`

```
// the world is bigger than the screen. The camera follows the player, but
// never shows
// anything outside the map; the physics world is the size of the map too
const width = map.widthInPixels;
const height = map.heightInPixels;
this.physics.world.setBounds(0, 0, width, height);
this.cameras.main.setBounds(0, 0, width, height);
this.cameras.main.startFollow(this.player, true); // true: round to
whole pixels - no shimmer
```

- `startFollow(target)` moves the camera every frame to keep the target in the middle of the screen. It changes the camera's `scrollX` and `scrollY` - the world position of the screen's top left corner. (Extra arguments, `lerpX` and `lerpY` between 0 and 1, make it catch up gently instead of sticking exactly)
- `setBounds(x, y, width, height)` stops the camera at the edges of the map. Without it, walking to the edge would show empty space beyond
- `this.physics.world.setBounds(...)` : the physics world is 800 x 600 unless you say otherwise. Here the walls keep the player in anyway, but anything with `setCollideWorldBounds(true)` would otherwise bounce off an invisible wall in the middle of the map

The HUD must not scroll with the world:

```
this.hudText.setScrollFactor(0);           // stays put on the screen while the
camera moves
```

A **scroll factor** of 1 (the default) moves an object fully with the camera; 0 not at all - it is fixed to the screen; 0.5 moves it half as far, which is how parallax backgrounds are made.

A minimap is just another camera

A scene can have several cameras, each drawing the same world into its own rectangle of the screen. The minimap is a second camera in the top right corner, zoomed out to a tenth:

```
const minimap = this.cameras.add(MINI_X, MINI_Y, MINI_WIDTH, MINI_HEIGHT);
minimap.setZoom(MINI_ZOOM);
minimap.setBounds(0, 0, width, height);
minimap.centerOn(width / 2, height / 2);
minimap.setBackgroundColor(0x000000);

// a big red dot on the player: only the minimap shows it
this.marker = this.add.circle(this.player.x, this.player.y, MARKER_RADIUS,
0xe63946);
this.cameras.main.ignore(this.marker);
```

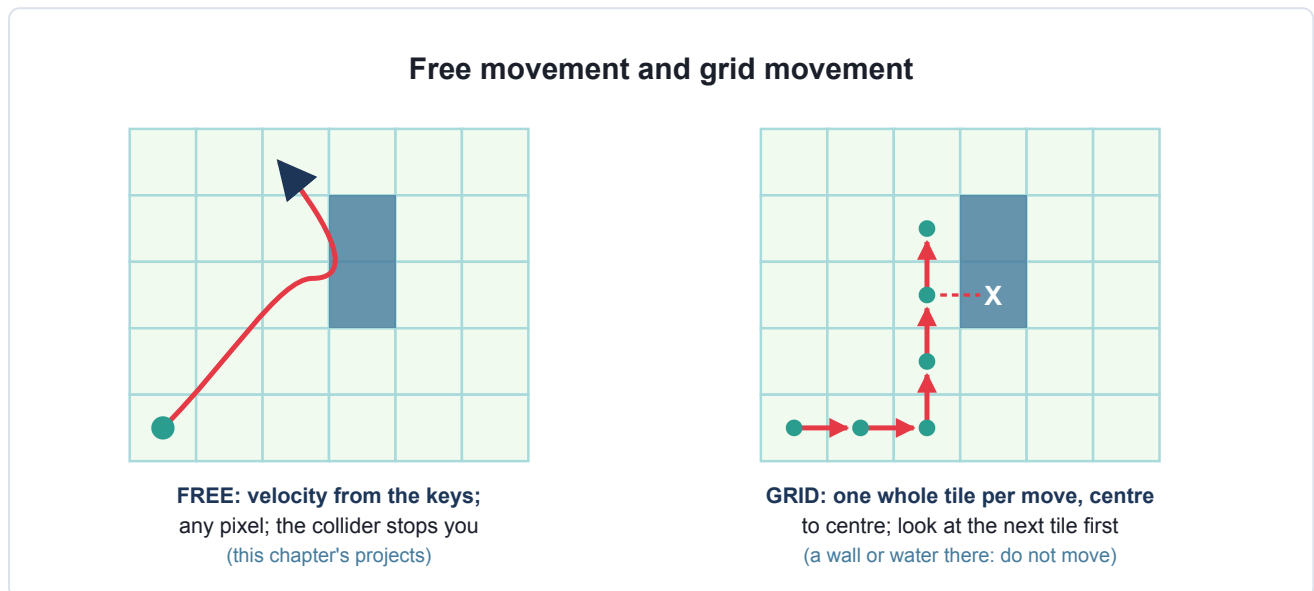
At zoom 0.1, the 1920 x 1280 world fits exactly into 192 x 128 pixels. Every camera draws every game object - unless told to `ignore` it. The player, at a tenth of its size, would be a 3-pixel speck on the minimap, so a big red circle follows the player (in `update()`) and the **main** camera ignores it: it is seen only on the minimap. The other way round, the minimap ignores the HUD text and its own frame, which belong to the main camera's screen.

Starting again

R restarts the scene, and `create()` makes a new `Cave`. Chapter 2's lesson applies: the scene object is reused, so `init()` resets the gem count. And the player is the **same** `Player` **class** as in `ch10_array_map`, unchanged - it never knew how big the world was, or that a camera was following it.

Free movement and grid movement

All three projects use **free movement**: the player can be at any pixel, moves smoothly, and the physics collider stops it at walls. Many tile games - Pokémon, Sokoban, most roguelikes - use **grid movement** instead: the player is always on a tile, and each move goes one whole tile.



Grid movement needs no physics at all. The player's position is really a column and a row. To move, you work out the **next** tile, look at it (`getTileAt`) and decide - is it a wall, water, a door?

- and only then move. The move itself is usually a short **tween** (Chapter 7) from one tile's centre to the next, with input ignored until it finishes, so the player glides rather than teleports.

Which to choose? Free movement suits action games, where fine positioning and speed matter. Grid movement suits puzzles and turn-based games, where the rules are about tiles: "boxes can be pushed one square", "each monster moves one square after you". It is also much simpler to get exactly right - there are no half-overlaps with walls to worry about.

Common mistakes

Note - the map is invisible, but the player still bumps into invisible walls: the key given to `addTilesetImage` does not match a loaded image, so it returns `null` and the layer has no pictures. The browser's console says `Texture key "tilez" not found`. The walls still work because collision is about indexes, not pictures. Keep keys in constants.

Note - the player walks straight through walls: either `setCollision` was not called (or was given the wrong indexes), or there is no `this.physics.add.collider(player, layer)`. You need both. Press D in `ch10_array_map` to see which tiles collide.

Note - the player stands in the corner of a tile, not the middle: `tileToWorldXY` gives the tile's **top left** corner. Add half a tile.

Note - do not name a scene method `load`, `add`, `make`, `physics`, `cameras`, `time` or `input`. They are already fields of every scene, and TypeScript says so, in several confusing ways at once - for a `load()` method, one of them is `Property 'load' in type 'PainterScene' is not assignable to the same property in base type 'Scene'`. That is why the painter's methods are `saveMap()` and `loadMap()`.

Summary

- a **tileset** is one picture of many equal tiles; each tile has an **index**, counting from 0. A map is a grid of indexes; `-1` means an empty cell
- `this.make.tilemap({ data, tileWidth, tileHeight })` makes the map's data from a 2D array, written row by row: `data[row][column]`
- `map.addTilesetImage(key)` says which image to use; `map.createLayer(0, tileset, x, y)` makes the layer that draws it; `map.createBlankLayer(name, tileset)` makes an empty layer for a map made in code

- `layer.setCollision([indexes])` (or `setCollisionByExclusion`) plus `this.physics.add.collider(sprite, layer)` makes tiles solid
- `getTileAtWorldXY` and `getTileAt` find a tile; `worldToTileXY` and `tileToWorldXY` convert between pixels and tiles (the tile's top left corner)
- `putTileAt`, `removeTileAt`, `putTilesAt`, `fill` and `weightedRandomize` change the map at any time
- for a world bigger than the screen: `camera.setBounds`, `camera.startFollow`, `physics.world.setBounds`, and `setScrollFactor(0)` for anything fixed to the screen
- a second camera (`this.cameras.add`, `setZoom`, `ignore`) makes a minimap
- keep generated levels as plain data, separate from the tilemap that draws them

Challenges

1. **A new island** (*ch10_array_map*) - Change the map: add a second bridge across the river, a maze of walls in the bottom half, and a moat of water round the house (with a sand path to the door). Make the player start somewhere else. You only need to change `level.ts` - or paint the map in `ch10_tilePainter` and paste it in.
2. **Sand slows you down** (*ch10_array_map*) - Make the player walk at half speed on sand. The `Player` class should not have to know about tiles: give it a way to be told how fast to go, and let the scene tell it.
3. **Eyedropper** (*ch10_tilePainter*) - Right-clicking a tile on the map should *choose* that tile (as if its number key had been pressed) instead of painting. Right-clicking an empty cell chooses the eraser. The browser's own right-click menu should not appear over the game.
4. **Treasure tiles** (*ch10_scrolling_world*) - Replace the gem sprites with **chest tiles** (`dungeon_tiles.png` index 8) put into the map itself. Walking into a chest collects it: it disappears from the map, the coin sound plays, and the HUD counts it. *Hint:* the chests could go on the wall layer, where they would stop the player - look up `setTileIndexCallback`, or the callback a collider can take. Or put them on a third layer and use `overlap`.
5. **A map from strings** (*ch10_array_map*) - Numbers are hard to read. Write the level as an array of strings instead - one string per row, one character per tile: `#` wall, `~` water, `.` grass, `s` sand, and `@` for where the player starts (on grass). Turn it into the `number[]`

[] the tilemap needs, and find the start position from the `@`. *Hint:* a `Record<string, number>` can map characters to indexes; `row.split("")` turns a string into an array of characters. Check that every row is the same length.

6. **Grid movement** (*ch10_array_map*) - Change the player to move one whole tile at a time: tap an arrow key to move one tile; hold it to keep moving. The player glides from tile centre to tile centre, cannot move onto walls or water, and cannot change direction halfway between tiles. *Hint:* physics is no longer needed. Keep the player's column and row; before moving, look at the next tile with `getTileAt`; move with a tween (Chapter 7) and ignore the keys until it completes. `tileToWorldXY` gives the corner of the tile to move to.

Previous: [Chapter 9 - 2D physics](#) · Next: [Chapter 11 - Tilemaps with Tiled](#)

Tilemaps with Tiled

In Chapter 10 you built maps by typing numbers into arrays. That works for a small room, but nobody designs a real level that way. In this chapter you draw levels in **Tiled**, a free map editor, and load them into Phaser: tiles you paint with the mouse, solid ground chosen by a property rather than by a list of numbers, and the player, coins, enemies and doors placed in the editor instead of in the code.



What you will learn

- what Tiled is, and how to make a map in it: tilesets, tile layers, object layers and custom properties
- where the files go: the map's source (`.tmx`) and the JSON file the game loads (`.tmj`)
- what is inside an exported `.tmj` file, and what a **GID** is
- how to load a Tiled map with `tilemapTiledJSON` , `make.tilemap` , `addTilesetImage` and `createLayer` - and why the names must match exactly
- how to make tiles solid with `setCollisionByProperty` , and read any tile's properties
- three ways to read an object layer - `findObject` , `createFromObjects` and `getObjectLayer(...).objects` - and how to read custom properties on objects and maps

- how to join rooms together with doors that name the map they lead to

The projects

Project	What it shows
<code>ch11_tiled_level</code>	a platform level from Tiled: a ground layer made solid by a <code>collides</code> property, dangerous tiles with a <code>hazard</code> property, a background layer, and the player, coins, enemies and finish placed in an object layer
<code>ch11_tiled_doors</code>	two top-down rooms made in Tiled with <code>dungeon_tiles.png</code> ; doors with <code>target</code> and <code>entrance</code> properties load the next map and put the player at the right place

Why use a map editor?

A Chapter 10 map is a grid of numbers: to move a platform you count along a row. A map editor lets you **see** the level while you make it - pick a tile, paint it, and that is what the game will show. It also lets you put more in the map than tiles:

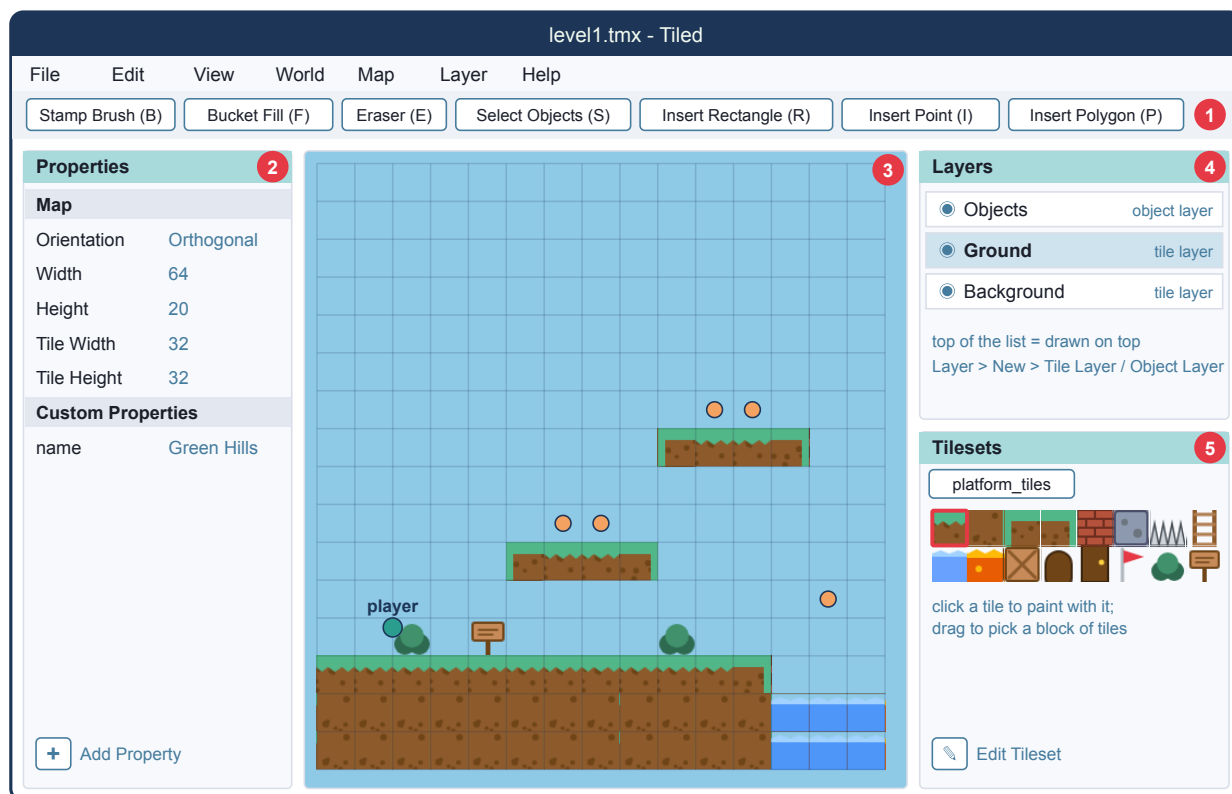
- **objects** - points and rectangles with names, such as "the player starts here", "a coin here", "this rectangle is a door"
- **custom properties** - your own named values, on a tile ("this tile is solid"), on an object ("this enemy walks at 100 pixels per second", "this door leads to the cellar") or on the whole map ("this level is called Green Hills")

The **level becomes data, not code**. The code says what a coin *does*; the map says where the coins *are*. A designer who never opens the code can make a new level.

[Tiled](#) is the editor almost everyone uses for 2D maps. It is free (you can donate), it runs on Windows, macOS and Linux, and Phaser can read its JSON format directly. The steps below use the menus of Tiled 1.12 (checked against 1.12.2); other versions may move things a little.

Tiled in seven steps

This section walks through making `level1`, the map in `ch11_tiled_level1`. The finished map is in the project, but open it in Tiled as you read, and try each step on a map of your own. Here is Tiled's window, with the level open (a drawing - yours will look a little different):



- 1 Tools: paint tiles (Stamp, Bucket, Eraser) or make objects (Rectangle, Point, Polygon)
- 2 Properties: whatever is selected - the map, a layer, a tile or an object - and its custom properties
- 3 The map itself: 32 x 32 tiles; the grid and the objects are only drawn in the editor
- 4 Layers: tile layers and object layers, top of the list drawn on top
- 5 Tilesets: the tiles you paint with

Where the files go

Each project keeps its map in two places:

```

ch11_tiled_level/
  tiled/
    level1.tmx
Tiled
  public/
    assets/
      maps/
        level1.tmj
game loads
  tilesets/
    platform_tiles.png

```

the map's SOURCE - open and change this in

the map EXPORTED as JSON - the file the

the tileset picture, used by both

`.tmx` is Tiled's own format (XML); `.tmj` is its JSON format, which Phaser reads. You work on the `.tmx`, and export the `.tmj` when you want the game to see your changes - just as you edit `.ts` files and build `game.js`. (Tiled can also save straight to `.tmj`, with no `.tmx`; that works too.)

Step 1 - a new map

File > New > New Map... opens the New Map dialog:

New Map (File > New > New Map...)

Map

Orientation: Orthogonal

Tile layer format: CSV

Tile render order: Right Down

Map size

☒ Fixed ☐ Infinite

Width: 64 tiles

Height: 20 tiles (2048 x 640 pixels)

Tile size

Width: 32 px

Height: 32 px

Cancel OK

New Tileset (File > New > New Tileset...)

Tileset

Name: platform_tiles

Type: Based on Tileset Image

☒ Embed in map

Image

Source: ...tilesets/platform_tiles.png

☐ Use transparent color (Browse...)

Tile width: 32 px

Tile height: 32 px

Margin: 0 px

Spacing: 0 px

Cancel OK

1 a fixed size, in tiles - the map is 64 tiles across and 20 down

2 Embed in map: the tileset is saved INSIDE the map file. Phaser cannot read a separate (external) tileset

3 the name you will pass to `addTilesetImage()` in Phaser - Tiled suggests the picture's file name

- **Orientation: Orthogonal** - an ordinary grid of squares (Tiled also does isometric and hexagonal maps)
- **Tile layer format: CSV** - how the tiles are written in the file. CSV is readable, and Phaser reads it. (Avoid the compressed Base64 formats: Phaser cannot read those)

- **Tile render order: Right Down** - leave it
- **Map size: Fixed, 64 x 20 tiles** - the level is 64 tiles across and 20 down. Not "Infinite": a game level has edges, and the camera and physics world need to know where they are
- **Tile size: 32 x 32 px** - the size of one tile in `platform_tiles.png`

Press **OK**. The new map opens, untitled: use **File > Save As...** to save it as `tilted/level1.tmx` inside the project.

Step 2 - a tileset

A **tileset** is the picture the tiles come from, cut into squares. **File > New > New Tileset...** (or the New Tileset button at the bottom of the Tilesets panel) opens the second dialog above:

- **Name: platform_tiles** - Tiled suggests the picture's file name. Remember this name: the code needs it
- **Type: Based on Tileset Image** - one picture cut into tiles
- **Embed in map: ticked** - this matters. An embedded tileset is saved *inside* the map file. Phaser cannot read a separate tileset file (`.tsx` or `.tsj`). While the box is unticked, the dialog's button says **Save As...** (to save a separate tileset file); ticking it changes the button to **OK**
- **Source** - browse to `public/assets/tilesets/platform_tiles.png` . Tiled stores the path relative to the map file
- **Tile width and height: 32 px, margin and spacing 0** - this picture's tiles are packed edge to edge

Press **OK**, and the tiles appear in the **Tilesets** panel.

Step 3 - tile layers, and painting

A new map starts with one tile layer, "Tile Layer 1". Double-click its name in the **Layers** panel and rename it **Ground**. Then **Layer > New > Tile Layer** makes a second one: call it **Background**, and drag it below Ground in the list. In the Layers panel, the layer at the **top of the list is drawn on top** - so Background is drawn first, and Ground over it.

Why two layers? The bushes, sign and flag are scenery, not ground: a separate layer keeps them apart, and lets one place hold two tiles - a bush in front of a wall, say.

To paint, click a layer in the Layers panel (it is easy to paint on the wrong one), click a tile in the Tilesets panel, and use the tools:

Tool	Key	Does
Stamp Brush	B	paints the chosen tile (or block of tiles - drag in the Tilesets panel to choose several)
Bucket Fill Tool	F	fills an area
Eraser	E	removes tiles

With the Stamp Brush, right-click on the map to pick up the tile under the pointer. **Ctrl+Z** (Cmd+Z on a Mac) undoes.

Step 4 - custom properties on tiles

Which tiles are solid? In Chapter 10 the code had a list: `layer.setCollision([2])`. With Tiled you can put the answer in the tileset instead, as a **custom property**.

Click **Edit Tileset** under the Tilesets panel. The tileset opens in a tab of its own. Select the solid tiles - grass, dirt, the two grass edges, brick, stone and crate - with Ctrl-click (Cmd-click on a Mac). Then, in the **Properties** panel, click the **+** button (**Add Property**), type the name **collides**, choose the type **bool**, and press OK. Tick the new property's box: now all seven tiles have `collides = true`.

The tileset, with a custom property on some of its tiles

☒ collides = true (solid: the player stands on these)
☐ hazard = true (touch one and you start again)

Tiles with no box have no custom properties at all.
 The numbers are tile INDEXES within the tileset, from 0 (see the GIDs diagram).

Properties (7 tiles selected)

Tile	
ID	0
Class	
Probability	1
Custom Properties	
collides	☒ true

1 tick it - the value is true

2 select several tiles first to add it to them all

+ Add Property: name collides, type bool

Do the same for the spikes, water and lava, with a bool property called **hazard**. Tiles with no property simply do not have it - the bushes, sign, flag and ladder have none.

Properties can be `bool` , `int` , `float` , `string` , `color` , `file` or `object` , and the names are up to you: nothing in Tiled knows what "collides" means. Only your code gives it a meaning.

Step 5 - an object layer

Some things in a level are not tiles: where the player starts, where the coins and enemies are, the area that counts as the finish. **Layer > New > Object Layer** makes a layer for **objects**; call it **Objects**.

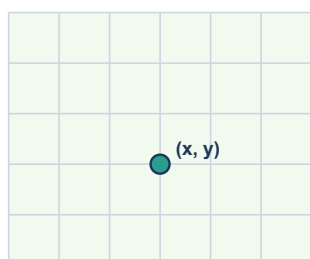
The object tools appear in the tool bar when an object layer is selected:

Tool	Key	Makes
Insert Point	I	a point: a single position
Insert Rectangle	R	a rectangle
Insert Polygon	P	a shape with many corners - or, finished without closing it, a line of points (a polyline)
Select Objects	S	select, move and resize objects

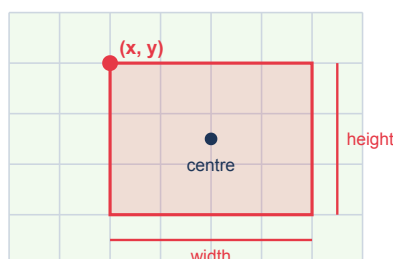
Click with Insert Point to place the player's start. In the Properties panel, give it the **Name** `player` . Place a point for each coin, each named `coin` , and one for each enemy, named `enemy` . Draw a rectangle round the flag and name it `goal` .

Each enemy also has a custom property, added with **+** as for tiles: **speed**, a **float** - 60 for the first slime, 100 for the second. Tiled measures each kind of object differently, which matters when the code places things:

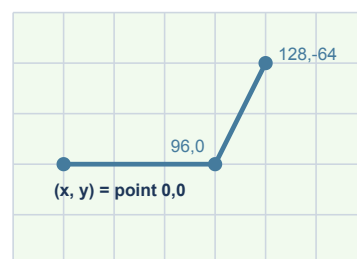
Where Tiled measures each kind of object from



Point: `x` and `y` are the point.
A sprite placed there is centred on it.



Rectangle: `x, y` is the TOP-LEFT corner.
A Zone is placed by its centre:
 $(x + \text{width} / 2, y + \text{height} / 2)$



Polyline: a list of points,
each measured FROM `(x, y)`.

Note - Tiled also has a **Class** field for every object (it was called **Type** in older versions). This chapter uses only the **Name**, because the name is saved the same way by every version of Tiled.

Step 6 - map properties

Map > Map Properties... shows the properties of the whole map in the Properties panel. Add a string property called **name**, with the value `Green Hills`. The game shows it in the corner of the screen.

Step 7 - save, and export

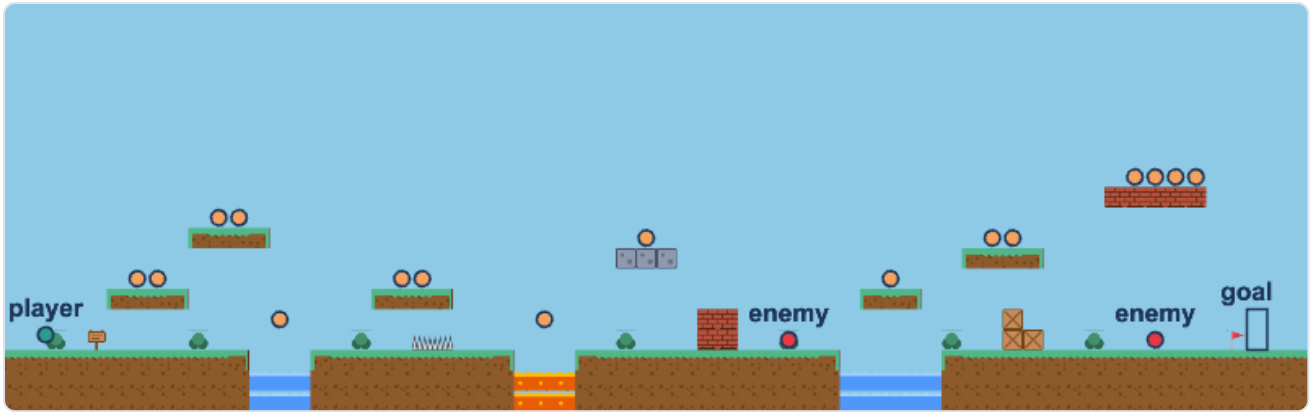
File > Save (Ctrl+S) saves `level1.tmx`. That is your work - but the game cannot read it. To make the file the game loads, use **File > Export As...**, choose the type **JSON map files (*.tmj *.json)**, and save it as `public/assets/maps/level1.tmj`.

Tiled remembers where you exported to. After the first time, **File > Export** (Ctrl+E, or Cmd+E on a Mac) writes the `.tmj` again without asking. (In **Edit > Preferences** - in the **Tiled** menu on a Mac - you can also tell Tiled to repeat the last export every time you save.) Then build the game, and refresh.

Note - If your tileset is *not* embedded, tick **Embed tilesets** under **Export Options** in Preferences before exporting, or use the **Embed Tileset** button at the bottom of the Tilesets panel. Otherwise Phaser shows *External tilesets unsupported. Use Embed Tileset and re-export* in the console, and the game stops with an error.

The maps in this chapter's projects were written by a small script, `tools/make_maps.ts`, so that they are exactly what the text describes. They open in Tiled like any other map. (The drawings of Tiled's windows in this chapter come from `tools/make_diagrams.ts`.)

Here is the whole level, with its objects drawn over it as Tiled shows them:



Inside a .tmj file

Open `public/assets/maps/level1.tmj` in Celbridge. It is long, but simple:

level1.tmj - the map

```
"type": "map",
"orientation": "orthogonal",
"width": 64, "height": 20,
"tilewidth": 32,
"tileheight": 32,
"infinite": false,
"properties": [ ... ],
"tilesets": [ ... ],
"layers": [ ... ],
"nextlayerid": 4,
"nextobjectid": 21,
"version": "1.10"
```

Keys are written in alphabetical order;
"..." marks what has been left out here.

Phaser reads all of it with
`this.load.tilemapTiledJSON(key, file)`
and builds the map with
`this.make.tilemap({ key })`

tilesets[0] - embedded in the map

```
"name": "platform_tiles", <- addTilesetImage("platform_tiles", ...)
"firstgid": 1,
"image": "../tilesets/platform_tiles.png",
"columns": 8, "tilecount": 16, "tilewidth": 32, ...
"tiles": [ { "id": 0, "properties": [
  { "name": "collides", "type": "bool", "value": true } ] }, ... ]
```

layers[0] and [1] - tile layers

```
"name": "Ground", "type": "tilelayer", <- createLayer("Ground", ...)
"width": 64, "height": 20,
"data": [ 0, 0, 0, ... one GID per tile, row by row:
  1, 1, 1, 1, 4, 0, 0, 0, 3, ... ] 64 x 20 = 1280 numbers
```

layers[2] - the object layer

```
"name": "Objects", "type": "objectgroup", <- getObjectLayer("Objects")
"objects": [
  { "id": 1, "name": "player", "point": true, "x": 64, "y": 520, ... },
  { "id": 2, "name": "coin", "point": true, "x": 208, "y": 432, ... },
  { "id": 18, "name": "enemy", "point": true, "x": 1232, "y": 528,
    "properties": [ { "name": "speed", "type": "float", "value": 60 } ] },
  { "id": 20, "name": "goal", "x": 1952, "y": 480,
    "width": 32, "height": 64, ... }
]
```

- the **map**: its size in tiles, the size of a tile, and its custom properties
- **tilesets**: each embedded tileset, with its name, picture, size, and the custom properties of the tiles that have any
- **layers**, in drawing order: a tile layer is a `data` list of numbers, one for each place in the map, row by row; an object layer is a list of `objects`, each with an `id`, a `name`, a position, and any custom properties

Custom properties are written as a list, each with a `name` , `type` and `value` .

Tile numbers: GIDs

The numbers in a tile layer's `data` are not quite the tile numbers from Chapter 10. They are **GIDs** - global tile IDs:

Tile index (in the tileset) and GID (in the map)

tileset												
index	0	1	2	3	4	5	6	7	8	9	10	11
GID	1	2	3	4	5	6	7	8	9	10	11	12

GID = firstgid + index. This tileset's firstgid is 1, so GID 0 is left free to mean "no tile".
A second tileset in the same map would start where this one ends - firstgid 17.

row 17 of the Ground layer, columns 7 to 16, as saved in the .tmj:

	1	1	1	1	4	0	0	0	3	1
drawn										
tile.index	1	1	1	1	4	-1	-1	-1	3	1

In Phaser a Tiled tile's index IS its GID - grass top is 1, not 0 - and an empty place is -1.
So `setCollision([1, 2, 3])` means GIDs. Choosing tiles by property avoids counting at all.

A map can use several tilesets, so Tiled numbers every tile in the map in one sequence. The first tileset's tiles start at its **firstgid**, 1 - which leaves **0 to mean "no tile here"**. A second tileset would carry on from 17. So grass top, which is tile 0 in the picture, is GID 1 in the map.

Phaser keeps the GIDs: a tile from a Tiled map has `tile.index` equal to its GID, and an empty place has index -1. If you ever choose tiles by number in a Tiled map - `setCollision([1, 2, 3])` - they are GIDs. That is one reason to choose them by property instead, as the next section does: then nobody has to count.

Loading the map in Phaser

Two files to load

```
src/scenes/GameScene.ts
```

```

preload(): void {
    // the map (JSON), and the picture its tileset uses - two separate files
    this.load.tilemapTiledJSON(MAP_KEY, MAP_FILE);
    this.load.image(TILES_KEY, TILES_FILE);
}

```

`tilemapTiledJSON` loads the `.tmj` into Phaser's tilemap cache, under a key. It does **not** load the tileset's picture, even though the map names it - you load the picture yourself, as an ordinary image. The keys and file names are in `src/assets.ts`:

`src/assets.ts`

```

// the level, exported from Tiled as JSON (the Tiled source is
// tiled/level1.tmx)
export const MAP_KEY = "level1";
export const MAP_FILE = "assets/maps/level1.tmj";

// the tileset's picture. TILESET_NAME is the name the tileset has INSIDE
// the map - the name shown in
// Tiled's Tilesets panel. Phaser needs both: the name to find the tileset
// in the map, and the key to
// find the picture it loaded.
export const TILES_KEY = "platform_tiles";
export const TILES_FILE = "assets/tilesets/platform_tiles.png";
export const TILESET_NAME = "platform_tiles";

// the names of the layers, exactly as they are spelled in Tiled's Layers
// panel
export const BACKGROUND_LAYER = "Background";
export const GROUND_LAYER = "Ground";
export const OBJECTS_LAYER = "Objects";

```

Four steps to a map on screen

`src/scenes/GameScene.ts`

```

private createMap(): void {
    // 1. the map: its layers, tileset and objects, all read from the JSON
    file loaded in preload()
    this.map = this.make.tilemap({ key: MAP_KEY });

    // 2. join the tileset in the map (by its NAME in Tiled) to the picture
    Phaser loaded (by its KEY)
    const tiles = this.map.addTilesetImage(TILESET_NAME, TILES_KEY);
    if (tiles === null) {
        throw new Error(`No tileset "${TILESET_NAME}" in the map, or no picture
        "${TILES_KEY}" loaded`);
    }

    // 3. a game object for each tile layer, by its name in Tiled. They are
    drawn in the order they
    // are made, so the background goes first. (createLayer can also make a
    GPU layer, which is a
    // different type - we did not ask for one, so this is an ordinary
    TilemapLayer.)
    this.map.createLayer(BACKGROUND_LAYER, tiles);
    this.ground = this.map.createLayer(GROUND_LAYER, tiles) as
    Phaser.Tilemaps.TilemapLayer;

    // 4. every tile whose "collides" property is ticked in Tiled's tileset
    becomes solid
    this.ground.setCollisionByProperty({ collides: true });
}

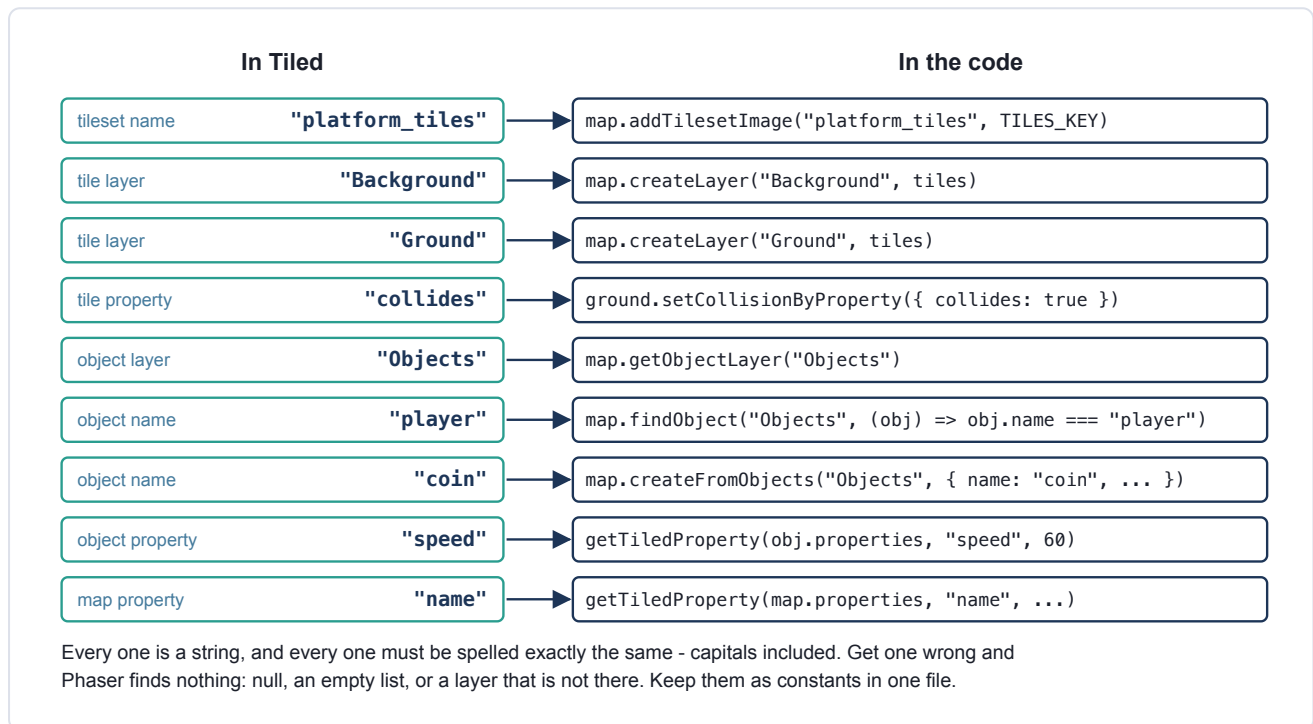
```

1. `this.make.tilemap({ key })` - in Chapter 10 you passed `data`, an array. Pass a `key` instead and Phaser builds the `Tilemap` from the loaded JSON: every layer, tileset and object in it. A `Tilemap` is not drawn; it is the data
2. `addTilesetImage(name, key)` joins the tileset **named in the map** to the picture **loaded under a key**. Here both are `"platform_tiles"`, but they are two different things - the first comes from Tiled, the second from your `load.image` call. If the map has no tileset with that name - or no picture was loaded with that key - you get `null`
3. `createLayer(name, tileset)` makes a `TilemapLayer` - a game object that draws one layer. Layers are drawn in the order you make them, so make the background first. The `as` is needed because `createLayer` is typed to return either an ordinary layer or a `TilemapGPULayer` (a faster, more limited kind you get by passing `true` as a fifth argument)

4. `setCollisionByProperty({ collides: true })` finds every tile in the layer whose Tiled properties include `collides: true`, and makes it solid. After that, a collider works just as in Chapter 10:

```
this.physics.add.collider(this.player, this.ground);
```

Every name in that code came from Tiled, and every one must be spelled exactly the same - capital letters too:



That is why they are constants in `assets.ts`: each name is written once. When one is wrong, Phaser does not stop with a clear error - it writes a warning in the browser's console (press F12) and hands back `null` or nothing, and the game fails a line or two later. Check the console first.

Seeing what is solid

Press **D** in the game. `toggleDebug()` calls `this.ground.renderDebug(graphics, colours)`, which draws the layer's colliding tiles:



Every red tile is one with `collides` ticked in Tiled. The spikes, water and lava are not red - the player falls into them.

Reading a tile's properties

The `hazard` property is not used by Phaser at all. The scene reads it itself, every frame:

```
// Is the player touching a tile with the "hazard" property (spikes, water,
// lava)? Phaser has
// turned each tile's Tiled properties into an object, so it is just
// tile.properties.hazard.
private touchingHazard(): boolean {
    const body = this.player.body as Phaser.Physics.Arcade.Body;
    // the tiles under the player's body - trimmed by 2 pixels, so only a real
    // touch counts
    const tiles = this.ground.getTilesWithinWorldXY(body.x + 2, body.y + 2,
    body.width - 4, body.height - 2);
    return tiles.some((tile) => tile.properties.hazard === true);
}
```

`getTilesWithinWorldXY(x, y, width, height)` returns every tile in a rectangle of the world - here, the player's body. Each `Tile` has a `properties` object holding its custom properties from Tiled, so a spike tile has `properties.hazard === true` and a grass tile has `properties.collides === true`. `some(...)` is true if the function says yes for at least one tile - like Java's `stream().anyMatch(...)`.

Reading the object layer

Phaser gives you three ways into an object layer. The level uses all three, one for each kind of thing.

findObject - one particular object

src/scenes/GameScene.ts

```
// findObject: the first object in the layer that the function says yes to
const spawn = this.map.findObject(OBJECTS_LAYER, (obj) => obj.name ===
  "player");
if (spawn === null) {
  throw new Error('The map has no object called "player" in its Objects
  layer');
}
// x and y are optional in Phaser's type for a Tiled object, so give them a
  fallback
this.spawnX = spawn.x ?? 0;
this.spawnY = spawn.y ?? 0;
```

Each object is a plain object (Phaser's type is `Phaser.Types.Tilemaps.TiledObject`) with the fields from the `.tmj`: `name`, `x`, `y`, `width`, `height`, `properties`.

`findObject` returns the first one your function accepts - or `null`, which TypeScript makes you deal with.

createFromObjects - a sprite for every matching object

```
// createFromObjects: a Sprite for every object called "coin", placed where
  the object is, with
// the texture we ask for
const coinSprites = this.map.createFromObjects(OBJECTS_LAYER, { name:
  "coin", key: COIN_KEY });
this.coinsTotal = coinSprites.length;

// a static physics group gives each coin a body that never moves, so the
  player can touch it
this.coins = this.physics.add.staticGroup(coinSprites);
this.coins.playAnimation(COIN_SPIN);
```

`createFromObjects` does the whole job for simple things: it makes a `Sprite` for each object called `"coin"`, at the object's position, with the texture you name, already added to the scene. (You can also choose objects by `id`, `gid` or `type`, and ask for your own class with `classType`.) It copies each object's custom properties too: one whose name matches a sprite field (`alpha`, `angle` ...) sets that field; any other goes into the sprite's **data**, read back with `sprite.getData("name")`.

getObjectLayer - every object, to do as you like with

Enemies need more than a sprite: each one is a `Slime`, with a speed from Tiled. Here the scene looks through the objects itself:

```
// getObjectLayer(...).objects: every object in the layer, to look through ourselves
const objectLayer = this.map.getObjectLayer(OBJECTS_LAYER);
if (objectLayer === null) {
    throw new Error(`The map has no object layer called "${OBJECTS_LAYER}"`);
}

const slimes: Slime[] = [];
for (const obj of objectLayer.objects) {
    if (obj.name === "enemy") {
        // a custom property, set on each enemy in Tiled
        const speed = getTiledProperty(obj.properties, "speed",
DEFAULT_SLIME_SPEED);
        slimes.push(new Slime(this, obj.x ?? 0, obj.y ?? 0, speed,
this.ground));
    }
}
```

Reading an object's custom properties

There is one untidy thing to know. Phaser turns a **tile's** properties into a handy object (`tile.properties.hazard`), but leaves an **object's** properties - and the **map's** - just as Tiled wrote them: a list of `{ name, type, value }`. So the project has a small function to look one up:

```
src/tiled.ts
```

```

export function getTiledProperty<T>(
  properties: unknown,
  name: string,
  fallback: T,
): T {
  if (!Array.isArray(properties)) {
    return fallback;
  }
  const found = (properties as TiledProperty[]).find((property) =>
property.name === name);
  if (found === undefined || typeof found.value !== typeof fallback) {
    return fallback;
  }
  // safe: the line above checked that the value is the same kind
(string/number/boolean) as T
  return found.value as T;
}

```

`getTiledProperty(obj.properties, "speed", 60)` gives the enemy's speed - or 60 if the designer forgot to set one, or typed "fast" into it. A map is made by a person: the game should cope with mistakes.

Java and TypeScript - `<T>` makes this a **generic** function, just like a generic method in Java: `<T> T getTiledProperty(Object properties, String name, T fallback)`. TypeScript works out `T` from the fallback - a number here, a string for a door's target - so the result needs no cast. And, as in Java, generic types are **erased** when the code runs: the function cannot ask "is this a `T`?". Instead it compares `typeof` the value with `typeof` the fallback - "number", "string" or "boolean" - which the running program *can* see.

The level's name uses the same function, on the map's properties:

```

// a custom property on the MAP itself (Map > Map Properties in Tiled)
const levelName = getTiledProperty(this.map.properties, "name", "Unnamed
level");

```

Rectangles become zones

The finish is a rectangle object. A `Zone` is an invisible game object with a size - just right for "is the player in here?" - but remember from the objects diagram that Tiled measures a rectangle from its top-left corner, and a zone is placed by its centre:

```
// Tiled measures a rectangle from its top-left corner; a Zone - like most  
game objects - is  
placed by its centre. A Zone is an invisible rectangle: just right for  
"is the player here?"  
const zone = this.add.zone(x + width / 2, y + height / 2, width, height);  
this.physics.add.existing(zone, true);  
this.physics.add.overlap(this.player, zone, () => {  
    this.finish();  
});
```

An enemy that reads the map

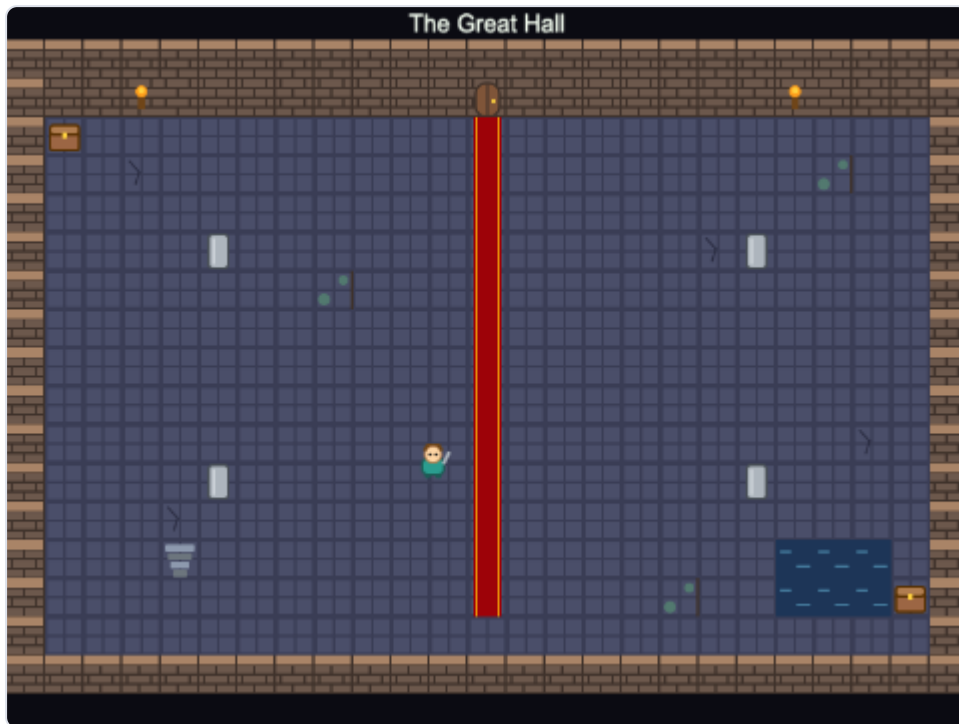
The `Slime` uses the tile layer too. Each frame it looks at the tile just past its front foot; if there is no tile there, or it is not solid, it would walk off an edge - so it turns round:

src/objects/Slime.ts

```
const aheadX = this.direction < 0 ? body.left - 1 : body.right + 1;  
const tileAhead = this.ground.getTileAtWorldXY(aheadX, body.bottom + 1);  
if (tileAhead === null || !tileAhead.collides) {  
    this.direction = -this.direction;  
}
```

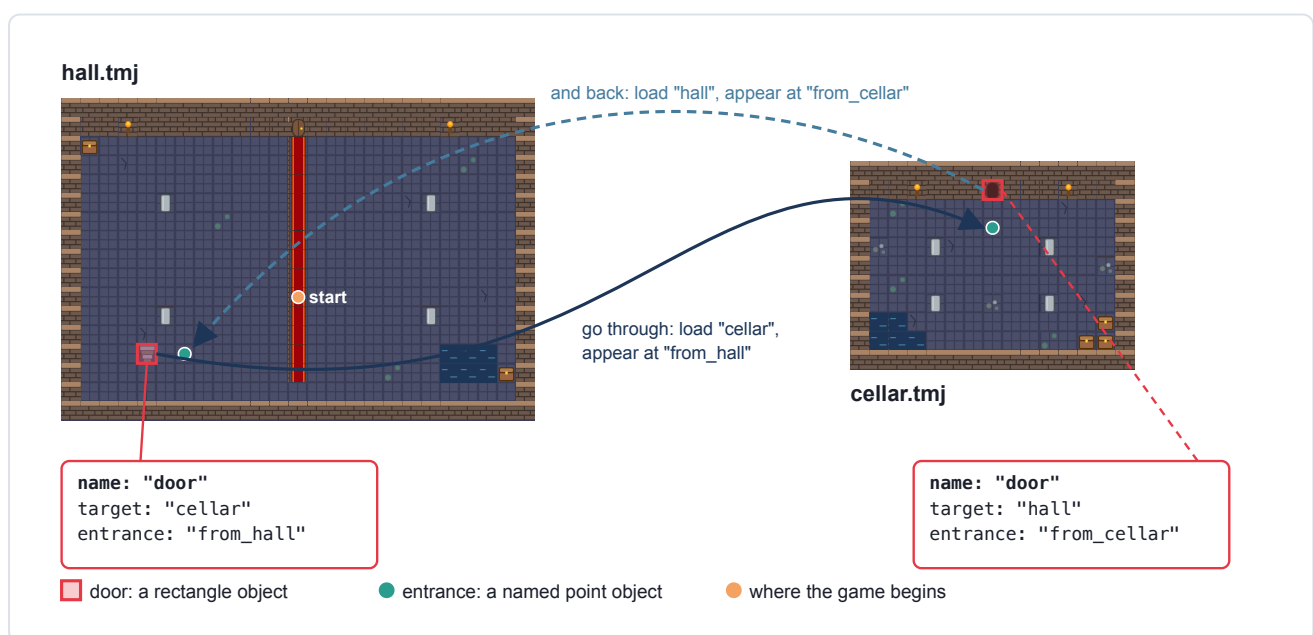
`tile.collides` is `true` for the tiles `setCollisionByProperty` made solid, so the slimes can patrol any platform a designer adds.

Doors between maps



`ch11_tiled_doors` has two rooms, each a Tiled map: `hall.tmj` and `cellar.tmj`. Walk onto the stairs in the hall and you go down to the cellar; walk through the cellar's door and you come back up, beside the stairs.

Each doorway is a rectangle object called `door`, with two custom string properties. Each place a player can arrive is a point object with a name of its own:



- `target` - the map the door leads to

- `entrance` - the name of the point object *in that map* where the player appears

Nothing in the code lists the rooms or says how they join. That is all in the maps.

One scene, restarted with a different map

There is only one scene. What it shows is decided by the data it is started with:

`src/scenes/RoomScene.ts`

```
// which map to show, and the name of the point object in it where the  
player appears  
export interface RoomData {  
  map: string;  
  entrance: string;  
}  
  
const FIRST_ROOM: RoomData = { map: "hall", entrance: "start" };
```

```
// The first time the game starts, Phaser passes no data - an empty object -  
so each value falls back  
// to the first room. Partial<RoomData> is RoomData with every field  
optional.  
init(data: Partial<RoomData>): void {  
  this.room = {  
    map: data.map ?? FIRST_ROOM.map,  
    entrance: data.entrance ?? FIRST_ROOM.entrance,  
  };  
  this.leaving = false;  
}
```

`Partial<RoomData>` is one of TypeScript's built-in **utility types**: `RoomData` with every field optional - exactly what `init` receives, nothing the first time and a full `RoomData` after that.

The scene loads only the map it needs:

```
// Only the map for THIS room. The map's name is also its key. The first visit to a room loads its file; after that it is already in the cache, and the loader does not load it again.
this.load.tilemapTiledJSON(this.room.map, MAPS_FOLDER + this.room.map + MAP_EXTENSION);
```

`preload()` runs every time the scene starts, but files whose keys are already in the cache are skipped. So the cellar's map is loaded the first time the player heads for the cellar, and never again (Chapter 3 has more on loading later).

`create()` then builds the room just as the level did - `make.tilemap`, `addTilesetImage`, `createLayer`, `setCollisionByProperty` - and puts the player at the entrance point it was given:

```
// the entrance is a point object whose NAME is the one we were given
const entrance = map.findObject(OBJECTS_LAYER, (obj) => obj.name === this.room.entrance);
```



A door is a zone that knows where it goes

`src/objects/Door.ts`


```

export class Door extends Phaser.GameObjects.Zone {
  public readonly target: string;
  public readonly entrance: string;

  constructor(scene: Phaser.Scene, obj: Phaser.Types.Tilemaps.TiledObject) {
    const x = obj.x ?? 0;
    const y = obj.y ?? 0;
    const width = obj.width ?? 32;
    const height = obj.height ?? 32;

    // Tiled measures a rectangle from its top-left corner; a Zone is placed
by its centre
    super(scene, x + width / 2, y + height / 2, width, height);

    this.target = getTiledProperty(obj.properties, "target", "");
    this.entrance = getTiledProperty(obj.properties, "entrance", "");
  }
}

```

`readonly` is like Java's `final` on a field: set in the constructor, never changed. The scene makes a `Door` for every object called `door` - `filterObjects` is `findObject`'s sibling, returning *all* the matches:

```

// filterObjects: every object in the layer that the function says yes to
const doorObjects = map.filterObjects(OBJECTS_LAYER, (obj) => obj.name ===
"door") ?? [];
for (const obj of doorObjects) {
  const door = new Door(this, obj);
  this.physics.add.overlap(this.player, door, () => {
    this.goThrough(door);
  });
}

```

Going through

```
private goThrough(door: Door): void {
    // overlap fires every frame the player is in the doorway: only act on the
    first
    if (this.leaving || !door.isOpen()) {
        return;
    }
    this.leaving = true;
    this.player.freeze();
    this.sound.play(DOOR_SOUND);

    // fade to black, THEN start this scene again with the next room (see
    Chapter 4 for camera fades)
    this.cameras.main.fadeOut(FADE_TIME);
    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, ()
=> {
        const next: RoomData = { map: door.target, entrance: door.entrance };
        this.scene.restart(next);
    });
}
```

`this.scene.restart(data)` shuts the scene down and starts it again, passing `data` to `init()` - exactly like `this.scene.start` in Chapter 2, but for the scene that is already running. Every room is built by the same `create()`.

Notice that the **entrance points are a tile or two away from the doors**, so the player does not arrive standing in a doorway and bounce straight back. The rooms are smaller than the screen, so the camera is pointed at the middle of each:

```
this.cameras.main.centerOn(map.widthInPixels / 2, map.heightInPixels / 2).
```

Common mistakes

Note - A name does not match. `addTilesetImage("platform", ...)` when the tileset is called `platform_tiles` gives *Tilemap has no tileset "platform". Its tilesets are ['platform_tiles']* in the console, and `null . createLayer("ground", ...)` when the layer is `Ground` gives *Invalid Tilemap Layer ID: ground* and *Valid tilelayer names: ['Background', 'Ground']*, then `null` - and the next line fails with *Cannot read properties of null (reading 'setCollisionByProperty')*. Phaser's warnings list the names it does have: compare them with yours.

Note - The map file is not there. A wrong path in `tilemapTiledJSON` gives a 404 in the browser's Network tab, then *No map data found for key level1* in the console, and every layer is missing. Did you export the map into `public/assets/maps/`, and build?

Note - Loading the .tmx. `tilemapTiledJSON` given `level1.tmx` fails with *SyntaxError: Unexpected token '<', "<?xml vers"... is not valid JSON*: Phaser reads the JSON export, not Tiled's XML file.

Note - You changed the map but the game did not. The game loads the `.tmj`, not the `.tmx`. Export (File > Export), build, then refresh.

Note - Everything falls through the floor. No tile has `collides` ticked, the property is spelled differently (`Collides` , `collide`), or it is a `string` property holding `"true"` instead of a `bool`. Press D to see which tiles are solid.

Note - Things are half a tile out. A rectangle's `x` and `y` are its top-left corner; a point's are the point. Add half the width and height to place a centred game object on a rectangle.

Summary

- Tiled is a free map editor. A map has tilesets, tile layers, object layers and custom properties - on tiles, objects and the map itself
- keep the source as `tilled/<name>.tmx` ; export JSON to `public/assets/maps/<name>.tmj` for the game. Use an **orthogonal**, **fixed-size** map, **CSV** tile data, and an **embedded** tileset
- the numbers in a Tiled tile layer are **GIDs**: firstgid + tile index, with 0 for "no tile". In Phaser `tile.index` is the GID, and an empty place is -1
- `this.load.tilemapTiledJSON(key, file)` loads the map; load the tileset's picture with `this.load.image`
- `this.make.tilemap({ key })`, then `map.addTilesetImage(nameInTiled, imageKey)`, then `map.createLayer(layerName, tileset)` - the names must match Tiled exactly
- `layer.setCollisionByProperty({ collides: true })` makes tiles solid by property; any tile's properties are in `tile.properties`
- read objects with `map.findObject` (one), `map.filterObjects` (all that match), `map.createFromObjects` (sprites, made for you) or `map.getObjectLayer(name).objects` (the lot). Object and map properties are a list of `{ name, type, value }`
- when the level is data, a new level - or a new room behind a door - needs a new map, not new code

Challenges

1. **Above the player** (*ch11_tiled_doors*) - In Tiled, add a third tile layer, **Above**, to the hall. Paint something on it that the player should walk *behind* - wall tops along the bottom wall, say, or a row of pillars. Load it so it is drawn over the player, and check the player can walk behind it without bumping into it.
2. **Gems** (*ch11_tiled_level*) - Some coins should be worth more. In Tiled, give a few of the coin objects an **int** custom property called **value**, set to 5. Those coins should look different (use `gem.png` from the asset library) and add 5 to a score; ordinary coins add 1. Show the score as well as the coin count.

3. **A third room** (*ch11_tiled_doors*) - The hall has a closed door at the end of the red carpet. Open it: make a new room in Tiled - a library, a garden, a treasure room - with a door back to the hall, and make the hall's door lead to it. How much code did you have to change?
4. **Signposts** (*ch11_tiled_level*) - Add rectangle objects called **sign** to the level, each with a **string** property **message**. While the player is inside a sign's rectangle, show its message in a box on screen; when they leave, hide it. Put one by the sign at the start that explains the controls. *Hint:* `this.physics.overlap(a, b)` - with no callback - returns `true` or `false` there and then, so it can be asked in `update()`.
5. **Moving platforms** (*ch11_tiled_level*) - With the Insert Polygon tool, draw a **polyline** object called **platform** over one of the pits (finish it without closing the shape). Make a platform that travels along the line and back again, forever, carrying the player. *Hint:* the points in `obj.polyline` are measured from the object's `x` and `y`. A chain of tweens (`this.tweens.chain`) can visit them in turn. The platform needs a physics body that is not moved by gravity or by the player - look at `setImmovable` and `setAllowGravity`. Arcade Physics will push the player up with a platform moved by a tween, but not sideways: that part is up to you.
6. **Spinning coins, as tiles** (*ch11_tiled_level*) - Add `coin_spin.png` to the map as a *second* tileset, called **coins**. In Tiled's tileset editor, use the **Tile Animation Editor** to make its first tile an animation of all six frames. Paint animated coins on a new tile layer, **Pickups**, and let the player collect them by touching them. *Hint:* what is the second tileset's firstgid? `addTilesetImage` is called once per tileset. Phaser 4 plays Tiled's tile animations by itself. Look up `layer.removeTileAt(x, y)`, and remember that an overlap with a tile layer reports *every* tile under the player - empty places (index -1) included.

Previous: [Chapter 10 - Tilemaps](#) · Next: [Chapter 12 - Higher or lower](#)

Higher or lower

The first of the game chapters. A card is face up; will the next one be higher or lower? It is about the simplest card game there is, which makes it the right place to practise something every game needs: keeping the **rules** of the game apart from the **pictures** of it. You build the game three times - simple, intermediate and advanced - and each version grows out of the one before.



What you will learn

- what makes a guessing game worth playing: risk, streaks, and the moment of the reveal
- how to model playing cards with a `Card` data class, a `Deck` and plain rule functions - separate from the `CardSprite` that draws a card (model/view separation)
- how to shuffle properly with the Fisher-Yates shuffle, and why the popular `sort(() => Math.random() - 0.5)` trick is wrong
- how to describe a game's state with a union of string literals, and ignore input while a card is being revealed
- how to turn a card over with a two-part tween, and make a reusable button with hover states

- how a game grows: from one scene and a few fields to scoring, lives, a history row and saved records

The projects

Project	What it shows
<code>ch12_higher_or_lower_simple</code>	one scene: a <code>Card</code> class, card pictures, H and L keys, five right in a row to win, a wrong guess sends you back to zero
<code>ch12_higher_or_lower_intermediate</code>	a real shuffled <code>Deck</code> , a <code>CardSprite</code> that flips, clickable buttons, a streak display, sounds, title and game over scenes
<code>ch12_higher_or_lower_advanced</code>	one deck and three lives, a pot of points that pays more for risky guesses, cash out, a row of cards played, a best score and longest run saved in the browser, and polish

What makes a guessing game fun?

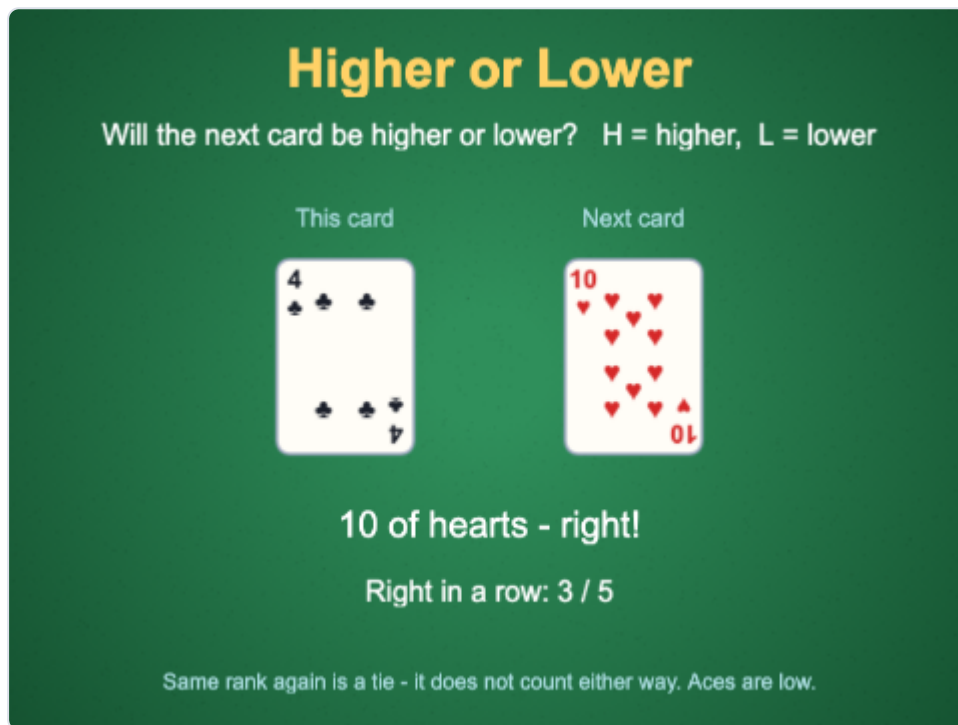
On paper, higher or lower is barely a game: the player picks one of two options and the deck decides. Yet people play it on television, in pubs and in every casino. Three things carry it:

- **Risk.** Guessing "higher" on a 2 is nearly safe; guessing "higher" on a jack is a gamble. A good version of the game makes the player *feel* the difference - and pays them for taking the risk.
- **Streaks.** One right guess is luck. Five in a row feels like skill (it mostly is not). Show the streak growing, and let the player lose it, and every guess matters more than the last.
- **The reveal.** The half-second while a card turns over is the whole game. Do not rush it, do not let the player skip it, and let the result land with a sound.

Watch those three grow through the chapter. The simple version has a streak and a reveal of sorts; the intermediate version makes the reveal a real moment; the advanced version adds risk and reward.

Two rules need deciding before any code is written. **Aces are low** - an ace is 1, below a 2. And **a tie does not count**: if the next card has the same rank, the guess was neither right nor wrong, and play carries on. (Some versions count a tie as a loss. Ours is kinder, and it is a one-line change - which is itself a good reason to keep the rules in one place.)

Version 1 - simple



`ch12_higher_or_lower_simple` is a complete game in one scene, `GameScene`, plus one small class. Press H or L; the next card is shown; after a moment it becomes the card to beat.

A card is data

`src/Card.ts`


```

export type Suit = "clubs" | "diamonds" | "hearts" | "spades";

// in the same order as the rows of cards.png
export const SUITS: Suit[] = ["clubs", "diamonds", "hearts", "spades"];
...
export class Card {
  // readonly: set once, in the constructor, and never changed - a card does
  // not turn into another
  public readonly suit: Suit;
  public readonly rank: number; // 1 = ace, 2 - 10, 11 = jack, 12 = queen,
  // 13 = king

  constructor(suit: Suit, rank: number) {
    this.suit = suit;
    this.rank = rank;
  }

  // what "higher" and "lower" compare. Aces are low, so the value is just
  // the rank.
  public get value(): number {
    return this.rank;
  }
}

```

Three TypeScript ideas in a few lines:

- `type Suit = "clubs" | "diamonds" | ...` is a **union of string literals**. A `Suit` is a string, but only one of those four. Write `new Card("heart", 3)` and the build fails. In Java you would reach for an `enum`; TypeScript has enums too, but a union of strings is lighter, and the value prints as itself in the console.
- `readonly` is Java's `final` for a field. A card never changes, so say so.
- `get value()` is a **getter**: a method you read like a field (`card.value`, no brackets). Right now it just returns the rank, so why not use `rank` everywhere? Because "the number the rules compare" and "which rank the card is" are different ideas that happen to be equal today. Make aces high and they stop being equal - and only `value` needs to change.

Notice what is **not** in `Card`: no `import Phaser`, no position, no picture, no "face up". A `Card` is what the rules care about. That is the first half of model/view separation.

`Card` also has a `static random()` method that makes any card at all. There is no deck in this version, so the same card can come up twice in a row - hold that thought.

Which picture shows a card?

The pictures are all in one sprite sheet, `cards.png` from the asset library: 80 x 112 pixels a card, thirteen to a row, one row per suit, then two card backs.

Frames of `cards.png`: $\text{frame} = \text{suit} * 13 + (\text{rank} - 1)$

	A (1)	2 (2)	3 (3)	4 (4)	5 (5)	6 (6)	7 (7)	8 (8)	9 (9)	10 (10)	J (11)	Q (12)	K (13)
0 clubs	0	1	2	3	4	5	6	7	8	9	10	11	12
1 diamonds	13	14	15	16	17	18	19	20	21	22	23	24	25
2 hearts	26	27	28	29	30	31	32	33	34	35	36	37	38
3 spades	39	40	41	42	43	44	45	46	47	48	49	50	51
	52	53	52 and 53: the two backs										

Queen of hearts: suit 2, rank 12, so frame $2 * 13 + (12 - 1) = 37$

`src/scenes/GameScene.ts`

```
// The picture of a card is frame suit * 13 + (rank - 1) of cards.png: one
// row per suit, ace to
// king. The Card does not know this - pictures are the scene's business,
// not the card's.
private frameFor(card: Card): number {
    return SUITS.indexOf(card.suit) * 13 + (card.rank - 1);
}
```

It is loaded with `this.load.spritesheet(CARDS_KEY, CARDS_FILE, { frameWidth: CARD_WIDTH, frameHeight: CARD_HEIGHT })` (Chapter 3), shown with `this.add.image(x, y, CARDS_KEY, frame)`, and changed with `setFrame(frame)`. The face-down card is frame 52, `BACK_FRAME`.

The rules, in one place

`src/scenes/GameScene.ts`

```

type Guess = "higher" | "lower";
type Outcome = "right" | "wrong" | "tie";
...
// Was the guess right? The rules of the game, in one place.
private judge(guess: Guess, current: Card, next: Card): Outcome {
    if (next.value === current.value) {
        return "tie";
    }
    const wentHigher = next.value > current.value;
    if ((guess === "higher" && wentHigher) || (guess === "lower" &&
!wentHigher)) {
        return "right";
    }
    return "wrong";
}

```

More unions. A method returning `Outcome` can only return one of three strings, and a `boolean` would not do: there are three answers, not two. `judge` looks only at cards - it plays no sound and changes no text. The caller decides what each outcome *looks* like.

Game state: when to ignore the player

The game is not always ready for a guess. While the result is on screen, pressing H again should do nothing - otherwise a quick player guesses against a card they have not seen yet. And once the game is won, H and L should do nothing at all. So the scene keeps track of what it is doing:

```

// What the game is doing right now. While a card is being revealed, H and L
are ignored - otherwise
// a fast player could guess again before seeing the result.
type GameState = "waiting" | "revealing" | "won";
...
private state: GameState = "waiting";

```

and the first thing `guess()` does is check it:

```

private guess(guess: Guess): void {
  if (this.state !== "waiting") {
    return; // a card is still being revealed, or the game is over
  }
  this.state = "revealing";

  // turn over the next card
  const next = Card.random();
  this.nextImage.setFrame(this.frameFor(next));
  this.sound.play(FLIP_KEY);

  const outcome = this.judge(guess, this.currentCard, next);
  if (outcome === "right") {
    this.inARow = this.inARow + 1;
    ...
  }
  this.showStreak();

  // leave the result on screen for a moment, then the next card becomes the
one to beat
  this.time.delayedCall(REVEAL_TIME, () => {
    this.moveOn(next);
  });
}

```

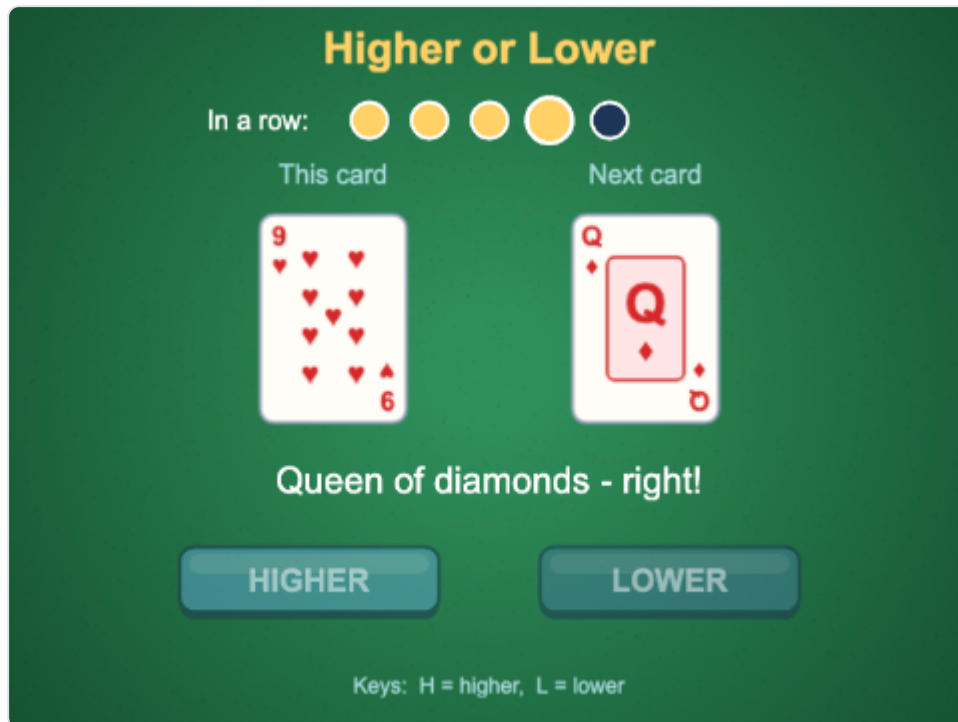
`moveOn()` puts the new card in the "this card" place, turns the next card face down, and sets `state` back to "waiting" - or to "won" after five in a row, when SPACE calls `this.scene.restart().init()` resets `state` and `inARow`, because a restarted scene is the same object as before (Chapter 2's trap).

This `state` field is a tiny **state machine**: a value that says which of a few situations the game is in, and code that behaves differently in each. Every game in the rest of the book has one.

What the simple version is missing

Play it for a minute and you will find its limits. The next card just *appears*. The same card can turn up twice. There is nothing to click, no title screen, and no way to lose. The rules and the pictures are both inside `GameScene`, which is fine at 175 lines but will not stay fine. The intermediate version fixes all of that.

Version 2 - intermediate

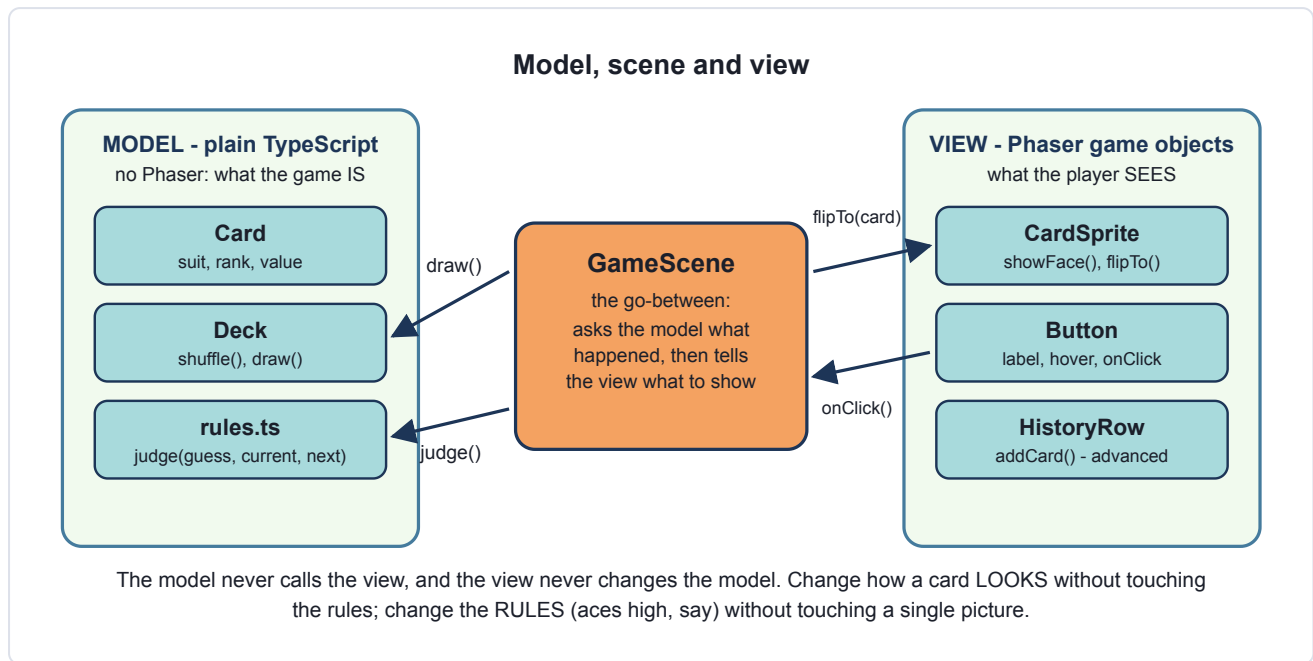


`ch12_higher_or_lower_intermediate` is a proper little game: a title screen, a shuffled deck, cards that flip, HIGHER and LOWER buttons, five lights for the streak, sounds, and a game over screen. Five right in a row wins; one wrong guess and you are out.

The code has spread out into files, and *where* each thing lives is the main lesson:

```
src/
  Card.ts           the model: plain TypeScript, no Phaser
  Deck.ts
  rules.ts
  assets.ts         every asset key and file
  objects/
    CardSprite.ts  the view: Phaser game objects
    Button.ts
  scenes/
    keys.ts  PreloadScene.ts  TitleScene.ts  GameScene.ts  GameOverScene.ts
```

Model and view



The **model** is what the game *is*: cards, a deck, the rules. The **view** is what the player *sees*: pictures, buttons, animations. The **scene** sits between them - it asks the model what happened, then tells the view what to show.

`judge()` has moved out of the scene into `src/rules.ts`, as a plain exported function, and the `Guess` and `Outcome` types moved with it. The frame arithmetic has moved the other way, out of the scene and into the view, `CardSprite`.

Why go to this trouble for a card game? Because the two halves change for different reasons. A designer wants the cards to lift as they turn: that is `CardSprite`, and the rules cannot break. A player wants aces high: that is `Card.value`, and no picture can break. You could test `judge()` and `Deck` with no browser at all. And when the next chapter needs a deck of cards, `Card.ts` and `Deck.ts` can be copied across unchanged - there is nothing in them about this game's screen.

Note - you will meet this idea under other names: MVC (model-view-controller), or "separating logic from presentation". The rule of thumb is the same: the model never imports Phaser, and the view never decides what the rules are.

A real deck

`src/Deck.ts`

```

export class Deck {
  private cards: Card[] = [];

  constructor() {
    for (const suit of SUITS) {
      for (let rank = 1; rank <= 13; rank++) {
        this.cards.push(new Card(suit, rank));
      }
    }
    this.shuffle();
  }

  // The Fisher-Yates shuffle. Work backwards through the deck; swap each
  // card with a card chosen
  // at random from the ones not yet placed (itself included). Every one of
  // the 52! possible orders
  // is exactly as likely as every other.
  public shuffle(): void {
    for (let i = this.cards.length - 1; i > 0; i--) {
      const j = Math.floor(Math.random() * (i + 1)); // 0 to i, inclusive
      const swap = this.cards[i];
      this.cards[i] = this.cards[j];
      this.cards[j] = swap;
    }
  }

  // take the top card off the deck
  public draw(): Card {
    const card = this.cards.pop();
    if (card === undefined) {
      throw new Error("The deck is empty");
    }
    return card;
  }
}

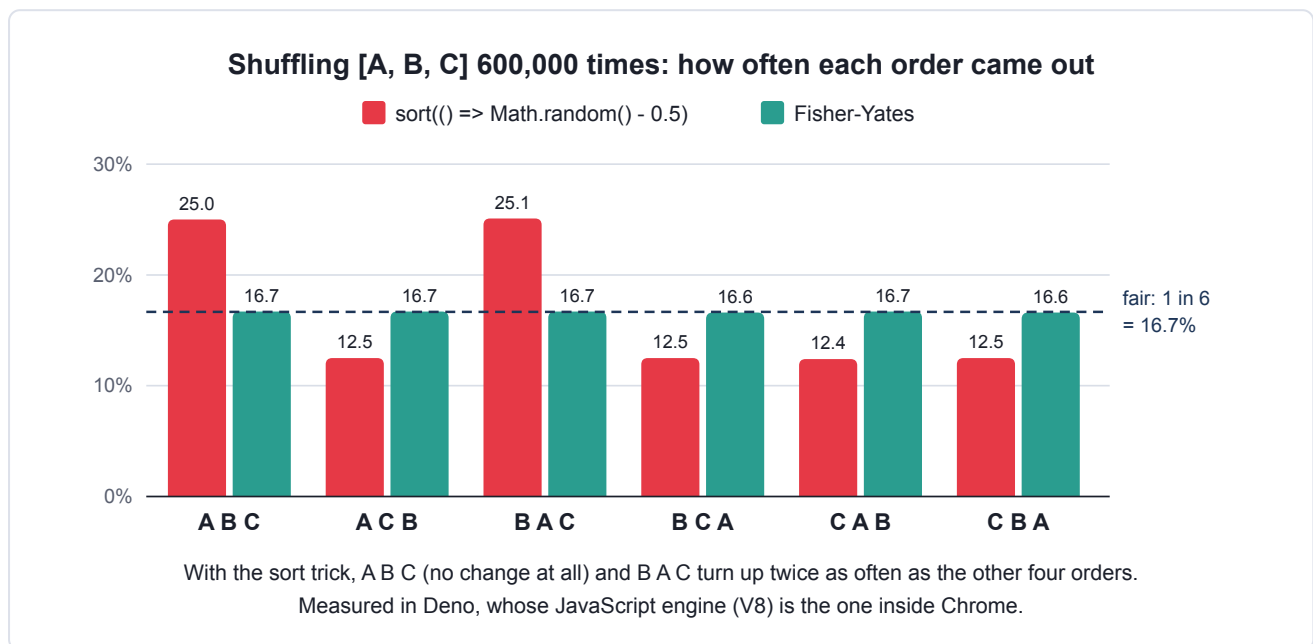
```

The **Fisher-Yates shuffle** is the one to know. Think of it as dealing the deck into a new order from the back: position 51 gets a card picked at random from all 52; position 50 gets one of the 51 left; and so on. Each step is a fair pick from what is left, so the whole order is fair, and it takes one pass through the deck.

Search online for "shuffle an array in JavaScript" and you will soon find this instead:

```
cards.sort(() => Math.random() - 0.5); // DON'T
```

It looks clever - "sort, but let each comparison come out at random" - and it is wrong. A sort function expects its comparisons to be consistent (if a is before b and b before c, then a is before c). Random answers break that promise, and what comes out depends on the order the sort happens to compare things in. It is not even close to fair:

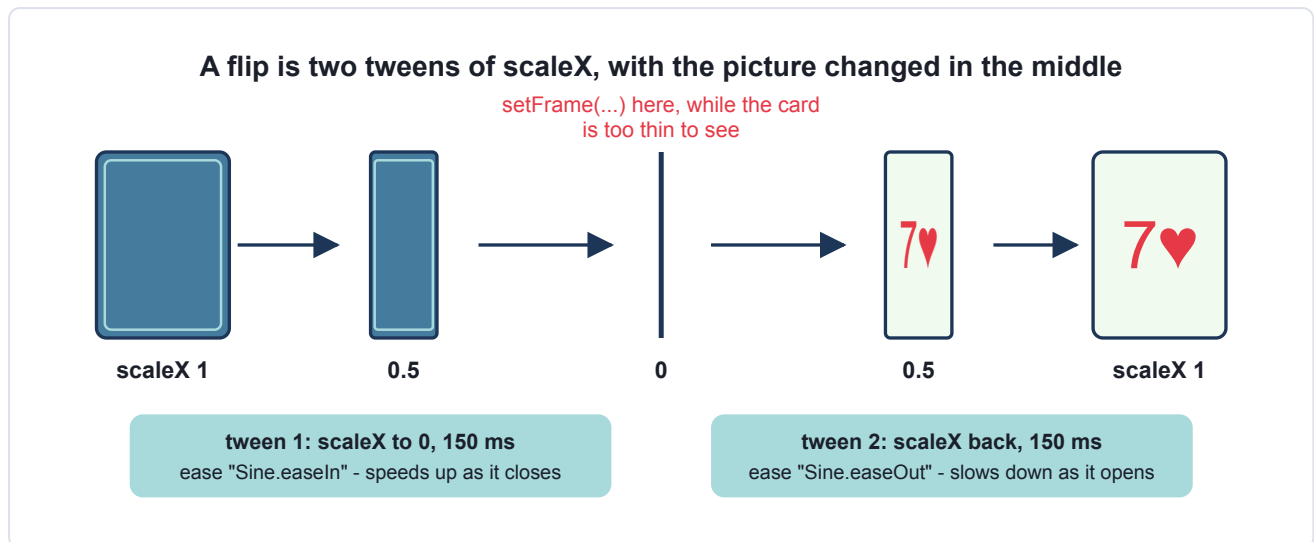


That chart is real: 600,000 shuffles of three cards in each way. With the sort trick, the order that is *not shuffled at all* comes out a quarter of the time instead of a sixth. On a full deck it is as bad: the top card stays on top about 6% of the time instead of 2%. In a card game, that is a bug a sharp player can feel.

Phaser has a correct shuffle built in - `Phaser.Utils.Array.Shuffle(array)` is Fisher-Yates too. We write our own because it is worth understanding, and because it keeps `Deck` free of Phaser.

`draw()` shows a small TypeScript safety net. `pop()` returns `Card | undefined` - an empty array has nothing to pop - so TypeScript will not let `draw()` promise a `Card` until the `undefined` case is dealt with. Throwing an error is right here: in this version the deck cannot run out (a game is over long before 52 cards), so an empty deck would mean a bug.

Flipping a card



A card on screen is flat, so it cannot really turn over. The trick is to squash it to nothing across, change the picture while it is invisible, and stretch it out again - and the eye sees a flip.

```
src/objects/CardSprite.ts
```

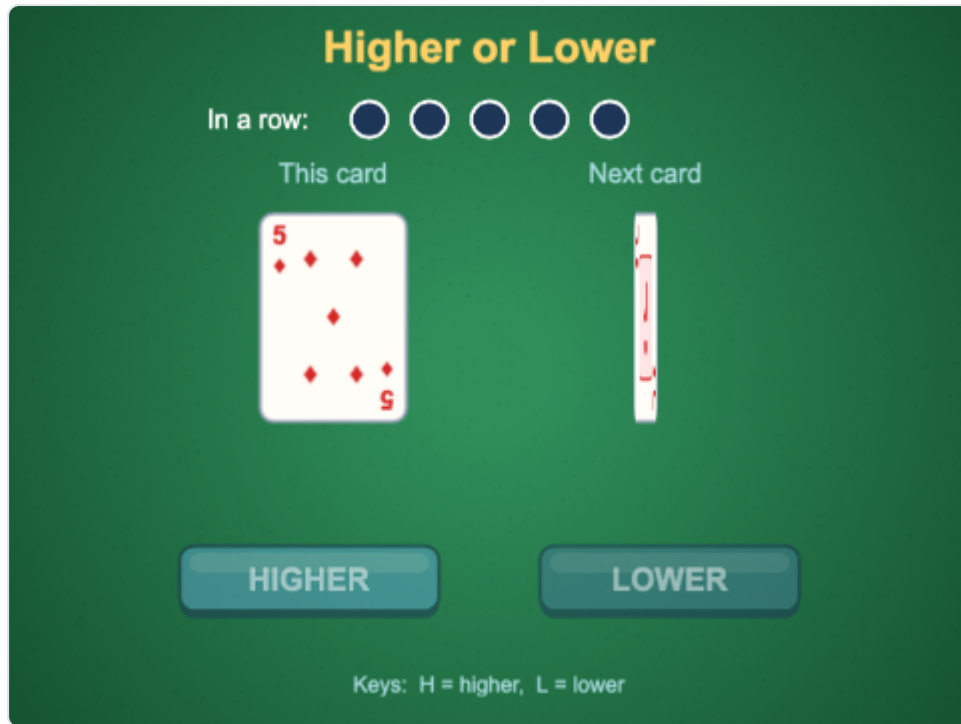
```

export class CardSprite extends Phaser.GameObjects.Image {
  ...
  public flipTo(card: Card | null, onComplete: () => void): void {
    this.scene tweens.chain({
      targets: this,
      tweens: [
        {
          scaleX: 0,
          duration: FLIP_TIME / 2,
          ease: "Sine.easeIn",
          onComplete: () => {
            if (card === null) {
              this.showBack();
            } else {
              this.showFace(card);
            }
          },
        },
        {
          scaleX: this.baseScale,
          duration: FLIP_TIME / 2,
          ease: "Sine.easeOut",
        },
      ],
      onComplete: onComplete,
    });
  }
}

```

`this.scene.tweens.chain(...)` (Chapter 7) plays the two tweens one after the other. The first has its own `onComplete`, which swaps the picture at the exact moment the card is edge-on. The chain has an `onComplete` too, which calls back whoever asked for the flip. `card: Card | null` means "a card to show, or `null` for the back".

The easing matters more than you would think. `Sine.easeIn` starts slowly and speeds up; `easeOut` does the opposite. Together they make the card snap through the edge-on moment, as a real card does. Try `"Linear"` for both and it looks like a door.



Why `scaleX` and not `scale`? `scale` sets both directions, so the card would shrink to a dot. The card's image keeps its origin in the middle (the default), so it narrows towards its centre line.

`CardSprite` also has `showFace(card)`, `showBack()`, and the static `frameFor(card)` that used to be in the scene. It extends `Image`, as Chapter 1's `Ball` did - it needs a picture and a frame, not animations. The name says what it is to the game, not which Phaser class it happens to extend.

Buttons

The keys still work, but a card game wants buttons. `src/objects/Button.ts` is a small class that every later chapter could use:

```

export class Button extends Phaser.GameObjects.Container {
  private background: Phaser.GameObjects.Image;
  private label: Phaser.GameObjects.Text;
  private enabled = true;

  constructor(scene: Phaser.Scene, x: number, y: number, text: string,
onClick: () => void) {
    super(scene, x, y);

    // positions inside a container are relative to the container: (0, 0) is
its centre
    this.background = scene.add.image(0, 0, BUTTON_KEY);
    this.label = scene.add.text(0, -2, text, { ... }).setOrigin(0.5);
    this.add([this.background, this.label]);

    this.background.setInteractive({ useHandCursor: true });

    // three pictures for three states: normal, pointer over it, pressed
    this.background.on("pointerover", () => {
      this.background.setTexture(BUTTON_OVER_KEY);
    });
    ...
    this.background.on("pointerup", () => {
      this.background.setTexture(BUTTON_OVER_KEY);
      if (this.enabled) {
        scene.sound.play(CLICK_KEY);
        onClick();
      }
    });

    scene.add.existing(this);
  }

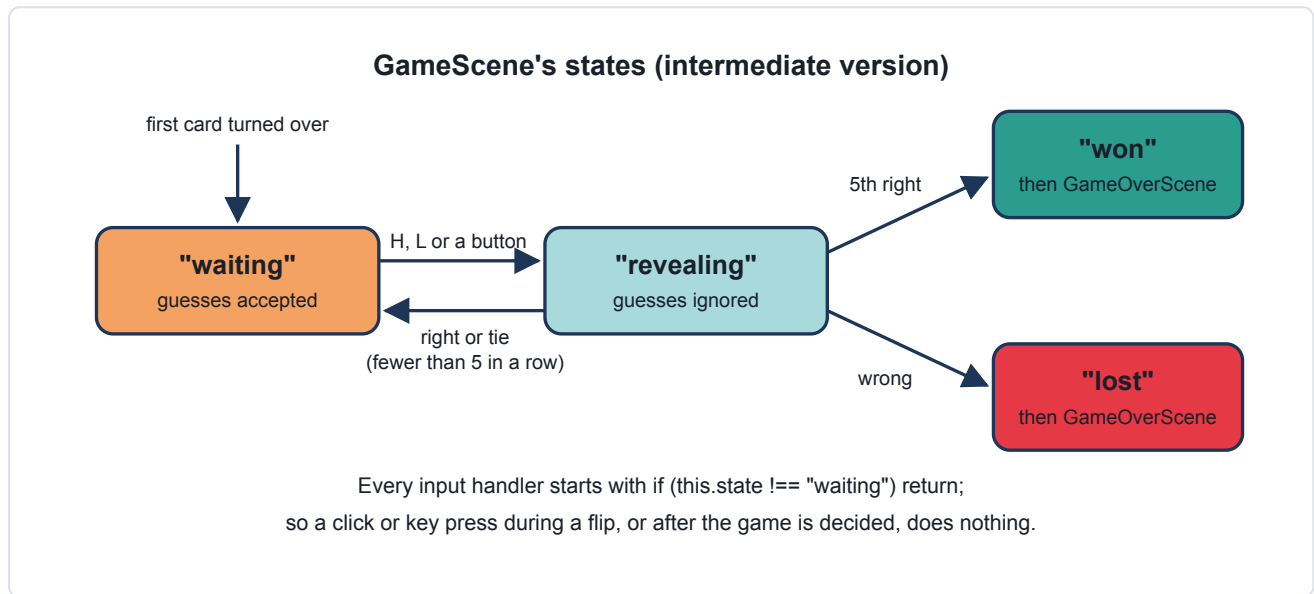
```

- A `Container` is a game object that holds other game objects - here a picture and a label - and moves, fades and hides them together. `setAlpha(0.5)` on the button fades both.
- The asset library has three button pictures: `button.png`, `button_over.png` and `button_down.png`. Swapping the texture on `"pointerover"`, `"pointerout"` and `"pointerdown"` is what makes a button feel alive.
- The action happens on `"pointerup"`, not `"pointerdown"`: that is how buttons behave everywhere else, and it lets a player press, change their mind, and slide off.

- The caller passes in `onClick`, a function. `Button` has no idea what it is for - the same class makes PLAY, HIGHER, LOWER, PLAY AGAIN and MENU.

`setEnabled(false)` fades the button and makes clicks do nothing. The game scene switches the buttons off while a card is turning, so the player can see that now is not the time to click.

The state machine, grown up



`src/scenes/GameScene.ts`

```
// What the game is doing right now. Only "waiting" accepts a guess.  
type GameState = "waiting" | "revealing" | "won" | "lost";
```

The guess now happens in steps, each started by the one before:

```
private guess(guess: Guess): void {
    if (this.state !== "waiting") {
        return; // a card is turning over, or the game is over: ignore the click
        or key
    }
    this.state = "revealing";
    this.setButtonsEnabled(false);
    this.messageText.setText("");

    const next = this.deck.draw();
    this.sound.play(FLIP_KEY);
    this.nextSprite.flipTo(next, () => {
        this.reveal(guess, next);
    });
}
```

The result is not judged until the flip has finished: `reveal()` runs from the flip's `onComplete`. So the player sees the card *before* the sound tells them whether they were right. That ordering is the reveal - get it wrong (play the sound first) and the tension is gone.

`reveal()` asks `judge()`, then does one of three things: on a wrong guess, sets "lost" and starts `GameOverScene` after a pause; on a fifth right guess, sets "won" and does the same; otherwise it waits a moment and calls `moveOn()`, which shows the new card in the "this card" place and flips the next card face down again - and when *that* flip finishes, `waitForGuess()` sets "waiting" and switches the buttons back on.

Because the H key, the L key and both buttons all call the same `guess()`, one check at the top covers them all. If input were handled in four places, there would be four places to forget it.

Streak, sounds and endings

The streak is five circles made with `this.add.circle(...)`. A right guess fills the next one with gold and gives it a quick `yoyo` tween to 1.4 times its size and back - a small thing that makes a right answer feel like progress.

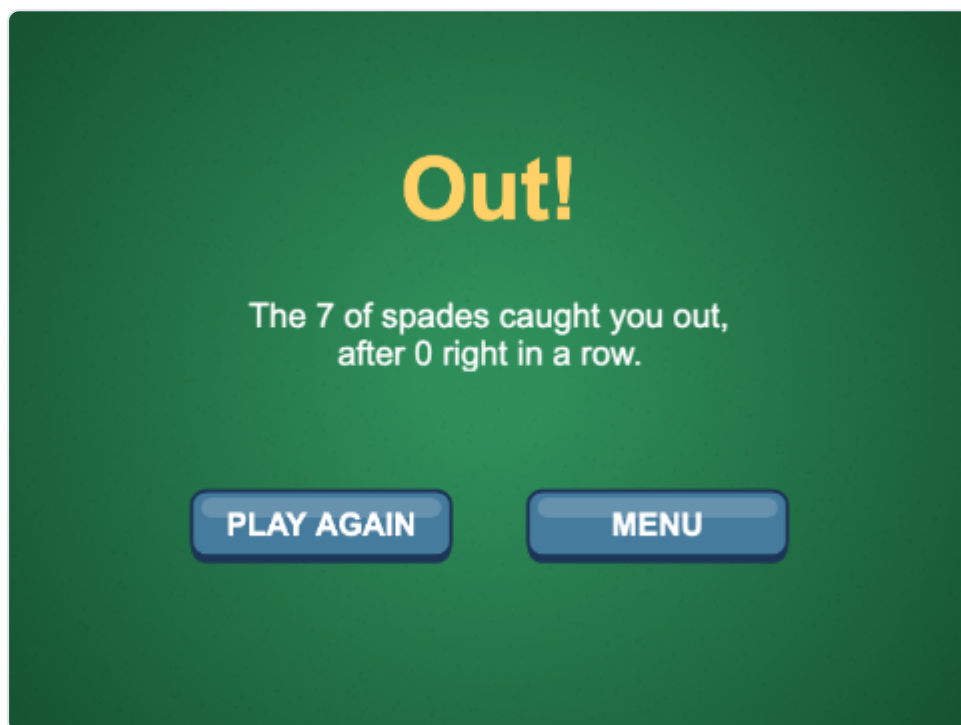
Sounds come from the asset library: `flip.wav` as a card turns, `correct.wav` or `wrong.wav` for the result, `card_place.wav` for a tie and as a card goes down, `click.wav` for buttons, and `win.wav` or `lose.wav` on the last screen (Chapter 5).

Winning and losing go to the **same** scene, `GameOverScene`, with an interface saying which:

src/scenes/GameOverScene.ts

```
export interface GameOverData {  
  won: boolean;  
  inARow: number;  
  lastCard: string;  
}
```

The two endings show the same things - a heading, a line about what happened, PLAY AGAIN and MENU - so one scene reading `won` is simpler than two scenes that are nearly the same.



Version 3 - advanced



`ch12_higher_or_lower_advanced` turns a coin-toss into a game of nerve. You play right through one deck, with three lives:

- every right guess adds points to a **pot** - more for a risky guess, and more the longer the run
- **CASH OUT** (or C) banks the pot into your score, where it is safe - but ends the run
- a **wrong** guess loses the whole pot **and** a life
- the game ends when the last life goes, or the deck runs out (and whatever is in the pot is banked)

The model gained scoring rules and a records class; the view gained a history row; the scene gained lives, a score, a pot, and polish. `Card`, `Deck` and `Button` barely changed.

Paying for risk

`src/rules.ts`


```

// Of the 13 ranks, how many would make this guess right? "Higher" on a 4 is
won by 5 to king: 9.
// (This treats every rank as equally likely - it does not look at which
cards are left in the deck.)
export function winningRanks(guess: Guess, current: Card): number {
  if (guess === "higher") {
    return 13 - current.value;
  }
  return current.value - 1;
}

// The riskier the guess, the more it pays: x1 if most ranks win, x2 for
about half, x3 for a long
// shot. 0 means the guess cannot win at all ("higher" on a king).
export function riskMultiplier(guess: Guess, current: Card): number {
  const wins = winningRanks(guess, current);
  if (wins === 0) {
    return 0;
  }
  if (wins >= 7) {
    return 1;
  }
  if (wins >= 4) {
    return 2;
  }
  return 3;
}

// What a right guess adds to the pot: more for a risky guess, and more the
longer the run.
export function pointsFor(multiplier: number, inARow: number): number {
  return BASE_POINTS * multiplier * inARow;
}

```

So three safe right guesses in a row are worth $10 + 20 + 30 = 60$ - and each one after that is worth more again. That is the streak doing its job: the longer you go, the more there is to lose, and the harder it is to stop.

The player should not have to do this arithmetic, so the buttons show it. Every time the game is ready for a guess:

```
src/scenes/GameScene.ts
```

```
private waitForGuess(): void {
    this.state = "waiting";
    this.messageText.setText(this.pot > 0 ? "Higher, lower - or cash out?" :
    "Higher or lower?");

    // show what each guess would pay; a guess that cannot win ("higher" on a
    king) is switched off
    const higher = riskMultiplier("higher", this.currentCard);
    const lower = riskMultiplier("lower", this.currentCard);
    this.higherButton.setText(higher > 0 ? `HIGHER x${higher}` : "HIGHER");
    this.lowerButton.setText(lower > 0 ? `LOWER x${lower}` : "LOWER");
    this.higherButton.setEnabled(higher > 0);
    this.lowerButton.setEnabled(lower > 0);
    this.cashOutButton.setEnabled(this.pot > 0);
}
```

Button gained a `setText()` method for this. Showing the stakes *before* the guess is what turns risk from a hidden number into a decision the player makes.

The pot, and cashing out

```
private cashOut(): void {
    if (this.state !== "waiting" || this.pot === 0) {
        return;
    }
    const banked = this.pot;
    this.score = this.score + this.pot;
    this.pot = 0;
    this.inARow = 0; // cashing out ends the run: the next right guess starts
    the pot again
    this.sound.play(COIN_KEY);
    this.sparkles.explode(30, this.scoreText.x + this.scoreText.width / 2,
    this.scoreText.y);
    this.pulse(this.scoreText);
    this.updateHud();
    this.waitForGuess();
    this.messageText.setText(`${banked} banked! Higher or lower?`);
}
```

The same `state` check guards it - C in the middle of a flip does nothing. A wrong guess goes the other way, in `loseLife()` : the pot goes to zero, a life is lost, the camera shakes

(`this.cameras.main.shake(250, 0.01)`) and the heart for that life grows and fades away.

Two kinds of points - `score` (banked, safe) and `pot` (at risk) - are what make the game. With only a score, the best play is to guess the likely way for ever. With a pot, *when to stop* is the real question, and every player answers it differently.

Dealing, and two sprites that swap jobs

Cards now come from somewhere: a face-down deck on the left. Each new card slides from the deck to its place, then flips.

```
// Deal a card: it slides face down from the deck to x, then flips over. It  
is brought to the top  
// of the display list, so it passes OVER the card to beat on its way  
across.  
private dealTo(sprite: CardSprite, x: number, card: Card, onComplete: () =>  
void): void {  
    this.children.bringToTop(sprite);  
    sprite.setPosition(DECK_X, CARD_Y).setVisible(true);  
    sprite.showBack();  
    this.sound.play(PLACE_KEY);  
    sprite.slideTo(x, CARD_Y, () => {  
        this.sound.play(FLIP_KEY);  
        sprite.flipTo(card, onComplete);  
    });  
}
```

`CardSprite` gained `slideTo(x, y, onComplete)`, a single tween of `x` and `y`, and its flip now also stretches `scaleY` a little at the halfway point, so the card seems to lift off the table.

After the reveal, the revealed card slides left to become the card to beat. Rather than destroy one `CardSprite` and make another, the scene just **swaps** which sprite does which job:

```

private moveOn(next: Card): void {
    this.history.addCard(this.currentCard);
    this.currentCard = next;

    const oldCurrent = this.currentSprite;
    this.currentSprite = this.nextSprite;
    this.nextSprite = oldCurrent;
    this.nextSprite.setVisible(false);

    this.currentSprite.slideTo(CURRENT_X, CARD_Y, () => {
        if (this.lives === 0) {
            this.endGame("Out of lives!");
        } else if (this.deck.isEmpty()) {
            this.endGame("That was the last card!");
        } else {
            this.waitForGuess();
        }
    });
}

```

`currentSprite` and `nextSprite` are just *references*, as in Java - swapping them swaps the names, not the objects. Two sprites do all the dealing for the whole game.

The history row

The old card to beat goes into a row of small cards along the bottom, so the player can see what has gone - useful for anyone counting cards.

`src/objects/HistoryRow.ts`

```

// (Container already has a method called add(), for adding game objects -
// hence addCard.)
public addCard(card: Card): void {
    // the new card starts one place to the right of the end of the row,
    // invisible...
    const image = this.scene.add.image(this.images.length * SPACING, 0,
    CARDS_KEY, CardSprite.frameFor(card))
        .setScale(SCALE)
        .setAlpha(0);
    this.add(image);
    this.images.push(image);

    // ...the oldest fades out if there are too many...
    if (this.images.length > MAX_CARDS) {
        const oldest = this.images.shift()!; // shift() takes the first item off
        // an array
        ...
    }

    // ...and every card slides into its place, fading in
    this.images.forEach((cardImage, index) => {
        this.scene.tweens.add({
            targets: cardImage,
            x: index * SPACING,
            alpha: 1,
            duration: SLIDE_TIME,
            ease: "Cubic.easeOut",
        });
    });
}

```

Rather than work out which cards need to move, every card is simply told where it should be. Cards already in place tween nowhere; the rest slide along. "Tell everything where it belongs" is often simpler than "work out what changed".

`CardSprite.frameFor()` is `static` so that `HistoryRow` can use it without a `CardSprite`: one formula, in one place, used by everything that draws a card.

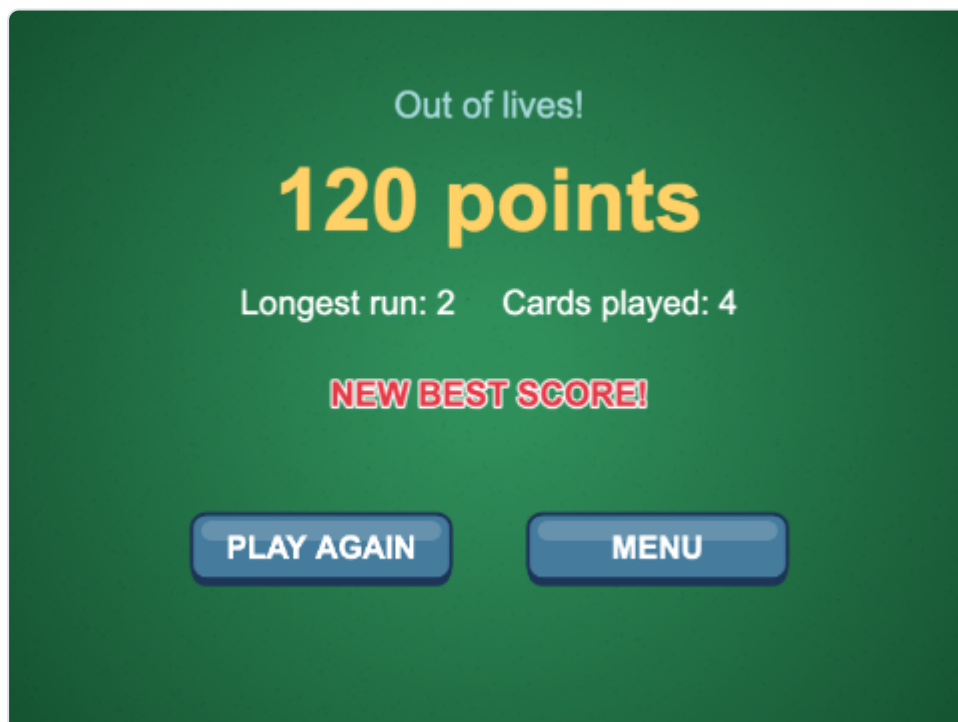
Records that last

`src/Records.ts` keeps the best score and the longest run in the browser's local storage, the way Chapter 6 kept its high scores:

```
export class Records {
  private data: RecordData = { bestScore: 0, bestStreak: 0 };

  constructor() {
    // local storage only holds strings, and may hold anything - or be
    // switched off. If it cannot
    // be read, start from nothing rather than crash.
    try {
      const saved = localStorage.getItem(STORAGE_KEY);
      if (saved !== null) {
        this.data = { ...this.data, ...JSON.parse(saved) };
      }
    } catch {
      // keep the empty records
    }
  }
}
```

`{ ...this.data, ...JSON.parse(saved) }` starts from the defaults and lays the saved values over them, so a record saved by an older version of the game, missing a field, still loads. `submit(score, streak)` saves anything beaten and returns which records were new (`newBestScore` and `newBestStreak`), and `GameScene.endGame()` passes them straight on to the game over screen by spreading them into its data with `...newRecords`.



Polish

None of these change the rules; all of them change how the game feels. Each is a few lines in `GameScene` or the title screen:

Polish	How
"+30" rises and fades over a right card	<code>floatText()</code> : a text with a tween of <code>y</code> and <code>alpha</code> , destroyed in <code>onComplete</code>
the pot and score jump when they change	<code>pulse()</code> : <code>scale 1.25</code> with <code>yoyo: true</code>
a burst of sparkles when you cash out	a particle emitter with <code>emitting: false</code> , then <code>explode(30, x, y)</code> (Chapter 7)
the screen shakes on a wrong guess	<code>this.cameras.main.shake(250, 0.01)</code>
a heart pops and fades when a life goes	a tween of <code>scale</code> and <code>alpha</code> on that heart
the four aces fan out on the title screen	one tween per ace, each with a longer <code>delay</code>
impossible guesses are greyed out	<code>setEnabled(false)</code> when <code>riskMultiplier</code> is 0
the deck disappears with its last card	<code>this.deckPile.setVisible(!this.deck.isEmpty())</code>

How the versions compare

	simple	intermediate	advanced
cards come from	<code>Card.random()</code>	a shuffled <code>Deck</code>	one <code>Deck</code> , played right through
the rules live in	<code>GameScene</code>	<code>rules.ts</code>	<code>rules.ts</code> , with scoring
a card on screen	an <code>Image</code> and <code>setFrame</code>	a <code>CardSprite</code> that flips	a <code>CardSprite</code> that is dealt, flips and lifts
input	H, L	H, L, buttons	H, L, C, buttons showing the stakes
states	waiting, revealing, won	waiting, revealing, won, lost	dealing, waiting, revealing, over
win / lose	5 in a row; wrong = back to 0	5 in a row; wrong = out	a score; 3 lives; a pot to cash out
scenes	1	4	4
remembers	nothing	nothing	best score and longest run

Notice what did not change. `judge()` is the same function in all three. `Card` only lost its `random()` method. That is the reward for separating the model: the game changed a lot, the rules hardly at all.

Common mistakes

Note - the frame is one out. Forget the `- 1` in `suit * 13 + (rank - 1)` and every card shows as the next rank up - and the king of spades shows as frame 52, a card back. If the pictures look *nearly* right, check the arithmetic.

Note - guessing during the flip. Leave out `if (this.state !== "waiting")` `return;` and a fast double-press draws two cards and judges both against the same card to beat. Nothing crashes; the game is just wrong. Every way in - keys, buttons - must go through the one check.

Note - function in a tween callback. Write `onComplete: function () { this.showFace(card); }` and the build is happy, but the game stops at the first flip with `TypeError: this.showFace is not a function`. A function gets its own `this` (here, the tween); an arrow function `() => { ... }` keeps the `this` of the code around it. Use arrow functions for callbacks.

Note - naming a Container method `add`. `Container` already has `add()`, for adding game objects, so a `HistoryRow.add(card)` fails the build with `Property 'add' in type 'HistoryRow' is not assignable to the same property in base type 'Container'`. Choose a name of your own, such as `addCard`.

Summary

- a guessing game lives on **risk**, **streaks** and **the reveal**: show the stakes, show the streak, and never rush the moment the card turns
- keep the **model** (`Card`, `Deck`, `rules.ts` - no Phaser) apart from the **view** (`CardSprite`, `Button`, `HistoryRow`); the scene asks one and tells the other
- a union of string literals (`"higher" | "lower"`) is a small, safe set of values; use one for suits, guesses, outcomes and game states
- shuffle with **Fisher-Yates**; `sort(() => Math.random() - 0.5)` is biased
- a `state` field, checked at the top of every input handler, stops the player acting while the game is busy
- a flip is two tweens of `scaleX` in a chain, with the frame changed in between
- a `Container` groups game objects into one - a button is a picture and a label
- scores become interesting when the player can lose what they have not banked yet

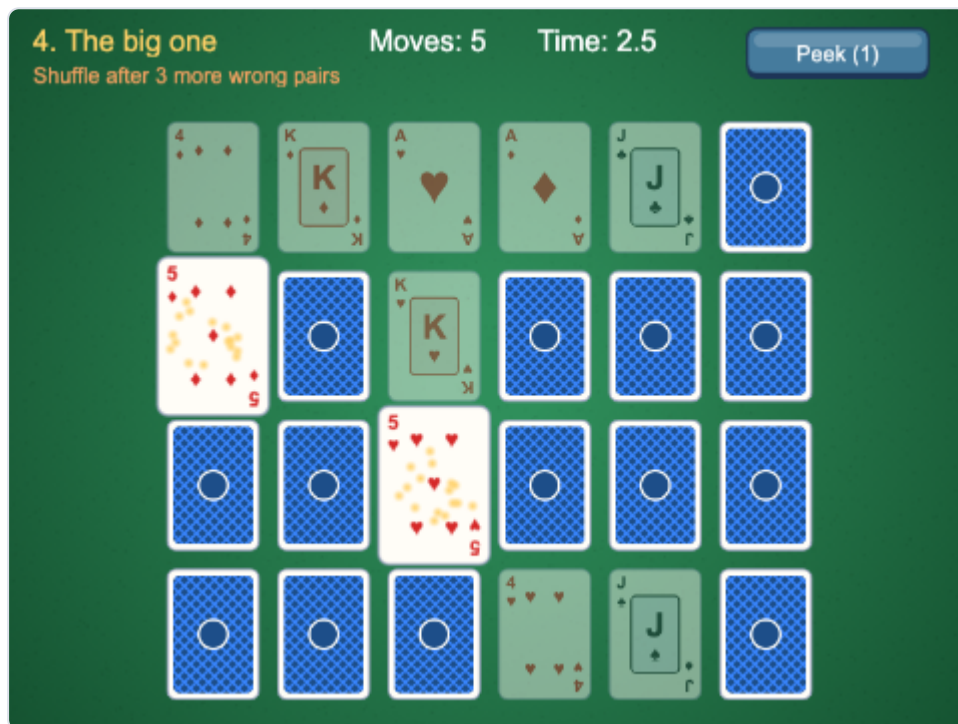
Challenges

1. **Aces high** (*ch12_higher_or_lower_simple*) - Make aces the highest card instead of the lowest: an ace beats a king. Change the rule shown at the bottom of the screen to match. The ace must still show the ace picture.
2. **Cards left** (*ch12_higher_or_lower_intermediate*) - Show how many cards are left in the deck, under the next card, and keep it up to date as cards are dealt. It should say 51 when the game starts.
3. **Probability hint** (*ch12_higher_or_lower_intermediate*) - Under each button, show the chance (as a percentage) that the guess will be right, worked out from the cards really left in the deck - not from the 13 ranks. Hide the hints while a card is being revealed, and update them for every new card to beat.
4. **Jokers wild** (*ch12_higher_or_lower_intermediate*) - Add two jokers to the deck. If the next card is a joker, the guess counts as right, whatever it was. You never guess against a joker: when a joker would become the card to beat, deal a fresh card on top of it instead (and never start a game with a joker). *Hint:* `cards.png` has no joker, so draw one with `Graphics` and save it with `generateTexture(key, width, height)` once everything is loaded. A joker is a *different texture*, not just a different frame - look up `setTexture(key, frame)`.
5. **Double or nothing** (*ch12_higher_or_lower_advanced*) - When there is a pot, let the player press D to gamble it instead of cashing out: one card is dealt from the deck; if it is red, the pot is doubled and banked; if it is black, the pot is lost (but not a life). Either way the run ends, and the card joins the history row. *Hint:* the game needs a new state while the gamble card is being dealt, so that nothing else can happen. `dealTo()` will deal the card for you.
6. **The computer plays the odds** (*ch12_higher_or_lower_advanced*) - Pressing A hands the game to the computer (and pressing it again takes it back). The computer counts the cards really left in the deck and guesses whichever way more of them go. If even that is not very likely - say less than a 65% chance - and there is something in the pot, it cashes out instead. It should wait a little before each move, so you can watch it play. *Hint:* the game tells you when it is ready for a move - it calls `waitForGuess()`. Start a timer from there. Remember the player may switch the computer off while it is thinking.

Previous: [Chapter 11 - Tilemaps with Tiled](#) · Next: [Chapter 13 - Memory match](#)

Memory match

Turn over two tiles. If they match, they stay; if not, they turn back, and you try to remember where they were. Memory match (you may know it as *Concentration* or *Pairs*) is one of the simplest games there is to play, and a surprisingly good one to write: it needs a grid laid out in code, a fair shuffle, animation that feels right, and - the heart of this chapter - a small **state machine** that stops the player clicking while tiles are showing. You will build it three times, each version growing from the one before.



What you will learn

- how to lay out a grid of game objects with two loops, and centre it in the game
- how to make shuffled pairs, and deal them onto the grid
- how to show one frame of a sprite sheet on an `Image`, and change it with `setFrame`
- how to write game rules as a **state machine** - a union type and a `switch` - and use it to lock the board
- how to animate a flip with two tweens, and wait for it to finish with a **callback**
- how moves, time and a star rating turn a puzzle into a game
- how an `interface` lets one game play with different sets of tiles, and how to describe levels as data

The projects

Project	What it shows
<code>ch13_memory_simple</code>	the whole game in one scene: a 4 x 3 grid, click to turn over, a wrong pair turns back after a second
<code>ch13_memory_intermediate</code>	flip tweens, a moves counter and a clock, sounds, a title screen and a win screen with stars
<code>ch13_memory_advanced</code>	four levels on bigger boards, a menu, playing-card tiles that match by rank and colour, a peek power-up, a shuffle twist, and best results saved in the browser

Play all three before reading on. Notice what changes as the versions grow - and notice that the simple one, with no animation at all, is already a real game.

What makes a memory game work

A memory game is a game about **information**. At the start you know nothing; every tile you turn over tells you something; the skill is in keeping it. A few things follow from that:

- **The player must see the wrong pair.** Turn a wrong pair back too quickly and the player never had the chance to remember it. Too slowly, and the game drags. About a second is right
- **The board must be fair.** While a wrong pair is showing, a quick player could click a third and a fourth tile and see four at once. The game must *lock* the board until the pair has turned back. This is the most important rule in the code, and the state machine exists to enforce it
- **Score what matters.** Finding every pair is guaranteed in the end - so the score is *how efficiently* you did it: moves (pairs of tiles turned over), and time
- **Feedback.** A flip, a sound, a little celebration for a pair and a shake for a miss. None of it changes the rules; all of it changes how the game feels

The simple version



`ch13_memory_simple` is one scene and one game object class. It is worth reading all of it before going on: about 230 lines in three files, a good part of them comments.

A tile: one frame of a sprite sheet

All thirteen pictures a tile can show are in one file, `memory_tiles.png`. It is a **sprite sheet**: a row of 100 x 100 frames. Frame 0 is the back of every tile (the purple "?"), and frames 1 to 12 are twelve different faces.

[image: `memory_tiles.png`: frame 0 is the back, frames 1-12 are the faces]

A sprite sheet is loaded with `load.spritesheet`, which cuts the picture into frames and numbers them from 0, left to right:

`src/scenes/GameScene.ts`

```
preload(): void {  
    // a sprite sheet: one picture, cut into equal frames, numbered from 0,  
    // left to right  
    this.load.spritesheet(TILES_KEY, TILES_FILE, { frameWidth: TILE_SIZE,  
    frameHeight: TILE_SIZE });  
}
```

A `Tile` is an `Image` that starts on frame 0, and remembers which face it hides:

`src/objects/Tile.ts`

```
export class Tile extends Phaser.GameObjects.Image {
    // which picture this tile hides (1-12). Two tiles on the board have each
    // face.
    // readonly: set once, in the constructor, and never changed - like final
    // in Java
    public readonly face: number;

    private faceUp = false;

    constructor(scene: Phaser.Scene, x: number, y: number, face: number) {
        super(scene, x, y, TILES_KEY, BACK_FRAME);
        this.face = face;
        scene.add.existing(this);
    }
}
```

The fourth argument to an `Image`'s constructor is the **frame**. You might expect a sprite sheet to need a `Sprite` - but an `Image` can show any single frame. A `Sprite` adds animation (playing frames one after another, Chapter 7), which a tile does not need.

Turning a tile over is just changing its frame:

```
public showFace(): void {
    this.faceUp = true;
    this.setFrame(this.face);
}

public hideFace(): void {
    this.faceUp = false;
    this.setFrame(BACK_FRAME);
}
```

`readonly` is TypeScript's `final`: `face` can be set in the constructor and never again, so no code anywhere can quietly change what a tile hides. `faceUp` is private, and read through `isFaceUp()`. The tile knows *what it is*; it does not know the rules of the game. The scene decides when a tile turns over and what a pair means - the same division as Chapter 2's `Ball`, which never knew it could be clicked.

Making the pairs

Six pairs need each of the faces 1 to 6 twice, in a random order:

src/scenes/GameScene.ts

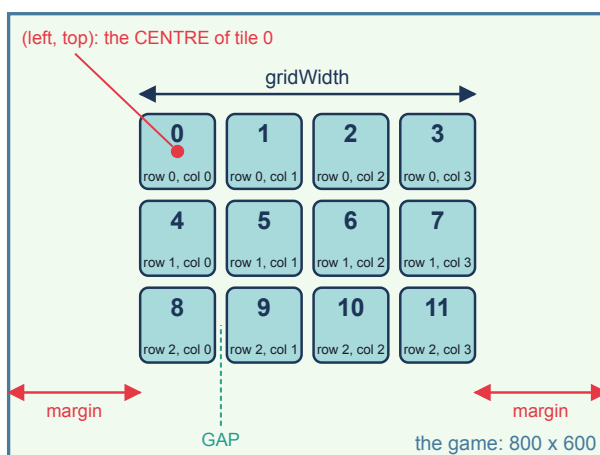
```
// [1, 1, 2, 2, 3, 3, ...] - each face twice - in a random order
private makeFaces(pairs: number): number[] {
    const faces: number[] = [];
    for (let face = 1; face <= pairs; face++) {
        faces.push(face, face);
    }
    // Phaser's Fisher-Yates shuffle (Chapter 12 writes one by hand). It
    // shuffles the array
    // in place, and returns it too.
    return Phaser.Utls.Array.Shuffle(faces);
}
```

`push` takes any number of values, so `faces.push(face, face)` adds the pair in one go. Chapter 12 explained why a proper shuffle matters and wrote Fisher-Yates in a `Deck` class; here we use the same algorithm from Phaser's toolbox. `Phaser.Utls.Array` has many more small helpers worth a look - `GetRandom`, `RemoveRandomElement`, `NumberArray`.

Laying out the grid

Now the faces are dealt onto a grid, row by row. The grid should sit in the middle of the game whatever its size, so its position is worked out, not typed in:

Laying out a 4 x 3 grid, centred in the game



The sums in layOutGrid()

```
gridWidth = COLS * TILE_SIZE + (COLS - 1) * GAP
    four tiles across, but only three gaps between them

left = (800 - gridWidth) / 2 + TILE_SIZE / 2
    two equal margins, so the grid is centred - and
    + TILE_SIZE / 2, because a tile's origin is its centre

x = left + col * (TILE_SIZE + GAP)
y = top + row * (TILE_SIZE + GAP)
    each step across or down is one tile and one gap

face = faces[row * COLS + col]
    the shuffled faces are dealt out row by row: the
    number on each tile is its index in the array
```



```

// ROWS x COLS tiles, with GAP pixels between them, centred in the game
private layOutGrid(faces: number[]): void {
  const gridWidth = COLS * TILE_SIZE + (COLS - 1) * GAP;
  const gridHeight = ROWS * TILE_SIZE + (ROWS - 1) * GAP;

  // the CENTRE of the top-left tile (a tile's origin is its centre)
  const left = (this.scale.width - gridWidth) / 2 + TILE_SIZE / 2;
  const top = (this.scale.height - gridHeight) / 2 + TILE_SIZE / 2;

  for (let row = 0; row < ROWS; row++) {
    for (let col = 0; col < COLS; col++) {
      const x = left + col * (TILE_SIZE + GAP);
      const y = top + row * (TILE_SIZE + GAP);
      const face = faces[row * COLS + col];

      const tile = new Tile(this, x, y, face);
      tile.setInteractive({ useHandCursor: true });
      tile.on("pointerdown", () => {
        this.tileClicked(tile);
      });
      this.tiles.push(tile);
    }
  }
}

```

Three things to notice:

- **Gaps are one fewer than tiles.** Four tiles across have three gaps between them. Forgetting the `- 1` is the classic off-by-one, and the grid ends up slightly off-centre
- **Origins are centres.** A tile's `x` and `y` are its middle (Chapter 1), so the first tile's centre is half a tile in from the grid's edge
- `row * COLS + col` turns a row and column into a position in a flat list: row 0 is items 0-3, row 1 is items 4-7, and so on. You will use this sum in every grid game you ever write

The `ROWS`, `COLS` and `GAP` constants are at the top of the file. Change them and the grid lays itself out again - as long as `ROWS * COLS` is even, since every tile needs a partner.

The rules as a state machine

What should happen when a tile is clicked? It depends. If nothing is showing, turn it over. If one tile is showing, turn this one over and compare. If a wrong pair is showing, do *nothing*.

The answer depends on **what state the game is in** - and that is exactly what a state machine is: a list of states the game can be in, and for each one, what each event does.

Chapter 12 kept a game's state in a union of string literals, and used it to ignore the player while a card was being revealed. Here that idea is the heart of the game. This game has three states:

```
// the three states, as a union of string literals: `state` can hold these  
three strings and  
// nothing else - a typo such as "oneup" is a build error  
type State = "idle" | "oneUp" | "twoUp";
```

and the click handler is a `switch` on the state:

```
private tileClicked(tile: Tile): void {  
    // a tile that is already face up - the first of this pair - cannot be  
    picked again  
    if (tile.isFaceUp()) {  
        return;  
    }  
  
    switch (this.state) {  
        case "idle":  
            tile.showFace();  
            this.firstTile = tile;  
            this.state = "oneUp";  
            break;  
  
        case "oneUp":  
            tile.showFace();  
            this.checkPair(this.firstTile!, tile);  
            break;  
  
        case "twoUp":  
            // a wrong pair is showing: ignore the click. Without this "lock", a  
            quick player could  
            // turn over a third and fourth tile while the first two are still up  
            break;  
    }  
}
```

`firstTile` is declared as `Tile | null` - a tile, or nothing. In the `"oneUp"` state there is always a first tile, but the compiler cannot know that, so `this.firstTile!` tells it "trust me, this is not null". (If you were wrong, you would get an error when the game runs - use `!` only where the state guarantees it.)

`checkPair` either keeps the pair or schedules the turn back:

```
private checkPair(first: Tile, second: Tile): void {
    this.firstTile = null;

    if (first.matches(second)) {
        first.setMatched();
        second.setMatched();
        this.pairsFound = this.pairsFound + 1;
        this.state = "idle";

        if (this.pairsFound === this.tiles.length / 2) {
            this.win();
        }
    } else {
        // leave the wrong pair showing long enough to remember, then turn both
        // back
        this.state = "twoUp";
        this.time.delayedCall(MISMATCH_DELAY, () => {
            first.hideFace();
            second.hideFace();
            this.state = "idle";
        });
    }
}
```

`this.time.delayedCall(ms, callback)` runs the callback once, after a delay, on the scene's clock. For that second, the game is in `"twoUp"`, and the `switch` ignores every click. That is the **lock**: one line, and it is what makes the game fair.

Why not a boolean `locked`? For three states you could get away with `firstTile` and a `locked` flag. But every new feature adds another flag - `flipping`, `peeking`, `shuffling` - and soon there are combinations that should be impossible ("locked but not flipping but peeking?"). One `state` field that holds exactly one value can never be in two states at once. The next two versions add three more states; the pattern holds.

Winning, and playing again

```
private win(): void {
    this.messageText.setText("All pairs found! Press SPACE to play again");

    this.input.keyboard!.once("keydown-SPACE", () => {
        this.scene.restart();
    });
}
```

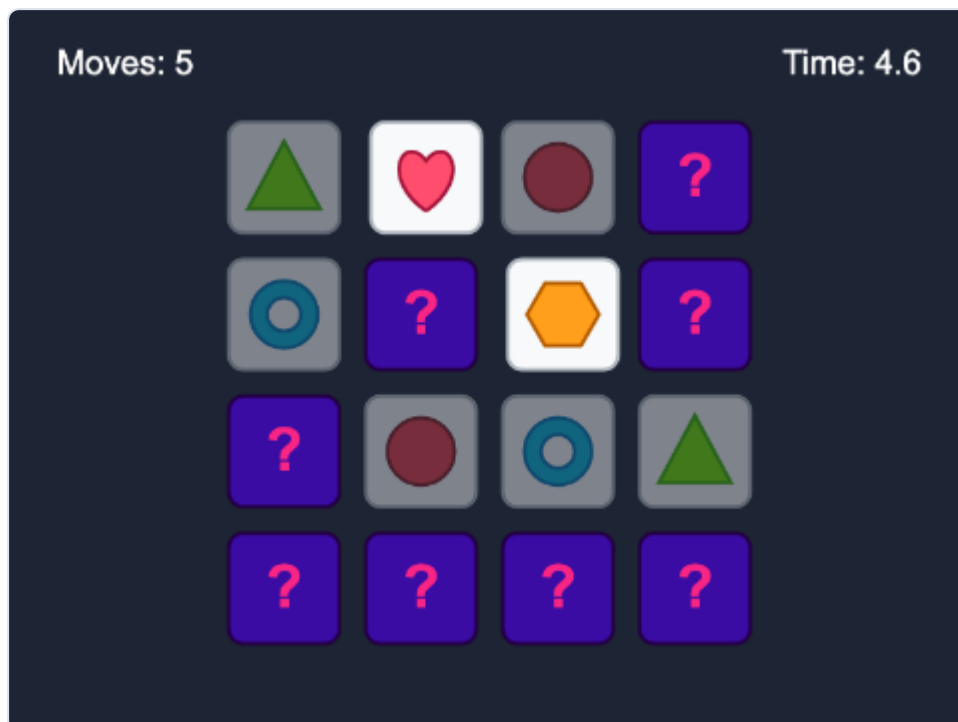
`this.scene.restart()` stops the scene and starts it again. It is the same scene object, so - as Chapter 2 warned - its fields keep their old values unless something resets them. `init()` does:

```
init(): void {
    this.tiles = [];
    this.state = "idle";
    this.firstTile = null;
    this.pairsFound = 0;
}
```

Forget `this.tiles = []` and the second game's array holds 24 tiles - twelve of them destroyed objects from the first game - and the game can never be won, because `pairsFound` never reaches 12.

Common mistake: clicking to play again. "Click to play again" seems more natural than SPACE, so you write `this.input.once("pointerdown", ...)` in `win()`. The game restarts *the instant you find the last pair*. The click that found the pair is still being handled: Phaser sends `pointerdown` to the tile first, then to the scene - and by then `win()` has added the new listener, so the same click triggers it. Either use a key, wait a moment (`delayedCall`), or move to another scene first, as the next version does.

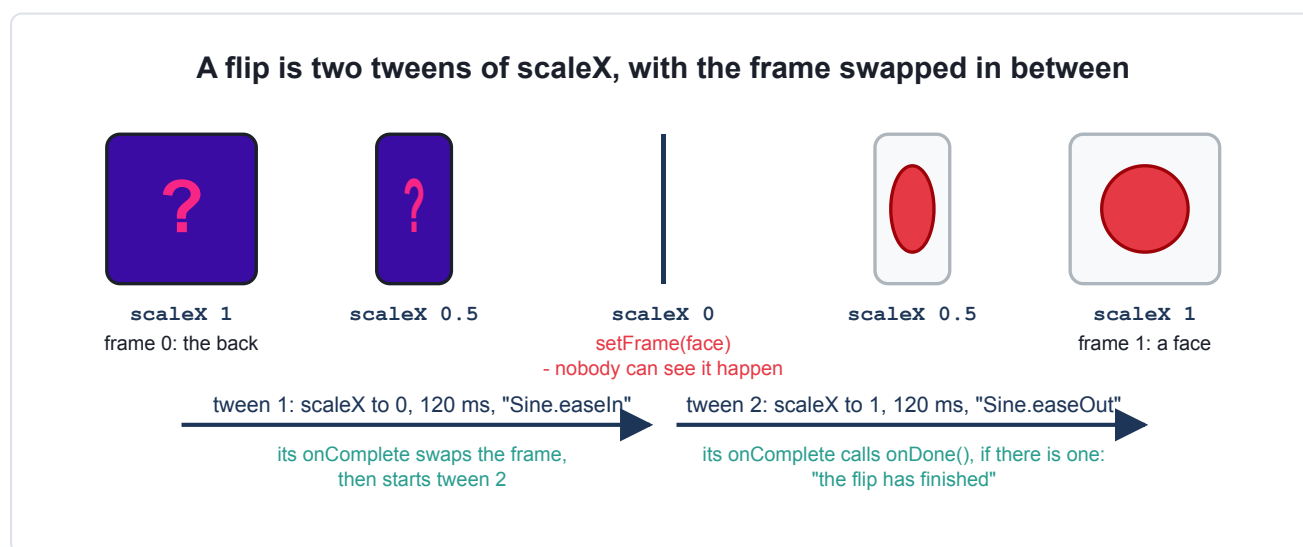
The intermediate version: making it feel good



`ch13_memory_intermediate` plays by exactly the same rules, on a 4 x 4 board. What it adds is *feel*: flips, sounds, a celebration, a clock, and a reason to play again - stars. It also has a title scene and a win scene, as in Chapter 2.

The flip

A card has no thickness, so to turn it over you can squash it to nothing across, swap the picture while it cannot be seen, and stretch it back. That is two tweens of `scaleX` :



`src/objects/Tile.ts`

```

// squash to nothing, swap the picture, stretch back - two tweens, one after the other
private flipTo(frame: number, onDone?: () => void): void {
    this.scene.tweens.add({
        targets: this,
        scaleX: 0,
        duration: HALF_FLIP,
        ease: "Sine.easeIn",
        onComplete: () => {
            this.setFrame(frame);
            this.scene.tweens.add({
                targets: this,
                scaleX: 1,
                duration: HALF_FLIP,
                ease: "Sine.easeOut",
                onComplete: () => {
                    // onDone is optional: call it only if one was given
                    if (onDone) {
                        onDone();
                    }
                },
            });
        },
    });
}

```

Chapter 12 flipped a card the same way. The easing matters more than you would think: `Sine.easeIn` then `Sine.easeOut` makes the tile speed up as it turns edge-on and slow down as it opens - which is how a real card moves. Try `"Linear"` for both and see how mechanical it looks.

The public methods set `faceUp` straight away, then start the flip:

```

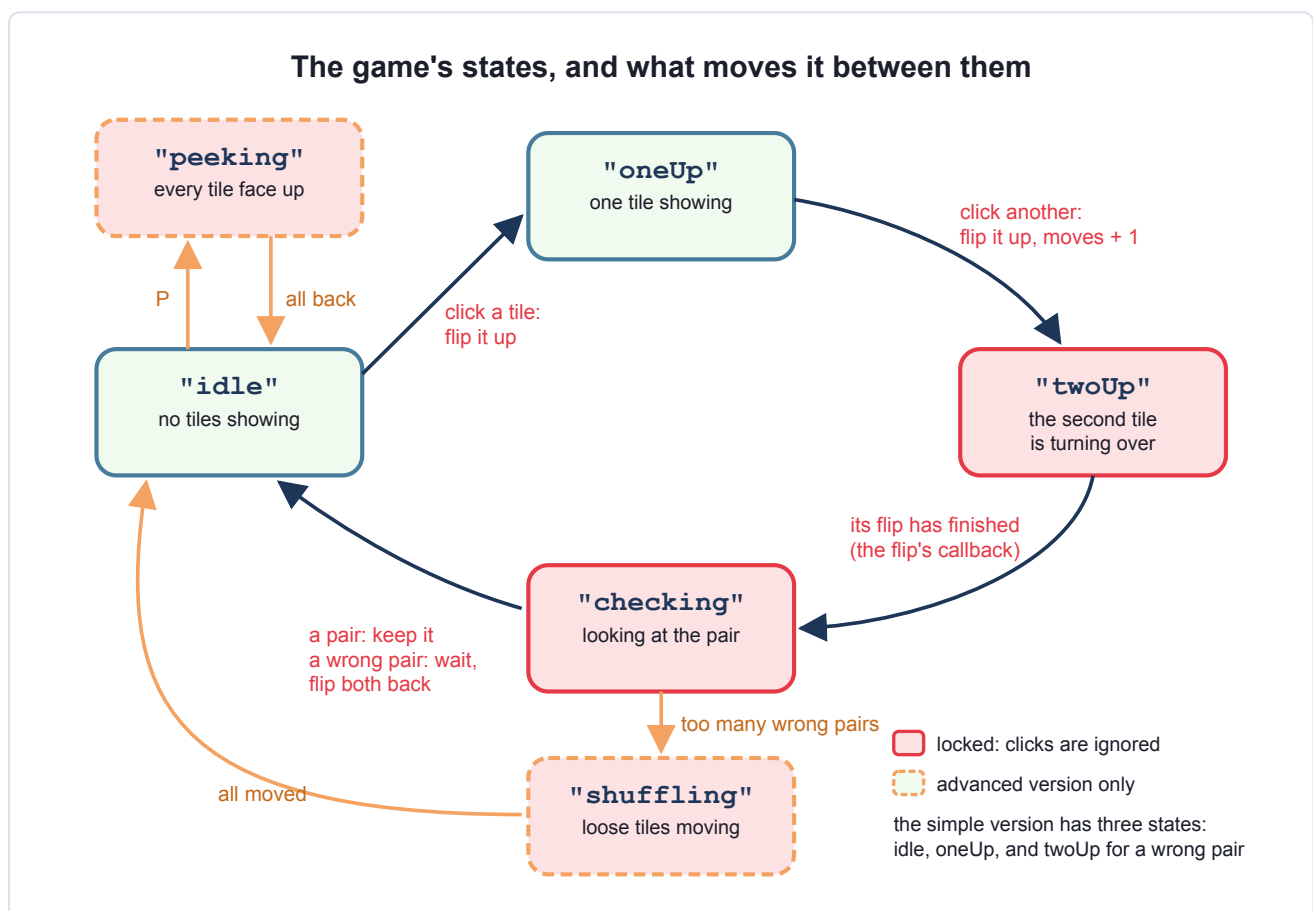
// faceUp changes AT ONCE, not when the flip ends - so a second click on a tile that is still
// turning over is already ignored
public flipUp(onDone?: () => void): void {
    this.faceUp = true;
    this.clearTint();
    this.flipTo(this.face, onDone);
}

```

Callbacks: "tell me when you have finished"

`onDone?: () => void` is a parameter that is a **function** - one that takes nothing and returns nothing - and the `?` makes it optional. The caller passes in "what to do when the flip has finished", and the tile calls it at the right moment. In Java you would pass a `Runnable` (or a lambda for one); in TypeScript, functions are values, and any arrow function will do.

The flip needs it because it takes time. The simple version compared the two tiles the instant the second was clicked. Here, the second tile is still turning over at that instant - showing the pair's verdict before the player can see the second face would feel like cheating. So the game has to wait, and that needs a **new state**:



`src/scenes/GameScene.ts`

```
type State = "idle" | "oneUp" | "twoUp" | "checking";
```

`"twoUp"` now means "the second tile is turning over"; `"checking"` means "both are showing and the game is dealing with them". Both are locked:

```

private tileClicked(tile: Tile): void {
    // "twoUp" and "checking" are the LOCKED states: the board ignores the
    player
    if (this.state === "twoUp" || this.state === "checking") {
        return;
    }
    if (tile.isFaceUp()) {
        return;
    }

    this.sound.play(FLIP_KEY);

    if (this.state === "idle") {
        if (!this.clockRunning) {
            this.clockRunning = true;
            this.startTime = this.time.now;
        }
        this.firstTile = tile;
        this.state = "oneUp";
        tile.flipUp();
    } else {
        // "oneUp": this is the second tile. Wait for its flip to finish before
        checking the pair
        this.secondTile = tile;
        this.state = "twoUp";
        this.moves = this.moves + 1;
        this.updateHud();
        tile.flipUp(() => {
            this.checkPair();
        });
    }
}

```

The first `flipUp()` has no callback - nothing needs to happen when it finishes. The second passes `() => { this.checkPair(); }`, so the pair is checked only once the player can see both faces.

In `checkPair`, the wrong-pair branch uses the callback again, so the board unlocks only when the tiles are face down:


```

} else {
  this.sound.play(WRONG_KEY);
  first.shake();
  second.shake();
  this.time.delayedCall(MISMATCH_DELAY, () => {
    first.flipDown();
    second.flipDown(() => {
      this.state = "idle"; // unlock only once both are face down again
    });
  });
}

```

Both tiles start flipping at the same moment and take the same time, so waiting for the second is waiting for both.

Celebrate and shake

A found pair pops and fades; a wrong pair shakes. Each is a few lines in `Tile`, and the scene just asks for them - `first.celebrate()`, `first.shake()`:

`src/objects/Tile.ts`

```

// part of a found pair: a quick "pop", then fade back - and no more clicks
public celebrate(): void {
  this.disableInteractive();
  this.scene tweens.chain({
    targets: this,
    tweens: [
      { scale: 1.2, duration: 120, ease: "Back.easeOut", yoyo: true },
      { alpha: MATCHED_ALPHA, duration: 300 },
    ],
  });
}

```

`tweens.chain` (Chapter 7) plays tweens one after another. The first grows the tile by a fifth and back (`yoyo`); the second fades it, so the board visibly empties as you play. The shake is a tween of `x` six pixels right and back, three times (`yoyo: true`, `repeat: 2`).

Tiles also light up under the pointer when they can be picked. That is `pointerover` and `pointerout` (Chapter 2) with `setTint` and `clearTint`, all inside `Tile`'s constructor.

Moves, time and stars

A **move** is one pair of tiles turned over, counted when the second tile is clicked. The clock starts at the first click, not when the board appears - the player should not lose time reading the board before they start. `update()` only works out the time while `clockRunning` is true.

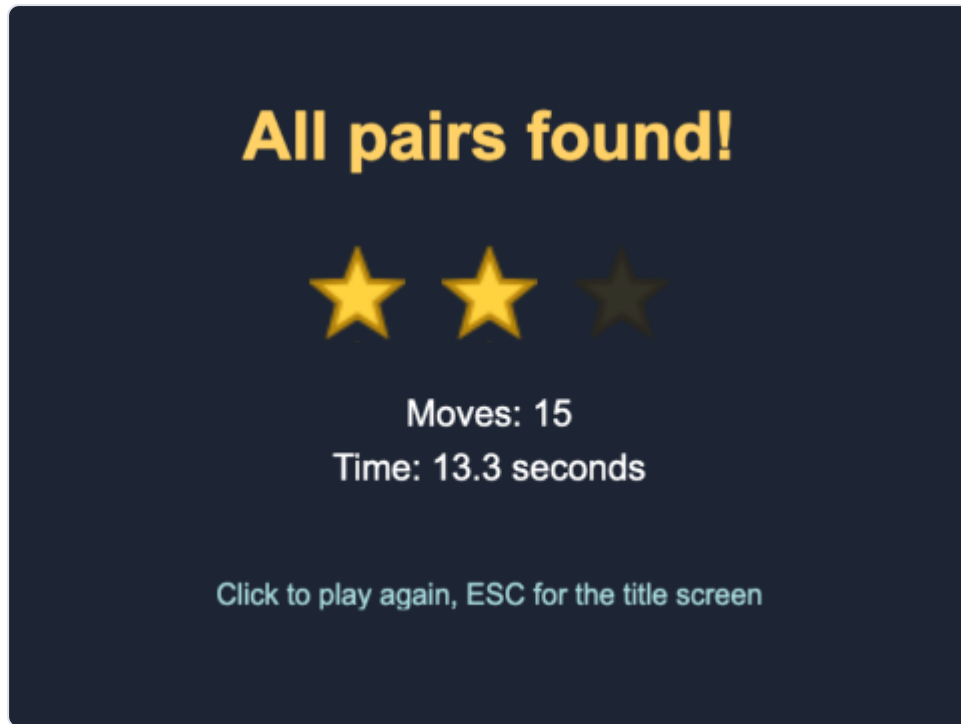
Moves become stars in a small function of their own:

`src/rating.ts`

```
const THREE_STARS = 1.5;    // up to 1.5 moves per pair: three stars
const TWO_STARS = 2.25;    // up to 2.25 moves per pair: two stars
                             (anything more: one)

export function starsFor(moves: number, pairs: number): number {
  if (moves <= Math.ceil(pairs * THREE_STARS)) {
    return 3;
  }
  if (moves <= Math.ceil(pairs * TWO_STARS)) {
    return 2;
  }
  return 1;
}
```

Not everything needs to be a class. A rule that turns numbers into a number is a plain function in its own module - easy to find, easy to test, and easy to tune. The limits are per pair, so the same function will work for any size of board. (For 8 pairs: 12 moves or fewer for three stars, 18 or fewer for two.)



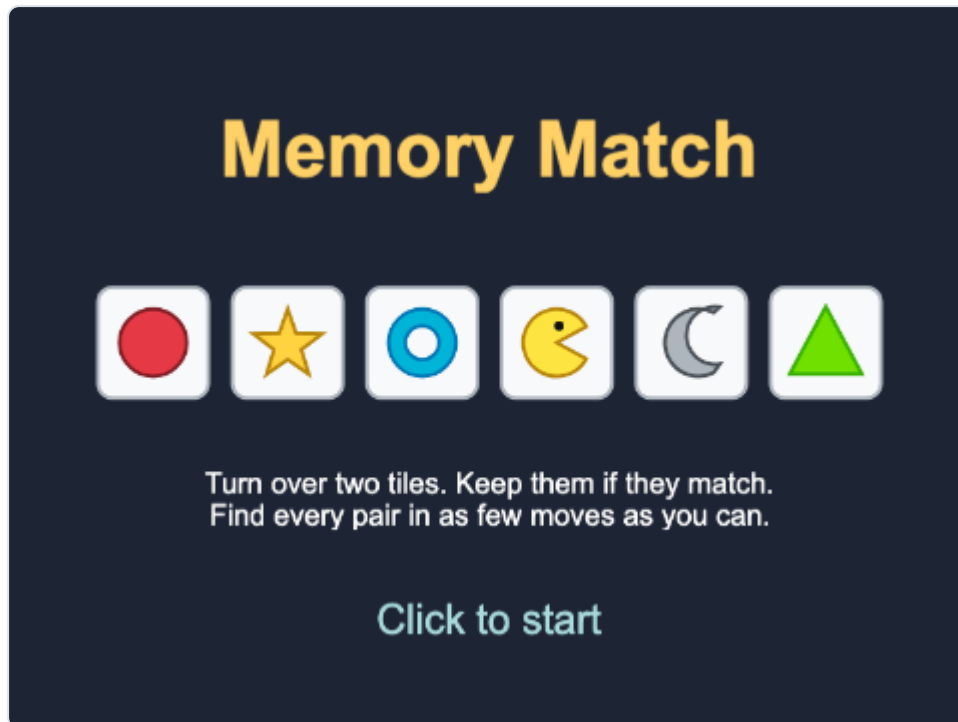
The win scene draws three stars from one picture. The earned ones start at scale 0 and grow in one after another, each with a slightly longer `delay` ; the others are tinted grey and faded:

`src/scenes/WinScene.ts`

```
for (let i = 0; i < 3; i++) {
  const star = this.add.image(centreX + (i - 1) * STAR_SPACING, 240,
STAR_KEY);
  if (i < stars) {
    // earned: grow from nothing, each a little later than the last
    star.setScale(0);
    this.tweens.add({
      targets: star,
      scale: STAR_SCALE,
      duration: 400,
      delay: 300 + i * 250,
      ease: "Back.easeOut",
    });
  } else {
    star.setScale(STAR_SCALE).setTint(UNEARNED_TINT).setAlpha(0.4);
  }
}
```

Showing the stars you *missed* is deliberate: two gold stars and a grey one says "you could do better" far more clearly than two stars alone.

A title screen that earns its place



The title scene loads everything and shows a row of tiles that drop in one after another - one tween with many targets, and `delay: this.tweens.stagger(100)` (Chapter 7). Its "Click to start" also does a job: browsers play no sound until the player has clicked or pressed a key (Chapter 5), so after that first click every flip can make a noise.

The advanced version: levels, themes and twists



`ch13_memory_advanced` turns the game into something you could put in front of people: a menu, four levels, two sets of tiles, a power-up, a twist on the harder levels, and best results that last. Its `GameScene` is bigger, but it is still the same state machine with the same four states - plus two more.

Levels as data

`src/levels.ts`

```
export interface LevelData {
  name: string;
  cols: number;
  rows: number;           // cols x rows must be even - every tile needs a
                           // partner
  shuffleAfter: number;   // after this many wrong pairs, the unmatched
                           // tiles swap places. 0 = never
}

export const LEVELS: LevelData[] = [
  { name: "Warm up", cols: 4, rows: 3, shuffleAfter: 0 },
  { name: "Four square", cols: 4, rows: 4, shuffleAfter: 0 },
  { name: "Shifting sands", cols: 5, rows: 4, shuffleAfter: 4 },
  { name: "The big one", cols: 6, rows: 4, shuffleAfter: 3 },
];
```

A level is not a class or a scene; it is a few numbers. The menu makes a button for each entry, the game scene is started with the level's index (`{ level: 2, theme: "cards" }`), and everything else

- the grid, the HUD, whether tiles shuffle - is worked out from the data. Adding a fifth level is adding a line.

A board that always fits

With levels of different sizes, and cards (80 x 112) as well as tiles (100 x 100), a fixed layout will not do. `layOutGrid` works out the grid's full-size width and height, then a **scale** that makes it fit the area below the HUD:

`src/scenes/GameScene.ts`

```
const gridWidth = cols * width + (cols - 1) * GAP;
const gridHeight = rows * height + (rows - 1) * GAP;
const scale = Math.min(1, AREA_WIDTH / gridWidth, AREA_HEIGHT / gridHeight);

const left = (this.scale.width - gridWidth * scale) / 2 + (width * scale) / 2;
const top = AREA_TOP + (AREA_HEIGHT - gridHeight * scale) / 2 + (height * scale) / 2;
```

`Math.min(1, ...)` takes the smallest of three numbers: 1 (never enlarge), and the scales at which the width and the height would just fit. Every tile is made at that scale, and every distance in the layout is multiplied by it. (All four levels happen to fit at full size; a 6 x 6 board would not, and would be shrunk.)

That creates a small trap. The flip tweened `scaleX` back to **1** - which would make a shrunk tile pop back to full size the first time it turned over. So the advanced `Tile` remembers its `baseScale`, as Chapter 12's `CardSprite` did, and every tween that returns to "normal" returns to that:

`src/objects/Tile.ts`

```
this.scene.tweens.add({
  targets: this,
  scaleX: this.baseScale,
  duration: HALF_FLIP,
  ease: "Sine.easeOut",
```

Themes: pictures or playing cards

The menu offers two sets of tiles. In the picture theme a pair is two identical pictures. In the card theme a pair is two cards of the **same rank and colour**: the seven of hearts matches the seven of diamonds, and the king of clubs matches the king of spades. It makes for a harder game - your eye is looking for identical pictures, and the pairs are never identical.

It also breaks an assumption. Until now, "match" meant "same frame". Two cards that match show *different* frames. So a tile's face becomes two things: what it shows, and what it means.

Same picture, or same meaning? A `TileFace` has both

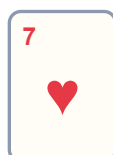


frame: 32
"7 red"



frame: 19
"7 red"

match



frame: 32
"7 red"



frame: 45
"7 black"

no match

frame
what the tile shows -
a different card for each

matchKey
what the tile means -
tiles match when these
are the same

src/themes/Theme.ts

```
// one tile's face: what it shows, and what it matches
export interface TileFace {
  frame: number;           // the sprite sheet frame shown when the tile is face
                           up
  matchKey: string;        // two tiles match when their matchKeys are the same
}

export interface Theme {
  readonly label: string;   // shown on the menu
  readonly texture: string; // the sprite sheet's key...
  readonly file: string;    // ...and file
  readonly tileWidth: number; // the size of one frame
  readonly tileHeight: number;
  readonly backFrame: number;
  readonly maxPairs: number; // how many different pairs the theme can
                           make
  readonly background?: string; // optional (the ?): a picture to put
                           behind the board

  // pairs x 2 faces, each pair next to each other (the game shuffles them)
  makeFaces(pairs: number): TileFace[];
}
```

`Theme` is an interface in the Java sense: a list of what a class must provide. Two classes provide it. `PictureTheme` picks some of the twelve pictures and makes each into a pair with the same frame. `CardTheme` does this:

src/themes/CardTheme.ts


```

public makeFaces(pairs: number): TileFace[] {
    // every pair there could be - 13 ranks x 2 colours - in a random order
    const kinds: { rank: number; red: boolean }[] = [];
    for (let rank = 1; rank <= RANKS; rank++) {
        kinds.push({ rank: rank, red: true }, { rank: rank, red: false });
    }
    Phaser.Utls.Array.Shuffle(kinds);

    const faces: TileFace[] = [];
    for (const kind of kinds.slice(0, pairs)) {
        const suits = kind.red ? [HEARTS, DIAMONDS] : [CLUBS, SPADES];
        const matchKey = `${kind.rank} ${kind.red ? "red" : "black"}`;
        for (const suit of suits) {
            faces.push({ frame: suit * RANKS + (kind.rank - 1), matchKey: matchKey
});
        }
    }
    return faces;
}

```

`{ rank: number; red: boolean }[]` is an array of objects of an **inline type** - handy when a shape is used in just one place. The frame sum is the one from Chapter 12: `cards.png` has 13 cards per row, one row per suit, so the seven of hearts (suit 2) is frame $2 \times 13 + 6 = 32$.

And the tile compares meanings, not pictures:

`src/objects/Tile.ts`

```

// compare what the tiles MEAN, not what they show - two cards can match
// with different pictures
public matches(other: Tile): boolean {
    return this.face.matchKey === other.face.matchKey;
}

```

The game scene never mentions pictures or cards. It asks its `Theme` for faces, a texture and a size, and plays. (The preload scene loads every theme's sprite sheet in a loop, using each theme's own frame size - so it does not know about pictures or cards either.) The themes are kept by name:

`src/themes/themes.ts`

```
export type ThemeName = "pictures" | "cards";

export const THEMES: Record<ThemeName, Theme> = {
  pictures: new PictureTheme(),
  cards: new CardTheme(),
};
```

`Record<K, V>` is an object type whose keys are `K` and whose values are `V` - here, exactly one `Theme` for each `ThemeName`. Add `"animals"` to `ThemeName` and the compiler insists you add an animals theme to `THEMES`. A third theme is one new class and two short lines.

Note - `class CardTheme implements Theme` works as in Java, with one difference worth knowing. TypeScript checks the *shape*, not the name: any object with the right properties and methods is a `Theme`, whether or not its class says `implements`. Writing `implements` is still good practice - it makes the compiler check the class where it is written, and tells the reader what it is for.

Two more locked states: peek and shuffle

The advanced state machine has six states (see the diagram above - the dashed ones are new). Both new states are locked, and both are entered only from `"idle"`, when no tile is face up.

The peek is a power-up: once per level, press **P** (or click the button) and every unmatched tile turns face up for a second and a half. Useful - so it costs the third star.

```
src/scenes/GameScene.ts
```

```

private peek(): void {
  if (this.state !== "idle" || this.peeksLeft === 0) {
    return;
  }
  this.state = "peeking";
  this.peeksLeft = this.peeksLeft - 1;
  this.peeked = true;
  ...

  const hidden = this.tiles.filter((tile) => !tile.isMatched());
  for (const tile of hidden) {
    tile.flipUp();
  }
  this.time.delayedCall(PEEK_TIME, () => {
    hidden.forEach((tile, index) => {
      if (index === hidden.length - 1) {
        // the flips all take the same time: when the last has finished, so
        have the rest
        tile.flipDown(() => {
          this.state = "idle";
        });
      } else {
        tile.flipDown();
      }
    });
  });
}

```

The first line is the state machine doing its job again. Pressing P while a tile is showing, or during a shuffle, or twice quickly, does nothing - with no special cases, because every one of those situations is simply "not `idle`".

`filter` makes a new array of the items for which the function returns `true` (like Java's `stream().filter(...)`), and `forEach` gives each item *and* its index.

The shuffle is a twist on the later levels: after `shuffleAfter` wrong pairs in a row, every unmatched tile slides to where another one was. Everything you remembered is now wrong - but the tiles move slowly enough to follow, if you are quick.



```
// the twist: every tile not yet matched moves to where another unmatched
// tile was
private shuffleUnmatched(): void {
  this.state = "shuffling";
  this.wrongSinceShuffle = 0;

  const loose = this.tiles.filter((tile) => !tile.isMatched());
  const places = loose.map((tile) => ({ x: tile.x, y: tile.y }));
  Phaser.Utls.Array.Shuffle(places);

  this.sound.play(SHUFFLE_KEY);
  loose.forEach((tile, index) => {
    tile.slideTo(places[index].x, places[index].y, SHUFFLE_TIME);
  });
  this.flashMessage("Shuffle!");

  this.time.delayedCall(SHUFFLE_TIME, () => {
    this.state = "idle";
  });
}
```

The trick is to shuffle the **places**, not the tiles. `map` makes a list of where the loose tiles are now (`{ x: tile.x, y: tile.y }`) - the brackets tell TypeScript the `{` starts an object, not a block of code); shuffling that list and sending tile *i* to place *i* moves every tile to a

place that was occupied, so the board keeps its shape. `slideTo` is one tween of `x` and `y` with `"Cubic.easeInOut"`.

When does it happen? After a wrong pair has turned back:

```
// both tiles of a wrong pair are face down again: time for the twist, or  
back to the player  
private afterWrongPair(): void {  
    const shuffleAfter = this.level.shuffleAfter;  
    if (shuffleAfter > 0 && this.wrongSinceShuffle >= shuffleAfter) {  
        this.shuffleUnmatched();  
    } else {  
        this.state = "idle";  
    }  
    this.updateHud();  
}
```

A state machine makes this kind of rule easy to slot in: `"checking"` used to lead only to `"idle"`, and now it can lead to `"shuffling"` instead. Nothing else had to change.

Buttons

The menu, the HUD's peek button and the level-complete screen all use a `Button` - Chapter 12's idea: a `Container` holding the `button.png` picture and a text, which changes picture when hovered and pressed, and runs an `onClick` callback it was given. Because a container moves, scales and fades its contents as one, the HUD's smaller button is just `new Button(...).setScale(0.7).setEnabled(false)` fades the button and stops it listening, once the peek has been spent.

Remembering the best

The best result for each level, in each theme, is kept in the browser's local storage (Chapter 6), by a plain class, `BestResults`. What counts as "best" is a rule of its own:

```
src/BestResults.ts
```

```
// more stars is better; with the same stars, fewer moves; with the same
moves, less time
private static isBetter(a: BestResult, b: BestResult): boolean {
    if (a.stars !== b.stars) {
        return a.stars > b.stars;
    }
    if (a.moves !== b.moves) {
        return a.moves < b.moves;
    }
    return a.seconds < b.seconds;
}
```

When the results are read back, each one is checked - anything could be in storage, from an older version of the game or a curious player - and anything that does not look like a result is dropped rather than crashing the menu.

The chosen theme is kept in the **registry** (Chapter 2), so it survives going from the menu to a level and back, and the level-complete scene hands the theme on when you pick "Next level".

Common mistakes

The board is not locked. Every click handler should start by asking the state whether clicks are allowed. If turning three tiles over quickly is possible, a state is missing - or a callback sets "idle" too early.

Tweening back to 1. A tile made at `scale 0.8` and flipped back to `scaleX: 1` is suddenly wider than its neighbours. Tween back to the scale it started with.

Summary

- a grid is two loops; its size is `COLS * TILE_SIZE + (COLS - 1) * GAP`, and it is centred by splitting what is left into two equal margins. `row * COLS + col` finds a grid square in a flat list

- pairs are made by pushing each face twice and shuffling
(`Phaser.Utls.Array.Shuffle`)
 - an `Image` can show any frame of a sprite sheet; `setFrame` changes it
 - game rules are a **state machine**: a union type of states, and code that asks the state what an event should do. Locked states ignore the player
 - a flip is two `scaleX` tweens with `setFrame` between; a callback (`onDone?: () => void`) says when it has finished, and the game waits for it in a state of its own
 - moves and time are the score; a small function turns them into stars
 - an `interface` (`Theme`) lets one game play with any set of tiles; a `matchKey` separates what a tile shows from what it matches
 - levels are data; the board scales to fit; tweens return to a tile's `baseScale`
-

Challenges

1. **Choose the size** (*ch13_memory_simple*) - Let the player choose the board: keys **1**, **2** and **3** start a new game on a 4 x 3, 4 x 4 or 6 x 4 board, and SPACE after a win plays again at the same size. Show the size in the message at the top.
2. **Against the clock** (*ch13_memory_intermediate*) - Give the player 60 seconds. Show the time *left*, counting down, and turn it red for the last ten seconds. If it runs out, lock the board, turn every tile face up, and go to the win scene - which should say "Out of time!" and show no stars.
3. **A hint** (*ch13_memory_intermediate*) - Press **H** to make one unmatched pair flash (without turning over) so the player knows where it is. A hint costs two moves, and only works when no tile is face up.
4. **Two players** (*ch13_memory_intermediate*) - Players 1 and 2 take turns. A pair wins a point and another go; a wrong pair passes the turn. Show whose turn it is and both scores, and have the win scene announce the winner (or a draw). *Hint*: there is exactly one place in `GameScene` where the game knows a turn has ended. Think about what the stars mean now - perhaps nothing.
5. **Triple match** (*ch13_memory_simple*) - Make it a game of threes: each face appears three times on a 4 x 3 board, and you turn over up to three tiles. Two that do not match are a wrong guess straight away; three the same stay up. *Hint*: replace `firstTile` with an

array of the tiles showing, and let the states be about how many are up. How big can the board be with twelve faces?

6. **Keyboard control** (*ch13_memory_advanced*) - Play the whole game without a mouse: a cursor (a highlighted outline) moves round the grid with the arrow keys, and SPACE or ENTER turns over the tile under it. It must work on every level, in both themes, and after a shuffle. *Hint*: a cursor is a column and a row, not a tile. After a shuffle the tiles have moved - so find the tile *at* the cursor's position when SPACE is pressed, rather than remembering one.

Previous: [Chapter 12 - Higher or lower](#) · Next: [Chapter 14 - Catch, avoid and shoot](#)

Catch, avoid and shoot

A player at the bottom of the screen, moving left and right. Things come down from the top: some to catch, some to dodge, some to shoot. From *Space Invaders* and *Galaxian* to a hundred phone games, it is one of the oldest shapes a game can have. In this chapter you build it three times - a fruit catcher, a space shooter, and a space shooter with levels, bosses and power-ups - and on the way meet the tools every action game needs: spawn timers, **object pools**, waves, fire rates, invulnerability, particles, and a difficulty that keeps rising.



What you will learn

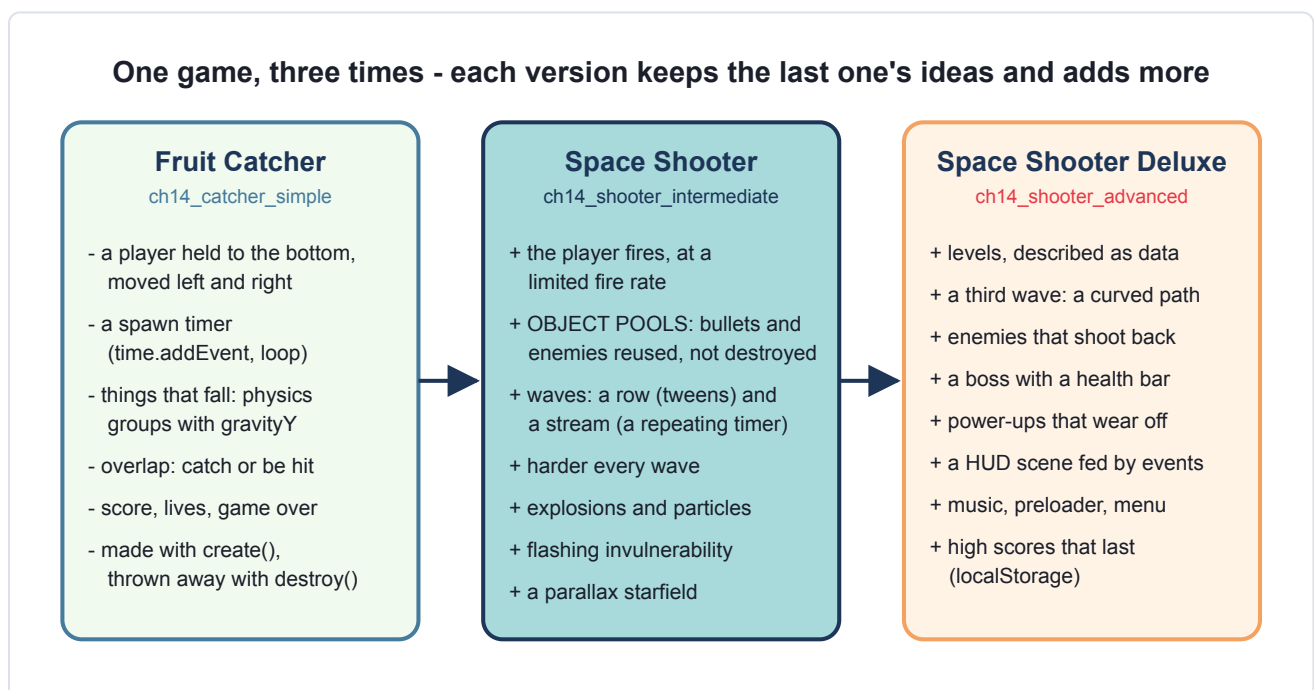
- how to keep a player on one line and move it with physics, held on screen by the world's edges
- how to spawn things on a timer with `this.time.addEvent({ delay, loop, callback })`, and stop it
- what an **object pool** is, why shooters need one, and how to build one with a physics group, `get()`, `enableBody` and `disableBody` - and why `killAndHide` alone is not enough
- how to send enemies in **waves** with patterns made from tweens, timers and curved paths

- a fire rate, lives, flashing invulnerability, explosions with particles, and a parallax starfield
- how to grow a small game into a big one: levels as data, a boss, power-ups, a HUD scene fed by events, music and high scores

The projects

Project	What it shows
<code>ch14_catcher_simple</code>	catch the fruit, dodge the bombs: a spawn timer, falling physics groups, overlaps, score and lives
<code>ch14_shooter_intermediate</code>	a ship that fires, pooled bullets and enemies, two wave patterns, explosions, a starfield
<code>ch14_shooter_advanced</code>	three levels with a boss each, enemies that shoot back, a curved wave, power-ups, a HUD scene, music, high scores

Each is a complete game. Play all three before reading on - it is much easier to follow the code when you know what it does.

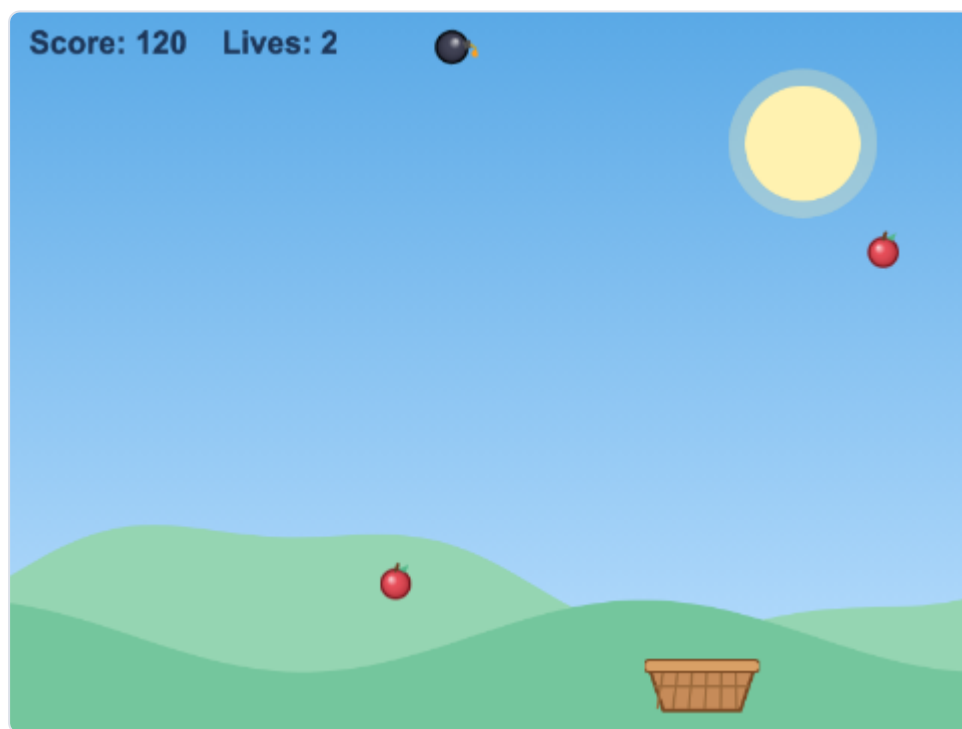


What makes the genre work

The player can do almost nothing: move along one line, and perhaps press one button. Everything interesting comes from what comes down the screen, and when - so that is where most of the code goes. Three things make it work:

- **readable threats** - a fruit looks like a fruit, a bomb like a bomb; enemy bullets are a different colour from yours
- **rising pressure** - easy at first, harder and harder, so beginners survive and experts are tested
- **fairness** - hit boxes a little smaller than the pictures, a moment of safety after a hit, and enemies that stop firing when they are too close to dodge

Version 1: Fruit Catcher



`ch14_catcher_simple` is one scene and one small class - under 300 lines, comments and all. Fruit and bombs fall; catch the fruit for 10 points; catching a bomb, or letting a fruit hit the ground, costs one of three lives.

A player held to the bottom

The basket is an Arcade Physics image (Chapter 8) - an image with a **body**:

src/objects/Basket.ts

```
scene.add.existing(this);           // show it
scene.physics.add.existing(this);   // give it a body

this.setBodySize(CATCH_WIDTH, CATCH_HEIGHT, false);
this.setOffset(CATCH_OFFSET_X, CATCH_OFFSET_Y);

// the edges of the game stop the basket: no checks of our own needed
this.setCollideWorldBounds(true);
```

- the basket **never changes its y**: nothing gives it a vertical velocity, and the world has no gravity, so it stays on its line without being told
- `setCollideWorldBounds(true)` stops it at the edges - no checks of our own, unlike Chapter 1's ball
- the body is **not** the whole picture: `setBodySize(84, 16, false)` and `setOffset(6, 4)` make it a thin strip across the top (`false` : "do not centre it; I will place it"). A fruit that brushes the side does not count - it has to fall *in*

The basket's only method is `move(direction)` , which sets its x velocity to -1, 0 or 1 times its speed. Every frame, `update()` checks `this.cursors.left.isDown` and `this.cursors.right.isDown` - `this.input.keyboard!.createCursorKeys()` makes those key objects - and calls it. `isDown` is the tool for **holding** a key; `keyboard.on("keydown-...")` (Chapter 1) is the tool for a single press.

Spawning on a timer

Something has to drop from the sky every 700 milliseconds. You could count time in `update()` , but the scene's clock will do it for you:

src/scenes/GameScene.ts

```
// the spawner: call dropSomething() every SPAWN_DELAY milliseconds, for
// ever (loop)
this.spawnTimer = this.time.addEvent({
  delay: SPAWN_DELAY,
  loop: true,
  callback: () => {
    this.dropSomething();
  },
});
```

`this.time.addEvent(config)` makes a **TimerEvent**. With `loop: true` it goes off every `delay` milliseconds until it is removed; with `repeat: 4` instead it would go off five times (once, plus four repeats) and then stop by itself. Keep the `TimerEvent` in a field if you will need it again: the game-over code calls `this.spawnTimer.remove()` to stop the drops.

You met `this.time.delayedCall(ms, callback)` in Chapter 2. It is just `addEvent` with no repeats - one call, after a wait.

The timer belongs to the scene's clock, so it pauses when the scene pauses and is thrown away when the scene shuts down. You never have to clean it up when the scene restarts.

Things that fall

`src/scenes/GameScene.ts`

```
// Two physics groups. Every member they make gets a body, and the settings
// here -
// gravityY - are given to each new member as it is made.
this.fruit = this.physics.add.group({ gravityY: FALL_GRAVITY });
this.bombs = this.physics.add.group({ gravityY: FALL_GRAVITY });
```

There is no gravity in the game's config - if there were, the basket would fall too. Instead each group gives its own members gravity as they are made. So a fruit starts still, just above the screen, and speeds up as it falls, the way a real one would (Chapter 9).

The timer's callback makes the fruit and bombs:

`src/scenes/GameScene.ts`

```

if (Math.random() < BOMB_CHANCE) {
    // group.create() makes a physics sprite, adds it to the scene AND to the
    // group
    const bomb: Phaser.Physics.Arcade.Sprite = this.bombs.create(x, DROP_Y,
    BOMB_KEY);
    bomb.setAngularVelocity(Phaser.Math.Between(-180, 180)); // a slow
    tumble, in degrees per second
} else {
    this.fruit.create(x, DROP_Y, FRUIT_KEY);
}

```

`Math.random()` is a number from 0 up to (not including) 1, so `Math.random() < 0.3` is true 30% of the time - the usual way to say "sometimes".

Catching, and missing

src/scenes/GameScene.ts

```

this.physics.add.overlap(this.basket, this.fruit, (_basket, fruit) => {
    this.catchFruit(fruit as Phaser.Physics.Arcade.Sprite);
});
this.physics.add.overlap(this.basket, this.bombs, (_basket, bomb) => {
    this.catchBomb(bomb as Phaser.Physics.Arcade.Sprite);
});

```

One overlap against a **whole group** covers every fruit there will ever be, including the ones not made yet (Chapter 8). The callback is handed the two objects that touched; Phaser types them loosely, so `as` says which class they really are - a physics group makes `Phaser.Physics.Arcade.Sprite`s unless it is told otherwise.

Things that fall past the bottom are finished with. `update()` checks every member of each group:

src/scenes/GameScene.ts

```

for (const fruit of this.fruit.getChildren() as
Phaser.Physics.Arcade.Sprite[]) {
    if (fruit.y > GONE_Y) {
        fruit.destroy();
        this.loseLife();           // a dropped fruit costs a life
    }
}

```

`getChildren()` returns a **copy** of the group's members as an array, so destroying some of them inside the loop is safe. (Changing a list while you walk through it is as bad an idea in TypeScript as it is in Java.)

Lives and game over

src/scenes/GameScene.ts

```

private endGame(): void {
    this.gameOver = true;
    this.sound.play(LOSE_SOUND);

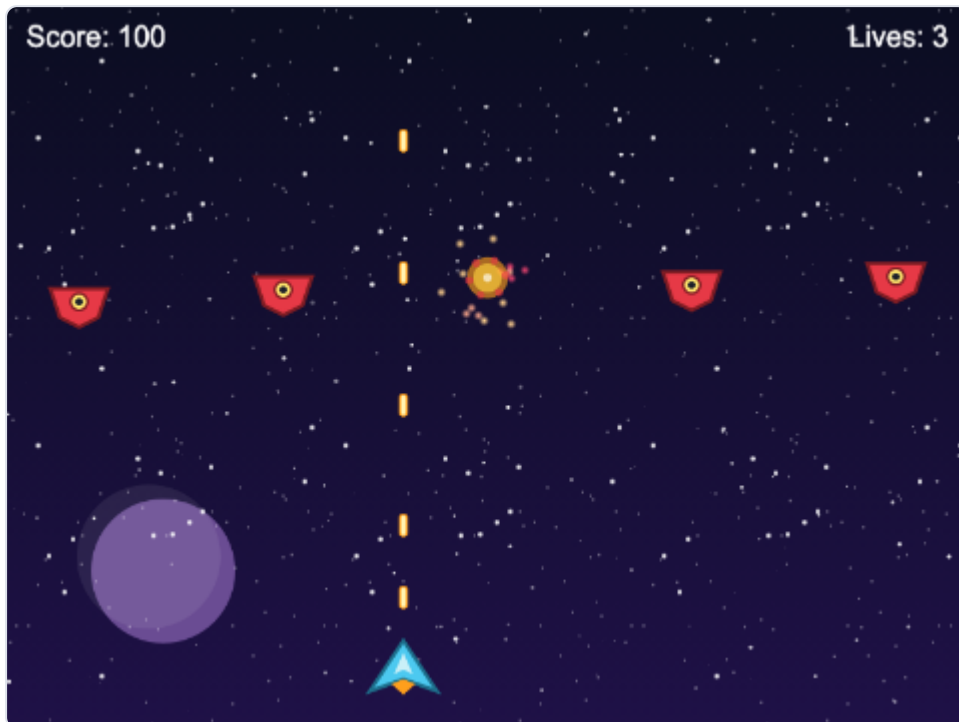
    // stop the world: no more drops, and everything freezes where it is
    this.spawnTimer.remove();
    this.physics.pause();
    this.basket.setTint(0x777777);
}

```

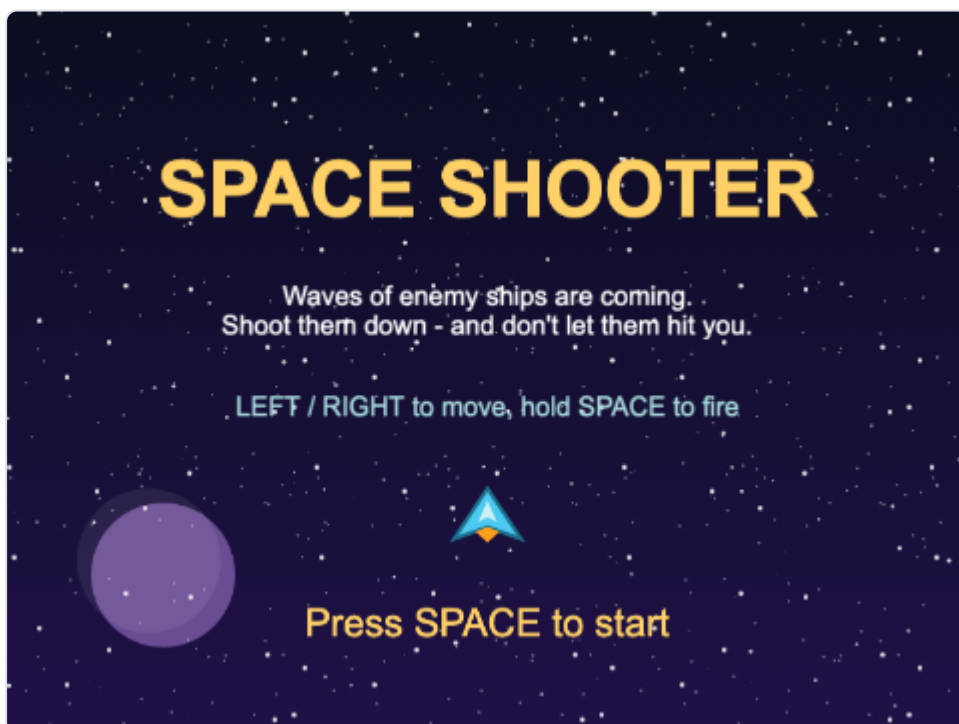
`this.physics.pause()` stops every body in the scene, and with them every overlap check - so nothing can be caught after the game is over. A GAME OVER message goes up, and SPACE calls `this.scene.restart()`, which shuts the scene down and starts it again. The score and lives are fields, and a restarted scene keeps its fields (Chapter 2's trap) - so `init()` puts them back.

That is a whole game: a player, a spawner, two kinds of falling thing, a score and a way to lose. Everything else in the chapter is built on it.

Version 2: Space Shooter



`ch14_shooter_intermediate` turns the basket into a ship, and the fruit into enemies that have to be shot. Look at what stays the same: `PlayerShip.move()` is `Basket.move()`; the game is still a scene with a timer, some groups and some overlaps. What is new is **how many** objects there are, and how often they come and go.



A fire rate

Hold SPACE and the ship fires about five shots a second - not sixty. The ship remembers the time of its next allowed shot:

src/objects/PlayerShip.ts

```
public tryToFire(time: number): boolean {
  if (time < this.nextFireTime) {
    return false;
  }
  this.nextFireTime = time + FIRE_DELAY;
  return true;
}
```

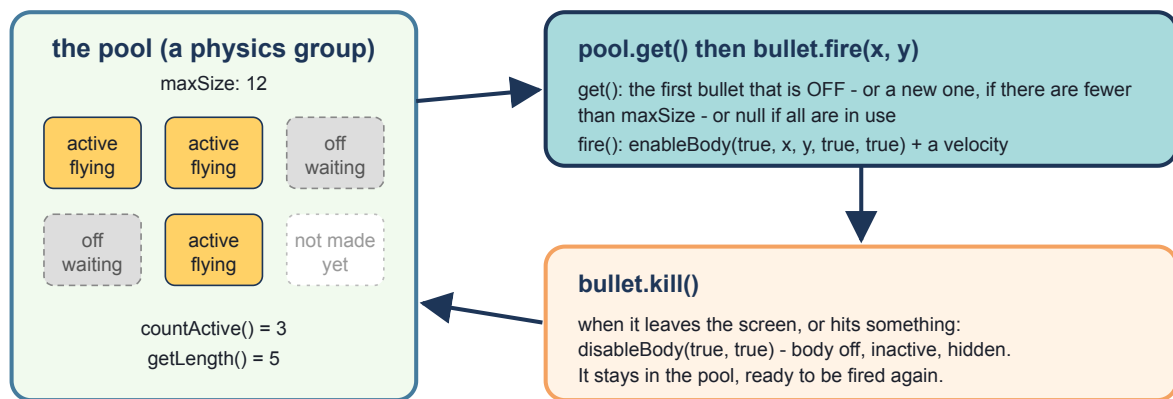
The scene's `update(time)` fires when `this.fireKey.isDown && this.ship.tryToFire(time)` - `time` is the scene's clock, in milliseconds. A stored "next allowed" time is the standard way to limit how often anything happens: shots, jumps, footstep sounds.

Why pool?

Hold fire for a minute and the ship fires over 300 bullets. In the catcher, every fruit was made with `create()` and thrown away with `destroy()`. Do that with bullets and the game builds hundreds of objects a minute - each a game object with a physics body - and leaves every one for JavaScript's **garbage collector** to clear up. The collector runs when it decides to, and when it runs a lot, the game stutters.

The fix is an **object pool**: make a few bullets, and **reuse** them. A bullet that leaves the screen is not destroyed - it is switched off, and waits to be fired again. After the first few seconds the game makes no new bullets at all.

An object pool: the same few bullets, switched on and off again and again



No new objects while the game runs, and nothing for the garbage collector to clear up - just switches.

A pool in Phaser

A physics group can be a pool. Give it a `classType` - the class it should make its members from - and a `maxSize` :

src/scenes/GameScene.ts

```
// POOLS: physics groups that make members of our own classes, up to maxSize
// of them. Asking a
// pool for an object reuses a switched-off one if it can, and only makes a
// new one if it must.
this.bullets = this.physics.add.group({ classType: Bullet, maxSize:
BULLET_POOL_SIZE });
this.enemies = this.physics.add.group({ classType: Enemy, maxSize:
ENEMY_POOL_SIZE });
```

The bullet class knows how to switch itself on and off:

src/objects/Bullet.ts

```

export class Bullet extends Phaser.Physics.Arcade.Image {
    // The pool makes new bullets itself, calling `new Bullet(scene, x, y,
    ...)` . The pool also adds
    // them to the scene and gives them a body, so there is nothing else to do
    here.
    constructor(scene: Phaser.Scene, x: number, y: number) {
        super(scene, x, y, BULLET_KEY);
    }

    // Switch on: body enabled and reset to (x, y), active (so preUpdate
    runs), and visible.
    public fire(x: number, y: number): void {
        this.enableBody(true, x, y, true, true);
        this.setVelocityY(-SPEED);
    }

    // Switch off: body disabled (so it hits nothing), inactive (so preUpdate
    stops), and hidden.
    // It stays in the pool, ready for next time.
    public kill(): void {
        this.disableBody(true, true);
    }
}

```

and its `preUpdate()` calls `kill()` once the bullet is above the top of the screen. Compare the constructor with `Basket` 's: no `scene.add.existing` , no `scene.physics.add.existing` . The **pool** makes bullets, so the pool adds them to the scene and gives them bodies.

A game object has three separate switches, and a pooled object needs all three:

Switch	Set by	Off means
<code>active</code>	<code>setActive(false)</code>	Phaser stops calling its <code>preUpdate()</code> ; the pool treats it as free
<code>visible</code>	<code>setVisible(false)</code>	it is not drawn
<code>body.enable</code>	<code>disableBody()</code>	physics stops moving it, and it overlaps nothing

`disableBody(true, true)` turns off all three at once (the two `true` s mean "and make it inactive", "and hide it"). `enableBody(true, x, y, true, true)` turns them all back on, and moves the body to `(x, y)` with its old velocity cleared.

Firing asks the pool for a bullet:

src/scenes/GameScene.ts

```
private fire(): void {
    // get() finds a switched-off bullet in the pool (or makes one, if the
    pool is not full yet).
    // It returns `any`, because a group can hold anything - we know ours
    holds Bullets. It returns
    // null when all BULLET_POOL_SIZE bullets are already on screen.
    const bullet = this.bullets.get() as Bullet | null;
    if (bullet === null) {
        return;
    }
    bullet.fire(this.ship.x, this.ship.y - MUZZLE_OFFSET);
    this.sound.play(SHOOT_SOUND, { volume: 0.4 });
}
```

`get()` looks for the first member whose `active` is `false`. If there is none and the group has fewer than `maxSize` members, it makes a new one; if the pool is full, it gives back `null`. The type `Bullet | null` makes the compiler insist that you check - forget, and a burst of fire with a full pool ends the game with `TypeError: Cannot read properties of null (reading 'fire')`.

You can watch the pool work. We played for a minute, holding fire the whole time, and then asked the game how many bullets it had ever made - `this.bullets.getLength()` - and the answer was **9**. Nine bullet objects, fired over three hundred times.

Note - `killAndHide` is not enough. Groups have a method `group.killAndHide(child)`, and you will see it in many Phaser examples. It sets `active` and `visible` to `false` - but it does not touch the **body**. The body stays enabled, keeps its velocity, and keeps overlapping things. We tried it: a bullet switched off with `killAndHide`, invisible and inactive, carried on up the screen and shot down an enemy. For physics objects in a pool, use `disableBody(true, true)`. (For objects without bodies - a pool of text pop-ups, say - `killAndHide` is exactly right.)

When is `destroy()` still the right choice? When objects are **rare**. The shooter's explosions are not pooled - there are a few a second at most, each lives for less than half a second, and the code stays simpler:

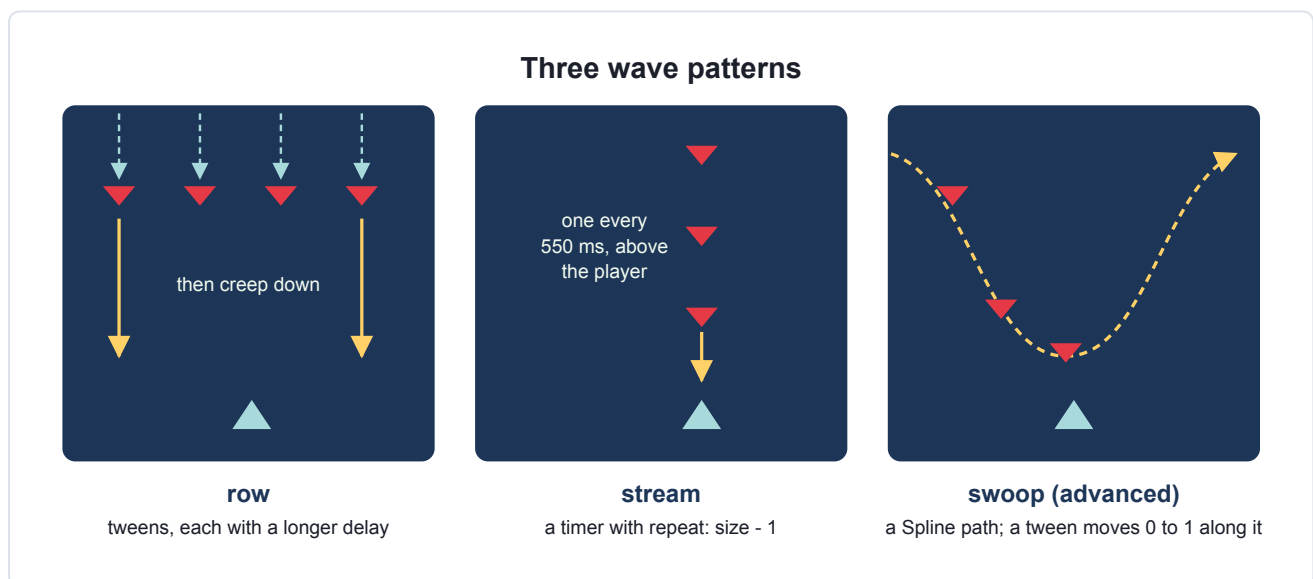
```
src/scenes/GameScene.ts
```

```
private explode(x: number, y: number): void {  
    const boom = this.add.sprite(x, y, EXPLOSION_KEY).play(EXPLODE_ANIM);  
    boom.once(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {  
        boom.destroy();  
    });  
    this.sparks.explode(SPARK_COUNT, x, y);  
    this.sound.play(EXPLOSION_SOUND, { volume: 0.5 });  
}
```

Pool what you make **many of, often**. The explosion is an animated sprite (Chapter 7) that destroys itself when its animation ends, plus a burst from **one** particle emitter made in `create()` with `emitting: false`. `explode(count, x, y)` fires that many particles at that point - one emitter serves every explosion in the game.

Waves

The catcher's timer dropped things at random. A shooter is more fun when enemies come in **waves**, each with a shape the player learns to read. This version has two:



```
src/scenes/GameScene.ts
```

```

private startNextWave(): void {
  this.wave = this.wave + 1;
  const size = Math.min(FIRST_WAVE_SIZE + this.wave - 1, MAX_WAVE_SIZE);
  const extraSpeed = (this.wave - 1) * SPEED_PER_WAVE;
  this.enemiesLeft = size;
  this.showBanner(`WAVE ${this.wave}`);

  if (this.wave % 2 === 1) {
    this.rowWave(size, ROW_SPEED + extraSpeed);
  } else {
    this.streamWave(size, STREAM_SPEED + extraSpeed);
  }
}

```

This is the game's **difficulty ramp**: wave 1 has five ships, and every wave has one more (up to twelve) and flies a little faster. The ramp is two lines of arithmetic, driven by named constants at the top of the file - which makes it easy to tune. Play, change a number, play again.

A **row** wave uses tweens (Chapter 7). Every ship starts above the screen; each one's tween waits a little longer (`delay: i * ROW_STAGGER`), so they swoop into line one after another. `Back.easeOut` overshoots and settles, which reads as "arriving". When a ship's tween completes, it starts to creep down:

`src/scenes/GameScene.ts`

```

this.tweens.add({
  targets: enemy,
  y: ROW_Y,
  duration: ROW_ENTRY_TIME,
  ease: "Back.easeOut",
  delay: i * ROW_STAGGER,
  onComplete: () => {
    enemy.setVelocityY(speed);
  },
});

```

Tweening a physics object's position is fine: each frame, before it moves, an Arcade body copies its position from its game object. While the tween runs the ship has no velocity, and the tween moves it; after, the velocity does.

A **stream** wave uses a timer with `repeat`, dropping one ship at a time above wherever the player is - so standing still is not an option:

src/scenes/GameScene.ts

```
this.time.addEvent({
  delay: STREAM_GAP,
  repeat: size - 1,
  callback: () => {
    const x = Phaser.Math.Clamp(this.ship.x, EDGE, this.scale.width - EDGE);
    const enemy = this.spawnEnemy(x, SPAWN_Y);
    enemy?.setVelocityY(speed);
  },
});
```

`enemy?.setVelocityY(speed)` is **optional chaining**: "if `enemy` is not `null` (or `undefined`), call the method; otherwise do nothing". It is a short way of writing the `if (enemy !== null)` check.

How does the game know a wave is over? It counts. `enemiesLeft` is set to the wave's size, and every way an enemy can leave - shot down, crashed into the ship, flown off the bottom - calls one method:

src/scenes/GameScene.ts

```
// one fewer enemy in this wave; when there are none left, the next wave comes after a pause
private enemyGone(): void {
  this.enemiesLeft = this.enemiesLeft - 1;
  if (this.enemiesLeft === 0 && !this.gameOver) {
    this.time.delayedCall(WAVE_PAUSE, () => {
      this.startNextWave();
    });
  }
}
```

Why not just ask the pool `this.enemies.countActive() === 0`? Because in a stream wave, the ships arrive over several seconds - after the first ship is shot, there are no active enemies at all, but the wave is far from over.

Pooled objects and tweens

A pooled enemy can be shot down in the middle of its entrance tween. If nothing stopped the tween, it would carry on. We tried it: we switched an enemy off halfway into its swoop and at once handed it out again at (700, 500) - and the old tween dragged it straight back up into the row, then set it creeping down at the row's speed. So `kill()` does more than switch the enemy off:

`src/objects/Enemy.ts`

```
public kill(): void {  
    // A pooled object must be switched off COMPLETELY: a tween still moving  
    // this enemy would carry  
    // on moving it while it waits in the pool, and move it again when it is  
    // reused.  
    this.scene tweens.killTweensOf(this);  
    this.disableBody(true, true);  
}
```

This is the price of pooling: an object that is reused must be **fully reset**, every time. Anything you attach to it - a tween, a timer, a tint, a changed scale - comes back with it next time unless `kill()` or `spawn()` puts it right.

Getting hit: invulnerability

When an enemy crashes into the ship, the player loses a life - and for the next two seconds the ship flashes and cannot be hurt again. Without that moment, one bad crash in the middle of a stream could cost every life at once.

`src/objects/PlayerShip.ts`


```

public makeInvulnerable(): void {
    this.invulnerable = true;
    this.scene.tweens.add({
        targets: this,
        alpha: 0.15,
        duration: FLASH_TIME,
        yoyo: true,
        repeat: INVULNERABLE_TIME / (FLASH_TIME * 2) - 1,
        onComplete: () => {
            this.setAlpha(1);
            this.invulnerable = false;
        },
    });
}

```

One tween does both jobs. `yoyo: true` fades out and back in (200 ms), `repeat` does that ten times (two seconds), and `onComplete` ends the safe time. The overlap uses a **process callback** (Chapter 8), which Phaser asks before calling the overlap callback:

src/scenes/GameScene.ts

```

this.physics.add.overlap(this.ship, this.enemies, (_ship, enemy) => {
    this.enemyHitsShip(enemy as Enemy);
}, () => {
    return !this.ship.isInvulnerable();
});

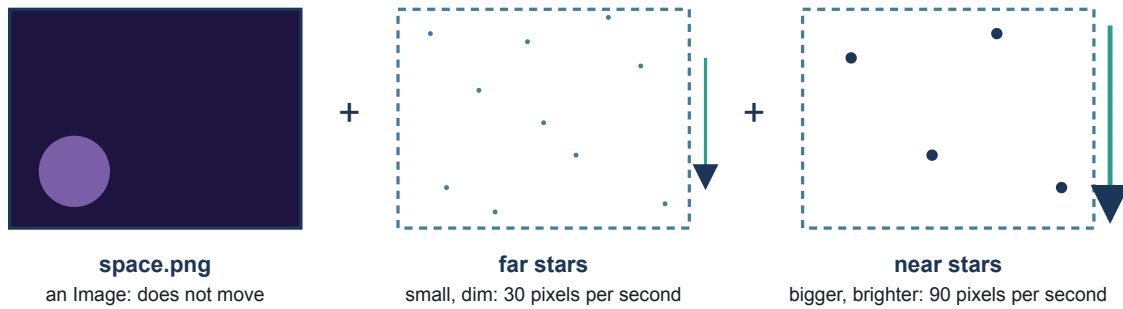
```

While the ship is flashing, the process callback says `false`, and enemies pass straight through it.

A parallax starfield

A still background makes a ship look parked. The shooter's sky scrolls - in three layers:

A starfield in three layers - a picture and two TileSprites



Each TileSprite repeats a 256 x 256 star picture; changing `tilePositionY` slides the picture inside it.

`space.png` does not move. Over it are two **TileSprites**. A `TileSprite` is a rectangle filled with a picture repeated over and over, like tiles on a floor; changing its `tilePositionY` slides the picture inside the rectangle, and the repeats mean it never runs out.

`src/objects/StarLayer.ts`

```
export class StarLayer extends Phaser.GameObjects.TileSprite {
  ...
  // TileSprite has a preUpdate of its own, so call it too (Chapter 4)
  protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);
    // a smaller tilePositionY slides the picture DOWN, which looks like
    // flying UP
    this.tilePositionY = this.tilePositionY - this.speed * delta / 1000;
  }
}
```

The far layer moves at 30 pixels a second, the near layer at 90. Things far away seem to move slowly and things close by quickly - the effect you see from a train window, called **parallax** - so two layers at two speeds are enough to give the sky depth.

The star pictures are not files: the start scene **draws** them, once, with `Graphics`, and saves the drawing as a texture:

`src/objects/StarLayer.ts`

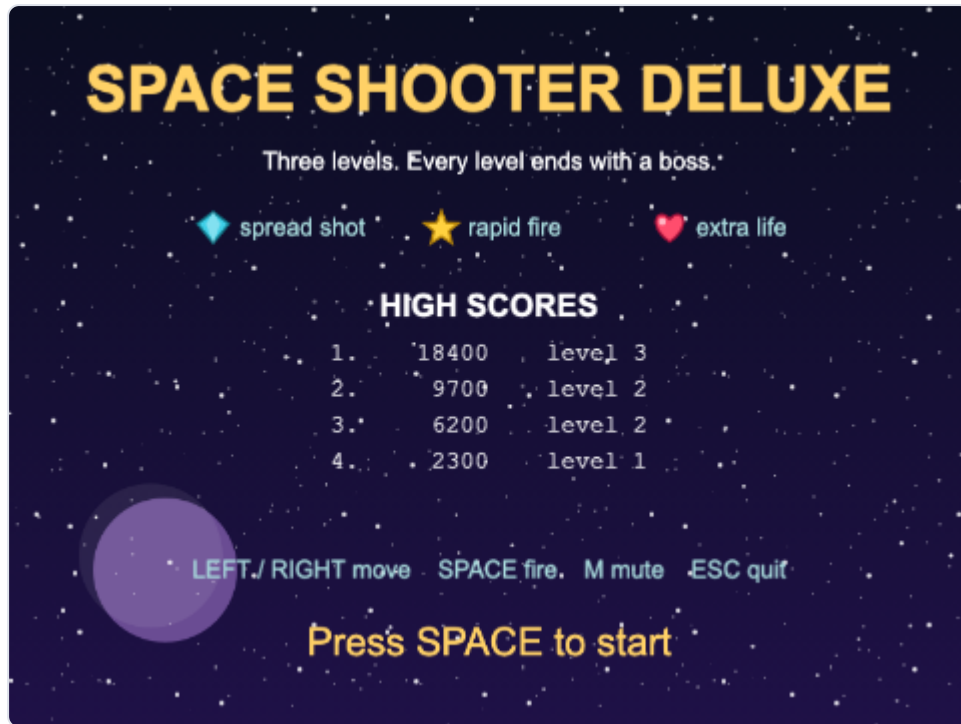
```
const graphics = scene.make.graphics({}, false); // false: draw it, but
not on screen
...
graphics.fillCircle(x, y, radius);
...
graphics.generateTexture(key, TILE_SIZE, TILE_SIZE);
graphics.destroy(); // the texture is saved; the Graphics is no
longer needed
```

`generateTexture(key, width, height)` turns what the Graphics has drawn into a texture called `key`, which any scene can then use like a loaded picture. `makeTexture` is `static` - it belongs to the class, not to one star layer, just as in Java - and it checks `scene.textures.exists(key)` first, so coming back to the title screen does not draw the stars again.

Version 3: Space Shooter Deluxe



`ch14_shooter_advanced` is the same shooter, grown into a game you could hand to a friend: three levels, a boss at the end of each, enemies that shoot back, power-ups, a menu, music and a high-score table. It is more than twice the size of the intermediate version, so it is split into more classes - but much of it is the intermediate version's code, moved to a better home.



Levels as data

In the intermediate version, the waves are worked out by arithmetic. Here each level is described by **data**:

src/levels.ts

```
// how a wave of enemies moves (see WaveSpawner)
export type WavePattern = "row" | "stream" | "swoop";

export interface WaveData {
  pattern: WavePattern;
  size: number;           // how many ships
  speed: number;          // pixels per second (for "swoop": how long each
  // ship's flight lasts, in ms)
}

export interface LevelData {
  waves: WaveData[];
  enemyFireChance: number; // 0 to 1: how likely a chosen enemy is to fire,
  // each time one is chosen
  bossHealth: number;      // hits it takes to destroy the boss
  bossFireDelay: number;   // milliseconds between the boss's volleys
  bossSweepTime: number;   // milliseconds for the boss to cross the screen
}
```

and `LEVELS` is an array of three `LevelData` object literals - level 1 starts with `{ pattern: "row", size: 6, speed: 50 }`.

`WavePattern` is a **union of string literals** - the only strings allowed are those three. Type `"sweep"` by mistake and the build fails. It does the job of a Java `enum`, with less ceremony.

Why data? Because a level designer - you, next week - can add a fourth level, or make level 2 harder, by editing a table, without reading any game code. The game scene never says how many levels there are; it asks `LEVELS.length`.

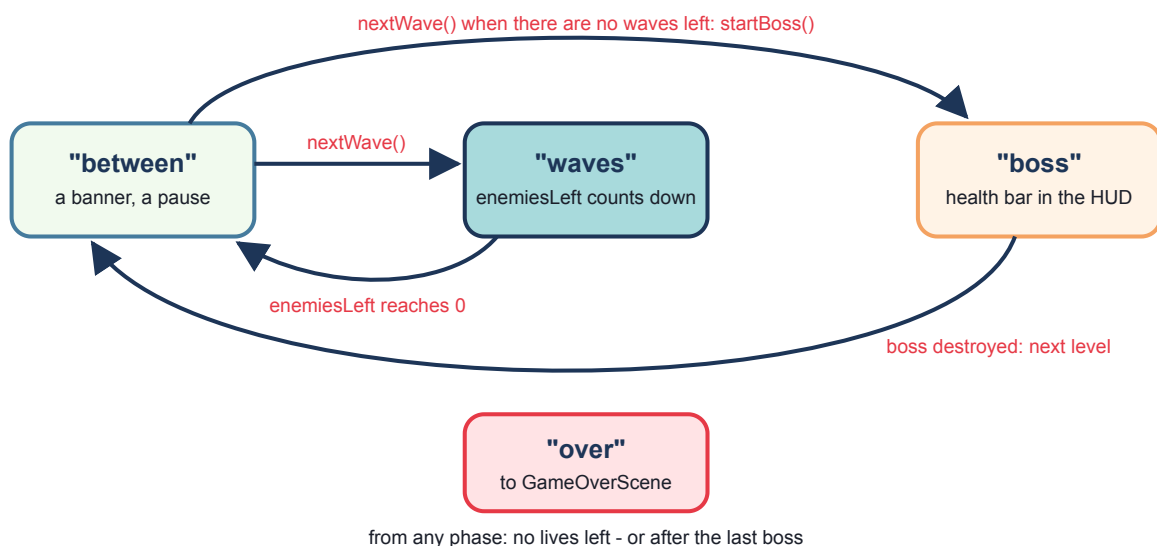
The phase

The scene now has more to keep track of - waves, then a boss, then a pause, then the next level - so it keeps a **phase**, a small state machine like the ones in Chapters 12 and 13:

```
src/scenes/GameScene.ts
```

```
// what the scene is doing - which decides what happens when the last enemy of a wave goes  
type Phase = "waves" | "between" | "boss" | "over";
```

GameScene's phase, through one level (advanced version)



```
src/scenes/GameScene.ts
```

```

private nextWave(): void {
  const level = LEVELS[this.levelIndex];
  this.waveIndex = this.waveIndex + 1;
  if (this.waveIndex >= level.waves.length) {
    this.startBoss();
    return;
  }
  const wave = level.waves[this.waveIndex];
  this.phase = "waves";
  this.enemiesLeft = wave.size;
  this.spawner.start(wave);
}

```

`enemyGone()` is the intermediate version's, with one change: it only starts the next wave if the phase is `"waves"`. When the game is over, or the boss is on screen, a stray enemy leaving must not start anything.

The WaveSpawner, and a curved path

The wave patterns have moved out of the scene into a class of their own, `WaveSpawner`. The scene says *which* wave to start; the spawner knows *how*. `row` and `stream` are the intermediate version's code, unchanged. `swoop` is new - ships that follow one another along a curve:

`src/WaveSpawner.ts`

```

private swoop(size: number, duration: number): void {
  const width = this.scene.scale.width;
  const fromRight = Math.random() < 0.5;
  const points = SWOOP_POINTS.map((value, index) => {
    const isX = index % 2 === 0;
    return fromRight && isX ? width - value : value;
  });
  const path = new Phaser.Curves.Spline(points);
  const start = path.getStartPoint();

  for (let i = 0; i < size; i++) {
    const enemy = this.spawnEnemy(start.x, start.y);
    enemy?.followPath(path, duration, i * SWOOP_GAP);
  }
}

```

A **spline** is a smooth curve through a list of points - here six points, from off the left edge, down across the screen and off the right edge (`SWOOP_POINTS` holds them as x, y pairs). Half the time every x is mirrored (`width - value`), so the swoop comes from the right.

How does a ship follow a curve? A curve can tell you the point any fraction of the way along it: `path.getPointAt(0)` is the start, `getPointAt(0.5)` is halfway, `getPointAt(1)` is the end. So the enemy has a number, `pathProgress`, that a tween moves from 0 to 1, and every frame it puts itself at that point:

`src/objects/Enemy.ts`

```
// fly along `path`, taking `duration` milliseconds, starting after `delay` milliseconds
public followPath(path: Phaser.Curves.Curve, duration: number, delay: number): void {
    this.path = path;
    this.pathProgress = 0;
    this.scene tweens.add({
        targets: this,
        pathProgress: 1,
        duration: duration,
        delay: delay,
    });
}
```

`src/objects/Enemy.ts`

```
preUpdate(_time: number, _delta: number): void {
    if (this.path !== null) {
        // getPointAt measures along the curve's length, so the ship moves at an even speed
        this.path.getPointAt(this.pathProgress, this.point);
        this.setPosition(this.point.x, this.point.y);
    }
}
```

A tween can move **any** number property of any object - not just `x`, `y` and `alpha`. Tweening a number of your own, and using it to work out something else, is one of the most useful tricks there is. And because the tween's target is the enemy, `kill()`'s `killTweensOf(this)` still stops it.

(Phaser also has a `PathFollower` game object, but one pooled enemy class that can fly any pattern is simpler than two kinds of enemy.)

Enemies that shoot back

Enemy bullets are a third pool, of `EnemyBullet` s. A looping timer (every 400 ms) might make one enemy fire:

`src/scenes/GameScene.ts`

```
const shooter: Enemy = Phaser.Utls.Array.GetRandom(shooters);
const bullet = this.enemyBullets.get() as EnemyBullet | null;
if (bullet === null) {
    return;
}
const angle = Phaser.Math.Angle.Between(shooter.x, shooter.y, this.ship.x,
this.ship.y);
bullet.fire(shooter.x, shooter.y, angle, ENEMY_BULLET_SPEED);
```

`Phaser.Math.Angle.Between(x1, y1, x2, y2)` is the angle, in radians, from the first point to the second, so the bullet flies at where the ship **is now**. (Keep moving and it misses.) Before this, `enemyFires()` rolls against the level's `enemyFireChance`, and chooses `shooters` only from enemies that are on screen and not too low - an enemy firing from right above the ship would be impossible to dodge, which is not hard, just unfair.

`EnemyBullet.fire()` turns the angle into a velocity with `cos` and `sin`, as Chapter 2 did for the ball.

The boss

After a level's last wave comes the boss: the enemy picture, scaled up, tinted, and much tougher. There is only ever one, so it is not pooled - the scene makes it once, switched off, and switches it on at the end of each level:

`src/objects/Boss.ts`


```
onComplete: () => {
  this.scene tweens.add({
    targets: this,
    x: RIGHT_X,
    duration: sweepTime,
    ease: "Sine.easeInOut",
    yoyo: true,
    repeat: -1,
  });
},
```

That is the end of `appear()` : a first tween flies the boss down into place, and when it completes, a second sweeps it across and back (`yoyo`) for ever (`repeat: -1`). The `Sine.easeInOut` ease slows the boss at each end, the way a pendulum slows.

A hit makes the boss flash white:

`src/objects/Boss.ts`

```
// Phaser 4: a FILL tint paints the whole picture in the tint colour (here,
// white)
this.setTint(0xffffffff).setTintMode(Phaser.TintModes.FILL);
this.scene.time.delayedCall(HIT_FLASH_TIME, () => {
  this.setTint(BOSS_TINT).setTintMode(Phaser.TintModes.MULTIPLY);
});
```

Note - Phaser 3 code does this with `setTintFill(0xffffffff)` , and you will find it in many tutorials. Phaser 4 removed `setTintFill` : a tint now has a colour (`setTint`) and a **mode** (`setTintMode`). `MULTIPLY` is the ordinary tint that darkens the picture; `FILL` paints over it.

The boss fires a fan of five bullets every `bossFireDelay` milliseconds, from a timer the scene keeps in `bossFireTimer` so it can `remove()` it when the boss is destroyed. The boss's health goes to the HUD, which draws the bar.

One detail: when an overlap is between a single object and a group, list the single object **first** - `overlap(this.boss, this.bullets, ...)` - so you know the callback gets (boss, bullet) in that order.

Power-ups

A destroyed enemy sometimes (15% of the time) drops a power-up. There are three kinds:

src/objects/PowerUp.ts

```
export type PowerUpKind = "spread" | "rapid" | "life";

// which picture shows which kind - a Record is an object with one entry for
// every kind, and the
// compiler checks that none is missing
const TEXTURES: Record<PowerUpKind, string> = {
  spread: GEM_KEY,
  rapid: STAR_KEY,
  life: HEART_KEY,
};
```

`Record<K, V>` is TypeScript's type for "an object with a `V` for every `K`" - here, a picture key for every kind of power-up. Add a fourth kind to `PowerUpKind` and forget its picture, and the build fails. In Java you would reach for an `EnumMap`.

Power-ups are rare, so they are not pooled; they go into an ordinary physics group, `this.physics.add.group({ velocityY: POWERUP_FALL_SPEED })`, which gives every member the same falling speed.

Spread shot and rapid fire **wear off** after eight seconds. Like the fire rate, the ship keeps them as times on the scene's clock: picking up spread shot sets `this.spreadUntil = time + POWER_UP_TIME`, and it is on while `time < this.spreadUntil`. There is no timer to cancel, nothing to clean up if the game ends, and a second spread shot just moves the time later. `tryToFire()` uses a shorter delay while rapid fire is on; with spread shot, `fire()` asks the pool for three bullets and fires them at -12, 0 and 12 degrees.

Collecting one shows off TypeScript's **narrowing**:

src/scenes/GameScene.ts

```
private collect(powerUp: PowerUp): void {
  this.sound.play(POWERUP_SOUND);
  if (powerUp.kind === "life") {
    this.lives = Math.min(this.lives + 1, MAX_LIVES);
    this.events.emit(LIVES_CHANGED, this.lives);
  } else {
    // here TypeScript knows kind is "spread" or "rapid" - the only other
    // possibilities
    this.ship.powerUp(powerUp.kind, this.time.now);
  }
  powerUp.destroy();
}
```

`ship.powerUp()` only accepts `"spread" | "rapid"`. Inside the `else`, the compiler has worked out that `kind` cannot be `"life"`, so it allows the call. Move that line above the `if` and the build fails.

A HUD scene, fed by events

The score, hearts, level, power-ups and boss bar are drawn by `HudScene`, which the game scene launches to run on top of itself (Chapter 4):

`src/scenes/GameScene.ts`

```
// the HUD runs on top of this scene, starting from these values
const hudData: HudData = { score: this.score, lives: this.lives, level: 1 };
this.scene.launch(HUD_SCENE, hudData);
```

After that, the game scene never touches the HUD. When something changes, it **emits an event**

- `this.events.emit(SCORE_CHANGED, this.score)` - and the HUD, which listens, redraws (Chapter 6). The event names are constants in `src/events.ts`, for the same reason scene keys are.

`src/scenes/HudScene.ts`

```

const gameEvents = this.scene.get(GAME_SCENE).events;
gameEvents.on(SCORE_CHANGED, this.showScore, this);
gameEvents.on(LIVES_CHANGED, this.showLives, this);
gameEvents.on(LEVEL_CHANGED, this.showLevel, this);
gameEvents.on(WEAPON_CHANGED, this.showWeapon, this);
gameEvents.on(BOSS_HEALTH_CHANGED, this.showBossHealth, this);

this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  gameEvents.off(SCORE_CHANGED, this.showScore, this);
});

```

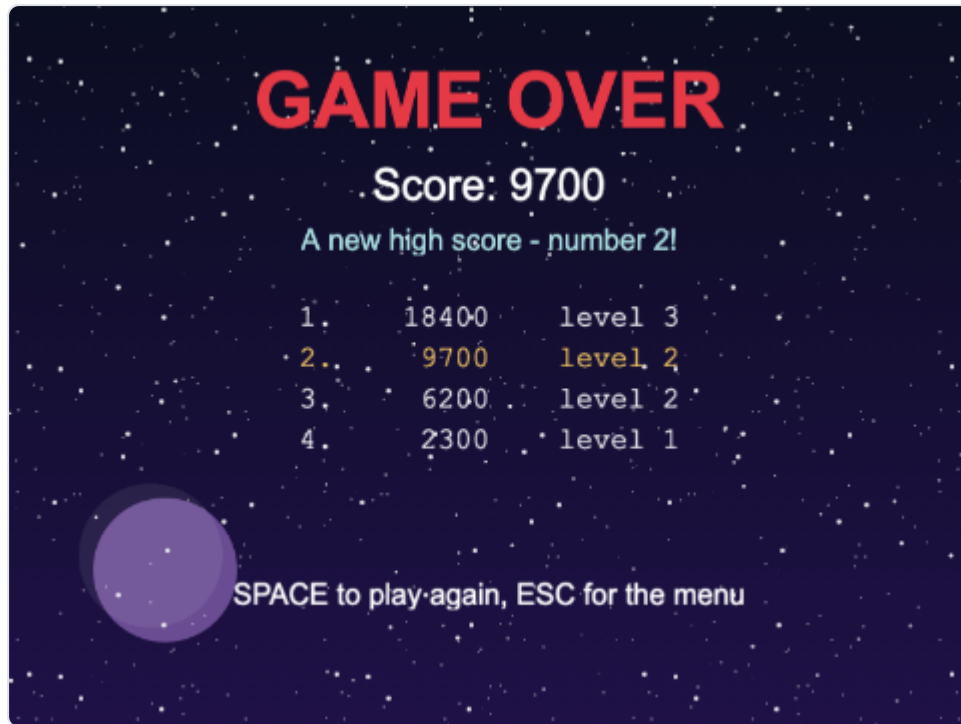
...and so on for the other four. The HUD listens on **another scene's** emitter, which outlives the HUD: every new game launches a new HUD. Without the `off` calls, each game would add another set of listeners, all still attached to HUDs that are long gone (Chapter 4's listener leak). Passing the method with its `this` (`this.showScore, this`) to both `on` and `off` is what lets `off` find the right listener to remove.

The boss bar is a `Container` of three objects, shown and hidden as one; its red fill has its origin at its left end, so `setScale(health / maxHealth, 1)` shrinks it towards the left.

Music, and scores that last

Two more things come from earlier chapters and need no new ideas:

- `src/music.ts` plays one music track at a time. The sound manager belongs to the game, not a scene, so `playMusic()` checks whether a track is already playing before starting it (Chapter 5)
- `src/HighScores.ts` keeps the best five scores in `localStorage`, and checks what it loads back: anything that is not a list of `{ score, level }` objects is treated as an empty table rather than crashing the game (Chapter 6)



Common mistakes

Note - Four bugs that every shooter-writer meets:

- **a pooled object that keeps something from last time** - an old angle, an old tween, an old tint. Whatever `kill()` does not undo, `spawn()` or `fire()` must set. If an object misbehaves only the *second* time it is used, this is why
- **`get()` returning `null`** - type it as `Bullet | null`, not as `Bullet`, and let the compiler make you check
- **a body left switched on** - `killAndHide` and `setVisible(false)` leave the body working. Invisible things that still hit things are nearly always this
- **timers that outlive their purpose** - a spawner never removed keeps spawning after the game is over. `remove()` it, or check the game's state at the top of the callback, as `enemyFires()` does

Summary

- the genre is simple on the player's side; the design is in what comes down the screen, how fast, and how readable and fair it is
- `this.time.addEvent({ delay, loop: true, callback })` spawns on a timer; `repeat: n` runs it `n + 1` times; `remove()` stops it
- a physics group with `gravityY` or `velocityY` gives every member the same motion
- an **object pool** reuses objects instead of making and destroying them: a physics group with `classType` and `maxSize`, `get()` (which may return `null`), `enableBody(true, x, y, true, true)` to switch on and `disableBody(true, true)` to switch off. `killAndHide` leaves the body on
- pool what you make many of, often; `destroy()` is fine for the rare things
- a fire rate is a stored "next allowed" time; so is a power-up that wears off
- waves: tweens with staggered delays, timers with `repeat`, and a tween moving an object along a `Spline`; count the ships in a wave to know when it is over
- invulnerability is a flashing tween plus a process callback; a parallax starfield is two `TileSprite`s scrolling at different speeds
- as a game grows: levels as data, a phase, classes with one job each, and a HUD scene that only listens to events

Challenges

1. **Pointer control** (*ch14_catcher_simple*) - Let the player move the basket with the mouse, or a finger on a touch screen, as well as the arrow keys. The basket should slide towards the pointer at its usual speed - not jump to it - and stop when it gets there. The arrow keys should still work.
2. **Combo scoring** (*ch14_catcher_simple*) - Reward catching fruit in a row. Each fruit caught without a miss in between is worth more: 10, then 20, then 30, up to 50 a fruit. Dropping a fruit or catching a bomb breaks the combo. Show the current combo on screen, and pop up a short floating "+30" where each fruit was caught.
3. **Zig-zag enemies** (*ch14_shooter_intermediate*) - Add a third kind of wave in which the ships zig-zag from side to side as they come down the screen. Make it every third wave (row, stream, zig-zag, row, ...), and make the zig-zag grow wilder as the waves go on.

4. **Shield power-up** (*ch14_shooter_advanced*) - Add a fourth power-up: a shield. While the ship has it, a bubble shows round the ship, and the next hit - from an enemy or a bullet - pops the shield instead of costing a life. The shield does not wear off with time. Show it on the menu's power-up list too. *Hint:* follow `PowerUpKind` through the code, and let the compiler show you everywhere a new kind needs handling. The picture can be drawn with `Graphics` and `generateTexture` .
5. **Smart bomb** (*ch14_shooter_advanced*) - The player starts each game with three bombs. Pressing B uses one: every enemy on screen explodes (and scores), every enemy bullet disappears, the screen flashes, and the boss - if it is there - loses a chunk of health but survives. Show the bombs left in the HUD. *Hint:* the enemies are in a pool - which method gives you the ones that are really on screen? And think carefully about `enemiesLeft` when many ships are destroyed at once. The camera can `flash(duration)` .
6. **Second player** (*ch14_shooter_intermediate*) - Add a second ship for a second player, moved with A and D and firing with W, in a different colour. Each player has their own score and lives (show both); a player with no lives left disappears, and the game ends when both are out. The game over screen should say who won. *Hint:* the ship's class should not need to know which keys move it. Can both ships share the bullet pool? And what should the stream wave aim at now?

Previous: [Chapter 13 - Memory match](#) · Next: [Chapter 15 - Platformer](#)

Platformer

Run, jump, land on things, and do not fall down the hole. The platformer is the genre where physics matters most: the whole game is how the hero moves. In this chapter you build one three times - a single screen of platforms and coins, then a scrolling level made in Tiled with enemies and hazards, then a three-level game with ladders, moving platforms, checkpoints and a level select - and see how each version grows out of the one before.



What you will learn

- how gravity, `body.blocked.down` and a jump velocity make a hero that runs and jumps
- what makes a jump *feel* right: acceleration and drag, a variable jump height, coyote time and jump buffering
- how to build a level in Tiled whose tiles say what they are - solid, deadly, climbable - and read its objects to place the hero, coins, enemies and the goal
- how enemies patrol, how the game tells a stomp from a hurt, and how hazards are fair
- one-way platforms, platforms that move and carry the hero, and ladders
- checkpoints, lives, a HUD scene and a level select that unlocks as you play

The projects

Project	What it shows
<code>ch15_platformer_simple</code>	one screen: gravity, a static group of platforms, an animated hero, coins to collect
<code>ch15_platformer_intermediate</code>	a scrolling Tiled level: a jump with good feel, slimes to stomp, spikes and lava, a flag to reach, a camera that follows
<code>ch15_platformer_advanced</code>	three levels and a level select; moving and one-way platforms, ladders, bats that swoop, checkpoints, lives and a HUD

What makes a platformer

A platformer is a game about **movement**. The player spends the whole game running and jumping, so if the jump feels floaty, or a jump pressed a moment too late is ignored, the game feels bad however good the levels are. So the three versions spend as much time on *how the hero moves* as on *what is in the level*:

	simple	intermediate	advanced
the level	platforms placed in code	a Tiled map, bigger than the screen	three Tiled maps, one scene
the hero	velocity straight from the keys	acceleration, variable jump, coyote time, buffering	... and ladders
danger	none	slimes, spikes, lava, gaps	... and swooping bats
when hurt	-	start the level again	lose a life, back to the checkpoint
around it	one scene	title, game and end scenes	preload, level select, game, HUD, result

This chapter uses ideas from all the chapters before it, and points back to them rather than explaining them again: sprite sheets and animations (Chapter 7), Arcade Physics bodies,

colliders and groups (Chapters 8 and 9), tilemaps (Chapter 10) and Tiled maps (Chapter 11).

Version 1: the simple platformer



Gravity

Physics is switched on for the whole game in the config, with gravity:

src/main.ts

```
physics: {  
  default: "arcade",  
  arcade: {  
    gravity: { x: 0, y: 900 },  
    debug: false, // true draws every body's outline - try it  
  },  
},
```

Every dynamic body is now pulled down, speeding up by 900 pixels per second every second. Nothing stays in the air unless something holds it up.

The hero

`Hero` extends `Phaser.Physics.Arcade.Sprite` - a Sprite (so it can play animations) that can be given a physics body:

`src/objects/Hero.ts`

```
export class Hero extends Phaser.Physics.Arcade.Sprite {
  // Phaser types `body` as "a dynamic body, a static body, or null",
  // because it cannot know which
  // this sprite will get. It is always a dynamic body (the constructor
  // makes it), and `declare`
  // tells TypeScript so. It adds no code - it only narrows the type.
  declare body: Phaser.Physics.Arcade.Body;

  private cursors: Phaser.Types.Input.Keyboard.CursorKeys;

  constructor(scene: Phaser.Scene, x: number, y: number) {
    super(scene, x, y, HERO_KEY, 0);

    scene.add.existing(this); // draw it, and call its preUpdate() every
    // frame
    scene.physics.add.existing(this); // give it a dynamic Arcade Physics
    // body

    // the body is centred across the picture, and its bottom is the bottom
    // of the feet
    this.body.setSize(BODY_WIDTH, BODY_HEIGHT);
    this.body.setOffset((this.width - BODY_WIDTH) / 2, this.height -
    BODY_HEIGHT);
    this.setCollideWorldBounds(true);
  }
}
```

Two things to notice:

- `declare body`. The type of `body` on an Arcade Sprite is `Body | StaticBody | null`, so without this line every `this.body.velocity` would be a type error ("possibly null"). `declare` re-states the type of an inherited field without adding any code. It is a promise - like `as` - so it is only right because the constructor really does give the sprite a dynamic body
- **the body is smaller than the picture**. The hero picture is 32 x 48 with space around the figure. A body of 18 x 42, standing on the same feet, means the hero does not bump its head on thin air or hang off a ledge by an empty corner. Turn on `debug: true` to see it

Like Chapter 1's `Ball`, the hero looks after itself every frame:

`src/objects/Hero.ts`

```
protected override preUpdate(time: number, delta: number): void {  
    // a Sprite's own preUpdate() moves its animation on - without this line  
    // the hero never animates  
    super.preUpdate(time, delta);  
  
    this.run();  
    this.jump();  
    this.animate();  
}
```

Standing on something

The platforms are a **static group** (Chapter 8): bodies that never move and cannot be pushed. `create()` on a static group makes a sprite and gives it a static body in one go:

`src/scenes/GameScene.ts`

```
const platforms = this.physics.add.staticGroup();  
platforms.create(400, 568, GROUND_KEY); // the ground strip, 800 x 64, along  
// the bottom  
for (const p of PLATFORMS) {  
    platforms.create(p.x, p.y, PLATFORM_KEY);  
}  
  
// --- the hero, who stands on the platforms  
Hero.createAnimations(this);  
this.hero = new Hero(this, 60, 480);  
this.physics.add.collider(this.hero, platforms);
```

Gravity pulls the hero into a platform; the collider pushes it back out and stops it. And as it does, it sets a flag on the hero's body: `body.blocked.down` is `true` when, in the last physics step, something stopped the body moving down. That one flag is the answer to the question every platformer asks all the time: *is the hero standing on something?*

Jumping

`src/objects/Hero.ts`

```
private jump(): void {
  const jumpPressed = Phaser.Input.Keyboard.JustDown(this.cursors.up) ||
    Phaser.Input.Keyboard.JustDown(this.cursors.space);

  if (jumpPressed && this.body.blocked.down) {
    this.setVelocityY(-JUMP_SPEED); // negative y is UP
    this.scene.sound.play(JUMP_SOUND);
  }
}
```

A jump is just an upward velocity. Gravity then takes it away, 900 pixels per second every second, until the hero stops rising and starts to fall: the arc comes for free. `JustDown` is true only on the frame the key went down, so holding the key does not bounce the hero up and down.

Running is even simpler: left and right set the horizontal velocity, and no key sets it to 0.

Choosing an animation

Chapter 7 built an animation state machine for this hero. Here the state is worked out from the body every frame - in the air or not, moving or not - and each state has one animation:

src/objects/Hero.ts

```
private animate(): void {
  const velocity = this.body.velocity;

  if (!this.body.blocked.down) {
    this.play(velocity.y < 0 ? JUMP : FALL, true);
  } else if (velocity.x !== 0) {
    this.play(RUN, true);
  } else {
    this.play(IDLE, true);
  }
}
```

`play(key, true)` means "unless it is already playing" - without the `true`, the run animation would restart from its first frame 60 times a second, and look frozen.

Coins: overlap, not collide

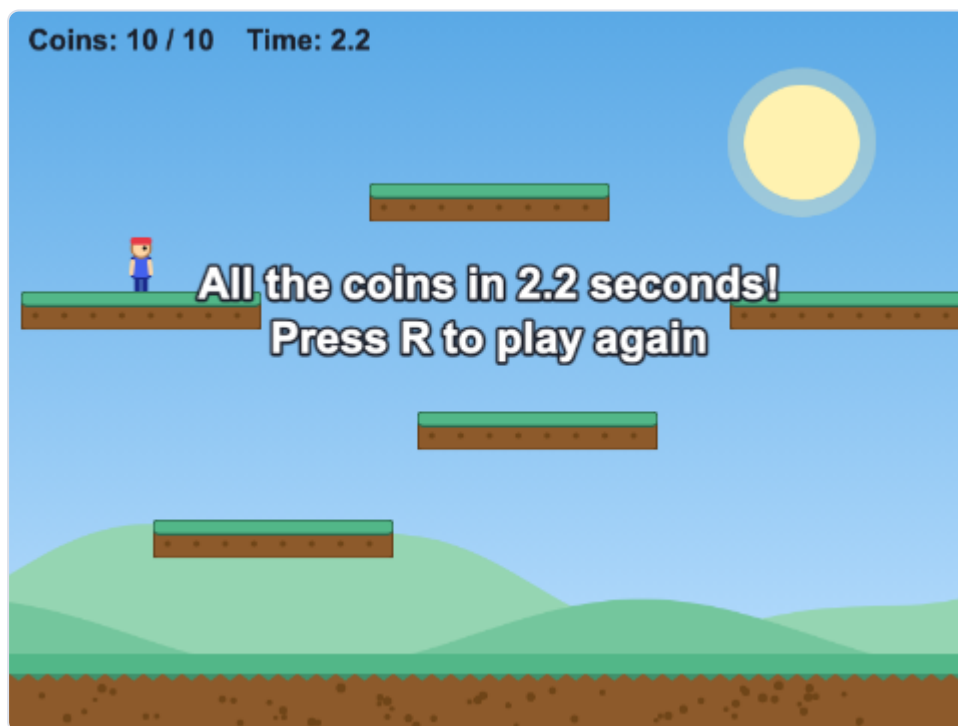
The coins are a static group too, but the hero must pass *through* them, so they use `overlap` :

`src/scenes/GameScene.ts`

```
// Phaser passes the two objects in the order they were given: hero first,
// then the coin.
// Its types cannot know which is which, so the coin is cast with `as`
this.physics.add.overlap(this.hero, coins, (_hero, coin) => {
  this.collectCoin(coin as Phaser.Physics.Arcade.Sprite);
});
```

`collectCoin()` destroys the coin - sprite and body together, so it cannot be collected twice - and counts it. Ten coins wins. R restarts the scene, and `init()` puts the count back to 0 (Chapter 2).

That is a complete platformer in about 350 lines, many of them comments. Play it, and you will notice what is missing: the hero starts and stops dead, every jump is the same height, and a jump pressed a fraction too late at the edge of a platform does nothing. The next version fixes all three.



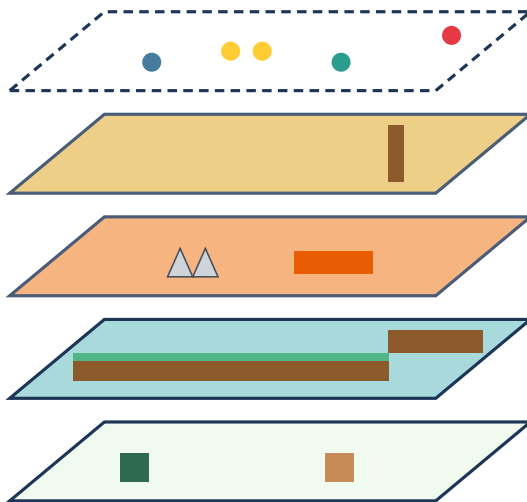
Version 2: a level from Tiled, and a jump that feels right



The level

The level is a Tiled map (Chapter 11), 80 tiles wide and 20 high - much bigger than the screen. It has three tile layers - Background, Ground and Hazards - and an Objects layer. (The advanced version's maps add a fourth tile layer, for ladders; the picture shows all five.)

One Tiled map, five layers - and what the game does with each



Objects

player, coins, slimes, bats, checkpoints, flag, platforms
`getObjectLayer("Objects").objects`

Ladders

tiles with `ladder = true` (advanced)
`getTileAtWorldXY(x, y)`

Hazards

tiles with `hazard = true`: overlap only
`physics.add.overlap(hero, hazards, ...)`

Ground

tiles with `collides = true`
`setCollisionByProperty({ collides: true })`

Background

bushes and signs - drawn, never touched

The properties belong to the TILES, in the tileset embedded in the map - so every spike in every level is a hazard.

The important idea is in the tileset: its tiles carry **custom properties**. Grass, dirt, brick and stone have `collides = true`; spikes, lava and water have `hazard = true`; the ladder has `ladder = true`. The code never mentions a tile number - it asks for tiles by what they are:

`src/scenes/GameScene.ts`

```
this.map = this.make.tilemap({ key: MAP_KEY });
const tileset = this.map.addTilesetImage(TILESET_NAME, TILES_KEY);
if (tileset === null) {
    // null means the name did not match the tileset's name inside the map
    throw new Error(`The map has no tileset called "${TILESET_NAME}"`);
}
this.map.createLayer("Background", tileset);
const ground = this.createLayer("Ground", tileset);
const hazards = this.createLayer("Hazards", tileset);

// collide with every tile that has collides = true in Tiled - no tile
// numbers in the code
ground.setCollisionByProperty({ collides: true });
```

`addTilesetImage` and `createLayer` quietly give back `null` when a name does not match the map - and the game then fails somewhere else, with a confusing error. Checking for `null` straight away, and saying *which* name was wrong, turns a puzzle into a one-line fix.

Notice `TILES_KEY` : the tileset picture was loaded as a **sprite sheet** of 32 x 32 frames. A tilemap only needs the picture, so a sprite sheet works; and the same key can show a single tile as a sprite - the flag is `this.physics.add.staticImage(x, y, TILES_KEY, FLAG_FRAME)` .

Note - The maps in this chapter were not drawn by hand in Tiled: a small script, `chapters/ch15_platformer/tools/make_levels.ts` , turns levels drawn as text into `.tmj` files - see "The level generator" near the end of the chapter. They are ordinary Tiled maps, with the tileset embedded: each project has a copy in `tilled/` to open in Tiled, and the game loads the one in `public/assets/maps/` .

Changing a level in Tiled. Open the map from the project's `tilled/` folder, make your changes, and save. Then **File > Export As...**, choose **JSON map files**, and save it over the map of the same name in `public/assets/maps/` - that is the file the game loads. (The export dialog starts in `tilled/` , so move to `public/assets/maps/` first. After the first time, **File > Export** repeats it.) Build, and refresh the game. Tiled writes the tileset's picture path relative to wherever you export to; Phaser never reads that path - the code names the picture with `addTilessetImage` - so it only has to be right for Tiled.

A world bigger than the screen

`src/scenes/GameScene.ts`

```
const width = this.map.widthInPixels;
const height = this.map.heightInPixels;
this.physics.world.setBounds(0, 0, width, height);
this.physics.world.setBoundsCollision(true, true, false, false);
this.cameras.main.setBounds(0, 0, width, height);
```

The physics world and the camera become the size of the map (Chapter 10).

`setBoundsCollision` takes left, right, up, down: the bottom of the world is **open**, so a hero who falls down a gap falls out of the world - and `update()` notices when the hero is 100 pixels below the map.

The camera follows the hero with `startFollow(this.hero, true, 0.1, 0.1)` : the last two numbers are the *lerp* - each frame the camera moves a tenth of the way to the hero, which smooths out every jump and landing. The sky has `setScrollFactor(0)` so it never

moves, and the clouds `0.3`, so they drift slowly past: far things seem to move more slowly, and this **parallax** makes the flat picture look deep.

Things placed in Tiled

The Objects layer holds points: where the hero starts, and where every coin, slime and the flag go. Each point is named by its `type` - in Tiled, that is the object's **Class** field (the Objects panel shows it in its Class column). The **Name** is only a label: an object whose Name says "slime" but whose Class says `coin` becomes a coin. A point marks where the thing's *feet* go:

src/scenes/GameScene.ts

```
// every object on the Objects layer with this type
private objectsOfType(type: string): Phaser.Types.Tilemaps.TiledObject[] {
    return this.map.getObjectLayer("Objects")!.objects.filter((o) => o.type
=== type);
}
```

src/scenes/GameScene.ts

```
const start = this.findObject("player");
this.hero = new Hero(this, start.x!, start.y!);
this.physics.add.collider(this.hero, ground);
```

A `TiledObject`'s `x` and `y` are typed `number | undefined`, because a few kinds of Tiled object have no position; ours always do, hence the `!`. The hero calls `this.setOrigin(0.5, 1)`, so its position *is* its feet, and it stands exactly on the point. (Arcade bodies follow the origin, so the body stays in the right place.)

Game feel 1: speeding up and slowing down

The simple hero went from standing to full speed in one frame. Now the keys set an **acceleration**, and **drag** slows the hero when no key is held:

src/objects/Hero.ts

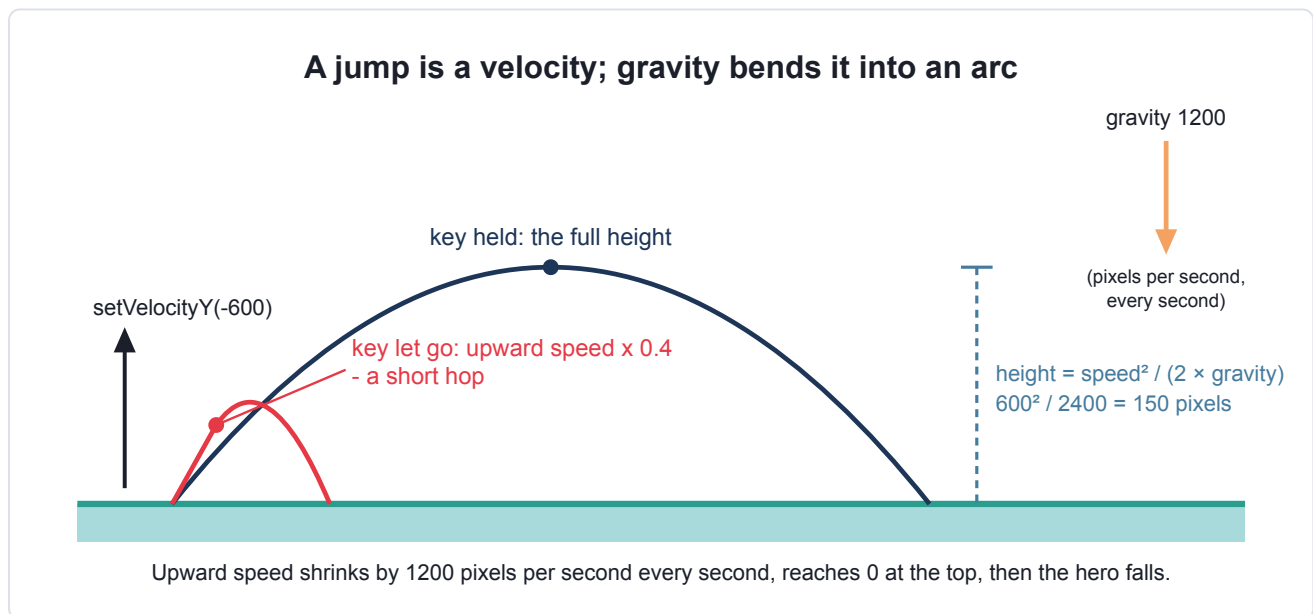
```
// drag slows the hero only while its acceleration is zero - that is, when
no key is held
this.setDragX(RUN_DRAG);
this.setMaxVelocity(MAX_RUN_SPEED, MAX_FALL_SPEED);
```

src/objects/Hero.ts

```
private run(): void {
  if (this.cursors.left.isDown) {
    this.setAccelerationX(-RUN_ACCELERATION);
  } else if (this.cursors.right.isDown) {
    this.setAccelerationX(RUN_ACCELERATION);
  } else {
    this.setAccelerationX(0); // let drag bring the hero to a stop
  }
}
```

With an acceleration of 1400 the hero reaches full speed (220) in about a sixth of a second: quick enough to feel responsive, slow enough to feel like it has weight. `setMaxVelocity` caps the running speed - and the falling speed too, so a long fall can never be fast enough to pass through a floor in a single step. These numbers are the *feel* of your game: change them and play.

Game feel 2: a jump you control



In almost every platformer, a tap makes a hop and holding the button makes a full jump. The trick is not to keep pushing the hero up while the key is held - it is to **cut the jump short** when the key is let go:

src/objects/Hero.ts

```
// a short tap makes a short hop: letting go while still going up cuts the
speed
const released = Phaser.Input.Keyboard.JustUp(this.cursors.up) ||
    Phaser.Input.Keyboard.JustUp(this.cursors.space);
if (released && this.body.velocity.y < 0) {
    this.setVelocityY(this.body.velocity.y * JUMP_CUT);
}
```

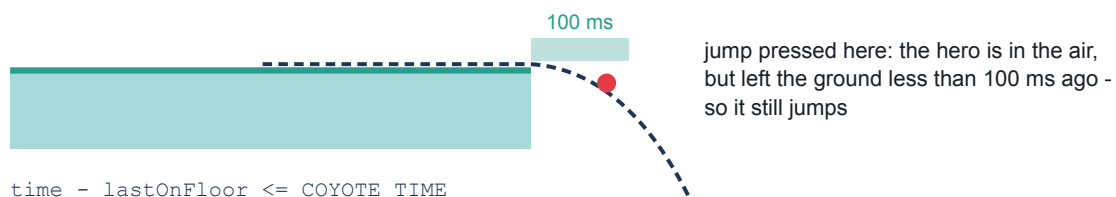
Every jump starts at the same speed. Letting go while the hero is still rising keeps only 40% of the upward speed, so the hero peaks soon after. Held all the way, the jump is 150 pixels high - about four and a half tiles; a quick tap is under 40.

Game feel 3: coyote time and jump buffering

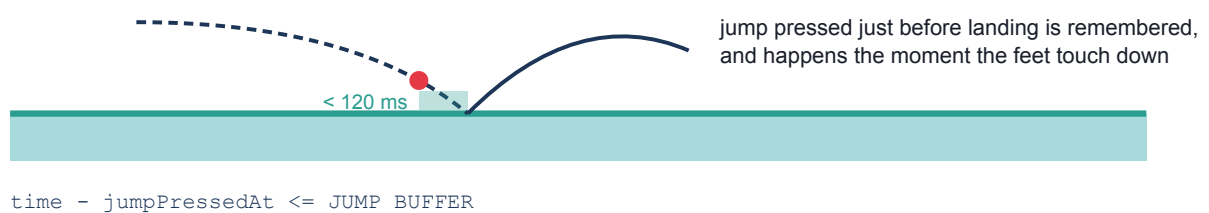
Two small timers make a big difference, and players never notice them - they only notice when they are missing:

Two forgiving timers: remember WHEN, not just WHETHER

Coyote time - jump a moment AFTER running off a ledge



Jump buffering - jump a moment BEFORE landing



- **coyote time:** like the cartoon coyote who runs off a cliff and does not fall until he looks down, the hero can still jump for 100 ms after running off a ledge. Players press jump when they *see* the edge, which is often a frame or two after the feet left it
- **jump buffering:** a jump pressed up to 120 ms *before* landing is remembered, and happens the moment the hero lands. Without it, a jump pressed slightly early is simply lost

Both come from the same change of thinking: instead of "is the hero on the floor *now*?" and "is jump pressed *now*?", remember *when* each last happened:

src/objects/Hero.ts

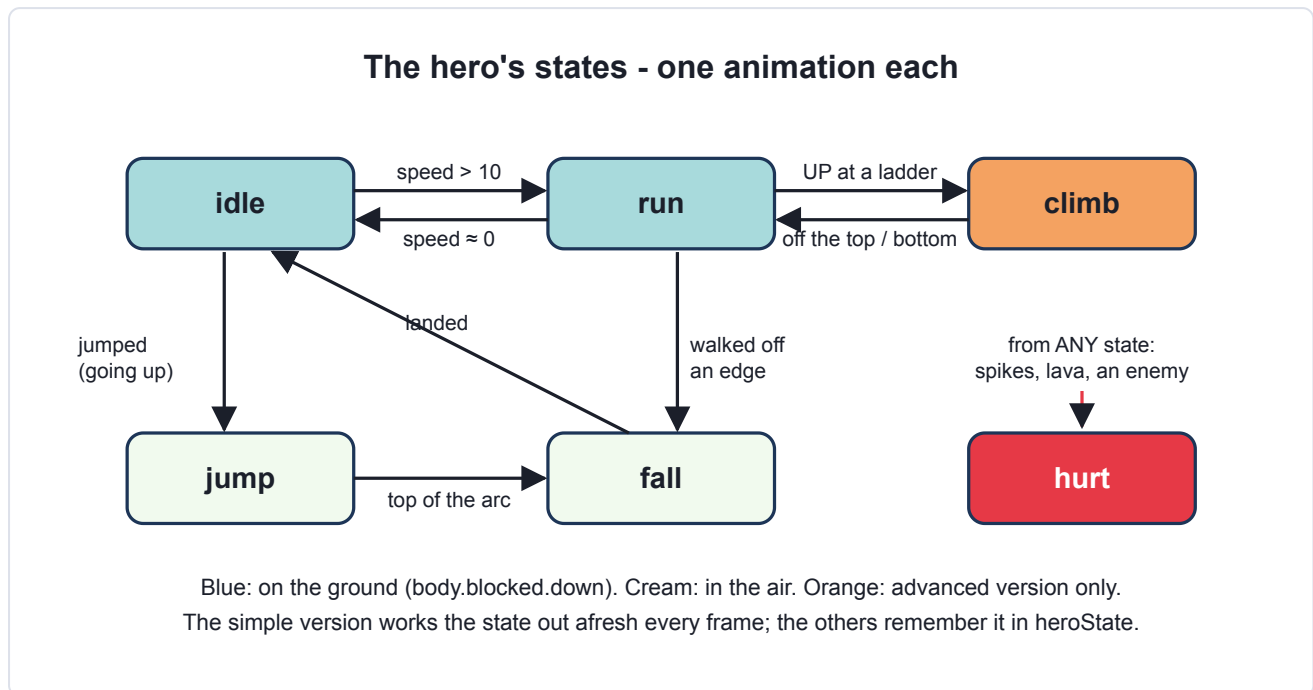
```
private jump(time: number): void {
    // remember WHEN things happened, rather than only whether they are
    // happening now
    if (this.body.blocked.down) {
        this.lastOnFloor = time;
    }
    if (Phaser.Input.Keyboard.JustDown(this.cursors.up) ||
        Phaser.Input.Keyboard.JustDown(this.cursors.space)) {
        this.jumpPressedAt = time;
    }

    const recentlyOnFloor = time - this.lastOnFloor <= COYOTE_TIME;
    const recentlyPressed = time - this.jumpPressedAt <= JUMP_BUFFER;

    if (recentlyOnFloor && recentlyPressed) {
        this.setVelocityY(-JUMP_SPEED);
        this.scene.sound.play(JUMP_SOUND);
        // use both up, so one press cannot make two jumps
        this.lastOnFloor = -1000;
        this.jumpPressedAt = -1000;
    }
}
```

`time` is the `time` given to `preUpdate()`, in milliseconds. The last two lines matter: without them, the frame after a jump would still count as "recently on the floor" *and* "recently pressed", and the hero would jump again.

The state machine, remembered



The simple hero worked out its state every frame and called `play()`. This one also *remembers* its state, as a union of string literals (Chapter 12):

src/objects/Hero.ts

```
type HeroState = "idle" | "run" | "jump" | "fall" | "hurt";
```

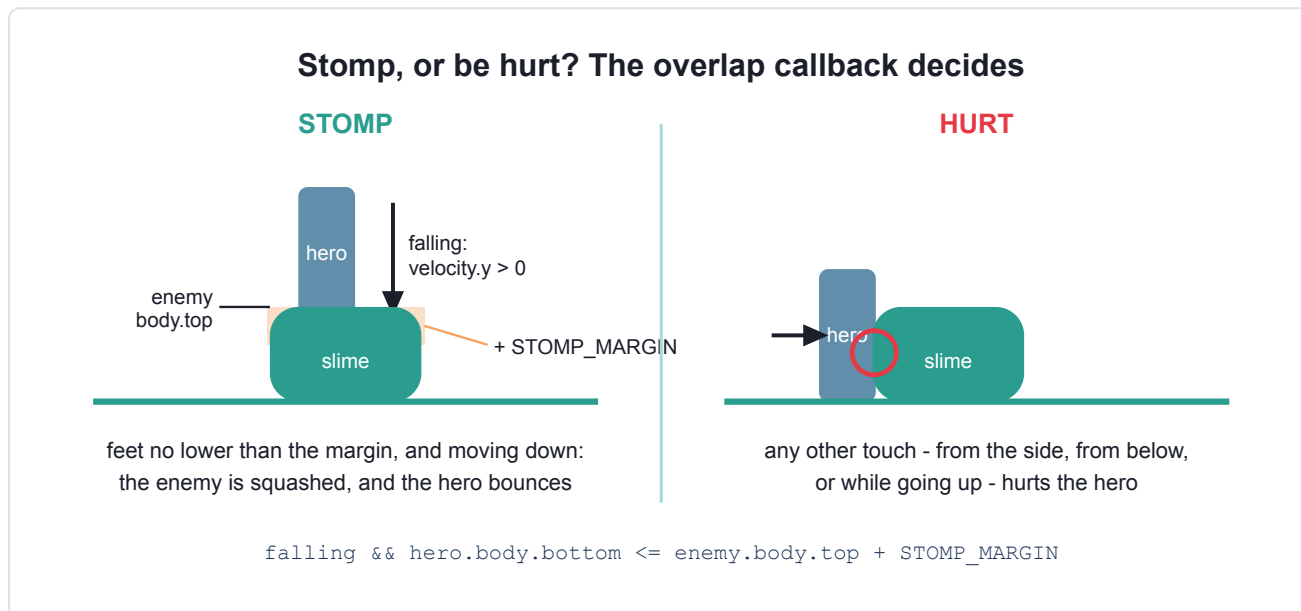
src/objects/Hero.ts

```
// every change of state goes through here, so each state's animation is set in one place
private setHeroState(next: HeroState): void {
  if (next !== this.heroState) {
    this.heroState = next;
    const animations: Record<HeroState, string> = { idle: IDLE, run: RUN,
    jump: JUMP, fall: FALL, hurt: HURT };
    this.play(animations[next]);
  }
}
```

Remembering it means the hero can have a state that the body cannot tell you about - "hurt" - and behave differently in it: once hurt, `preUpdate()` returns before reading any keys. (The field is called `heroState` because every game object already has a field called

state.) `Record<HeroState, string>` is an object with exactly one entry for each state: add a state to `HeroState` and forget its animation, and the build fails.

Slimes that patrol



A slime walks one way until it should turn round. Walls are easy - the collider sets `body.blocked.left` or `blocked.right`. The edge of a platform, or a spike, it has to *look for*, by asking the tilemap what is just in front of its feet:

`src/objects/Slime.ts`

```
// look just beyond the leading edge of the body: no ground under that spot,
// or a hazard in it,
// means turn round
private shouldTurn(): boolean {
  const aheadX = this.direction < 0 ? this.body.left - 2 : this.body.right +
2;
  const noGround = !this.ground.hasTileAtWorldXY(aheadX, this.body.bottom +
4);
  const hazard = this.hazards.hasTileAtWorldXY(aheadX, this.body.bottom -
4);
  return noGround || hazard;
}
```

The slime is given the two layers in its constructor. It only checks while standing on something (`blocked.down`), so a slime that is falling does not spin round in mid-air.

Stomp, or be hurt?

The hero and the slimes only overlap; the callback decides what the touch means:

src/scenes/GameScene.ts

```
// landing on top of a slime squashes it; any other touch hurts the hero
private heroMeetsSlime(slime: Slime): void {
    if (this.hero.isHurt() || slime.isSquashed()) {
        return;
    }
    const falling = this.hero.body.velocity.y > 0;
    const feetOnTop = this.hero.body.bottom <= slime.body.top + STOMP_MARGIN;

    if (falling && feetOnTop) {
        slime.squash();
        this.hero.bounce();
        this.sound.play(STOMP_SOUND);
    } else {
        this.hurtHero();
    }
}
```

Why the margin? Physics runs in steps. In the step when they first touch, a falling hero is usually a few pixels *into* the slime already, so "feet exactly on top" would almost never be true. Twelve pixels of forgiveness is fair; and "falling" stops a hero who jumps up into a slime from below from counting as a stomp. `squash()` switches the slime's body off at once - so it cannot hurt the hero in the next frame - then tweens it flat and destroys it.

Hazards, fairly

Spikes and lava are tiles, and the hero only overlaps them. But there is a surprise in overlapping a tilemap layer: Phaser calls the callback for **every tile near the body - including empty ones**. A process callback (Chapter 8) decides which tiles count:

src/scenes/GameScene.ts


```
// hazards only OVERLAP. Phaser checks every tile near the hero - even empty
// ones - so the
// process callback decides which count: a hazard tile, with the feet well
// into it
this.physics.add.overlap(this.hero, hazards, () => this.hurtHero(), (_hero,
tile) => {
    return this.isDeadly(tile as Phaser.Tilemaps.Tile);
});
```

src/scenes/GameScene.ts

```
private isDeadly(tile: Phaser.Tilemaps.Tile): boolean {
    return tile.properties.hazard === true && this.hero.body.bottom >
tile.pixelY + HAZARD_MARGIN;
}
```

`tile.properties` is the tile's custom properties from Tiled. The spikes are drawn in the bottom two thirds of their square, so the hero must be 12 pixels *into* the square before they hurt: brushing the top corner of an empty-looking square is not a death. Fair hit boxes are a big part of what makes a hard game feel fair rather than cheap.

Being hurt

When the hero is hurt it plays the classic platformer "ouch": a hop, then a fall straight through the floor - `hurt()` sets `body.checkCollision.none = true` so it collides with nothing. The camera stops following, and a moment later the scene restarts itself, passing on how many attempts there have been:

src/scenes/GameScene.ts

```
this.time.delayedCall(RESTART_DELAY, () => {
    const data: GameData = { attempt: this.attempt + 1 };
    this.scene.restart(data);
});
```

`scene.restart(data)` hands `data` to `init()`, just like `scene.start` (Chapter 2). Reaching the flag fades the camera out and starts `EndScene` with the coins, the time and the number of attempts.



Version 3: three levels, ladders and moving platforms



One scene, every level

All three levels are Tiled maps with the same layers, so one `GameScene` can play any of them. What is different about each level is a list:

`src/config/levels.ts`

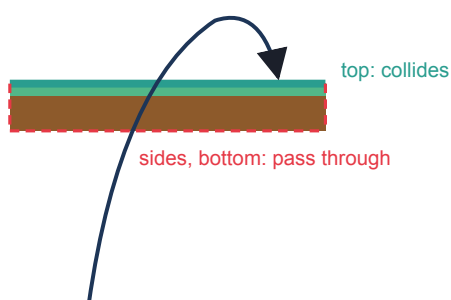
```
export const LEVELS: LevelInfo[] = [
  { key: "level1", file: "assets/maps/level1.tmj", name: "Green Hills",
    skyTint: 0xffffffff },
  { key: "level2", file: "assets/maps/level2.tmj", name: "Deep Caves",
    skyTint: 0x7080a0 },
  { key: "level3", file: "assets/maps/level3.tmj", name: "Lava Castle",
    skyTint: 0xff9070 },
];
```

`PreloadScene` loads every map in the list; `GameScene` is started with `{ level: index }` and builds whichever it is told. A fourth level would be one more map and one more line - no new code. This is **data-driven** design: the code says *how* a level works, the data says *which* levels there are.

One-way platforms

Solid from above only

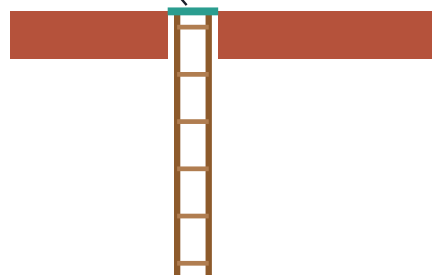
A one-way platform



```
body.checkCollision.down = false;
body.checkCollision.left = false;
body.checkCollision.right = false;
```

The top of a ladder

`tile.setCollision(false, false, true, false)`



Stand on it, walk across it - but while the hero is climbing, the collider's process callback says "no":
`() => !this.hero.isClimbing()`

Rectangles on the Objects layer become `MovingPlatform`s. Each is solid on its top face only:

`src/objects/MovingPlatform.ts`

```
// one-way: only the TOP collides. Jump up through it from underneath, or
// walk through
// its ends; land on it from above
this.body.checkCollision.down = false;
this.body.checkCollision.left = false;
this.body.checkCollision.right = false;
```

`checkCollision` says which faces of a body take part in collisions. With only `up` left, a hero jumping from below passes straight through and lands on top. (Chapter 8 did the same thing another way, with a process callback.)

Platforms that move - and carry the hero

A platform given `moveX` and `moveY` in Tiled goes there and back for ever. It moves by **velocity**, turning round at each end:

`src/objects/MovingPlatform.ts`

```
// called every frame (Phaser calls preUpdate on anything added with
// add.existing that has one)
protected preUpdate(): void {
  if (this.length === 0) {
    return; // a platform that stays still
  }
  // turn round at each end. Comparing distances from the two FIXED ends
  // means small overshoots never add up, however long the game runs
  if (this.outward && this.start.distance(this) >= this.length) {
    this.outward = false;
    this.setVelocity(-this.velocity.x, -this.velocity.y);
  } else if (!this.outward && this.end.distance(this) >= this.length) {
    this.outward = true;
    this.setVelocity(this.velocity.x, this.velocity.y);
  }
}
```

It is `immovable` (the hero cannot push it down) and ignores gravity. Now the physics engine does something clever: when a body stands on an immovable body that moves, the engine moves the rider by the same distance each step. So the hero rides along with no code of ours at all - and a platform rising into the hero lifts it, because the collider pushes the hero out of the way.

Note - Why not a tween, which would give a nicer ease in and out? A tween moves the *picture*, and the physics engine only notices afterwards. Phaser has

`body.setDirectControl(true)` for exactly this case - the engine then works out a speed from how far the tween moved the body - but the engine can take two or three physics steps in one slow frame, and the hero then slides about on the platform. With velocity, the engine moves the platform itself, and knows exactly how far.

Ladders

A ladder needs three things: a way to know the hero is at one, a `"climb"` state, and a top you can stand on.

Finding ladders. The hero should not need to know about tilemaps, so it is given a *function* to ask - the type of a function is written like this in TypeScript:

`src/objects/Hero.ts`

```
// "is there a ladder at (x, y)?" - the answer is the ladder's centre x, or
// null for no ladder
export type LadderFinder = (x: number, y: number) => number | null;
```

`src/scenes/GameScene.ts`

```
this.hero = new Hero(this, start.x!, start.y!, (x, y) => this.findLadder(x,
y));
```

`src/scenes/GameScene.ts`

```
// the function the hero is given: the middle of the ladder at (x, y), or
// null
private findLadder(x: number, y: number): number | null {
  const tile = this.ladders.getTileAtWorldXY(x, y);
  if (tile === null) {
    return null;
  }
  return tile.getCenterX();
}
```

In Java this would be an interface with one method; in TypeScript you can pass the function itself. The same `Hero` would work with ladders made of sprites, or of anything

else.

Climbing. UP with a ladder behind the hero, or DOWN while standing on top of one, switches to the `"climb"` state: gravity off, the hero lined up with the ladder, and the arrow keys now move it up and down:

src/objects/Hero.ts

```
this.setHeroState("climb");
this.x = ladderX; // line up with the ladder, so the body fits the hole it goes through
this.body.setAllowGravity(false);
this.setAcceleration(0, 0);
this.setVelocity(0, 0);
return true;
```

The hero leaves the ladder by jumping, stepping sideways, climbing off the top, or reaching the floor at the bottom - and gravity comes back on.

The top. Each ladder's top square is made solid on its top face only - the tile version of a one-way platform - so the hero can stand on it and walk off onto the ledge beside it:

src/scenes/GameScene.ts

```
private makeLadderTopsSolid(): void {
  this.ladders.forEachTile((tile) => {
    const above = this.ladders.getTileAt(tile.x, tile.y - 1);
    if (tile.index !== -1 && above === null) {
      tile.setCollision(false, false, true, false);
    }
  });
}
```

But a climbing hero must pass *through* that top, going up or down. A process callback on the collider switches it off while climbing:

src/scenes/GameScene.ts

```
// ladder tops are solid - except while climbing, when the hero must pass through them
this.physics.add.collider(this.hero, this.ladders, undefined, () =>
  !this.hero.isClimbing());
```

Bats that swoop

A bat hovers, bobbing gently, until the hero passes underneath - then swoops at where the hero *was*, and flies home. It is a second small state machine:

src/objects/Bat.ts

```
if (this.batState === "hover") {
  this.setVelocity(0, Math.cos(time / 250) * BOB_SPEED);
  if (time > this.stateEnds && this.canSeeTarget()) {
    // aim at where the hero is NOW - if the hero keeps moving, the bat
    misses
    this.batState = "swoop";
    this.stateEnds = time + SWOOP_TIME;
    this.scene.physics.moveToObject(this, this.target, SWOOP_SPEED);
  }
} else if (this.batState === "swoop") {
  if (time > this.stateEnds) {
    this.batState = "return";
    this.scene.physics.moveTo(this, this.home.x, this.home.y, RETURN_SPEED);
  }
}
```

`physics.moveToObject` sets a velocity towards a target at a given speed - the bat does not steer after that, which is what makes it dodgeable. Bats have `setAllowGravity(false)` and no collider with the ground: they fly through walls.

Slimes and bats are both Enemies

The scene treats a slime and a bat the same way when the hero touches one. An **interface** says what they have in common:

src/objects/Enemy.ts

```
export interface Enemy {
  body: Phaser.Physics.Arcade.Body;
  isSquashed(): boolean;
  squash(): void;
}
```

`class Slime extends Phaser.Physics.Arcade.Sprite implements Enemy` - exactly as in Java, and the compiler checks that the class really has everything the interface lists.

Then one handler serves both (it is the intermediate version's `heroMeetsSlime`, with `Enemy` where `Slime` was):

`src/scenes/GameScene.ts`

```
// one overlap, and one handler, for both kinds: Slime and Bat both  
implement Enemy, and  
// that is all heroMeetsEnemy() needs to know  
this.physics.add.overlap(this.hero, [...slimes, ...bats], (_hero, enemy) =>  
{  
    this.heroMeetsEnemy(enemy as Slime | Bat);  
});
```

A collider or overlap takes a game object, a group, or - as here - an array of game objects.

Checkpoints, lives and coming back

Touching a signpost `Checkpoint` moves the **respawn point** there. Being hurt now costs a life (kept in the registry, so the HUD can show it), and then - unless that was the last life - the hero comes back at the respawn point instead of the whole level restarting:

`src/scenes/GameScene.ts`

```
const lives: number = this.registry.get(LIVES) - 1;  
this.registry.set(LIVES, lives);  
  
this.time.delayedCall(RESPAWN_DELAY, () => {  
    if (lives > 0) {  
        this.hero.respawn(this.respawnPoint.x, this.respawnPoint.y);  
        this.cameras.main.startFollow(this.hero, true, 0.1, 0.1);  
    } else {  
        this.endLevel("gameOver");  
    }  
});
```

`respawn()` puts the hero back with `body.reset(x, y)` - which moves the sprite *and* its body and stops it - and gives it a moment of flashing invulnerability, so a bat hovering near the checkpoint cannot hurt it again before the player has even seen where it is. Squashed enemies and collected coins stay gone.

The HUD, fed by the registry



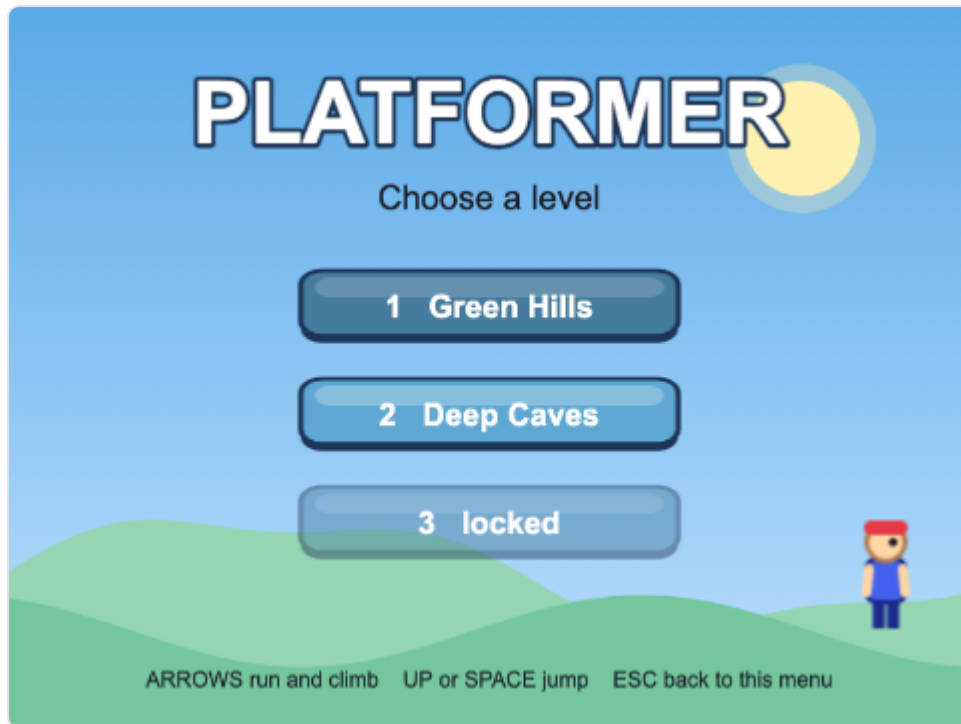
The hearts, coins and level name are drawn by `HudScene`, launched on top of the game (Chapter 4). It never talks to `GameScene`: the game puts numbers in the registry, and the registry announces every change (Chapter 6):

`src/scenes/HudScene.ts`

```
this.refresh();
this.registry.events.on("changedata", this.refresh, this);

// the registry outlives this scene: stop listening when the scene stops, or
// the old
// listener would try to update texts that no longer exist (Chapter 4)
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  this.registry.events.off("changedata", this.refresh, this);
});
```

A level select that unlocks



`MenuScene` shows a button for each level in `LEVELS`; the ones not reached yet are dimmed and do nothing. Which levels are open is remembered in `localStorage` by a small plain class, `Progress` - checked on the way back in, because anything in storage might be missing or have been edited (Chapter 6). Finishing a level calls `Progress.unlockAfter(level, LEVELS.length)` in `ResultScene`. Lives carry from one level into the next; choosing a level from the menu is a new game, with full lives. And the music is started once, in the menu, only if it is not already playing (Chapter 5).

The level generator

Tiled is the right tool for making levels, and you should use it for yours. For this chapter, though, the levels were drawn as text, one character per tile, and turned into Tiled maps by a script - which keeps them easy to read in a book, and easy to change:

```
tools/make_levels.ts
```

```
".....===.....H.....#
###.....",
".....H.....===.....
.....XX.....",
"..P..*..cc.....*..s.....H.....k.....^^...s.....s.....
.....XX...s...*....F....",
"#####.....#####.....#####
#####",
```

is ground (the top of a run becomes grass), H a ladder, ^ spikes, = a platform, and the letters are objects: P the player, c coins, s slimes, k a checkpoint, F the flag. The script writes each map twice - into the project's `public/assets/maps/` for the game, and into its `tilled/` folder with the tileset path Tiled expects - with the tileset and its `collides`, `hazard` and `ladder` properties embedded. Run it from the guide's folder with `deno run -A chapters/ch15_platformer/tools/make_levels.ts`.

Common mistakes

Note - The hero falls through the floor. The collider is missing (`this.physics.add.collider(hero, ground)`), or the layer has no colliding tiles: a Tiled map needs `setCollisionByProperty({ collides: true })` and the property set on the tiles in the tileset. Turn on `debug: true` to see the bodies.

Note - The hero cannot jump. `body.blocked.down` is only set by a collider or the world bounds. If the hero stands on something with `overlap` instead of `collider`, it is never "blocked", and never allowed to jump.

Note - The animation is frozen on its first frame. Either `play(key)` is called every frame without `true` (so it restarts every frame), or a `preUpdate()` override forgot `super.preUpdate(time, delta)`.

Note - Touching any empty square near the spikes kills the hero. Overlapping a tilemap layer calls the callback for empty tiles too; check the tile in a process callback.

Summary

- Arcade gravity in the config pulls every dynamic body down; a jump is an upward velocity, and the arc comes for free
- `body.blocked.down` means "standing on something": only jump when it is true - or was, very recently (coyote time)
- acceleration and drag, a jump cut when the key is released, coyote time and jump buffering are what make the movement feel good; tune the numbers by playing
- a hero's state (`"idle" | "run" | ...`) chooses its animation; remembering it allows states the body cannot show, such as hurt and climbing
- tiles with custom properties (`collides` , `hazard` , `ladder`) let the code ask what a tile is; the Objects layer places everything else
- stomp or hurt is decided in the overlap callback, from the direction and position of the hero; hazards overlap, with a process callback to skip empty tiles and be fair
- `checkCollision` makes one-way platforms; immovable bodies moved by velocity carry whatever stands on them
- one data-driven scene can play every level; lives, checkpoints and a registry-fed HUD scene turn levels into a game

Challenges

1. **Double jump** (*ch15_platformer_simple*) - Let the hero jump once more while in the air. The second jump should be a little lower than the first, and the hero gets it back on landing. Two jumps in the air must not be possible.
2. **Springboard** (*ch15_platformer_simple*) - Add a springboard on the ground that throws the hero high into the air - high enough to reach the top platform straight from the ground. There is no picture for it in the asset library, so draw one with `Graphics` and `generateTexture` .
3. **Coin record** (*ch15_platformer_advanced*) - Show the most coins ever collected in each level on its button in the level select, e.g. "1 Green Hills 15/21", remembered between visits to the page. A level not yet finished shows no record.
4. **Wall jump** (*ch15_platformer_intermediate*) - When the hero is in the air and pressing into a wall, it slides down the wall slowly; pressing jump then pushes it up and away from the

wall. Build a tall shaft into the level (in Tiled, or in the map file) that can only be climbed this way. *Hint:* `body.blocked.left` and `body.blocked.right` say the hero is against a wall. For a moment after a wall jump, ignore the arrow keys, or the player will pull the hero straight back.

5. **Medal times** (*ch15_platformer_advanced*) - Give each level a gold, silver and bronze time. Show a running clock in the HUD; on the result screen, show the medal won (if any), and on the level select show the best medal for each level. *Hint:* the times belong in `LevelInfo`. The HUD can read the start time from the registry and work out the clock in its own `update()`.
6. **A new enemy** (*ch15_platformer_advanced*) - Add ghosts (`ghost.png` in the asset library): they drift slowly towards the hero, through walls, and **cannot** be stomped - landing on one hurts. Place some in the maps. *Hint:* the scene decides what a touch means using only the `Enemy` interface, so the interface will need to say whether an enemy can be stomped. Add points of type `"ghost"` to the Objects layer of a map in Tiled (`tilled/level11.tmj`), and save a copy over the one in `public/assets/maps/`.

Previous: [Chapter 14 - Catch, avoid and shoot](#)